

ĐI TÌM THỬ THÁCH 2

Các bài toán từ Kỳ thi Lập trình
Sinh viên Ba Lan 2011–2014

Bản dịch tiếng Việt
44 bài toán, 2011–2014

Bản dịch phục vụ mục đích giáo dục.

BẢN THẢO • 2026

Ghi chú về bản dịch

Bản PDF này là bản thảo tiếng Việt gồm đủ 44 bài toán và lời giải. Nội dung được ưu tiên dịch từ bản tiếng Anh. Với 33 lời giải chưa có trong bản tiếng Anh, bản tiếng Ba Lan năm 2019 (phiên bản 2.1) sẽ là nguồn chính.

Tên biến, công thức, độ phức tạp và dữ liệu vào/ra được giữ nguyên về ngữ nghĩa. Các chương trong bản xem trước vẫn cần một vòng soát kỹ thuật độc lập trước khi được xem là hoàn chỉnh.

Mục lục

Ghi chú về bản dịch	ii
1 Hình chữ nhật cấp số cộng ★★ ★	1
1.1 Đề bài	1
1.1.1 Dữ liệu vào	2
1.1.2 Dữ liệu ra	2
1.1.3 Ví dụ	2
1.2 Lời giải	2
1.2.1 Tai nạn hàng không	3
1.2.2 Mảnh đất	4
1.2.3 Hình chữ nhật cấp số cộng	7
2 Cuộc đua đường phố Bytean ★★ ★ ★	11
2.1 Đề bài	11
2.1.1 Dữ liệu vào	12
2.1.2 Dữ liệu ra	12
2.1.3 Ví dụ	13
2.2 Lời giải	13
2.2.1 Cấu trúc của các đường trái và phải	16
2.2.2 Lời giải nhanh hơn	17
3 Liệu nó có dừng?	20
3.1 Đề bài	20

3.1.1	Dữ liệu vào	21
3.1.2	Kết quả	21
3.1.3	Ví dụ	21
3.2	Lời giải	21
3.2.1	Chứng minh tính đúng đắn	23
3.2.2	Độ phức tạp	23
3.2.3	Một phép kiểm tra bằng toán tử bit	24
3.2.4	Bài toán Collatz	24
4	Cây và đàn kiến ★★★★★	26
4.1	Đề bài	26
4.1.1	Dữ liệu vào	28
4.1.2	Dữ liệu ra	28
4.1.3	Ví dụ	28
4.2	Lời giải	29
5	Diệt sóc đất ★★	32
5.1	Đề bài	32
5.1.1	Dữ liệu vào	33
5.1.2	Dữ liệu ra	33
5.1.3	Ví dụ	34
5.2	Lời giải	34
5.2.1	Các cấu trúc dữ liệu	35
5.2.2	Chèn một máy CD	35
5.2.3	Xóa một máy CD	36
5.2.4	Thuật toán	36
6	Máy giặt ★★	38
6.1	Đề bài	38
6.1.1	Dữ liệu vào	39
6.1.2	Dữ liệu ra	39
6.1.3	Ví dụ	39
6.2	Lời giải	40

6.2.1	Thuật toán tham lam	40
6.2.2	Chứng minh tính đúng đắn	41
6.2.3	Cài đặt hiệu quả	43
7	Bộ sinh bit ★★★★★	44
7.1	Đề bài	44
7.1.1	Dữ liệu vào	45
7.1.2	Dữ liệu ra	45
7.1.3	Ví dụ	45
7.2	Lời giải	45
7.2.1	Từ điển các đoạn cơ sở	46
7.2.2	Automaton tìm mẫu trong cây	48
8	Trà chiều ★	51
8.1	Đề bài	51
8.1.1	Dữ liệu vào	52
8.1.2	Dữ liệu ra	52
8.1.3	Ví dụ	52
8.2	Lời giải	52
8.2.1	Nhìn vào phần còn lại	52
8.2.2	Quan sát quyết định	53
8.2.3	Thuật toán	54
9	Chỉ số thông minh ★★★★★	55
9.1	Đề bài	55
9.1.1	Dữ liệu vào	56
9.1.2	Dữ liệu ra	56
9.1.3	Ví dụ	56
9.2	Lời giải	57
9.2.1	Đồ thị bù và tập độc lập	57
9.2.2	Quy về bài toán luồng cực đại	58
9.2.3	Lấy lại các sinh viên được chọn	59
9.2.4	Chứng minh tính đúng đắn	61

9.2.5	Độ phức tạp	62
9.3	Ghi chú nguồn	62
10	Hang động ★★★	63
10.1	Đề bài	63
10.1.1	Dữ liệu vào	64
10.1.2	Dữ liệu ra	64
10.1.3	Ví dụ	64
10.2	Lời giải	64
10.2.1	Cách làm trực tiếp	65
10.2.2	Đếm đồng thời cho mọi kích thước	66
10.2.3	Phân tích độ phức tạp	67
10.2.4	Cải tiến thêm	68
10.2.5	Một ghi chú về số lượng ước	68
11	Nhện chữ thập ★★★	70
11.1	Đề bài	70
11.1.1	Dữ liệu vào	70
11.1.2	Dữ liệu ra	71
11.1.3	Ví dụ	71
11.2	Lời giải	71
11.2.1	Tìm ba điểm không thẳng hàng	72
11.2.2	Kiểm tra các điểm còn lại	73
11.2.3	Tính đúng đắn	73
11.2.4	Độ phức tạp	73
11.3	Lưu ý về kiểu dữ liệu	74
12	Máy bán hàng tự động ★★★	75
12.1	Đề bài	75
12.1.1	Dữ liệu vào	76
12.1.2	Dữ liệu ra	76
12.1.3	Ví dụ	76
12.2	Lời giải	76

12.2.1	Thứ tự mua và cách nhìn từ phải sang trái . . .	76
12.2.2	Các tổng tiền tổ trên các hàng	78
12.2.3	Độ phức tạp của quy hoạch động cơ bản . . .	79
12.3	Lời giải nhanh hơn	80
12.3.1	Tính cực đại trượt	81
13	Chuyển xe buýt ★★★	83
13.1	Đề bài	83
13.1.1	Dữ liệu vào	84
13.1.2	Dữ liệu ra	84
13.1.3	Ví dụ	84
13.2	Lời giải	85
13.2.1	Đồ thị có hướng ẩn trong đề bài	86
13.2.2	Một pha: đi sang phải và lên trên	87
13.2.3	Ba hướng còn lại và sắp xếp hiệu quả	89
14	Dãy số	90
14.1	Đề bài	90
14.1.1	Dữ liệu vào	90
14.1.2	Dữ liệu ra	91
14.1.3	Ví dụ	91
14.2	Lời giải	91
14.2.1	Biến dãy thành dãy tuần hoàn	92
14.2.2	Ví dụ minh họa	94
14.2.3	Chứng minh tính đúng đắn	94
14.2.4	Độ phức tạp	95
15	DNA ★★	96
15.1	Đề bài	96
15.1.1	Dữ liệu vào	97
15.1.2	Dữ liệu ra	97
15.1.3	Ví dụ	97
15.2	Lời giải	97

15.2.1 Chứng minh tính đúng đắn	98
15.2.2 Thuật toán	99
15.2.3 Độ phức tạp	99
15.3 Lời cuối	99
16 Tính giá trị biểu thức ★★★★★	101
16.1 Đề bài	101
16.1.1 Dữ liệu vào	102
16.1.2 Dữ liệu ra	102
16.1.3 Ví dụ	102
16.2 Lời giải	103
16.2.1 Phân tích cú pháp và cây biểu thức	103
16.2.2 Quy hoạch động trên cây	104
16.2.3 Vì sao cách trực tiếp chưa đủ nhanh?	106
16.2.4 FFT cho nút cộng	106
16.2.5 Biến phép nhân thành phép cộng bằng generator	107
16.2.6 Tìm generator	108
16.2.7 Chứng minh tính đúng đắn	109
16.2.8 Độ phức tạp	109
17 Công thức 1 ★★★★★	110
17.1 Đề bài	110
17.1.1 Dữ liệu vào	111
17.1.2 Dữ liệu ra	111
17.1.3 Ví dụ	111
17.2 Lời giải	112
17.2.1 Số lần vượt tối đa của một xe	112
17.2.2 Điều kiện cần và xe tới hạn	113
17.2.3 Thuật toán quyết định	114
17.2.4 Chứng minh tính đúng đắn	115
17.2.5 Độ phức tạp	117
18 Cứu khủng long ★★★★★	118

18.1	Đề bài	118
18.1.1	Dữ liệu vào	119
18.1.2	Dữ liệu ra	119
18.1.3	Ví dụ	119
18.2	Lời giải	120
18.2.1	Vùng được bảo vệ là bao lồi	120
18.2.2	Diện tích sau khi thêm một điểm	120
18.2.3	Quét góc để tìm các điểm tiếp xúc	121
18.2.4	Thuật toán và độ phức tạp	123
19	Hydra ★★	125
19.1	Đề bài	125
19.1.1	Dữ liệu vào	126
19.1.2	Dữ liệu ra	126
19.1.3	Ví dụ	126
19.2	Lời giải	127
19.2.1	Một loại đầu có thể tính ngay	127
19.2.2	Các pha của thuật toán	127
19.2.3	Cài đặt hiệu quả	128
20	Nghịch thế ★	130
20.1	Đề bài	130
20.1.1	Dữ liệu vào	131
20.1.2	Dữ liệu ra	131
20.1.3	Ví dụ	132
20.2	Lời giải	132
20.2.1	Thành phần chứa a_1	132
20.2.2	Thuật toán	134
21	Đẻ mai làm ★	136
21.1	Đề bài	136
21.1.1	Dữ liệu vào	137
21.1.2	Dữ liệu ra	137

21.1.3 Ví dụ	137
21.2 Lời giải	137
21.2.1 Thứ tự công việc tối ưu	138
21.2.2 Cách 1: quét từ đầu và theo dõi thời gian rồi .	138
21.2.3 Rút gọn thành một công thức	140
21.2.4 Cách 2: đảo chiều thời gian	140
21.2.5 Độ đúng đắn	141
21.2.6 Độ phức tạp	142
22 Thỏ ★★★	143
22.1 Đề bài	143
22.1.1 Dữ liệu vào	144
22.1.2 Dữ liệu ra	144
22.1.3 Ví dụ	144
22.2 Lời giải	144
22.2.1 Phân tích trường hợp tổng quát	145
22.2.2 Quy hoạch động	147
23 Đường cao tốc ★★★	150
23.1 Đề bài	150
23.1.1 Dữ liệu vào	151
23.1.2 Dữ liệu ra	151
23.1.3 Ví dụ	151
23.2 Lời giải	152
23.2.1 Diễn giải hình học	153
23.2.2 Lời giải ngắn hơn	155
23.2.3 Lời giải nhanh hơn	156
24 Bytehattan ★★★	158
24.1 Đề bài	158
24.1.1 Dữ liệu vào	159
24.1.2 Dữ liệu ra	159
24.1.3 Ví dụ	159

24.2 Lời giải	160
24.2.1 Các mặt của đồ thị phẳng	161
24.2.2 Có thể làm gì thêm?	162
25 Thợ mộc ★★★★★	164
25.1 Đề bài	164
25.1.1 Dữ liệu vào	165
25.1.2 Dữ liệu ra	165
25.1.3 Ví dụ	165
25.2 Lời giải	166
25.2.1 Hình dạng của hai mảnh	166
25.2.2 Các tam giác cực đại	167
25.2.3 Tam giác cực đại chạm các đường	168
25.2.4 Các tam giác còn lại	169
25.2.5 Tổng kết	170
26 Biểu tình ★★	171
26.1 Đề bài	171
26.1.1 Dữ liệu vào	172
26.1.2 Dữ liệu ra	172
26.1.3 Ví dụ	172
26.2 Lời giải	173
26.2.1 Tính các giá trị t_1 và t_2	174
26.2.2 Bài toán đồ thị	175
26.3 Lời cuối	176
27 Kỳ thi ★	177
27.1 Đề bài	177
27.1.1 Dữ liệu vào	177
27.1.2 Dữ liệu ra	178
27.1.3 Ví dụ	178
27.2 Lời giải	178
27.2.1 Thuật toán	180

27.2.2 Độ phức tạp	181
28 Máy bắn tốc độ ★★	182
28.1 Đề bài	182
28.1.1 Dữ liệu vào	183
28.1.2 Dữ liệu ra	183
28.1.3 Ví dụ	183
28.2 Lời giải	183
28.2.1 Các lớp của cây và thuật toán tham lam	184
28.2.2 Cài đặt trong thời gian tuyến tính	186
28.2.3 Chứng minh tối ưu	186
29 Trò chơi bi ★★★★★	189
29.1 Đề bài	189
29.1.1 Dữ liệu vào	190
29.1.2 Dữ liệu ra	190
29.1.3 Ví dụ	190
29.2 Lời giải	191
29.3 Chứng minh tính đúng đắn	192
29.3.1 Giảm về các cấu hình nhỏ	193
29.3.2 Kiểm chứng các cấu hình bị chặn bởi 6	195
29.3.3 Độ phức tạp	197
30 Người hùng ★★	198
30.1 Đề bài	198
30.1.1 Dữ liệu vào	199
30.1.2 Dữ liệu ra	200
30.1.3 Ví dụ	200
30.2 Lời giải	201
30.2.1 Vì sao không thể dùng Dijkstra trực tiếp? . . .	201
30.2.2 Định là một khoảng thời gian được phép . . .	202
30.2.3 Quan sát giúp thuật toán đủ nhanh	203
30.2.4 Cài đặt và phân tích độ phức tạp	203

31 Kỹ thuật di truyền ★★	205
31.1 Đề bài	205
31.1.1 Dữ liệu vào	206
31.1.2 Dữ liệu ra	206
31.1.3 Ví dụ	207
31.2 Lời giải	207
31.2.1 Tìm độ dài tối ưu	207
31.2.2 Biết phần đuôi còn làm được bao nhiêu	208
31.2.3 Cài đặt dừng dây	209
31.2.4 Độ phức tạp	210
31.3 Mẹo cài đặt: mảng có thể xóa về 0 tức thời	211
32 Janošík ★	212
32.1 Đề bài	212
32.1.1 Dữ liệu vào	213
32.1.2 Dữ liệu ra	213
32.1.3 Ví dụ	213
32.2 Lời giải	214
32.2.1 Mô phỏng trực tiếp	214
32.2.2 Không làm cùng một việc hai lần	215
32.2.3 Giải hệ thức truy hồi	216
32.2.4 Rút gọn tổng	217
33 Chặn đi biên ★★★	219
33.1 Đề bài	219
33.1.1 Dữ liệu vào	220
33.1.2 Dữ liệu ra	220
33.1.3 Ví dụ	220
33.2 Lời giải	221
33.2.1 Đòi thứ tự cộng	221
33.2.2 Quét theo trục x	222
33.2.3 Đóng góp khi thêm một chiếc chặn	222
33.2.4 Cấu trúc dữ liệu	223

33.2.5 Độ phức tạp và độ chính xác	224
34 Luật sư ★	225
34.1 Đề bài	225
34.1.1 Dữ liệu vào	226
34.1.2 Dữ liệu ra	226
34.1.3 Ví dụ	226
34.2 Lời giải	227
34.2.1 Thuật toán	228
34.2.2 Độ phức tạp	229
35 Xăng ★★★	230
35.1 Đề bài	230
35.1.1 Dữ liệu vào	231
35.1.2 Dữ liệu ra	232
35.1.3 Ví dụ	232
35.2 Lời giải	232
35.2.1 Đặc trưng của một đường đi tốt	233
35.2.2 Thuật toán offline	234
35.3 Trả lời truy vấn trực tuyến	235
35.3.1 Xây dựng rừng	236
35.3.2 Giới hạn chiều cao	238
36 Giá cả ★★	239
36.1 Đề bài	239
36.1.1 Dữ liệu vào	240
36.1.2 Dữ liệu ra	240
36.1.3 Ví dụ	240
36.2 Lời giải	241
36.2.1 Lời giải thứ nhất	241
36.2.2 Biểu diễn tập con	242
36.2.3 Các tập con của một tập con	243
36.2.4 Một cải tiến nhỏ và lời giải chuẩn	245

36.2.5 Lời giải thay thế	246
37 Ước số ★	249
37.1 Đề bài	249
37.1.1 Dữ liệu vào	249
37.1.2 Dữ liệu ra	250
37.1.3 Ví dụ	250
37.2 Lời giải	250
37.2.1 Thuật toán	252
37.2.2 Chứng minh tính đúng đắn	253
37.2.3 Độ phức tạp	254
37.3 Lời kết	254
38 Nim Euclid ★★	256
38.1 Đề bài	256
38.1.1 Dữ liệu vào	257
38.1.2 Dữ liệu ra	257
38.1.3 Ví dụ	257
38.2 Lời giải	258
38.2.1 Phân tích các trường hợp	259
38.2.2 Thuật toán	263
38.2.3 Chứng minh tính đúng đắn	264
38.2.4 Độ phức tạp	264
39 Cột trụ ★★★	266
39.1 Đề bài	266
39.1.1 Dữ liệu vào	267
39.1.2 Dữ liệu ra	267
39.1.3 Ví dụ	268
39.2 Lời giải	268
39.2.1 Chia mặt bằng thành các mảnh	269
39.2.2 Cách dựng đơn giản hơn	271
39.2.3 Độ phức tạp	272

40 Nóng lên toàn cầu ★★★	273
40.1 Đề bài	273
40.1.1 Dữ liệu vào	274
40.1.2 Dữ liệu ra	274
40.1.3 Ví dụ	274
40.2 Lời giải	275
40.2.1 Cố định vị trí của cực tiểu duy nhất	275
40.2.2 Sáu vị trí biên	276
40.3 Cài đặt	277
40.3.1 Tìm $l_{i,1}$ và $r_{i,1}$	278
40.3.2 Tìm bốn vị trí của cực đại	278
40.3.3 Hiệu chỉnh hai phía	279
40.3.4 Thuật toán hoàn chỉnh	279
40.3.5 Độ phức tạp	280
41 Bản hit của mùa ★★★★★	281
41.1 Đề bài	281
41.1.1 Dữ liệu vào	282
41.1.2 Dữ liệu ra	282
41.1.3 Ví dụ	282
41.2 Lời giải	283
41.2.1 Kiểm tra một khuôn phủ	284
41.2.2 Lời giải ngây thơ	285
41.2.3 Cài đặt hiệu quả hơn	285
41.2.4 Thuật toán phân nhánh	287
41.2.5 Lời giải chuẩn	288
41.3 Lời kết	290
42 Dàn dựng ★★★★★	291
42.1 Đề bài	291
42.1.1 Dữ liệu vào	292
42.1.2 Dữ liệu ra	293
42.1.3 Ví dụ	293

42.2 Lời giải	293
42.2.1 Một chu trình với lịch bắn cố định	294
42.2.2 Hỗ trợ các thay đổi	295
42.2.3 Cài đặt	296
43 Hang động ★★	298
43.1 Đề bài	298
43.1.1 Dữ liệu vào	299
43.1.2 Dữ liệu ra	300
43.1.3 Ví dụ	300
43.2 Lời giải	300
43.2.1 Lời giải đơn giản	301
43.3 Chia để trị	301
43.4 Lời giải nhanh hơn	303
43.4.1 Cài đặt thuật toán tuyến tính	305
44 Thuyền trưởng ★★	306
44.1 Đề bài	306
44.1.1 Dữ liệu vào	307
44.1.2 Dữ liệu ra	307
44.1.3 Ví dụ	307
44.2 Lời giải	308
44.2.1 Chi phí của một chặng	309
44.2.2 Chèn các điểm trung gian mà không tăng chi phí	311
44.2.3 Chỉ cần nối các điểm kề nhau theo từng tọa độ	313
44.2.4 Dựng đồ thị và chạy Dijkstra	313
44.2.5 Độ phức tạp	314
Thông tin nguồn	315

Hình chữ nhật cấp số cộng



Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Radoszewski

Bộ nhớ: 128 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/ary>

1.1 Đề bài

Cho một bảng gồm $n \times m$ ô vuông đơn vị. Mỗi ô chứa một số nguyên. Ta quan tâm đến các **hình chữ nhật cấp số cộng** trên bảng: đó là những hình chữ nhật ghép từ các ô vuông đơn vị sao cho các số trên mỗi hàng và mỗi cột đều tạo thành một cấp số cộng. Nhắc lại rằng trong một cấp số cộng, hiệu của hai số hạng liên tiếp luôn không đổi.

Hãy tìm hình chữ nhật cấp số cộng lớn nhất, tức là hình chứa nhiều ô vuông đơn vị nhất. Chẳng hạn, trong bảng dưới đây, hình chữ nhật cấp số cộng lớn nhất gồm chín ô:

5	3	5	7
2	4	4	4
3	5	3	1

6 3 2 4

1.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên t ($1 \leq t \leq 10\,000$), là số bộ dữ liệu. Mỗi bộ bắt đầu bằng hai số nguyên n, m ($1 \leq n, m \leq 3\,000$). Mỗi dòng trong n dòng tiếp theo chứa m số nguyên thuộc đoạn $[0, 10^9]$, lần lượt là các số trong các ô của bảng. Kích thước mỗi tệp dữ liệu vào không vượt quá 20 MB.

1.1.2 Dữ liệu ra

In ra t dòng, mỗi dòng là đáp án của một bộ dữ liệu. Đáp án là số ô vuông đơn vị nằm trong hình chữ nhật cấp số cộng lớn nhất của bảng tương ứng.

1.1.3 Ví dụ

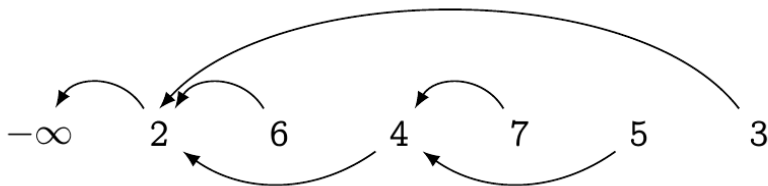
Input	Output
2	9
4 4	6
5 3 5 7	
2 4 4 4	
3 5 3 1	
6 3 2 4	
2 3	
0 1 2	
1 2 3	

1.2 Lời giải

Để trình bày lời giải, ta dùng hai bài toán phụ quen thuộc hơn. Mỗi bài toán sau sẽ được quy về bài toán đứng ngay trước nó.

1.2.1 Tai nạn hàng không

Cho dãy n số nguyên $a_1, \dots, a_n$. Với mỗi phần tử, ta muốn tìm phần tử nhỏ hơn gần nhất nằm bên trái nó. Chính xác hơn, với mỗi $i \in \{1, \dots, n\}$, cần tìm chỉ số j lớn nhất thỏa $j < i$ và $a_j < a_i$. Để kết quả luôn tồn tại, thêm phần tử canh $a_0 = -\infty$.

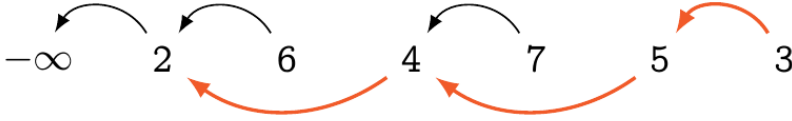


Hình 1.1: Hình 1. Các mũi tên tới phần tử nhỏ hơn gần nhất của dãy 2, 6, 4, 7, 5, 3, với phần tử canh $-\infty$.

Tên gọi của bài toán bắt nguồn từ một cách phát biểu khá nghiệt ngã: biết số người thiệt mạng trong các tai nạn máy bay qua từng năm; với mỗi tai nạn, cần xác định đã bao lâu rồi mới lại có một tai nạn nghiêm trọng đến thế. Trong biến thể đó, ta tìm phần tử lớn hơn gần nhất bên trái. Sau đây ta quay lại phát biểu ban đầu.

Bài toán được giải trong thời gian tuyến tính bằng cách “đi theo các mũi tên”. Duyệt dãy từ trái sang phải và gán cho mỗi phần tử một mũi tên trỏ đến phần tử nhỏ hơn gần nhất. Khi xử lý a_i , trước hết kiểm tra $a_{i-1} < a_i$. Nếu đúng, mũi tên từ a_i trỏ thẳng đến a_{i-1} . Nếu không, đi theo mũi tên của a_{i-1} đến phần tử nhỏ hơn gần nhất của nó. Tiếp tục như vậy cho đến khi gặp một phần tử nhỏ hơn a_i .

Để chứng minh thời gian chạy tuyến tính, chỉ cần chỉ ra rằng mỗi mũi tên được đi qua nhiều nhất một lần. Giả sử khi xác định mũi tên cho a_i , ta đi qua mũi tên xuất phát từ a_j , với $j < i$. Khi đó $a_j \geq a_i$; nếu không thì a_j đã là một ứng viên cho kết quả của a_i . Gọi a_k là phần tử cuối cùng tìm được cho a_i . Khi xử lý các phần tử $a_{i+1}, a_{i+2}, \dots$, ta sẽ không bao giờ gặp lại a_j trong quá trình đi theo mũi tên. Muốn đến a_j , ở một thời điểm nào đó phải nhảy qua a_i ;



Hình 1.2: Hình 2. Quá trình xác định mũi tên cho phần tử cuối $a_6 = 3$; các mũi tên màu cam là những mũi tên được lần ngược.

nhưng mũi tên nhảy qua a_i phải đi vào một phần tử nhỏ hơn a_i , và phần tử gần nhất như vậy là a_k . Vì thế mũi tên ấy không thể đi đến a_j , vốn nằm bên phải a_k .

1.2.2 Mảnh đất

Xét bảng nhị phân A gồm n hàng và m cột. Cần tìm một hình chữ nhật toàn số 1 có diện tích lớn nhất. Ta gọi đây là bài toán tìm **mảnh đất tối ưu**, theo tên một bài của Olympic Tin học Ba Lan lần thứ 9.

Bài toán có lời giải $O(nm)$. Ký hiệu (i, j) là ô ở hàng thứ i từ trên xuống và cột thứ j từ trái sang. Với mỗi ô, tính

$$D[i, j] = \text{số ô 1 liên tiếp đi xuống dưới, kể cả ô } (i, j).$$

Duyệt từ dưới lên: nếu $A[i, j] = 1$ thì $D[i, j] = D[i + 1, j] + 1$, ngược lại $D[i, j] = 0$. Tất cả các giá trị được tính trong $O(nm)$.

Quan sát chủ chốt là mảnh đất tối ưu có thể thu được như sau. Chọn một ô (i, j) , dựng hình chữ nhật có ô ấy trên cạnh trên, chiều cao $D[i, j]$, rồi kéo dài sang trái và phải tối đa cho đến biên bảng hoặc đến một ô 0. Ta sẽ thử mọi cách chọn (i, j) .

Thật vậy, một hình chữ nhật tối ưu có thể được giả sử là cực đại: mỗi cạnh của nó hoặc trùng với biên bảng, hoặc kề một ô 0. Đặc biệt điều này đúng với cạnh dưới. Chọn một ô i_0, j ở hàng dưới cùng kề ô 0 (hoặc chọn tùy ý nếu cạnh dưới trùng biên bảng), và gọi i là hàng trên cùng. Hình chữ nhật có chiều cao $D[i, j]$, chứa (i, j) trên cạnh trên, và không kéo dài thêm được về trái hay phải. Nó đúng là một trong các hình mà cách dựng trên xét tới.

1	0	0	1	1
0	1	1	1	1
1	1	1	1	0
1	1	1	1	0
0	1	1	0	1

Hình 1.3: Hình 3. Một bảng nhị phân và mảnh đất tối ưu gồm chín ô.

1	0	0	4	2
0	4	4	3	1
2	3	3	2	0
1	2	2	1	0
0	1	1	0	1

Hình 1.4: Hình 4. Mảng phụ D của bảng trong hình 3; khung đậm minh họa hình chữ nhật tối ưu.

Có thể phát biểu quan sát ấy theo cách khác. Với mỗi ô (i, j) có $D[i, j] > 0$, tìm hai chỉ số gần nhất $j' < j < j''$ sao cho

$$D[i, j'] < D[i, j], \quad D[i, j''] < D[i, j],$$

trong đó j' lớn nhất và j'' nhỏ nhất. Hai cột này nằm ngay ngoài hai cạnh dọc của mảnh đất. Quy ước $D[i, 0] = D[i, m + 1] = 0$, diện tích ứng với ô (i, j) là

$$(j'' - j' - 1)D[i, j].$$

Hai chỉ số j' và j'' được tìm bằng đúng thuật toán “tai nạn hàng không”: một lượt từ trái sang phải và một lượt từ phải sang trái trên mỗi hàng của D . Tổng thời gian vẫn là $O(nm)$.

1.2.3 Hình chữ nhật cấp số cộng

Quay lại bảng $n \times m$ các số nguyên không âm. Trước hết xử lý riêng các hình chữ nhật có một cạnh bằng 1 hoặc 2.

Với hình cao 1, mỗi hàng có thể được chia thành các cấp số cộng cực đại, mỗi dãy dài ít nhất hai và hai dãy liên tiếp có đúng một phần tử chung. Chẳng hạn, dãy $(6, 3, 0, 4, 8)$ được chia thành $(6, 3, 0)$ và $(0, 4, 8)$; dãy $(1, 3, 7, 11, 15)$ được chia thành $(1, 3)$ và $(3, 7, 11, 15)$. Nhờ đó có thể tìm hình cao 1 dài nhất trong $O(nm)$. Làm tương tự cho hình rộng 1, cao 2 và rộng 2.

Từ đây chỉ xét các hình có cả hai cạnh dài ít nhất 3 và quy bài toán về “mảnh đất”. Đánh dấu một ô nếu hình vuông 3×3 nhận ô đó làm tâm là một hình chữ nhật cấp số cộng.

Một hình chữ nhật có hai cạnh ít nhất 3 là hình chữ nhật cấp số cộng khi và chỉ khi mọi ô nằm trong nó, trừ vành biên trong rộng một ô, đều được đánh dấu.

- Chiều thuận là hiển nhiên: nếu hình chữ nhật là cấp số cộng thì mọi hình vuông 3×3 nằm trong nó cũng vậy.
- Ở chiều ngược lại, khi hai ô được đánh dấu kề nhau, các cấp số cộng trong hai hình vuông 3×3 tương ứng ghép lại thành

1	3	7	11	15
2	4	6	8	10
5	5	5	5	5
8	6	4	2	0
6	3	0	4	8

Hình 1.5: Hình 5. Một hình chữ nhật cấp số cộng cực đại có diện tích 16.

1	3	7	11	15
2	4	6	8	10
5	5	5	5	5
8	6	4	2	0
6	3	0	4	8

Hình 1.6: Hình 6. Các ô được khoanh là tâm của những hình vuông cấp số cộng 3×3 .

những cấp số cộng dài hơn. Lập luận điểm này cho thấy mọi dải cao 3 chạy hết chiều rộng và mọi dải rộng 3 chạy hết chiều cao đều là cấp số cộng. Vì vậy mỗi hàng và mỗi cột của toàn hình đều là một cấp số cộng.

Như vậy, sau khi xử lý riêng các hình “mỏng”, việc tìm hình chữ nhật cấp số cộng đủ dày có diện tích lớn nhất được quy về tìm hình chữ nhật toàn ô đánh dấu.

Còn một chi tiết nhỏ. Trong bài toán mảnh đất, ta chỉ cần hình có diện tích ab lớn nhất. Sau khi thêm lại vành biên rộng một ô, diện tích trở thành $(a+2)(b+2)$. Có thể xảy ra $ab > cd$ nhưng $(a+2)(b+2) < (c+2)(d+2)$, nên mảnh đất có diện tích lớn nhất chưa chắc cho hình chữ nhật cấp số cộng lớn nhất.

Tuy nhiên, hình chữ nhật cấp số cộng tối ưu chắc chắn ứng với một **mảnh đất cực đại**, tức là không thể kéo dài theo bất kỳ hướng nào. Thuật toán của bài toán mảnh đất thực chất đã xét mọi mảnh đất cực đại khi tìm đáp án. Chỉ cần đánh giá diện tích sau khi thêm vành cho từng ứng viên, thay vì chỉ giữ ứng viên có diện tích lõi lớn nhất.

Do đó, toàn bộ bài toán được giải trong thời gian tối ưu $O(nm)$.

Cuộc đua đường phố Bytean



Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Bộ nhớ: 64 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/baj>

2.1 Đề bài

Ngày mai, Cuộc đua đường phố Bytean sẽ diễn ra tại trung tâm thành phố Bytetown. Các con phố của Bytetown tạo thành một lưới đều: tất cả đều chạy theo hướng nam–bắc hoặc tây–đông. Người tham gia chỉ được phép chạy trên một số đoạn đường đã cho.

Byteasar có nhiệm vụ đặt biểu ngữ của các nhà tài trợ tại một số giao lộ. Để làm việc đó, anh phải nghiên cứu bản đồ đường chạy. Trên bản đồ có n giao lộ và m đoạn đường thẳng đứng hoặc nằm ngang được đánh dấu. Mỗi đoạn bắt đầu và kết thúc tại một giao lộ, không chứa giao lộ nào khác ở bên trong. Các đoạn đường chỉ có thể cắt nhau tại giao lộ.

Các giao lộ được đánh số từ 1 đến n . Cuộc đua bắt đầu tại giao lộ 1 và kết thúc tại giao lộ n . Các vận động viên được tự chọn lộ trình,

nhưng chỉ được chạy về phía nam và phía đông, đồng thời chỉ được đi trên các đoạn đã đánh dấu. Những đoạn đường trên bản đồ được chọn sao cho, nếu tuân thủ các quy tắc, từ mọi vị trí đều có thể tới đích và mọi vị trí đều có thể đi tới từ giao lộ xuất phát.

Byteasar muốn đặt các biểu ngữ sao cho không vận động viên nào nhìn thấy biểu ngữ của cùng một nhà tài trợ hai lần. Vì vậy, với một số cặp giao lộ, anh cần kiểm tra liệu có lộ trình của một vận động viên đi qua cả hai giao lộ hay không. Cuộc đua diễn ra ngay ngày mai, nên anh rất cần một chương trình hỗ trợ công việc này.

2.1.1 Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, k ($2 \leq n \leq 100\,000$, $1 \leq m \leq 200\,000$, $1 \leq k \leq 300\,000$), lần lượt là số giao lộ trên đường đua, số đoạn đường được đánh dấu và số cặp giao lộ cần kiểm tra.

n dòng tiếp theo mô tả vị trí các giao lộ. Dòng thứ i chứa hai số nguyên x_i, y_i ($-10^9 \leq x_i, y_i \leq 10^9$), là tọa độ của giao lộ i . Ngoài ra, $x_1 \leq x_n$ và $y_1 \geq y_n$. Không có hai giao lộ nào cùng nằm tại một điểm. Các trục tọa độ tương ứng với các hướng địa lý theo cách tự nhiên: trục OX hướng về phía đông, còn trục OY hướng về phía bắc.

Mỗi dòng trong m dòng tiếp theo chứa hai số nguyên a_i, b_i ($1 \leq a_i, b_i \leq n$, $a_i \neq b_i$), mô tả một đoạn nối hai giao lộ mang số hiệu ấy. Tất cả các đoạn đều thẳng đứng hoặc nằm ngang và chỉ có thể giao nhau tại đầu mút chung, tức các giao lộ.

k dòng sau cùng mô tả các cặp cần kiểm tra. Dòng thứ i chứa hai số nguyên p_i, q_i ($1 \leq p_i, q_i \leq n$, $p_i \neq q_i$).

2.1.2 Dữ liệu ra

In ra k dòng. Dòng thứ i chứa TAK (nghĩa là “có” trong tiếng Ba Lan) nếu tồn tại lộ trình của một vận động viên đi qua cả hai giao lộ p_i, q_i , theo thứ tự bất kỳ. Nếu không, in NIE (“không”).

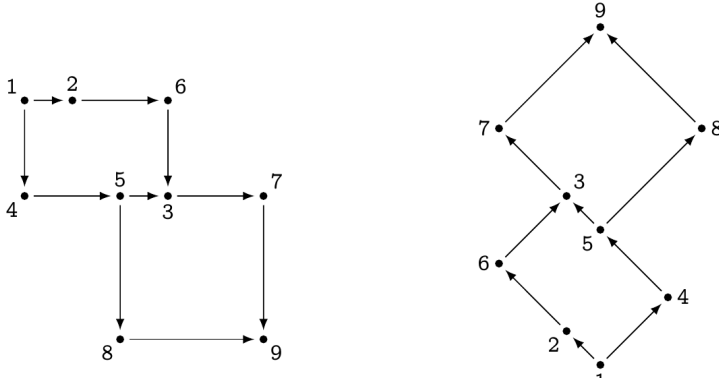
2.1.3 Ví dụ

Input	Output
9 10 4	TAK
1 6	NIE
2 6	NIE
4 4	TAK
1 4	
3 4	
4 6	
6 4	
3 1	
6 1	
1 2	
4 1	
2 6	
2 6	
3 6	
5 4	
5 3	
5 8	
3 7	
7 9	
9 8	
4 8	
2 5	
8 7	
7 6	

2.2 Lời giải

Trước hết, hãy quay bản đồ Bytetown 135° sang trái. Khi đó giao lộ xuất phát nằm ở đáy bản đồ, giao lộ đích nằm trên đỉnh, và các đường rời một giao lộ theo hai hướng chéo: lên trái hoặc lên phải. Phép quay không làm thay đổi đáp án, nhưng cách nhìn này thuận tiện hơn.

Ta xem bản đồ là một đồ thị có hướng: n đỉnh biểu diễn các giao lộ và m cạnh biểu diễn các đoạn đường có thể chạy qua.



Hình 2.1: Hình 1. Bản đồ trong ví dụ trước và sau khi quay 135° . Sau phép quay, đỉnh 1 ở dưới cùng và đỉnh 9 ở trên cùng.

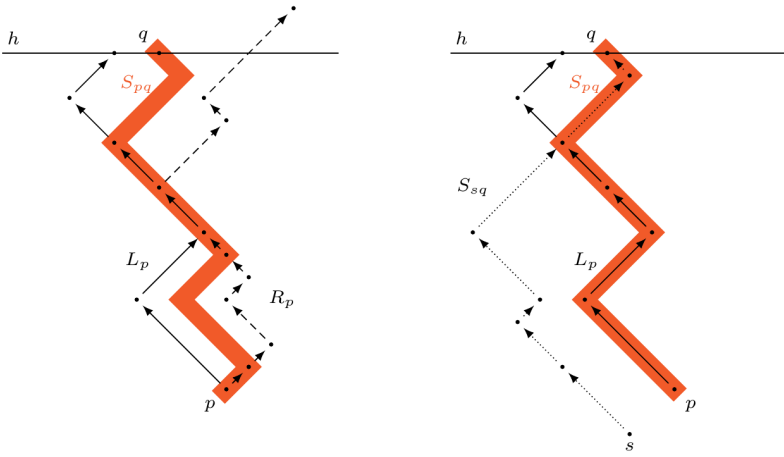
Với một đỉnh v , gọi **đường trái** xuất phát từ v là đường đi bắt đầu tại v , rồi tại mỗi đỉnh tiếp theo luôn rẽ sang trái nếu có thể. Nếu đỉnh chỉ có cạnh sang phải thì đường trái đi theo cạnh ấy. Đường trái kết thúc tại đỉnh đích. **Đường phải** được định nghĩa tương tự. Ký hiệu hai đường này là L_v và R_v . Trong ví dụ ở hình 1, L_4 đi qua 4, 5, 3, 7, 9, còn R_4 đi qua 4, 5, 8, 9.

Giả sử cần kiểm tra có thể đi từ đỉnh $p = (x_p, y_p)$ tới đỉnh $q = (x_q, y_q)$ hay không. Kẻ đường thẳng ngang h qua q . Ta giả sử p nằm dưới h ; nếu không thì hiển nhiên không tồn tại đường từ p tới q , trừ trường hợp $p = q$. Vẽ hai đường L_p, R_p .

Điều kiện 1. Đường L_p cắt h tại q hoặc về bên trái q , đồng thời R_p cắt h tại q hoặc về bên phải q .

Nếu có đường S_{pq} từ p tới q , điều kiện này nhất thiết đúng: S_{pq} cắt h tại q ; đường trái từ p không bao giờ có thể nằm về bên phải S_{pq} , và tương tự, đường phải từ p không bao giờ nằm về bên trái S_{pq} .

Chiều ngược lại cũng đúng. Giả sử điều kiện 1 được thỏa mãn. Xét một đường S_{sq} từ đỉnh xuất phát s tới q ; đường này chắc chắn tồn tại vì từ s có thể tới mọi đỉnh. Mọi đường tới q bắt đầu tại p



Hình 2.2: Hình 2. Bên trái: đường S_{pq} nằm trong miền kẹp giữa L_p và R_p . Bên phải: S_{sq} cắt L_p , nhờ đó ghép được một đường từ p đến q .

hoặc ở phía dưới p đều phải cắt L_p hoặc R_p . Giả sử S_{sq} cắt L_p . Ta đi từ p theo L_p tới giao điểm, rồi chuyển sang S_{sq} và đi tiếp tới q . Trường hợp cắt R_p hoàn toàn tương tự. Vậy để kiểm tra khả năng đi từ p tới q , chỉ cần kiểm tra điều kiện 1.

2.2.1 Cấu trúc của các đường trái và phải

Các đường trái và phải có cấu trúc rất chặt chẽ. Muốn dựng đường trái từ v , ta đi theo cạnh đầu tiên — sang trái nếu có, nếu không thì sang phải — để tới w . Từ đó trở đi, phần còn lại của L_v trùng hoàn toàn với L_w .

Ta dựng mảng hai chiều `left` để di chuyển nhanh trên các đường trái bằng kỹ thuật **binary lifting**. Với mọi đỉnh v và $0 \leq i \leq \lfloor \log_2 n \rfloor$, `left[v][i]` là đỉnh đạt tới sau 2^i bước trên L_v . Mảng `right` cho các đường phải được dựng tương tự. Để tiện xử lý, thêm tại đỉnh đích một khuyên đi từ chính nó về nó; nhờ vậy mọi ô trong hai mảng đều được xác định.

Giá trị `left[v][0]` chính là đỉnh ở cuối cạnh đầu tiên của L_v . Sau đó tính lần lượt theo công thức

$$\text{left}[v][i] = \text{left}[\text{left}[v][i-1]][i-1].$$

Thật vậy, trước hết ta đi 2^{i-1} bước từ v để tới w , sau đó đi thêm 2^{i-1} bước trên L_w , vốn trùng với phần còn lại của L_v . Hai bảng `left` và `right` đều có $O(n \log n)$ ô, nên được dựng trong tổng thời gian $O(n \log n)$.

Sau khi có hai bảng, ta kiểm tra điều kiện 1 bằng một dạng tìm kiếm nhị phân. Chẳng hạn, cần tìm nơi L_p cắt đường h . Bắt đầu ở p , ta đi dọc L_p nhưng luôn bảo đảm chưa vượt lên trên h . Với $i = \lfloor \log_2 n \rfloor, \dots, 0$, xét đỉnh đạt được sau 2^i bước từ đỉnh hiện tại. Nếu đỉnh đó cao hơn h thì không đi; nếu nó nằm trên chính h hoặc ở dưới h thì thực hiện bước nhảy 2^i .

Kết thúc quá trình, ta ở đỉnh cao nhất w trên L_p mà không cao hơn h . Chỉ còn xác định cạnh đầu tiên của L_w cắt h về phía nào so với q . Làm tương tự với R_p là kiểm tra được điều kiện 1.

Như vậy, sau $O(n \log n)$ thời gian tiền xử lý, mỗi truy vấn được trả lời trong $O(\log n)$.

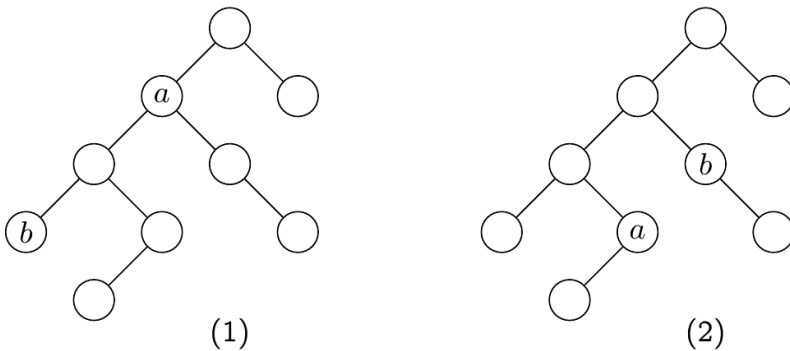
2.2.2 Lời giải nhanh hơn

Mặc dù lời giải trên đã đủ nhanh, ta còn có một thuật toán thú vị với thời gian tiền xử lý tuyến tính. Trước hết xét một bài toán tưởng như không liên quan.

Vị trí tương đối của hai đỉnh trong cây

Xét một cây nhị phân có gốc, trong đó mỗi đỉnh có không, một hoặc hai con. Nếu có hai con, ta phân biệt con trái và con phải. Chọn hai đỉnh khác nhau a, b , và gọi S_a, S_b là các đường từ gốc tới chúng.

Chỉ có đúng bốn khả năng: a nằm **trên** b (tức $a \in S_b$); a nằm **dưới** b (tức $b \in S_a$); a nằm **bên trái** b (khi đi từ a về gốc, ta nhập vào S_b từ bên trái); hoặc a nằm **bên phải** b . Hai trường hợp đầu đối xứng với nhau, hai trường hợp sau cũng vậy.



Hình 2.3: Hình 3. Hai trường hợp tiêu biểu: a nằm trên b , và a nằm bên trái b . Hai trường hợp còn lại thu được bằng cách đổi vai a, b .

Chạy DFS trên cây. Với mỗi đỉnh v , ghi thời điểm vào I_v và thời điểm ra O_v , tức thời điểm thăm v lần đầu và thời điểm ngay sau khi

đã duyệt xong toàn bộ cây con của v . Vị trí tương đối của hai đỉnh phân biệt a, b được xác định như sau:

- a nằm trên b khi và chỉ khi $I_a < I_b$ và $O_b < O_a$;
- a nằm dưới b khi và chỉ khi $I_b < I_a$ và $O_a < O_b$;
- a nằm bên trái b khi và chỉ khi $O_a < I_b$;
- a nằm bên phải b khi và chỉ khi $O_b < I_a$.

Vì thế, sau $O(n)$ thời gian tiền xử lý, ta xác định được vị trí tương đối của hai đỉnh trong $O(1)$.

Lời giải thời gian tuyến tính

Ta dựng đồ thị của các đường trái bằng cách lấy hợp của đường trái xuất phát từ mọi đỉnh. Đồ thị này có cùng n đỉnh với bản đồ Bytetown, nhưng từ mỗi đỉnh, trừ đỉnh đích, chỉ giữ đúng một cạnh: cạnh trái nếu có, nếu không thì cạnh phải.

Đồ thị thu được là một cây nhị phân có gốc tại đỉnh đích. Đây cũng là lý do ta quay bản đồ để đích ở trên: cây đường trái có gốc ở trên và mọc xuống dưới như cách thường vẽ cây. Gọi cây này là T_L . Tương tự, dựng cây đường phải T_R .

Một lần nữa, cần kiểm tra có đường từ p tới q hay không. Kẻ đường ngang h qua q và xét vị trí tương đối của p, q trong T_L :

- Nếu p nằm dưới q , đường cần tìm tồn tại ngay trong T_L .
- p không thể nằm trên q nếu về mặt hình học p ở dưới h , vì khi ấy p phải nằm phía trên h .
- Nếu p nằm bên trái q , hai đường L_p, L_q nhập lại tại một đỉnh, trong đó L_p đi vào từ trái và L_q đi vào từ phải. Lần ngược đồng thời hai đường xuống tới h , L_q gặp q , còn L_p gặp h bên trái q .
- Nếu p nằm bên phải q , L_p cắt h bên phải q .

Đây chính là một nửa của điều kiện 1. Do đó điều kiện về đường trái tương đương với: trong T_L , p nằm **dưới** hoặc **bên trái** q . Tương tự, điều kiện về đường phải tương đương với: trong T_R , p nằm **dưới** hoặc **bên phải** q .

Tóm lại, trước tiên dựng T_L, T_R , rồi chạy DFS trên mỗi cây để lấy thời điểm vào/ra. Cả hai bước tốn $O(n)$. Với mỗi cặp không có thứ tự trong đề, chọn đỉnh thấp hơn trên bản đồ đã quay làm p , đỉnh cao hơn làm q . Có thể so sánh độ cao này ngay trên tọa độ ban đầu bằng $x - y$, vì mỗi bước chạy về đông hoặc về nam đều làm đại lượng ấy tăng. Nếu hai đỉnh khác nhau có cùng độ cao thì chúng không thể cùng nằm trên một lộ trình. Sau đó đáp án là TAK khi và chỉ khi đồng thời:

1. p không nằm cao hơn q ;
2. trong T_L , p nằm dưới hoặc bên trái q ;
3. trong T_R , p nằm dưới hoặc bên phải q .

Mọi phép kiểm tra đều là $O(1)$. Tổng thời gian là $O(n + m + k)$ nếu tính cả việc đọc và định hướng m đoạn đường; phần tiền xử lý sau khi đã có đồ thị là $O(n)$, mỗi truy vấn $O(1)$, và bộ nhớ sử dụng là $O(n + m)$.

Kết quả tối ưu này có được nhờ cấu trúc đặc biệt của đồ thị: phẳng, không chu trình, chỉ có đúng một nguồn và một đích. Với đồ thị tổng quát, bài toán khó hơn nhiều: thuật toán được biết đến trong tài liệu nguồn để trả lời truy vấn trong $O(1)$ cần $O(n^{2.38})$ thời gian tiền xử lý và sử dụng phép nhân ma trận nhanh.

Liệu nó có dừng?

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 64 MB

Mã bài: czy

Nguồn: AMPPZ 2011¹

Bản dịch dựa trên bản tiếng Anh, các trang in 16–18
(trang PDF vật lý 30–32), của *Looking for a Challenge 2*.

3.1 Đề bài

Bajtazar đang đi dạo quanh thư viện của Đại học Warszawa thì nhìn thấy trên một mặt tiền của thư viện một đoạn chương trình kèm dòng chữ “Liệu nó có dừng?”. Câu hỏi có vẻ thú vị, nên sau khi về nhà, Bajtazar đã thử giải đáp nó. Không may, khi chép lại đoạn mã, cậu đã ghi nhầm thành:

Pseudocode

```
while n > 1 do
  if n mod 2 = 0 then
    n := n / 2
  else
    n := 3 · n + 3
```

¹<https://oi.edu.pl/en/archive/amppz/2011/czy>

Bajtazar muốn xác định những giá trị ban đầu nào của biến n làm chương trình mà cậu đã chép dừng lại. Ta giả sử biến n có kích thước không giới hạn, nghĩa là nó có thể nhận các giá trị lớn tùy ý.

3.1.1 Dữ liệu vào

Dòng đầu tiên và duy nhất chứa một số nguyên n ($2 \leq n \leq 10^{14}$). Hãy kiểm tra chương trình đã cho có dừng với giá trị n này hay không.

3.1.2 Kết quả

Trên dòng đầu tiên và duy nhất, in một từ TAK (nghĩa là “có” trong tiếng Ba Lan) nếu chương trình dừng với giá trị n đã cho; ngược lại, in NIE (nghĩa là “không”).

3.1.3 Ví dụ

Dữ liệu vào

4

Kết quả

TAK

3.2 Lời giải

Trong đề bài có một đoạn chương trình mà ta cần phân tích hành vi. Vì đã biết mã nguồn, ta có thể chép nguyên đoạn mã đó vào lời giải rồi chạy. Nếu với một giá trị n cho trước mà chương trình dừng, ta lập tức biết câu trả lời. Trường hợp ngược lại khó hơn một chút: làm sao có thể chắc chắn rằng đoạn mã đang chạy sẽ không bao giờ dừng?

Ta có thể bắt đầu bằng một giả định rất táo bạo — và không hẳn có căn cứ: nếu đoạn mã chưa dừng sau, chẳng hạn, 100 000 vòng lặp thì ta coi như nó sẽ không bao giờ dừng.

Cách làm ấy rất dễ cài đặt, nhưng có hai nhược điểm. Thứ nhất, dựa vào đâu ta biết 100 000 vòng lặp là đủ? Thứ hai, nếu biểu diễn n bằng số nguyên có dấu 64 bit (chẳng hạn kiểu long long trong C++), ta sẽ rơi vào một cái bẫy. Dù giá trị ban đầu của n theo đề bài không vượt quá 10^{14} , sau một số vòng lặp nó có thể trở nên lớn hơn rất nhiều. Ví dụ, với $n = 366\,713\,142\,269$, sau khoảng 160 vòng lặp, n đạt

$$10\,010\,331\,589\,553\,303\,736 > 10^{19},$$

trong khi số nguyên có dấu 64 bit chỉ biểu diễn được các số không lớn hơn

$$2^{63} - 1 \approx 9,2234 \cdot 10^{18}.$$

Hãy tiếp cận bài toán theo cách khác. Ta thử quan sát hành vi của đoạn mã với mọi n từ 1 đến 20; máy tính có thể giúp làm việc này. Rất nhanh chóng, ta nhận thấy chương trình dừng với n bằng 1, 2, 4, 8, 16. Với những giá trị còn lại, nó lặp vô hạn vì kể từ một thời điểm nào đó, n lần lượt nhận các giá trị

$$3, 12, 6, 3, 12, 6, 3, 12, 6, \dots$$

Có vẻ như chương trình dừng đúng với các giá trị là lũy thừa của 2. Thật vậy, nếu n là một lũy thừa của 2, đoạn mã sẽ liên tục chia n cho 2 cho đến khi $n = 1$. Do đó, trong trường hợp này chương trình chắc chắn dừng.

Ta có thể đoán rằng với mọi giá trị n còn lại, chương trình sẽ chạy mãi mãi. Phỏng đoán này đúng, nhưng vì sao?

Nếu giá trị ban đầu của n là số lẻ, phép gán

$$n := 3n + 3$$

sẽ được thực hiện. Dễ thấy giá trị mới của n chia hết cho 3. Hơn nữa, từ thời điểm đó trở đi, n luôn chia hết cho 3. Quả vậy, nếu

n chia hết cho 3 thì bất kể ta thực hiện phép gán $n := n/2$ hay $n := 3n + 3$, giá trị mới vẫn chia hết cho 3. Vì vậy, ta không bao giờ có thể đạt tới $n = 1$.

Chỉ còn trường hợp n chẵn nhưng không phải lũy thừa của 2. Khi đó, chương trình sẽ liên tục chia n cho 2 cho đến khi nó trở thành số lẻ. Số lẻ này phải lớn hơn 1, bởi chỉ khi giá trị ban đầu là một lũy thừa của 2 thì quá trình chia liên tiếp mới kết thúc ở 1. Một khi n đã trở thành số lẻ lớn hơn 1, theo phân tích ở trên, chương trình sẽ không dừng.

Vậy toàn bộ bài toán chỉ còn là kiểm tra n có phải lũy thừa của 2 hay không. Có thể kiểm tra điều kiện này rất dễ dàng: chừng nào n còn chia hết cho 2, ta tiếp tục chia nó cho 2. Nếu cuối cùng thu được $n = 1$, giá trị ban đầu là một lũy thừa của 2; ngược lại thì không phải.

3.2.1 Chứng minh tính đúng đắn

Ta chứng minh thuật toán in TAK khi và chỉ khi chương trình dừng.

- Nếu n là lũy thừa của 2, viết $n = 2^k$. Mỗi vòng lặp chia n cho 2, nên sau đúng k lần chia ta thu được $n = 1$ và vòng lặp dừng. Thuật toán cũng chia hết các thừa số 2 và nhận được 1, do đó in TAK.
- Nếu n không phải lũy thừa của 2, sau khi chia hết các thừa số 2, ta thu được một số lẻ lớn hơn 1. Ở vòng tiếp theo, giá trị mới là $3n + 3$, nên chia hết cho 3. Từ đó về sau, cả hai phép biến đổi $n := n/2$ (chỉ được thực hiện khi n chẵn) và $n := 3n + 3$ đều bảo toàn tính chia hết cho 3. Vì 1 không chia hết cho 3, chương trình không thể đạt tới 1 và do đó không dừng. Thuật toán nhận được một số khác 1 nên in NIE.

Vậy thuật toán luôn cho câu trả lời đúng.

3.2.2 Độ phức tạp

Thuật toán thực hiện nhiều nhất $\lfloor \log_2 n \rfloor$ phép chia, nên có độ phức tạp thời gian $O(\log n)$ và dùng $O(1)$ bộ nhớ.

3.2.3 Một phép kiểm tra bằng toán tử bit

Ta còn có thể kiểm tra nhanh hơn bằng các toán tử trên bit. Toán tử AND theo bit (ký hiệu & trong C++) cho kết quả có bit 1 đúng tại những vị trí mà cả hai toán hạng đều có bit 1. Toán tử XOR theo bit (ký hiệu ^) cho kết quả có bit 1 tại những vị trí mà hai toán hạng khác nhau.

Để kiểm tra n có phải lũy thừa của 2 hay không, chỉ cần kiểm tra

$$((n \text{ xor } (n - 1)) \text{ and } n) = n.$$

Một dạng tương đương và quen thuộc hơn, với $n > 0$, là

$$(n \text{ and } (n - 1)) = 0.$$

3.2.4 Bài toán Collatz

Trên mặt tiền thư viện của Đại học Warszawa thực sự có một đoạn mã gần giống đoạn mã trong đề. Tuy nhiên, đề bài nói rằng Bajtazar đã chép nhầm. Đoạn mã gốc trên tường thư viện là:

Pseudocode

```
while n > 1 do
  if n mod 2 = 0 then
    n := n / 2
  else
    n := 3 * n + 1
```

Ở dòng cuối là $n := 3n + 1$ thay vì $n := 3n + 3$. Dù hai đoạn mã chỉ khác nhau rất ít, khác biệt ấy lại có ý nghĩa lớn. Trong bài toán này, ta đã mô tả được tất cả các số làm chương trình dừng. Bài toán tương tự đối với đoạn mã trên tường thư viện khó hơn rất nhiều.

Năm 1937, nhà toán học người Đức Lothar Collatz đưa ra giả thuyết rằng đoạn mã trên dừng với mọi số nguyên dương n . Nhiều nhà toán học đã nghiên cứu giả thuyết này, nhưng tại thời điểm cuốn sách được biên soạn vẫn chưa ai chứng minh hoặc bác bỏ được nó. Bản tiếng Anh cho biết giả thuyết khi ấy đã được kiểm chứng bằng máy tính với mọi

$$n < 20 \cdot 2^{58} \approx 5,7646 \cdot 10^{18},$$

và thường được tin là đúng, nhưng chưa có một chứng minh hình thức. Vì vậy, nếu đề bài dùng đúng phép biến đổi của bài toán Collatz nguyên bản, câu hỏi “Liệu nó có dừng?” sẽ chưa thể được trả lời chắc chắn trong mọi trường hợp.

Cây và đàn kiến ★★★★★

Tác giả đề: Szymon Acedański

Tác giả lời giải: Szymon Acedański, Tomasz Idziaszek

Bộ nhớ: 6 MB

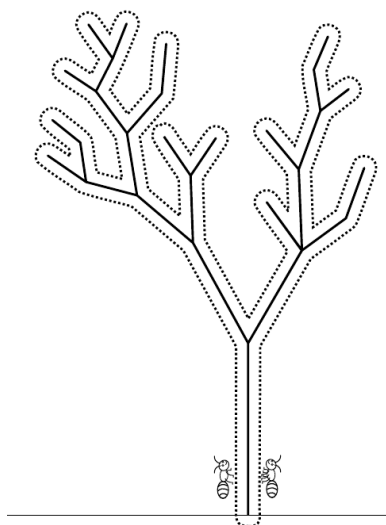
Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/drz>

4.1 Đề bài

Những người làm tin học thích cây. Kiến cũng thích cây. Vì vậy, ta được cho một cái cây có hai con kiến bò trên đó — Kiến Trái và Kiến Phải — theo cách thể hiện trong hình minh họa của đề gốc (chúng bò theo đường chấm). Chúng bắt đầu hành trình ở đầu dưới của thân cây, mỗi con ở một phía đối diện.

Khi đi xa dần khỏi gốc (đi lên), Kiến Trái mất 2 giây để đi hết một cạnh; khi đi về phía gốc (đi xuống), nó mất 1 giây. Kiến Phải nhanh gấp đôi Kiến Trái. Mỗi khi gặp nhau, cả hai con đều quay đầu và bắt đầu bò theo chiều ngược lại. Nếu một con bò từ cây xuống mặt đất, nó lập tức bắt đầu trèo lên phía bên kia của thân cây.

Ngoài ra, hai con kiến nhỏ đến mức ngay cả kính hiển vi cũng không nhìn thấy chúng (trong hình, chúng được cố ý vẽ lớn hơn rất nhiều). Hãy viết chương trình tính thời điểm hai con kiến quay đầu lần thứ hai.



Hình 4.1: Hai con kiến đi theo hai phía của đường bao quanh cây; đường chấm biểu diễn hành trình của chúng.

4.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên (t) ($(1 \leq t \leq 1000)$), là số bộ dữ liệu.

Mỗi bộ dữ liệu gồm hai dòng. Dòng đầu chứa số nguyên chẵn (n) ($(2 \leq n \leq 100,000,000)$), là số cạnh của cây. Dòng thứ hai chứa mô tả của cây: một xâu dài ($n/2$), biểu diễn một số nhị phân ($2n$) bit dưới dạng hệ thập lục phân, dùng các chữ số và các chữ cái thường từ a đến f.

Số này mô tả đường đi vòng quanh toàn bộ cây của Kiến Trái, giả sử Kiến Phải đứng yên. Đọc các bit từ trái sang phải, mỗi bit cho biết trên cạnh tiếp theo của hành trình, Kiến Trái đang đi xa khỏi gốc (bit 1) hay đi về phía gốc (bit 0). Cây có một thân, nghĩa là từ gốc chỉ có đúng một cạnh đi ra.

Kích thước tệp dữ liệu vào không vượt quá 50 MB. Lưu ý rằng con số này lớn hơn đáng kể lượng bộ nhớ dành cho chương trình.

4.1.2 Dữ liệu ra

In ra (t) dòng, lần lượt là đáp án cho các bộ dữ liệu. Mỗi đáp án là thời điểm (tính bằng giây) hai con kiến quay đầu lần thứ hai, viết dưới dạng phân số tối giản (p/q) (không có khoảng trắng quanh dấu /), trong đó (p, q) là các số nguyên dương. Nếu đáp án là số nguyên thì, hiển nhiên, ($q = 1$).

4.1.3 Ví dụ

Input	Output
1	282/5
28	
fb1da30d1b7230	

Bộ dữ liệu mẫu ứng với hình trong đề gốc và được chuyển thành dãy bit sau:

```
1111 1011 0001 1101 1010 0011 0000 1101 0001 1011 0111 0010
↪ 0011 0000
```

4.2 Lời giải

Giới hạn dữ liệu được chọn sao cho không thể lưu toàn bộ mô tả cây trong bộ nhớ. Lời giải chuẩn chỉ đọc mô tả đúng một lần và xử lý trực tuyến, không lưu lại. Phác thảo thuật toán như sau:

- Đọc kích thước cây; nhờ đó biết tổng số đoạn đi “lên” và đi “xuống”.
- Trong khi đọc mô tả, mô phỏng hành trình của Kiến Trái cho đến lần gặp thứ nhất. Trên mỗi cạnh, ta có thể kiểm tra cuộc gặp có xảy ra tại đó hay không: từ quãng đường Kiến Trái đã đi, ta suy ra được quãng đường Kiến Phải phải đi để tới điểm gặp.
- Sau cuộc gặp, cả hai con đi về phía gốc. Tiếp tục đọc dữ liệu, lần này mô phỏng hành trình của Kiến Phải. Điều có vẻ bất ngờ là Kiến Trái tới gốc trước, mặc dù Kiến Phải chạy nhanh hơn. Ta sẽ giải thích điều này ở dưới.
- Sau khi Kiến Trái tới gốc, tiếp tục mô phỏng Kiến Phải cho đến điểm gặp thứ hai.

Sau đây là mô tả chính thức hơn. Một điểm trên cây có thể được biểu diễn bởi cặp số thực $((a,k))$, trong đó:

- (a) là tổng số cạnh Kiến Trái phải đi qua từ gốc để tới điểm ấy; nếu một cạnh được đi qua theo cả hai chiều thì nó được tính hai lần;
- (k) là độ cao của điểm, tức số cạnh trên đường từ gốc đến điểm ấy.

Nếu Kiến Trái mất $(t_{\uparrow})$ giây cho một cạnh đi lên và $(t_{\downarrow})$ giây cho một cạnh đi xuống, thời gian để nó tới $((a,k))$ là

$$[a, k, t_{\uparrow}, t_{\downarrow}] := \frac{a+k}{2}t_{\uparrow} + \frac{a-k}{2}t_{\downarrow}.$$

Vì thế, điểm gặp thứ nhất a_1, k_1 thỏa

$$[a_1, k_1, 2, 1] = [2n - a_1, k_1, 1, \frac{1}{2}]. \quad (1)$$

Ta tìm điểm này bằng cách mô phỏng Kiến Trái trong lúc đọc từng cạnh. Giả sử Kiến Trái đang ở (a, k) , đầu mút của cạnh sắp xét. Đặt $b = 1$ nếu cạnh này đi lên và $b = -1$ nếu nó đi xuống. Nếu cuộc gặp xảy ra sau khi đi một đoạn dài ε trên cạnh thì

$$a_1 = a + \varepsilon, \quad k_1 = k + b\varepsilon.$$

Thế vào (1), ta được

$$\varepsilon = \frac{6n - 9a - k}{9 + b}.$$

Nếu $0 \leq \varepsilon < 1$, ta đã xác định được điểm gặp thứ nhất. Thời gian đi tới đó là

$$t_1 = [a_1, k_1, 2, 1] = \frac{3a_1 + k_1}{2}.$$

Giả sử sau cuộc gặp, ta để cả hai con tiếp tục quay về gốc. Kiến Trái sẽ tới gốc sau

$$[a_1, -k_1, 2, 1] = t_1 - k_1$$

giây tính từ lúc xuất phát, còn Kiến Phải sẽ tới gốc sau

$$[2n - a_1, -k_1, 1, \frac{1}{2}] = t_1 - \frac{k_1}{2}.$$

Do đó Kiến Trái luôn tới gốc trước Kiến Phải. Từ đây ta bắt đầu tìm điểm gặp thứ hai bằng cách mô phỏng hành trình của Kiến Phải.

Giả sử Kiến Phải đi một đoạn dài ε' trên cạnh bắt đầu tại a_0, k_0 , và tới điểm gặp thứ hai a_2, k_2 . Đặt $b' = 1$ nếu cạnh đang xét đi lên, $b' = -1$ nếu nó đi xuống. Khi đó

$$a_2 = a_0 + \varepsilon', \quad k_2 = k_0 + b'\varepsilon',$$

và điều kiện đồng bộ thời gian là

$$[a_1, -k_1, 2, 1] + [2n - a_2, k_2, 2, 1] = [a_2 - a_1, k_2 - k_1, 1, \frac{1}{2}].$$

Suy ra

$$\varepsilon' = \frac{12n - 9(a_0 - a_1) + (k_0 - k_1)}{9 - b'}.$$

Nếu $0 \leq \varepsilon' < 1$, ta đã tìm được điểm gặp thứ hai. Khoảng thời gian từ lần gặp thứ nhất đến điểm này là

$$t_2 = [a_2 - a_1, k_2 - k_1, 1, \frac{1}{2}] = \frac{3(a_2 - a_1) + (k_2 - k_1)}{4}.$$

Vì vậy, tổng thời gian từ lúc xuất phát đến lần gặp thứ hai là

$$t_1 + t_2 = \frac{3(a_1 + a_2) + k_1 + k_2}{4}.$$

Ta còn phải xử lý các phân số mà không dùng số thực. Giá trị ε là bội của $1/(9 + b)$; biến đổi đại số tương tự cho thấy ε' là bội của

$$\frac{9 - b}{(9 + b)(9 - b')}.$$

Vì $b, b' \in \{-1, 1\}$, suy ra a_1, a_2, k_1, k_2 đều là bội của $1/800$. Do đó, mọi phép tính có thể được thực hiện chính xác bằng số nguyên 64 bit: chỉ cần lưu các đại lượng sau khi nhân với 800.

Mỗi bước của thuật toán đọc đúng một cạnh kế tiếp và thực hiện một số hằng phép toán. Vì vậy, độ phức tạp thời gian là $O(n)$, còn bộ nhớ phụ là $O(1)$.

Diệt sóc đất ★★

Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 128 MB

Mã bài: eks

Nguồn: AMPPZ 2011¹

Bản dịch dựa trên bản tiếng Anh, các trang in 23–25
(trang PDF vật lý 37–39), của *Looking for a Challenge 2*.

5.1 Đề bài

Dick Dastardly muốn hành hạ những chú sóc đất Bytean tội nghiệp. Những sinh vật nhỏ hiền lành này sống trong các hang ở vùng cao của dãy núi High Bytemountains.

Dick tìm được một sống núi có n hang sóc đất nằm trên cùng một đường thẳng. Để đơn giản, các hang được đánh số từ 1 đến n , theo thứ tự từ tây sang đông. Dick định tra tấn chúng bằng nhạc rock and roll. Hắn đã mua m máy phát CD, cho vào mỗi máy một album khác nhau của nhóm Bytels và đặt các máy dọc theo sống núi. Biết rằng khi mở hết âm lượng, một máy CD làm phiền mọi chú sóc đất ở các hang có khoảng cách đến máy không vượt quá l mét.

¹<https://oi.edu.pl/en/archive/amppz/2011/eks>

Lo lắng, các chú sóc nhờ bạn xác định những hang mà chúng sẽ không thể ngủ ngon trong mùa đông này. Những sự độc ác của Dick vẫn chưa dừng lại ở đó...

Dick muốn gây rối hơn nữa nên thỉnh thoảng sẽ di chuyển các máy CD. Các chú sóc đã lấy được kế hoạch bí mật của hắn và biết chính xác rằng vào sáng ngày thứ i , Dick sẽ lấy máy CD đang cách hang số 1 một khoảng p_i mét và chuyển nó đến điểm cách hang đó r_i mét. Hãy giúp các chú sóc đếm số hang mà sau mỗi thao tác như vậy, chúng không thể ngủ được.

5.1.1 Dữ liệu vào

Dòng đầu chứa bốn số nguyên n, m, d, l ($2 \leq n, m \leq 500\,000$, $1 \leq d \leq 500\,000$, $1 \leq l \leq 10^9$), lần lượt là số hang sóc đất, số máy CD của Dick, số ngày và bán kính gây nhiễu của một máy CD.

Dòng thứ hai chứa $n - 1$ số nguyên $x_2, x_3, \dots, x_n$ ($0 < x_2 < x_3 < \dots < x_n \leq 10^9$), trong đó x_j là khoảng cách từ hang số j đến hang số 1.

Dòng thứ ba chứa m số nguyên $z_1, z_2, \dots, z_m$ ($0 \leq z_1 < z_2 < \dots < z_m \leq 10^9$), là khoảng cách từ hang số 1 đến các máy CD theo thứ tự. Tất cả các máy CD đều nằm về phía đông của hang này.

d dòng tiếp theo mô tả các lần di chuyển. Dòng thứ i chứa hai số nguyên p_i, r_i ($0 \leq p_i, r_i \leq 10^9$, $p_i \neq r_i$), có nghĩa là vào đầu ngày thứ i , Dick chuyển máy CD đang ở vị trí cách hang số 1 p_i mét đến vị trí cách hang đó r_i mét về phía đông. Có thể giả sử rằng trước thao tác luôn có một máy CD tại vị trí p_i , và không có máy CD nào tại vị trí r_i .

5.1.2 Dữ liệu ra

In ra $d + 1$ dòng. Dòng thứ i , với $1 \leq i \leq d$, chứa một số nguyên: số hang mà sóc đất không thể ngủ ngon trong đêm trước thao tác thứ i của Dick. Dòng cuối cùng chứa số đó sau thao tác cuối cùng.

5.1.3 Ví dụ

Dữ liệu vào

```
5 3 4 1
2 5 6 11
2 4 8
2 1
4 10
8 6
1 8
```

Kết quả

```
2
3
3
5
3
```

5.2 Lời giải

Mọi sự việc trong bài toán diễn ra trên một đường thẳng. Ta biểu diễn vị trí các hàng bằng các điểm. Đồng thời, mỗi máy CD tương ứng với một đoạn: nếu máy ở vị trí x , nó gây nhiễu trên đoạn

$$[x - l, x + l].$$

Vì mọi đoạn đều có cùng độ dài $2l$, để mô tả một đoạn chỉ cần lưu điểm giữa của nó, tức vị trí máy CD. Vị trí các hàng không thay đổi, còn Dick thay đổi vị trí các máy CD. Nhiệm vụ là mô phỏng các thay đổi ấy và sau mỗi lần thay đổi, tính có bao nhiêu **hàng khác nhau** được ít nhất một đoạn gây nhiễu phủ lên.

Điểm mấu chốt là luôn duy trì sẵn số hàng đang bị phủ; khi đó việc in đáp án rất đơn giản. Một lần di chuyển máy CD được tách thành hai thao tác: xóa máy khỏi vị trí cũ, rồi chèn máy vào vị trí mới. Mỗi thao tác chỉ ảnh hưởng cục bộ: những hàng bắt đầu được

phủ hoặc thôi không được phủ chỉ phụ thuộc vào những đoạn kề trực tiếp với đoạn của máy được chèn hay xóa. Tính chất này đúng vì tất cả các đoạn đều có cùng độ dài.

5.2.1 Các cấu trúc dữ liệu

Ta cần quản lý các máy CD bằng một cấu trúc hỗ trợ:

- chèn và xóa một phần tử;
- tìm máy CD gần nhất ở bên trái và ở bên phải của một vị trí.

Một tập có thứ tự, chẳng hạn set trong thư viện chuẩn C++, đáp ứng chính xác các yêu cầu này. Gọi tập lưu vị trí máy CD là M . Không có hai máy nào ở cùng một vị trí theo đề bài.

Các hàng không bao giờ di chuyển, nên chỉ cần lưu chúng vào mảng tăng dần N , đúng theo thứ tự đầu vào. Với một đoạn tọa độ $[A, B]$, số hàng thuộc đoạn đó có thể tìm bằng hai tìm kiếm nhị phân trong N :

$$\text{count}(A, B) = \text{lower_bound}(N, B + 1) - \text{lower_bound}(N, A).$$

5.2.2 Chèn một máy CD

Giả sử chuẩn bị chèn một máy tại vị trí x . Trước khi chèn, hai hàng xóm trực tiếp của x trong M nằm tại x' và x'' , với

$$x' < x < x''.$$

Sau khi chèn x , chỉ những hàng thuộc đoạn của x nhưng chưa bị hai hàng xóm này phủ mới là các hàng **mới** bị gây nhiễu. Các hàng đó chính xác là những hàng trong đoạn

$$[\max(x - l, x' + l + 1), \min(x + l, x'' - l - 1)]. \quad (1)$$

Thật vậy, đoạn của máy bên trái là $[x' - l, x' + l]$, nên phần không bị nó phủ bắt đầu từ $x' + l + 1$. Tương tự, phần không bị máy bên

phải phủ kết thúc ở $x'' - l - 1$. Giao phần còn lại với đoạn $[x - l, x + l]$ của máy mới cho công thức (1).

Do tọa độ đều nguyên, số hàng trong (1) được tính bằng hai lần `lower_bound` như trên. Ta cộng số đó vào đáp án đang duy trì, rồi chèn x vào M .

5.2.3 Xóa một máy CD

Thao tác xóa hoàn toàn tương tự. Giả sử máy tại x có hai hàng xóm trực tiếp x' và x'' . Trước khi xóa, các hàng trong đoạn (1) là đúng những hàng chỉ được máy tại x phủ; sau khi bỏ máy, chúng không còn bị phủ nữa. Vì vậy, ta trừ số hàng thuộc (1) khỏi đáp án, rồi xóa x khỏi M .

Để mỗi máy luôn có cả tiền nhiệm lẫn kế nhiệm, thêm vào M hai phần tử lính canh cố định ở các vị trí

$$-l - 1 \quad \text{và} \quad 10^9 + l + 1.$$

Chúng không thể gây nhiễu bất kỳ hàng nào vì mọi tọa độ trong đề đều thuộc $[0, 10^9]$, nhưng bảo đảm công thức vẫn dùng được tại hai đầu tập.

5.2.4 Thuật toán

1. Khởi tạo M với tất cả các vị trí máy CD và hai lính canh. Khởi tạo đáp án bằng cách lần lượt chèn các máy CD thực sự vào một tập chỉ gồm lính canh, áp dụng công thức (1) mỗi lần.
2. In đáp án ban đầu.
3. Với từng thao tác p_i, r_i : xóa máy ở p_i , chèn máy ở r_i , rồi in đáp án mới.

Mỗi lần chèn hoặc xóa cần $O(\log m)$ để tìm các hàng xóm trong tập M , và $O(\log n)$ cho hai tìm kiếm nhị phân trong mảng hàng N . Vì một lần di chuyển gồm một lần xóa và một lần chèn, độ phức tạp mỗi ngày vẫn là

$$O(\log m + \log n).$$

Khởi tạo mất $O(m(\log m + \log n))$ nếu chèn lần lượt, hoặc có thể tính theo cùng nguyên lý trong $O(m \log n)$ khi đã có tập vị trí ban đầu. Bộ nhớ sử dụng là $O(n + m)$.

Máy giặt ★★

Tác giả đề: Szymon Acedański

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 128 MB

Mã bài: fra

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/fra>

6.1 Đề bài

Một nhóm bạn quyết định giặt quần áo chung. Mọi người đều rất ngăn nắp: mỗi ngày, mỗi người dùng một đôi tất sạch và một chiếc áo sơ mi sạch. Họ đã bỏ tất và áo bẩn vào máy giặt, rồi bắt đầu tính cách phơi chúng.

Để mọi việc có trật tự, họ thống nhất rằng:

- mỗi **đôi tất** được kẹp lên dây bằng đúng một kẹp quần áo;
- mỗi **áo sơ mi** được kẹp bằng đúng ba kẹp;
- toàn bộ tất của cùng một người phải dùng các kẹp có cùng một màu;
- toàn bộ áo sơ mi của cùng một người phải dùng các kẹp có cùng một màu;
- quần áo của hai người khác nhau không được dùng kẹp cùng màu;

- ngoài các điều kiện trên, họ muốn dùng số **màu kẹp** ít nhất có thể.

Họ đã đồ toàn bộ kẹp xuống sàn và đếm được số kẹp của từng màu. Tuy nhiên, họ không biết nên phân các màu cho từng người như thế nào. Hãy viết chương trình giúp họ.

6.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n và k ($2 \leq n, k \leq 1\,000\,000$), lần lượt là số người bạn và số màu kẹp có sẵn.

Dòng thứ hai chứa n số nguyên $d_1, d_2, \dots, d_n$ ($1 \leq d_i \leq 1\,000\,000$). Số d_i là số ngày người thứ i đã tích quần áo cần giặt.

Dòng thứ ba chứa k số nguyên $l_1, l_2, \dots, l_k$ ($1 \leq l_j \leq 4\,000\,000$), trong đó l_j là số kẹp màu j .

6.1.2 Dữ liệu ra

In ra số màu kẹp ít nhất cần dùng để phơi hết quần áo. Nếu không thể phơi theo các quy tắc đã cho, in một từ duy nhất NIE (nghĩa là “không” trong tiếng Ba Lan).

6.1.3 Ví dụ

Dữ liệu vào

```
2 4
3 4
20 10 8 10
```

Dữ liệu ra

```
3
```

Dữ liệu vào

3 8

5 4 3

14 14 14 14 14 14 14 14

Dữ liệu ra

NIE

Trong ví dụ đầu, người thứ nhất cần $2 \cdot 3 = 6$ kẹp cho tất và $3 \cdot 3 = 9$ kẹp cho áo. Người thứ hai cần lần lượt 8 và 12 kẹp. Người thứ hai có thể dùng toàn bộ 20 kẹp màu thứ nhất cho cả tất lẫn áo. Người thứ nhất khi đó có thể dùng, chẳng hạn, hai màu thứ hai và thứ tư.

6.2 Lời giải

Người thứ i cần tổng cộng $5d_i$ kẹp. Có hai cách cấp màu cho người đó:

- dùng **một màu**, nếu một màu có ít nhất $5d_i$ kẹp; hoặc
- dùng **hai màu khác nhau**: một màu có ít nhất $3d_i$ kẹp cho áo và một màu có ít nhất $2d_i$ kẹp cho tất.

Không một màu nào được dùng cho hai người khác nhau. Do đó, một phương án hợp lệ dùng từ n đến $2n$ màu. Muốn tối thiểu hóa số màu thì ta phải làm cho số người dùng được đúng một màu là lớn nhất, đồng thời vẫn phải phát hiện trường hợp không có phương án hợp lệ.

6.2.1 Thuật toán tham lam

Xét những người theo thứ tự không tăng của d_i , tức người có nhiều quần áo hơn được xét trước. Khi tới người có d_i ngày quần áo:

1. Nếu tồn tại một màu còn chưa dùng có ít nhất $5d_i$ kẹp, cấp cho người này màu khả dụng có số kẹp **ít nhất** trong các màu đó.

2. Nếu không, nhưng có thể chọn hai màu còn chưa dùng — một màu có ít nhất $2d_i$ kẹp và một màu khác có ít nhất $3d_i$ kẹp — thì cấp hai màu khả dụng nhỏ nhất thỏa các ngưỡng tương ứng.
3. Nếu cả hai trường hợp đều không thể thực hiện, in NIE.

Sau khi một màu đã được cấp, loại toàn bộ màu đó khỏi tập màu còn có thể dùng. Số màu đã cấp là đáp án.

Việc ưu tiên người có nhiều quần áo là tự nhiên: họ có ít lựa chọn hơn. Nếu có thể dùng một màu thì luôn làm thế, vì đó là cách duy nhất để giảm số màu dùng. Cuối cùng, chọn màu vừa đủ nhỏ nhất để dành các màu lớn hơn cho những người khó tính hơn ở phía sau.

6.2.2 Chứng minh tính đúng đắn

Ta sẽ chứng minh rằng nếu có lời giải thì thuật toán tham lam tìm được một lời giải dùng số màu ít nhất.

Gọi ALG là cách cấp màu do thuật toán tạo ra; nó có thể mới là một phương án một phần nếu thuật toán dừng vì không thể cấp màu cho ai đó. Gọi OPT là một phương án hợp lệ tối ưu, nghĩa là dùng ít màu nhất. Đánh số lại người sao cho

$$d_1 \geq d_2 \geq \dots \geq d_n,$$

đúng theo thứ tự thuật toán xét.

Ta sẽ lần lượt biến đổi OPT sao cho ngày càng nhiều người đầu tiên được cấp giống hệt ALG, mà vẫn hợp lệ và không tăng số màu đã dùng. Giả sử i là người đầu tiên mà hai phương án cấp khác nhau.

Không thể có chuyện ALG không cấp được màu cho người i , bởi khi ấy ALG vẫn có thể lấy đúng các màu mà OPT đã cấp cho người i : ở các người trước đó, hai phương án hoàn toàn trùng nhau. Cũng không thể có chuyện OPT dùng một màu còn ALG dùng hai màu, vì thuật toán luôn dùng một màu nếu điều đó khả thi.

Hai phương án dùng cùng số màu cho người i

Trước hết, giả sử cả hai đều cấp một màu (trường hợp cả hai đều cấp hai màu tương tự). Gọi p là màu mà OPT cấp và a là màu mà ALG cấp. Vì thuật toán chọn màu phù hợp có số kẹp ít nhất, ta có

$$l_a \leq l_p.$$

Nếu OPT chưa dùng màu a , chỉ cần thay màu p của người i bằng a . Nếu OPT đã dùng màu a cho một người j , thì $j > i$, vì những người trước đó đã trùng với ALG. Hoán đổi màu p và a giữa hai người i, j . Màu a đủ cho người i theo cách ALG chọn; màu p đủ cho người j , bởi $l_p \geq l_a$ và $d_j \leq d_i$. Phương án sau hoán đổi vẫn hợp lệ và vẫn dùng cùng số màu.

Lập luận hoán đổi này áp dụng tương tự khi cả hai phương án dùng hai màu: thuật toán luôn lấy các màu khả dụng nhỏ nhất đáp ứng nhu cầu tất và áo, nên các màu OPT nhường lại luôn đủ cho những người xuất hiện muộn hơn.

OPT dùng hai màu, còn ALG dùng một màu

Đây là trường hợp còn lại. Gọi p là màu cấp cho áo, q là màu cấp cho tất trong OPT, và a là màu duy nhất ALG cấp cho người i .

Nếu màu a còn tự do trong OPT, ta thay p, q bằng a , làm giảm số màu đã dùng. Điều này trái với tính tối ưu của OPT. Vì thế, a phải đang được dùng cho một người $j > i$.

Xét cách OPT dùng màu a cho người j :

- Nếu j dùng a cho tất, thì $l_a \geq 2d_j$. Ta có thể thay màu a của j bằng q , vì $l_q \geq 2d_i \geq 2d_j$. Khi đó người i nhận a , còn số màu giảm đi một, mâu thuẫn với OPT tối ưu.
- Nếu j dùng a cho áo, tương tự thay a bằng p , vì $l_p \geq 3d_i \geq 3d_j$, lại dẫn đến mâu thuẫn.
- Khả năng duy nhất không mâu thuẫn là người j dùng riêng màu a cho cả áo lẫn tất, nên $l_a \geq 5d_j$. Khi đó cấp a cho người i , và thay cho người j bằng p cho áo cùng q cho tất. Các ngưỡng

đều đủ vì $d_j \leq d_i$. Số màu không tăng, và người i nay được cấp đúng như ALG.

Như vậy, ở mọi bước ta luôn có thể sửa một lời giải tối ưu để nó trùng ALG thêm một người đầu tiên. Suy luận lặp cho tới hết cho thấy ALG là một lời giải hợp lệ tối ưu. Đồng thời, nếu ALG không thể cấp màu ở một bước thì lời giải OPT giả định không thể tồn tại; do đó in NIE là chính xác.

6.2.3 Cài đặt hiệu quả

Sắp xếp n người theo d_i giảm dần trong $O(n \log n)$. Với các màu kẹ, cần một cấu trúc dữ liệu hỗ trợ:

- tìm màu có số kẹ nhỏ nhất nhưng không ít hơn x ;
- xóa một màu khi nó đã được dùng.

set của thư viện chuẩn C++ đáp ứng cả hai thao tác trong $O(\log k)$: lưu các cặp l_j, j , dùng `lower_bound(x, -∞)` để tìm màu mong muốn, rồi xóa phần tử đã cấp. Khi cần hai màu, hãy tìm màu cho tất và màu cho áo trong tập hiện thời, bảo đảm chúng khác nhau; có thể tạm xóa màu đầu tiên rồi tìm màu thứ hai, hoặc thử hai thứ tự nếu cần. Một cây đoạn tĩnh, cây đếm hoặc cây Fenwick cũng có thể hiện thực các thao tác này.

Khởi tạo cấu trúc cho k màu mất $O(k)$ (không kể chi phí chèn nếu dùng set). Tổng độ phức tạp của thuật toán là

$$O(k + n(\log k + \log n)),$$

và bộ nhớ sử dụng là $O(n + k)$.

Bộ sinh bit ★★★★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Bộ nhớ: 64 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/gen>

7.1 Đề bài

Byteasar thích nghịch bộ sinh bit ngẫu nhiên (thực ra là giả ngẫu nhiên) mà cậu tìm thấy trên máy tính. Bộ sinh hoạt động rất đơn giản. Khi máy tính khởi động, một số nguyên trong đoạn từ 0 đến $m - 1$ được chọn tự động. Số nguyên này được gọi là **hạt giống** (*seed*) của bộ sinh; ta ký hiệu nó bằng biến z . Mỗi lần cần sinh một bit, chương trình gọi hàm sau. Hàm cập nhật hạt giống rồi dùng giá trị mới để sinh ra một bit:

Pseudocode

```
z := floor((z · a + c) / k) mod m
if z < floor(m / 2) then
    return 0
else
    return 1
```

Các số a, c, k là những hằng số. Byteasar đã gọi hàm n lần và nhận

được dãy bit $b_1, b_2, \dots, b_n$. Cậu muốn biết có bao nhiêu giá trị khác nhau có thể là hạt giống ban đầu.

7.1.1 Dữ liệu vào

Dòng đầu chứa năm số nguyên a, c, k, m, n ($0 \leq a, c < m$, $1 \leq k < m$, $2 \leq m \leq 1\,000\,000$, $1 \leq n \leq 100\,000$). Dòng thứ hai chứa một xâu dài n , chỉ gồm các ký tự 0 và 1; ký tự thứ i biểu diễn bit b_i .

7.1.2 Dữ liệu ra

In một số nguyên: số lượng các số nguyên trong đoạn từ 0 đến $m - 1$ có thể đã là hạt giống ban đầu của bộ sinh.

7.1.3 Ví dụ

Input
3 6 2 9 2
10

Output
4

Các hạt giống ban đầu có thể là 1, 2, 7 hoặc 8.

7.2 Lời giải

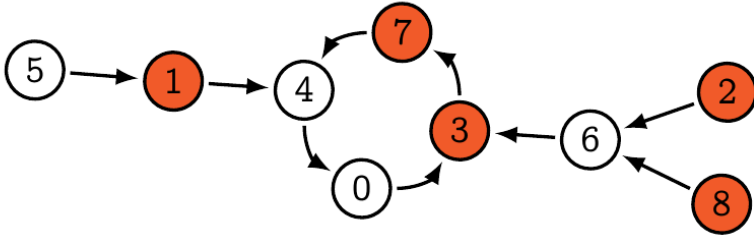
Thoạt nhìn, bài toán có vẻ chỉ là mô phỏng; thực ra lời giải sử dụng cả lý thuyết đồ thị lẫn thuật toán trên xâu. Ta mô tả không gian trạng thái của bộ sinh bằng đồ thị. Sau đó, để tìm những hạt giống phù hợp, ta dùng hoặc **từ điển các đoạn cơ sở** (*dictionary of basic factors*), hoặc một automaton tìm mẫu trong xâu để được lời giải nhanh hơn.

Trạng thái của bộ sinh là số nguyên z trong đoạn 0 đến $m - 1$. Mỗi lần gọi hàm, trạng thái chuyển thành

$$f(z) = \left\lfloor \frac{za + c}{k} \right\rfloor \bmod m,$$

và bit được sinh là $b(z) = 0$ nếu $f(z) < \lfloor m/2 \rfloor$, ngược lại là 1. Vì a, c, k cố định, cả bit sinh ra lẫn trạng thái mới chỉ phụ thuộc vào trạng thái hiện tại z .

Vì vậy, ta biểu diễn bộ sinh bằng một đồ thị có hướng. Đồ thị có m đỉnh $v_0, v_1, \dots, v_{m-1}$, tương ứng với mọi trạng thái có thể. Từ mỗi đỉnh v_i có đúng một cạnh đi tới $v_{f(i)}$, và v_i mang nhãn $b(i)$.



Hình 7.1: Hình 1. Đồ thị chín trạng thái của bộ sinh trong ví dụ; các đỉnh màu cam mang nhãn bit 1.

Gọi $p(v_i, l)$ là dãy nhãn trên đường đi gồm l đỉnh bắt đầu tại v_i . Đường đi này được xác định duy nhất, vì mỗi đỉnh chỉ có một cạnh đi ra. Bài toán trở thành: đếm số đỉnh v_i sao cho $p(v_i, n)$ đúng bằng dãy đầu vào $b_1, \dots, b_n$.

Lời giải hiển nhiên là thử từng đỉnh và sinh n bit, có thời gian $O(nm)$. Phần sau trình bày hai cách cải thiện.

7.2.1 Từ điển các đoạn cơ sở

Với mỗi lũy thừa hai 2^j , trong đó $0 \leq j \leq N = \lfloor \log_2 n \rfloor$, ta gán định danh cho các dãy nhãn dài 2^j , sao cho hai dãy có cùng định danh khi và chỉ khi chúng bằng nhau. Định danh là số nguyên trong đoạn 0 đến $m - 1$, nên có thể so sánh hai dãy trong $O(1)$. Ký hiệu $\text{ident}[v_i, 2^j]$ là định danh của $p(v_i, 2^j)$.

Vì cơ sở dữ liệu không phải một xâu mà là đồ thị, ta cũng phải nhảy nhanh trên đường đi. Ký hiệu $\text{step}[v_i, 1]$ là đỉnh đạt được sau khi đi theo l cạnh, xuất phát từ v_i . Ta tính step cho các l là lũy thừa hai bằng **binary lifting**.

Ta xây từ điển như sau. Với $j = 0$:

$$\text{ident}[v_i, 1] = b(i), \quad \text{step}[v_i, 1] = f(v_i).$$

Với $j = 1, \dots, N$, hãy gán định danh cho các dãy dài 2^j từ hai nửa dài 2^{j-1} . Trước hết, với mỗi đỉnh v_i , lập cặp định danh tạm

$$(\text{ident}[v_i, 2^{j-1}], \text{ident}[\text{step}[v_i, 2^{j-1}], 2^{j-1}]).$$

Phần tử thứ hai chính là định danh của nửa sau, bắt đầu sau 2^{j-1} nhãn đầu của $p(v_i, 2^j)$. Sắp xếp từ điển các cặp này; radix sort cho phép làm trong $O(m)$. Duyệt danh sách đã sắp để thay mỗi cặp bằng một định danh nguyên duy nhất, bảo đảm các cặp giống nhau nhận cùng định danh. Đồng thời tính

$$\text{step}[v_i, 2^j] = \text{step}[\text{step}[v_i, 2^{j-1}], 2^{j-1}].$$

Từ điển cho phép gán định danh cho một dãy dài đúng n . Định danh của $p(v_i, n)$ là cặp gồm định danh của 2^N nhãn đầu và 2^N nhãn cuối:

$$(\text{ident}[v_i, 2^N], \text{ident}[\text{step}[v_i, n - 2^N], 2^N]).$$

Vì $2^N + 2^N > n$, cặp này xác định duy nhất toàn bộ dãy. Giá trị $\text{step}[v_i, n - 2^N]$ được tính trong $O(\log n)$: bắt đầu tại v_i , rồi với mỗi bit 2^j có mặt trong biểu diễn nhị phân của $n - 2^N$, nhảy sang $\text{step}[v_i, 2^j]$.

Để áp dụng cho xâu đầu vào, cho thêm n đỉnh $u_1, \dots, u_n$, trong đó u_i có nhãn b_i và cạnh đi ra tới $u_{\min(i+1, n)}$. Tính các định danh cho cả $m + n$ đỉnh, rồi đếm các v_i có định danh của $p(v_i, n)$ trùng với định danh của $p(u_1, n)$.

Thời gian chạy là $O((m + n) \log n)$, nhưng nếu giữ toàn bộ bảng ident và step thì bộ nhớ cũng là $O((m + n) \log n)$, vượt giới hạn

đề bài. Ta giảm xuống $O(m + n)$ bằng cách, khi xây từ điển, chỉ giữ hai hàng mới nhất của bảng — ứng với 2^{j-1} và 2^j — và tính các giá trị $\text{step}[\cdot, n - 2^N]$ trực tiếp trong quá trình cần dùng.

7.2.2 Automaton tìm mẫu trong cây

Ta còn có lời giải $O(m + n)$. Trước hết, xét một mẫu cố định $x = x_1x_2 \dots x_n$ và một văn bản $y = y_1y_2 \dots y_m$. Ta xây automaton khớp mẫu, có $n + 1$ trạng thái $s_0, \dots, s_n$. Sau khi đọc $y_1 \dots y_j$, automaton ở trạng thái s_i khi hậu tố dài nhất của đoạn đã đọc đồng thời là tiền tố của x có độ dài i . Nói cách khác,

$$y_{j-i+1} \dots y_j = x_1 \dots x_i$$

và không có i lớn hơn thỏa điều kiện này.

Mọi thông tin chuyển trạng thái nằm trong bảng δ : khi ở s_i và đọc ký tự a , automaton chuyển sang $\delta(s_i, a)$. Đặc biệt, với $0 \leq i < n$, có $\delta(s_i, x_{i+1}) = s_{i+1}$. Khởi đầu ở s_0 , đọc văn bản lần lượt; mỗi khi đạt s_n , ta tìm được một lần xuất hiện của mẫu (ký tự vừa đọc là ký tự cuối của lần xuất hiện ấy).

Automaton này được xây trong $O(nA)$, với A là số ký tự khác nhau có thể xuất hiện. Đây là dạng hiện của ý tưởng Knuth–Morris–Pratt: KMP duy trì trạng thái khi chạy nhưng không dựng toàn bộ bảng chuyển. Khớp trên văn bản dài m mất $O(m)$.

Automaton cũng tìm mẫu hiệu quả trong cây có gốc. Nếu mỗi đỉnh cây mang một ký tự, ta cần tìm mọi đường đi bắt đầu ở một đỉnh bất kỳ, đi xuống, và có nhãn ghép thành x . Gán trạng thái automaton cho các đỉnh: nếu gốc mang nhãn a , trạng thái của gốc là $\delta(s_0, a)$. Đỉnh mang nhãn a , có cha ở trạng thái s_i , sẽ nhận trạng thái $\delta(s_i, a)$. Số lần xuất hiện của x là số đỉnh có trạng thái s_n . Thuật toán này mất $O(nA + m)$ thời gian và bộ nhớ.

Không nên chỉ chạy KMP trực tiếp trên cây. Dù tổng thời gian KMP trên một đường đi là tuyến tính, một lần chuyển sau khi đã khớp k ký tự có thể mất $\Theta(k)$. Với những cây có dạng phù hợp, cùng một bước chậm có

thể xảy ra tại nhiều đỉnh trên cùng một mức; tổng thời gian trên cả cây khi đó có thể là bậc hai. Vì vậy cần dựng bảng chuyển của automaton.

Quay lại bài toán. Mẫu cần tìm có độ dài n , nhãn chỉ là 0 và 1, do đó $A = 2$. Ta có thể xử lý từng thành phần liên thông yếu của đồ thị độc lập rồi cộng kết quả. Vì mỗi đỉnh có đúng một cạnh đi ra, mỗi thành phần gồm một chu trình với các cây gắn vào nó. Xét một thành phần có chu trình ℓ đỉnh, mang nhãn $y_1, y_2, \dots, y_\ell$. Ký hiệu T_i là cây gắn vào đỉnh thứ i của chu trình; ta coi chính đỉnh chu trình là gốc của T_i , nên cây có thể chỉ gồm một đỉnh.

Cố định T_i . Các cạnh trong cây hướng “lên trên”, tức hướng về chu trình. Vì vậy để dùng cách khớp mẫu trong cây, ta tìm mẫu đảo

$$x = b_n b_{n-1} \dots b_1.$$

Cách này tìm được các lần xuất hiện nằm hoàn toàn trong T_i . Để bắt cả những lần xuất hiện bắt đầu trong cây rồi đi tiếp qua chu trình, phải khởi tạo đúng trạng thái tại gốc. Ta cần độ dài j của hậu tố dài nhất của dãy vô hạn

$$\dots y_\ell \dots y_1 y_\ell \dots y_1 y_\ell \dots y_i$$

(tức dãy nhãn chu trình lặp vô hạn, cắt ngay sau y_i) mà đồng thời là tiền tố của mẫu x . Khi đó trạng thái khởi đầu ở gốc T_i là s_j .

Các trạng thái khởi đầu cho mọi cây trong một chu trình được tính hiệu quả. Nếu trạng thái cho T_i là s_j , thì trạng thái cho T_{i-1} là $\delta(s_j, y_{i-1})$, với chỉ số tuần hoàn (khi $i = 1$, dùng y_ℓ). Do đó chỉ cần tính trạng thái cho một cây, chẳng hạn T_1 .

Ta cần tìm j , tức độ dài hậu tố dài nhất của dãy

$$\dots y_\ell \dots y_1 y_\ell \dots y_1 y_\ell \dots y_1$$

trùng với tiền tố của x . Một cách trực tiếp là chạy automaton trên $Y = (y_\ell \dots y_1)^k$ với k đủ lớn, nhưng như vậy có thể quá chậm vì mỗi chu trình phải được duyệt nhiều vòng.

Trước hết chạy automaton trên hai vòng $y_\ell \dots y_1 y_\ell \dots y_1$, và giả sử trạng thái cuối là s_i . Nếu $i \leq \ell$, ta có ngay $j = i$. Nếu $i > \ell$, mẫu có thể quấn quanh chu trình nhiều lần. Gọi C là số lớn nhất sao cho tiền tố dài $C\ell$ của x khớp với chu trình; hay nói cách khác, x quấn được C vòng nhưng không quấn được $C + 1$ vòng. Chạy trên $Y = (y_\ell \dots y_1)^{C+1}$ sẽ kết thúc tại s_j .

Khi $C \geq 2$, sau 2ℓ ký tự automaton ở s_i . Nhờ tính tuần hoàn của tiền tố mẫu, sau $C\ell$ ký tự trạng thái là $s_{i+(C-2)\ell}$. Vì vậy, thay vì mô phỏng $C + 1$ vòng, ta khởi tạo trực tiếp ở trạng thái này rồi chỉ đọc thêm ℓ ký tự cuối.

Để tính nhanh C , dùng mảng Pref (còn gọi là z-array): $\text{Pref}[t]$ là độ dài tiền tố dài nhất của x xuất hiện tại vị trí t trong x . Mảng này tính được tuyến tính. Khi đã có nó,

$$C = \left\lfloor \frac{\text{Pref}[\ell + 1]}{\ell} \right\rfloor + 1$$

được suy ra trong $O(1)$.

Tóm lại, đầu tiên xây automaton khớp mẫu đảo $x = b_n \dots b_1$ và mảng Pref trong $O(n)$. Phân rã đồ thị thành các thành phần liên thông yếu. Trong mỗi thành phần, khởi tạo các trạng thái ở chu trình như trên, rồi duyệt các cây gắn vào chu trình để chạy automaton. Mỗi đỉnh và cạnh chỉ được xử lý một số lần hằng số, nên tổng thời gian là $O(m + n)$, bộ nhớ là $O(m + n)$.

Trà chiều ★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Jakub Łącki

Bộ nhớ: 32 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/her>

8.1 Đề bài

Trong chuyến thăm quần đảo Bytic, Byteasar rất thích thức uống truyền thống ở đây: trà pha sữa. Thức uống luôn được pha theo nghi thức cố định. Đầu tiên, một tách được rót đầy hỗn hợp gồm một nửa trà và một nửa sữa. Sau đó chọn một từ nghi lễ dài n , chỉ gồm hai ký tự H và M. Với $i = 1, 2, \dots, n$:

- nếu ký tự thứ i là H, Byteasar uống một nửa tách, rót thêm trà cho đầy tách rồi khuấy;
- nếu ký tự thứ i là M, làm tương tự nhưng rót thêm sữa.

Sau khi thực hiện xong mọi ký tự, phần chất lỏng còn lại bị đổ đi. Byteasar muốn biết tổng lượng mình đã uống: trà nhiều hơn, sữa nhiều hơn, hay hai loại bằng nhau.

8.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên n ($1 \leq n \leq 100\,000$). Dòng thứ hai chứa từ nghi lễ dài n , gồm các ký tự H và M.

8.1.2 Dữ liệu ra

In H nếu Byteasar đã uống nhiều trà hơn; M nếu đã uống nhiều sữa hơn; hoặc HM nếu hai lượng bằng nhau.

8.1.3 Ví dụ

Input

5

HMHHM

Output

H

Trong ví dụ, tổng lượng Byteasar đã uống là $37/64$ tách trà và $59/64$ tách sữa.

8.2 Lời giải

8.2.1 Nhìn vào phần còn lại

Thay vì cộng trực tiếp lượng Byteasar uống, hãy xét lượng **không** uống, tức lượng còn lại sau nghi thức. Tổng lượng đã rót của mỗi loại là biết được, nên từ lượng còn lại suy ra lượng đã uống.

Để tránh phân số, giả sử tách có dung tích 2^{n+1} ml. Ban đầu tách có 2^n ml trà và 2^n ml sữa. Xét 2^n ml trà được rót ở lượt đầu. Sau mỗi lượt tiếp theo, một nửa lượng này bị uống, nên sau $n - 1$ lần pha loãng còn lại 2 ml. Tương tự, phần được rót ở lượt thứ i đóng góp 2^i ml vào lượng cuối cùng. Ngoài ra, luôn còn 1 ml trà và 1 ml sữa từ lần rót ban đầu.

Vì thế, chẳng hạn với từ HMHHM, cuối cùng tách chứa

$$1 + 2^1 + 2^3 + 2^4$$

ml trà và

$$1 + 2^2 + 2^5$$

ml sữa. Ta có thể cộng các lũy thừa của 2 bằng số nguyên lớn, rồi trừ để có đáp án chính xác. Nhưng bài toán chỉ hỏi dấu của hiệu, nên có lời giải đơn giản hơn nhiều.

8.2.2 Quan sát quyết định

Lần rót cuối cùng không ảnh hưởng đến lượng đã uống, vì ngay sau đó Byteasar đổ phần còn lại đi. Do đó bỏ ký tự cuối của từ; từ nay chỉ xét $n - 1$ lượt rót, sau đó tách đầy một nửa.

Gọi t_p, m_p lần lượt là lượng trà, sữa đã rót vào tách, và t_c, m_c là lượng còn lại cuối nghi thức. Lượng đã uống là $t_p - t_c$ và $m_p - m_c$. Vì tách còn đầy một nửa,

$$t_c + m_c = 2^n.$$

Nếu trong $n - 1$ ký tự được xét có nhiều H hơn M, lượng trà đã rót lớn hơn lượng sữa đã rót ít nhất nửa tách:

$$t_p \geq m_p + 2^n.$$

Cuối cùng luôn còn ít nhất 1 ml sữa, nên $t_c \leq 2^n - 1$. Suy ra

$$t_p - t_c \geq t_p - 2^n + 1 \geq m_p + 1.$$

Tức là lượng trà đã uống còn lớn hơn cả **tổng** lượng sữa từng rót, nên chắc chắn lớn hơn lượng sữa đã uống. Lập luận đối xứng đúng khi M xuất hiện nhiều hơn.

Còn khi số H và M bằng nhau, tổng lượng hai loại đã rót bằng nhau. Khi đó loại nào còn ít hơn trong tách là loại đã được uống

nhiều hơn. Nếu $n = 1$, Byteasar chỉ uống một nửa hỗn hợp ban đầu, nên đáp án là HM.

Với $n > 1$, một nửa lượng chất lỏng cuối cùng đến từ lần rót áp chót (lần cuối cùng không bị bỏ qua). Nếu lượt áp chót rót sữa, riêng phần này cộng với 1 ml sữa ban đầu đã bảo đảm lượng sữa còn lại lớn hơn lượng trà; do đó Byteasar đã uống nhiều trà hơn. Lý luận đối xứng cho trường hợp lượt áp chót rót trà.

8.2.3 Thuật toán

1. Nếu $n = 1$, in HM.
2. Bỏ qua ký tự cuối; đếm số H và M trong $n - 1$ ký tự đầu.
3. Nếu một ký tự xuất hiện nhiều hơn, in chính ký tự đó.
4. Nếu hai số bằng nhau, in ký tự **khác** ký tự thứ $n - 1$ (ký tự của lượt áp chót).

Thuật toán dùng $O(n)$ thời gian và $O(1)$ bộ nhớ ngoài sâu đầu vào.

Chỉ số thông minh ★★ ★

Tác giả đề: Marek Cygan

Tác giả lời giải: Eryk Kopczyński

Bộ nhớ: 128 MB

Mã bài: ilo

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/ilo>

9.1 Đề bài

Ở Đại học Byteland chỉ có hai ngành: toán học và khoa học máy tính. Hiện có n sinh viên toán và m sinh viên khoa học máy tính. Hai ngành này khó đến mức không ai đồng thời theo học cả hai ngành.

Byteasar là hiệu trưởng trường. Ông muốn lập một đội sinh viên để giải những bài toán khó nhất của nhân loại. Biết chỉ số thông minh (IQ) của từng sinh viên, ông muốn tổng IQ của đội là lớn nhất có thể.

Tuy nhiên, IQ không phải tất cả: Byteasar còn yêu cầu mọi thành viên trong đội phải quen biết nhau. Mọi sinh viên toán đều quen mọi sinh viên toán khác; tương tự, mọi sinh viên khoa học máy tính đều quen nhau. Hãy giúp Byteasar lập một đội thỏa các điều kiện trên và có tổng IQ lớn nhất.

9.1.1 Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, k ($1 \leq n, m \leq 400, 0 \leq k \leq n \cdot m$), lần lượt là số sinh viên toán, số sinh viên khoa học máy tính, và số cặp sinh viên khác ngành quen nhau.

k dòng tiếp theo mô tả các cặp quen biết. Dòng thứ i chứa hai số nguyên a_i, b_i ($1 \leq a_i \leq n, 1 \leq b_i \leq m$): sinh viên toán số a_i quen sinh viên khoa học máy tính số b_i . Sinh viên trong mỗi ngành đều được đánh số từ 1.

Dòng kế tiếp chứa n số nguyên trong đoạn $[1, 10^9]$, là IQ của các sinh viên toán theo thứ tự đánh số. Dòng cuối chứa m số nguyên, là IQ của các sinh viên khoa học máy tính theo cùng cách.

9.1.2 Dữ liệu ra

Dòng đầu in tổng IQ lớn nhất của một đội thỏa yêu cầu.

Dòng thứ hai in số sinh viên toán được chọn; dòng thứ ba in các số hiệu của họ theo thứ tự bất kỳ. Nếu không chọn sinh viên toán nào, dòng thứ ba để trống.

Hai dòng tiếp theo mô tả tương tự số lượng và các số hiệu sinh viên khoa học máy tính được chọn.

Nếu có nhiều đáp án, được phép in bất kỳ đáp án nào.

9.1.3 Ví dụ

Dữ liệu vào

```
3 2 3
1 1
2 1
2 2
1 3 1
1 2
```

Kết quả có thể có

```
6
1
```

2

2

1 2

Ở đây chọn sinh viên toán số 2, cùng hai sinh viên khoa học máy tính số 1, 2. Ba sinh viên này đôi một quen biết và có tổng IQ $3 + 1 + 2 = 6$.

9.2 Lời giải

Gọi V_1 là tập sinh viên toán, V_2 là tập sinh viên khoa học máy tính. Dựng đồ thị hai phía

$$G = (V_1, V_2, E),$$

trong đó một cạnh biểu diễn một cặp sinh viên khác ngành quen nhau. Mỗi đỉnh v có trọng số $w(v)$, chính là IQ của sinh viên đó.

Trong mỗi phía, mọi cặp sinh viên vốn đã quen nhau. Vì vậy, một tập sinh viên là đội hợp lệ khi và chỉ khi mọi cặp gồm một đỉnh thuộc V_1 và một đỉnh thuộc V_2 trong tập đó đều là cạnh của G . Nói cách khác, ta cần tìm một **clique hai phía có tổng trọng số lớn nhất**.

9.2.1 Đồ thị bù và tập độc lập

Xét đồ thị hai phía bù

$$G' = (V_1, V_2, E'), \quad E' = (V_1 \times V_2) \setminus E.$$

Một cạnh của G' nối đúng hai sinh viên **không** quen nhau. Do đó, một đội hợp lệ trong G chính là một **tập độc lập** trong G' : trong tập được chọn không có cạnh nào của G' .

Phần bù của một tập độc lập là một **phủ đỉnh**. Một phủ đỉnh là tập $P \subseteq V_1 \cup V_2$ sao cho mọi cạnh của G' có ít nhất một đầu mút thuộc P . Vì IQ đều dương, và thực ra với mọi trọng số nói chung, ta có



Hình 9.1: Hình 1. Bên trái: đồ thị quen biết G với clique hai phía lớn nhất tô cam. Bên phải: đồ thị bù G' với một ghép cực đại (cạnh đậm), phủ đỉnh nhỏ nhất (đỉnh đen) và tập độc lập lớn nhất (đỉnh cam).

$$w(V_1 \cup V_2 \setminus P) = w(V_1 \cup V_2) - w(P).$$

Vì tổng trọng số của tất cả đỉnh là hằng số, tối đa hóa trọng số tập độc lập tương đương với tìm **phủ đỉnh có tổng trọng số nhỏ nhất** trong G' . Sau khi tìm được phủ đỉnh nhẹ nhất P , đội cần in chính là

$$(V_1 \cup V_2) \setminus P.$$

9.2.2 Quy về bài toán luồng cực đại

Ta tìm phủ đỉnh nhẹ nhất bằng luồng cực đại / lát cắt nhỏ nhất. Tạo mạng H như sau:

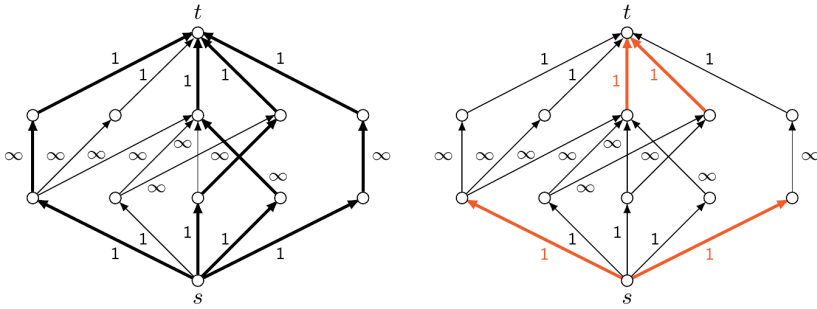
- Thêm nguồn s và đích t .
- Với mỗi $u \in V_1$, thêm cung $s \rightarrow u$ có dung lượng $w(u)$.
- Với mỗi $v \in V_2$, thêm cung $v \rightarrow t$ có dung lượng $w(v)$.
- Với mỗi cạnh $uv \in E'$, thêm cung $u \rightarrow v$ có dung lượng ∞ .

Ở đây ∞ chỉ cần lớn hơn tổng IQ của toàn bộ sinh viên; chẳng hạn đặt $\text{INF} = \text{totalIQ} + 1$ là đủ và tránh tràn số.

Xét một lát cắt hữu hạn của mạng. Nó không thể cắt bất kỳ cung dung lượng vô hạn nào. Do đó, với mỗi cạnh không-quen-biết $uv \in E'$, lát cắt bắt buộc phải cắt hoặc cung $s \rightarrow u$, hoặc cung $v \rightarrow t$. Diễn giải việc cắt $s \rightarrow u$ là đưa u vào phủ đỉnh, và việc cắt $v \rightarrow t$ là đưa v vào phủ đỉnh. Ta nhận được đúng điều kiện “mỗi cạnh của G' có một đầu mút được chọn”.

Ngược lại, từ mọi phủ đỉnh P của G' , cắt các cung $s \rightarrow u$ với $u \in P \cap V_1$ và các cung $v \rightarrow t$ với $v \in P \cap V_2$. Vì P che mọi cạnh $uv \in E'$, không còn đường nào từ s đến t . Giá trị lát cắt đúng bằng tổng IQ của các đỉnh trong P .

Vậy có tương ứng một-một giữa các lát cắt hữu hạn và các phủ đỉnh, đồng thời giá trị của chúng bằng nhau. Theo định lý luồng cực đại – lát cắt nhỏ nhất, giá trị luồng cực đại bằng trọng số phủ đỉnh nhẹ nhất.



Hình 9.2: Hình 2. Mạng luồng H : bên trái là một luồng cực đại (cạnh đậm), bên phải là lát cắt nhỏ nhất tương ứng (cạnh cam).

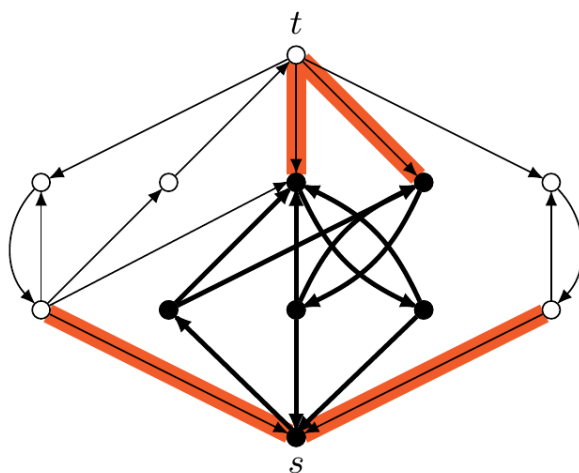
9.2.3 Lấy lại các sinh viên được chọn

Sau khi chạy thuật toán luồng cực đại, xét đồ thị dư. Gọi X là tập các đỉnh đi tới được từ s chỉ qua những cung dư có dung lượng dương.

Lát cắt nhỏ nhất tương ứng gồm các cung của mạng gốc đi từ X sang phần còn lại. Suy ra phủ đỉnh nhẹ nhất là

$$P = (V_1 \setminus X) \cup (V_2 \cap X).$$

Thật vậy, một đỉnh $u \in V_1$ thuộc phủ khi cung $s \rightarrow u$ đi từ X ra ngoài, tức $u \notin X$. Một đỉnh $v \in V_2$ thuộc phủ khi cung $v \rightarrow t$ đi từ X ra ngoài, tức $v \in X$.



Hình 9.3: Hình 3. Đồ thị dư của luồng ở hình 2; các đỉnh đen thuộc tập X , còn các cạnh cam tương ứng với lát cắt nhỏ nhất trong mạng gốc.

Vì đội là phần bù của phủ đỉnh, ta in:

$$\text{đội} \cap V_1 = V_1 \cap X, \quad \text{đội} \cap V_2 = V_2 \setminus X.$$

Tổng IQ đáp án bằng $\text{totalIQ} - \text{maxFlow}$ (tương đương tổng trọng số của các đỉnh vừa in).

9.2.4 Chứng minh tính đúng đắn

Ta chứng minh thuật toán luôn in một đội hợp lệ có tổng IQ lớn nhất.

Bổ đề 1. Một tập sinh viên là đội hợp lệ khi và chỉ khi nó là tập độc lập của G' .

Chứng minh. Trong cùng một ngành, mọi sinh viên quen nhau. Với hai sinh viên khác ngành, họ không quen nhau đúng khi và chỉ khi cặp tương ứng là cạnh của G' . Vì thế mọi thành viên đội một quen nhau đúng khi tập không chứa cả hai đầu mút của bất kỳ cạnh nào trong G' , tức là tập độc lập. $\square$

Bổ đề 2. Phần bù của một tập độc lập trong G' là một phủ đỉnh, và ngược lại.

Chứng minh. Nếu phần bù không che một cạnh, cả hai đầu mút của cạnh đó đều nằm trong tập độc lập — mâu thuẫn. Chiều ngược lại hoàn toàn tương tự: nếu phần bù của phủ đỉnh chứa một cạnh thì phủ đỉnh không che cạnh ấy. $\square$

Bổ đề 3. Giá trị lát cắt nhỏ nhất trong mạng H bằng trọng số phủ đỉnh nhẹ nhất của G' .

Chứng minh. Các cung có dung lượng vô hạn không xuất hiện trong một lát cắt tối ưu hữu hạn. Vì vậy, mọi lát cắt hữu hạn phải chọn, với từng cạnh $uv \in E'$, ít nhất một trong hai cung $s \rightarrow u$, $v \rightarrow t$; tập đầu mút được chọn là một phủ đỉnh và giá trị lát cắt bằng trọng số của nó. Ngược lại, mọi phủ đỉnh tạo ra một lát cắt chỉ gồm các cung tương ứng, có đúng cùng giá trị. Do đó hai giá trị tối ưu bằng nhau. $\square$

Bổ đề 4. Công thức từ tập đỉnh tới được X thu được chính xác một phủ đỉnh nhẹ nhất.

Chứng minh. Sau luồng cực đại, các cung gốc từ X sang phần bù của X tạo thành một lát cắt nhỏ nhất. Áp dụng tương ứng trong Bổ đề 3 cho những cung $s \rightarrow V_1$ và $V_2 \rightarrow t$, ta nhận được $P = (V_1 \setminus X) \cup (V_2 \cap X)$, là phủ đỉnh có trọng số nhỏ nhất. $\square$

Định lý. Đội mà thuật toán in ra là hợp lệ và có tổng IQ lớn nhất.

Chứng minh. Theo Bổ đề 4, thuật toán tìm được phủ đỉnh nhẹ nhất P . Theo Bổ đề 2, phần bù của P là một tập độc lập; theo Bổ đề 1, đó là một đội hợp lệ. Vì phần bù của phủ đỉnh có trọng số nhỏ nhất có trọng số lớn nhất, không đội hợp lệ nào có tổng IQ lớn hơn đội được in. $\square$

9.2.5 Độ phức tạp

Đồ thị bù có nhiều nhất $nm \leq 160\,000$ cạnh. Mạng luồng có

$$|V_H| = n + m + 2, \quad |E_H| = |E'| + n + m$$

cung gốc (mỗi cung thường được lưu cùng cung ngược trong cài đặt).

Có thể dùng Dinic với cận tổng quát $O(|V_H|^2 |E_H|)$, phù hợp với giới hạn của bài. Bản mô tả gốc cũng nêu thuật toán **scaling flow**: vì dung lượng hữu hạn nhỏ hơn 2^B , nó chạy trong $O(B|V_H|^3)$, với $B \leq 40$ cho các giá trị của bài. Bộ nhớ cần $O(|V_H| + |E_H|)$.

9.3 Ghi chú nguồn

Lời giải trên dịch và diễn giải từ phần *Intelligence Quotient* trong bản tiếng Anh của *Looking for a Challenge 2*. Bản gốc trước hết liên hệ trường hợp mọi trọng số bằng 1 với định lý König về matching cực đại và phủ đỉnh nhỏ nhất; cách xây dựng mạng luồng ở trên là dạng tổng quát trực tiếp cho trọng số IQ bất kỳ.

Hang động ★★ ★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Bộ nhớ: 256 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/jas>

10.1 Đề bài

Byteasar vừa khám phá một hang động gồm n phòng, được nối với nhau bằng các hành lang sao cho giữa hai phòng bất kỳ luôn có đúng một đường đi. Nói cách khác, các phòng và hành lang tạo thành một cây.

Hang động cần được khảo sát kỹ hơn, nên Byteasar nhờ các bạn giúp đỡ. Mọi người muốn chia thành các nhóm. Mỗi nhóm phải khảo sát cùng số lượng phòng; mỗi phòng phải được khảo sát bởi đúng một nhóm. Ngoài ra, để các nhóm không cản trở công việc của nhau, các thành viên của một nhóm phải có thể đi giữa mọi cặp phòng được giao cho nhóm đó mà không cần đi qua phòng của nhóm khác.

Có bao nhiêu số nhóm khác nhau mà các nhà thám hiểm có thể chia thành?

10.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n ($2 \leq n \leq 3\,000\,000$), là số phòng trong hang. Các phòng được đánh số từ 1 đến n .

$n - 1$ dòng tiếp theo mô tả các hành lang. Dòng thứ i trong số đó chứa một số nguyên a_i $1 \leq a_i \leq i$, biểu thị một hành lang nối phòng $i + 1$ với phòng a_i .

10.1.2 Dữ liệu ra

In trên một dòng tất cả các số nguyên k sao cho có thể chia các phòng thành k tập rời nhau, có cùng kích thước, và có thể đi giữa mọi cặp phòng thuộc cùng một tập chỉ qua các phòng của tập đó. Các số phải được in theo thứ tự tăng dần, cách nhau đúng một dấu cách.

10.1.3 Ví dụ

Dữ liệu vào

6
1
2
3
3
5

Kết quả

1 3 6

10.2 Lời giải

Ta dùng ngôn ngữ lý thuyết đồ thị. Hàng động là một cây liên thông không có chu trình. Điều kiện “đi giữa mọi cặp phòng của cùng một nhóm mà không rời nhóm” nói đúng rằng mỗi nhóm phải cảm sinh một cây liên thông.

Để tránh nhầm lẫn với k là **số nhóm** trong dữ liệu ra, trong lời giải này gọi s là số đỉnh của mỗi mảnh. Nếu có k nhóm thì

$$s = \frac{n}{k}, \quad k = \frac{n}{s}.$$

Vì vậy s bắt buộc là một ước của n . Một cách chia thành các mảnh có s đỉnh sẽ có đúng n/s mảnh. Muốn tách một cây thành n/s cây con, ta phải xóa đúng

$$\frac{n}{s} - 1$$

cạnh.

10.2.1 Cách làm trực tiếp

Trước hết, cố định một ước s của n và kiểm tra liệu có thể chia cây thành các mảnh liên thông, mỗi mảnh có s đỉnh hay không.

Thoạt nhìn, cách chia có thể tưởng như có nhiều lựa chọn; thực ra, nếu nó tồn tại thì được xác định duy nhất. Phác thảo chứng minh như sau. Xét một cách chia với $s < n$. Trong cây các mảnh, luôn có một mảnh lá P , tức nó chỉ nối với phần còn lại bằng đúng một cạnh e . Trong mọi cách chia hợp lệ với cùng s , cạnh e buộc phải nằm giữa hai mảnh: nếu không, phía chứa P sẽ có số đỉnh nhỏ hơn hoặc lớn hơn s . Xóa P và lặp lại lập luận với phần cây còn lại. Do đó từng cạnh cần cắt được quyết định lần lượt, nên cách chia là duy nhất.

Ta có thể kiểm tra bằng DFS hậu tự. Khi trở về từ các lời gọi đệ quy, tính kích thước các cây con. Nếu tại một đỉnh có cây con hiện thời đúng s đỉnh, cắt cạnh nối cây con ấy với phần còn lại; các đỉnh của nó sẽ không còn được tính trong những kích thước phía trên. Nếu cây thật sự chia được, ta sẽ cắt đúng $n/s - 1$ cạnh và phần còn lại ở gốc có đúng s đỉnh.

Chiều ngược lại cũng đúng: nếu đã cắt $n/s - 1$ lần, mỗi lần tách ra một cây có s đỉnh, phần còn lại cũng có s đỉnh vì toàn bộ cây có n đỉnh. Vậy ta đã thu được n/s mảnh hợp lệ.

Với một s cố định, DFS mất $O(n)$ thời gian. Nếu thử mọi ước của n , thời gian là

$$O(nd(n)),$$

trong đó $d(n)$ là số ước dương của n . Ta có thể duyệt $1, 2, \dots, n$ để nhận biết các ước trong $O(n)$ thời gian, nhưng điều đó vẫn chưa đủ nhanh: trong đoạn đến $3 \cdot 10^6$, số 2 882 880 có tới 336 ước. Trường hợp xấu này có trong bộ kiểm thử.

10.2.2 Đếm đồng thời cho mọi kích thước

Ta xét tất cả các s trong một lượt DFS. Cố định một cạnh e . Nếu xóa e , cây được tách thành hai phần có kích thước a và $n - a$. Khi nào cạnh này phải bị cắt để chia thành các mảnh kích thước s ?

Ta sẽ dùng nhận xét sau.

Mệnh đề 1. Cây chia được thành các mảnh có s đỉnh khi và chỉ khi có đúng $n/s - 1$ cạnh mà hai phía của cạnh đều có số đỉnh là bội của s .

Chứng minh. Nếu tồn tại cách chia, ta cắt đúng $n/s - 1$ cạnh. Mỗi phía của một cạnh bị cắt là hợp của một số nguyên các mảnh kích thước s , nên số đỉnh ở cả hai phía chia hết cho s .

Ngược lại, giả sử có đúng $n/s - 1$ cạnh thỏa điều kiện. Xóa toàn bộ các cạnh đó. Mỗi thành phần liên thông nhận được có số đỉnh chia hết cho s : có thể thấy điều này bằng cách xét tổng số đỉnh ở mỗi phía của các cạnh bị xóa. Ta thu được n/s thành phần không rỗng, tổng cộng n đỉnh, và mỗi thành phần có kích thước là một bội dương của s . Vì tổng kích thước là $(n/s)s$, mỗi thành phần bắt buộc có chính xác s đỉnh. $\square$

Như vậy, chỉ cần đếm số cạnh thỏa điều kiện với từng s . Gọi $\text{edg}[s]$ là số cạnh mà hai phía đều có kích thước là bội của s .

Với cạnh chia cây thành a và $n - a$ đỉnh, ta tăng $\text{edg}[s]$ với mọi s chia hết cả a lẫn $n - a$. Nói cách khác, s là một ước của

$$\text{gcd}(a, n - a).$$

Không nên duyệt ngay các ước ấy cho từng cạnh. Thay vào đó, dùng mảng phụ t . Khi cần tăng $\text{edg}[s]$ thêm 1 với mọi ước s của một số x , chỉ cần tăng $t[x]$ thêm 1. Vì vậy, khi DFS tính được kích thước a của một phía của mỗi cạnh, ta thực hiện

$$t[\text{gcd}(a, n - a)] += 1.$$

Sau khi xử lý tất cả cạnh, khôi phục các giá trị cần đếm bằng

$$\text{edg}[s] = \sum_{j: s|j} t[j].$$

Thật vậy, $t[j]$ đóng góp cho đúng mọi ước của j .

Cuối cùng, với mỗi ước s của n , nếu

$$\text{edg}[s] = \frac{n}{s} - 1,$$

thì kích thước mảnh s là khả thi theo Mệnh đề 1. Khi đó in số nhóm n/s . Sắp xếp các số được in theo thứ tự tăng dần (hoặc duyệt các ước s theo thứ tự giảm dần).

10.2.3 Phân tích độ phức tạp

Tính các kích thước cây con và điền t cần một DFS, ngoài các phép GCD. Tính $\text{edg}[s]$ bằng công thức trên duyệt các bội của s . Chỉ cần xét $s \mid n$, nên tổng số bước là

$$\sum_{s|n} \frac{n}{s}.$$

Khi s chạy qua các ước của n thì n/s cũng chạy qua đúng các ước ấy theo thứ tự ngược lại. Tổng trên bằng $D(n)$, tổng các ước của n . Ta có $D(n) = O(n \log \log n)$, nên phần này không chi phối thời gian chạy.

Phép toán đắt nhất là tính $\text{gcd}(a, n - a)$ cho $O(n)$ cạnh. Thuật toán Euclid cần $O(\log n)$ thời gian cho một GCD, vì thế tổng thời gian là

$$O(n \log n),$$

và bộ nhớ là $O(n)$.

10.2.4 Cải tiến thêm

Ta còn có thể tiền xử lý mọi giá trị

$$\gcd(i, n - i), \quad i = 1, \dots, n,$$

trong $O(n \log \log n)$ thời gian. Nếu d chia cả i lẫn $n - i$, thì d cũng chia n . Do đó GCD nói trên chính là ước lớn nhất của n chia i .

Khởi tạo mảng $\gcd$ rỗng. Duyệt các ước d của n theo thứ tự tăng dần; với mỗi d , duyệt các bội jd và gán

$$\gcd[jd] = d.$$

Do d được duyệt tăng dần, lần gán cuối cùng cho mỗi vị trí là ước lớn nhất của n chia vị trí đó, tức đúng bằng $\gcd(jd, n - jd)$. Tổng số lần cập nhật giống phần tính edg , do đó là $O(n \log \log n)$. Thay các phép Euclid bằng mảng này, toàn bộ thuật toán có thời gian $O(n \log \log n)$. Trong thực tế, mức cải thiện thường không đáng kể.

10.2.5 Một ghi chú về số lượng ước

Ta đã dùng hàm $d(n)$ đếm số ước. Có một cận đơn giản:

$$d(n) < 2\sqrt{n}.$$

Thật vậy, có nhiều nhất $\sqrt{n}$ ước không vượt quá $\sqrt{n}$, và mỗi ước lớn hơn $\sqrt{n}$ ghép cặp với một ước nhỏ hơn $\sqrt{n}$. Một cận tiệm cận mạnh hơn là

$$d(n) = n^{O(1/\log \log n)}.$$

Hàm $\log \log n$ tăng rất chậm: nếu logarit lấy cơ số 2 thì $\log \log(2^{32}) = 5$ và $\log \log(2^{64}) = 6$. Khi ước lượng thời gian

trong thực hành với các số không quá 2^{64} , có thể coi xấp xỉ trường hợp xấu của $d(n)$ là khoảng $3\sqrt{n}$. Xấp xỉ này không chặt chẽ theo nghĩa toán học, nhưng khá hữu ích trong thực tế.

Nhện chữ thập ★★

Tác giả đề: Szymon Acedański

Tác giả lời giải: Tomasz Idziaszek

Bộ nhớ: 32 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2011/krz>

11.1 Đề bài

Loài nhện chữ thập Bytean (*Araneida baitoida*) có một khả năng đáng kinh ngạc: nó có thể tức thời giăng một mạng nhện lớn tùy ý, miễn là toàn bộ các sợi tơ của mạng nằm trong cùng một mặt phẳng. Nhờ đó, nó không cần chờ ruồi mắc vào chiếc mạng đã có sẵn: chỉ cần biết vị trí hiện tại của một con ruồi, nó có thể lập tức giăng mạng để bắt con ruồi ấy.

Một con nhện chữ thập vừa phát hiện n con ruồi trong khu vườn của Byteasar. Mỗi con đang đứng yên tại một điểm trong không gian ba chiều. Nhện muốn biết liệu có thể bắt tất cả chúng bằng một mạng nhện hay không. Hãy viết chương trình trả lời câu hỏi đó.

11.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên n ($1 \leq n \leq 100\,000$).

n dòng tiếp theo mô tả các con ruồi trong không gian: dòng thứ i chứa ba số nguyên x_i, y_i, z_i ($-1\,000\,000 \leq x_i, y_i, z_i \leq 1\,000\,000$), là tọa độ của con ruồi thứ i trong không gian Euclid ba chiều. Không có hai con ruồi nào ở cùng một vị trí.

11.1.2 Dữ liệu ra

In một từ duy nhất TAK (nghĩa là “có” trong tiếng Ba Lan) nếu nhện có thể bắt tất cả ruồi bằng một mạng nhện. Nếu không, in NIE (nghĩa là “không”).

11.1.3 Ví dụ

Input

```
4
0 0 0
-1 0 -100
100 0 231
5 0 15
```

Output

TAK

Trong khi đó, với dữ liệu vào

```
4
0 1 0
-1 0 -100
100 0 231
5 0 15
```

kết quả đúng là

NIE

11.2 Lời giải

Bài toán yêu cầu kiểm tra xem n điểm trong không gian ba chiều có cùng nằm trên một mặt phẳng hay không. Ký hiệu điểm thứ i là

$$p_i = (x_i, y_i, z_i).$$

Ba điểm không thẳng hàng xác định duy nhất một mặt phẳng. Do đó, ta thực hiện hai bước:

1. Tìm ba điểm không thẳng hàng, nếu tồn tại.
2. Kiểm tra mọi điểm còn lại có thuộc mặt phẳng do ba điểm ấy xác định hay không.

Nếu $n \leq 3$, đáp án hiển nhiên luôn là TAK. Sau đây xét $n \geq 4$.

11.2.1 Tìm ba điểm không thẳng hàng

Không thể tùy tiện chọn p_1, p_2, p_3 , vì có thể chúng thẳng hàng. Chọn p_1, p_2 , rồi duyệt các điểm p_i , $i = 3, \dots, n$, để tìm một điểm không nằm trên đường thẳng qua p_1, p_2 .

Đặt

$$t_i = (p_2 - p_1) \times (p_i - p_1),$$

trong đó $\times$ là tích có hướng. Với hai vector

$$v = (x_v, y_v, z_v), \quad w = (x_w, y_w, z_w),$$

ta có

$$v \times w = (y_v z_w - y_w z_v, z_v x_w - z_w x_v, x_v y_w - y_v x_w).$$

Điểm p_i nằm trên đường thẳng qua p_1, p_2 khi và chỉ khi cả ba tọa độ của t_i đều bằng 0.

Nếu mọi t_i đều bằng vector không, toàn bộ các điểm cùng nằm trên một đường thẳng; đường thẳng ấy nằm trong một mặt phẳng nào đó, nên in TAK.

Ngược lại, với một i sao cho $t_i \neq 0$, ba điểm p_1, p_2, p_i không thẳng hàng và xác định một mặt phẳng. Vector t_i là vector pháp tuyến của mặt phẳng này.

11.2.2 Kiểm tra các điểm còn lại

Một điểm p_j thuộc mặt phẳng vừa tìm khi và chỉ khi vector $p_j - p_1$ vuông góc với pháp tuyến t_i . Ta kiểm tra điều đó bằng tích vô hướng. Với

$$v = (x_v, y_v, z_v), \quad w = (x_w, y_w, z_w),$$

định nghĩa

$$v \cdot w = x_v x_w + y_v y_w + z_v z_w.$$

Hai vector khác không vuông góc khi và chỉ khi tích vô hướng của chúng bằng 0. Vì vậy, với mỗi điểm p_j , chỉ cần kiểm tra

$$t_i \cdot (p_j - p_1) = 0.$$

Nếu có một điểm không thỏa mãn, in NIE; nếu tất cả đều thỏa mãn, in TAK. Có thể tổ chức việc duyệt sao cho mỗi điểm chỉ được kiểm tra một lần.

11.2.3 Tính đúng đắn

Nếu thuật toán in TAK ngay sau bước đầu, mọi điểm cùng thẳng hàng với p_1, p_2 , nên chúng đồng phẳng. Nếu thuật toán chọn được p_i , thì p_1, p_2, p_i không thẳng hàng, vì tích có hướng tương ứng khác 0; do đó chúng xác định duy nhất một mặt phẳng có pháp tuyến t_i . Tích vô hướng bằng 0 chính xác khi vector từ p_1 đến điểm đang xét vuông góc với pháp tuyến, tức điểm đó thuộc mặt phẳng này. Vì vậy thuật toán in TAK đúng khi và chỉ khi toàn bộ ruồi đồng phẳng.

11.2.4 Độ phức tạp

Mỗi điểm được xử lý số lần hằng số. Thời gian chạy là $O(n)$, và ngoài dữ liệu đầu vào thuật toán chỉ dùng $O(1)$ bộ nhớ.

11.3 Lưu ý về kiểu dữ liệu

Nên dùng số nguyên thay vì số thực để tránh sai số dấu phẩy động: tọa độ đầu vào là số nguyên và thuật toán chỉ dùng cộng, trừ, nhân.

Mỗi hiệu tọa độ có trị tuyệt đối không quá $2 \cdot 10^6$. Khi khai triển tích vô hướng ở trên, mỗi hạng là tích của ba hiệu tọa độ, nên trị tuyệt đối không vượt quá

$$(2 \cdot 10^6)^3 = 8 \cdot 10^{18}.$$

Một biến nguyên có dấu 64 bit vừa đủ chứa từng hạng, nhưng tổng sáu hạng có thể đạt $48 \cdot 10^{18}$, vượt miền biểu diễn 64 bit có dấu. Có ba cách xử lý:

1. Dùng số nguyên 128 bit nếu trình biên dịch hỗ trợ, chẳng hạn `__int128` của GCC.
2. Tự cài kiểu số nguyên đủ lớn; ở đây chỉ cần cộng và so sánh với 0.
3. Tính tổng hai lần, theo modulo m_1 và m_2 , trong đó m_1, m_2 nguyên tố cùng nhau và $m_1 m_2 > 48 \cdot 10^{18}$. Theo định lý phần dư Trung Hoa, cả hai kết quả bằng 0 khi và chỉ khi tổng thực sự bằng 0.

Máy bán hàng tự động ★★ ★

Tác giả đề: Jakub Pachocki

Tác giả lời giải: Eryk Kopczyński

Giới hạn bộ nhớ: 128 MB

Mã bài: aut

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/aut>

12.1 Đề bài

Byteasar học khoa học máy tính tại Đại học Bytean. Trong tòa nhà khoa của cậu có một máy bán đồ ăn vặt, cung cấp n loại đồ ăn, được đánh số từ 1 đến n . Các loại đồ ăn có thể có giá khác nhau vì khác kích cỡ và hương vị.

Gần đây Byteasar phát hiện máy bị hỏng. Khi mua một món thuộc loại i , máy còn nhả thêm một món của **mỗi** loại $1, 2, \dots, i - 1$, miễn là loại tương ứng vẫn còn trong máy. Nếu một loại nào đó đã hết thì máy đơn giản không nhả món thuộc loại ấy. Chỉ có thể mua món loại i khi máy còn ít nhất một món loại đó.

Byteasar muốn lợi dụng lỗi này. Với một số tiền cho trước, hãy xác định tổng giá trị lớn nhất — tổng các giá của mọi món nhận được — mà cậu có thể lấy từ máy. Cậu không bắt buộc phải tiêu hết tiền.

12.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n và k ($1 \leq n \leq 40$, $1 \leq k \leq 64\,000$), lần lượt là số loại đồ ăn và số tiền Byteasar có.

Dòng thứ hai chứa n số nguyên $c_1, \dots, c_n$ ($1 \leq c_i \leq 40$); c_i là giá của một món loại i .

Dòng thứ ba chứa n số nguyên $l_1, \dots, l_n$ ($0 \leq l_i \leq 40$); l_i là số món loại i hiện có trong máy.

12.1.2 Dữ liệu ra

In một số nguyên duy nhất: tổng giá trị lớn nhất của các món Byteasar có thể lấy với không quá k đơn vị tiền.

12.1.3 Ví dụ

Dữ liệu vào

```
6 8
7 2 3 5 7 2
1 3 0 3 2 1
```

Dữ liệu ra

```
30
```

Ta mua một món loại 6; máy đồng thời nhả ra mỗi loại một món loại 1, 2, 4, 5. Sau đó mua một món loại 4, và máy nhả thêm một món loại 2.

12.2 Lời giải

12.2.1 Thứ tự mua và cách nhìn từ phải sang trái

Trước hết, hãy xét thứ tự nên mua. Giả sử ta muốn mua các món loại i và j , với $i < j$. Tổng giá trị các món sẽ nhận được không phụ

thuộc vào thứ tự mua hai món này. Tuy nhiên, nếu máy chỉ còn đúng một món loại i , ta không thể bắt đầu bằng việc mua loại j : món loại i sẽ bị nhả kèm và sau đó không còn gì để mua. Vì vậy, có thể giả sử các món được mua theo thứ tự **không giảm** của số loại, tức từ trái sang phải.

Ta có thể thử đệ quy mọi chiến lược mua theo thứ tự ấy, nhưng số chiến lược là quá lớn. Ta sẽ dùng quy hoạch động, với một mẹo: tuy các món thực sự được mua từ trái sang phải, quyết định *mua những món nào sẽ được lập từ phải sang trái*.

Khi đang dự định mua một món loại j , ta biết chắc sẽ nhận được mọi món còn có thể nhận của các loại $i < j$. Chúng có thể được nhả ngay cùng món j , hoặc cùng một món khác sẽ mua trước đó, nhưng điều này không ảnh hưởng đến tổng giá trị.

Biểu diễn mỗi loại i bằng một cột cao l_i , và ghi c_i trong mọi ô của cột. Với dữ liệu ví dụ ta được:

3		2		5		
2		2		5	7	
1		7	2	5	7	2
+-----						
		1	2	3	4	5
						6

Chọn một món loại 4 và một món loại 6, như trong ví dụ. Nếu thực hiện từ trái sang phải, ta mua loại 4 trước và nhận tổng giá trị $7 + 2 + 5$; sau đó mua loại 6 và nhận thêm $2 + 5 + 7 + 2$. Đến lúc mua món thứ hai, loại 1 đã hết nên không nhận thêm món loại 1.

Ta có thể lập kế hoạch tương đương theo chiều ngược lại. Trước tiên chọn mua loại 6, khi đó chắc chắn nhận các món có giá $7 + 2 + 5 + 7 + 2$. Sau đó chọn mua loại 4, nhưng nhớ rằng món loại 1, vốn chỉ có một chiếc, đã được tính vào giá trị nhận được khi chọn loại lớn hơn nên không được tính lần nữa. Khi lập kế hoạch, thậm chí có thể chọn mua loại i dù trước đó đã dự định mua nhiều hơn l_i món có số loại lớn hơn; điều duy nhất bị cấm là **trực tiếp** mua quá l_i món loại i .

12.2.2 Các tổng tiền tố trên các hàng

Trong cách nhìn này, mỗi hàng chỉ chọn đúng một ô. Để tính giá trị nhanh, với ô ở cột i , hàng y , gọi $t[i, y]$ là tổng tiền tố trên hàng đó, từ cột 1 đến cột i . Với ví dụ:

$$t[6, 1] = 7 + 2 + 5 + 7 + 2 = 23, \quad t[4, 2] = 2 + 5 = 7.$$

Sau khi thay mỗi ô bằng tổng tiền tố tương ứng, bảng trở thành:

3		0	2	2	7	7	7
2		0	2	2	7	14	14
1		7	9	9	14	21	23
+-----							
		1	2	3	4	5	6

Do đó, đi từ hàng dưới lên, chiến lược chọn một ô (i, y) trên mỗi hàng, thỏa các điều kiện:

- ô được chọn ở hàng kế tiếp không được nằm bên phải ô đã chọn ở hàng bên dưới;
- cột i được chọn nhiều nhất l_i lần;
- tổng các giá c_i của các ô được chọn không vượt quá ngân sách k ;
- tổng các giá trị $t[i, y]$ tại những ô được chọn là lớn nhất có thể.

Ta chọn các ô theo thứ tự từ phải sang trái. Giả sử vừa xét xong cột i và đã chọn trong cột này một số ô từ 0 đến l_i . Từ lập luận trên, thông tin duy nhất về các ô đã chọn trong những cột $j \geq i$ cần cho các bước sau là:

- tổng số ô đã chọn, ký hiệu y ;
- ngân sách còn lại, ký hiệu b .

Vì thế ta dùng đệ quy có nhớ hoặc quy hoạch động. Mỗi trạng thái (i, y, b) chỉ cần được tối ưu một lần. Đặt $w[i, y, b]$ là giá trị tối ưu của trạng thái đó. Nếu chọn s ô trong cột i , ta có công thức:

$$w[i, y, b] = \max_{0 \leq s \leq l_i} \left\{ w[i + 1, y - s, b + s \cdot c_i] + \sum_{h=1}^s t[i, y - h + 1] \right\}. \quad (1)$$

Ở đây chỉ xét các lựa chọn s khiến chỉ số của trạng thái hợp lệ. Công thức đúng vì s ô mới được chọn là s hàng cao nhất chưa được xử lý trong cột i ; mỗi ô đóng góp đúng tổng tiền tố ở hàng tương ứng, còn phần bên phải cột i đã được tối ưu trong trạng thái kế tiếp.

12.2.3 Độ phức tạp của quy hoạch động cơ bản

Gọi C là giá lớn nhất của một món và L là số món nhiều nhất của cùng một loại. Chỉ số i nhận $O(n)$ giá trị, y nhận $O(L)$ giá trị — không có ích gì khi trực tiếp mua quá L món — và b nhận $O(k)$ giá trị. Bảng có $O(nLk)$ ô. Mỗi ô thử tối đa L giá trị của s , nên thời gian là

$$O(nL^2k).$$

Với các giới hạn ban đầu, đó xấp xỉ $40^3 \cdot 64\,000 \approx 4 \cdot 10^9$ phép toán, vẫn quá lớn. Nhưng giới hạn k lớn đến vậy không thực sự cần thiết. Nếu trực tiếp mua L món có số loại lớn nhất, ta nhận được toàn bộ món trong máy; chi phí nhiều nhất là LC , phần tiền thừa có thể đem làm từ thiện. Vì vậy có thể thay ngân sách bằng

$$K = \min(k, LC).$$

Khi đó thời gian giảm còn

$$O(nL^3C),$$

và với dữ liệu cực đại, số phép toán vào cỡ $40^5 \approx 10^8$. Bộ nhớ là $O(nL^2C)$. Chỉ cần giữ hai lớp liên tiếp theo i , nên giảm được thành $O(L^2C)$.

12.3 Lời giải nhanh hơn

Quy hoạch động trên đã đủ để được chấp nhận trong cuộc thi, kể cả khi không giảm bộ nhớ. Tuy nhiên, có một lời giải nhanh hơn; dưới đây là phác thảo ý tưởng.

Trong (1), ta phải tính rất nhiều giá trị cực đại. Với một i cố định, duyệt mọi giá trị có thể của b , và lần lượt tính

$$w[i, y, b - y \cdot c_i]$$

cho từng y .

Tạm bỏ qua hạng

$$\sum_{h=1}^s t[i, y - h + 1].$$

Hai giá trị liên tiếp

$$w[i, y, b - y \cdot c_i] \quad \text{và} \quad w[i, y + 1, b - (y + 1)c_i]$$

là cực đại trên gần như cùng một tập con của bảng w . Tập đầu là một đoạn dài l_i trên một lát cắt “chéo” một chiều của bảng; khi chuyển sang tập thứ hai, chỉ thêm một phần tử ở đầu phải và bỏ một phần tử ở đầu trái.

Do đó bài toán cốt lõi trở thành: cho mảng $M[1..N]$ và độ dài cửa sổ m , hãy nhanh chóng tính cực đại của mọi đoạn độ dài m , tức với mỗi $y = 1, \dots, N - m$, tính

$$\max M[y..y + m].$$

Hạng vừa tạm bỏ qua cũng dễ xử lý. Khi dịch cửa sổ để tính cực đại kế tiếp, mọi giá trị trong mảng M đều tăng cùng một lượng $t[i, y]$ nào đó. Cụ thể, thay vì lưu các giá trị

$$M[y] = w[i + 1, y, b - y \cdot c_i + s \cdot c_i] + \sum_{h=1}^s t[i, y - h + 1],$$

ta lưu một độ lệch chung δ , còn trong mảng chỉ lưu

$$M'[y] = M[y] - \delta.$$

Giá trị thực cần lấy cực đại là $M'[y] + \delta$. Khi cửa sổ dịch sang phải, thêm phần tử mới theo công thức

$$M'[y] = w[i + 1, y, b - y \cdot c_i],$$

cập nhật δ thêm $t[i, y]$, rồi giá trị $w[i, y, b - y \cdot c_i]$ chính là cực đại của đoạn cửa sổ tương ứng trong M' , cộng với δ .

12.3.1 Tính cực đại trượt

Ta còn cần tính cực đại của các cửa sổ độ dài m trong $M[1..N]$ trong $O(N)$. Dịch cửa sổ từ trái sang phải và duy trì tập S gồm những vị trí hiện tại hoặc có thể trở thành cực đại trong tương lai. Chính xác hơn, một vị trí thuộc S nếu ở bên phải nó, trong cửa sổ, không có giá trị nào lớn hơn hoặc bằng nó.

Nếu

$$S = \{x_1, \dots, x_r\},$$

thì dãy $x_1, \dots, x_r$ tăng dần, còn dãy $M[x_1], \dots, M[x_r]$ giảm dần. Cực đại của cửa sổ là $M[x_1]$.

Mỗi lần dịch cửa sổ một vị trí:

1. nếu cực đại cũ rơi ra ở mép trái, loại x_1 khỏi S ;
2. thêm vị trí mới ở mép phải, đồng thời loại ở cuối S mọi vị trí có giá trị không lớn hơn giá trị mới.

Mỗi vị trí được thêm đúng một lần và loại đúng một lần, nên tổng số thao tác là $O(N)$. Có thể cài đặt S bằng hàng đợi hai đầu trong mảng; mọi thao tác khi đó đều $O(1)$.

Nhờ cực đại trượt, toàn bộ bài toán giải được trong

$$O(nLk).$$

Thay k bằng $\min(k, LC)$ như trên cho thời gian cuối cùng

$$O(nL^2C).$$

Chuyến xe buýt ★★ ★

Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Łącki

Bộ nhớ: 128 MB

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/biu>

13.1 Đề bài

Byteasar là giáo viên một trường tiểu học ở Byteburg. Thời tiết hiện rất đẹp, nên anh muốn đưa lớp của mình đi xe buýt tới Bytetown — thủ đô của Byteland. Byteasar quyết định thuê một công ty du lịch để tổ chức chuyến đi.

Các con đường ở Bytetown tạo thành một lưới đều gồm các đường theo hướng đông–tây và bắc–nam. Khoảng cách giữa hai con đường song song kề nhau luôn là 1 kilômét. Tại một số giao lộ có các điểm tham quan. Các hướng dẫn viên Bytean gán cho mỗi điểm một **hệ số hấp dẫn**: hệ số càng lớn thì điểm đó càng thú vị với du khách. Vì học sinh của Byteasar dễ chán, anh yêu cầu các điểm tham quan được ghé thăm phải có hệ số hấp dẫn tăng dần theo đúng thứ tự ghé thăm.

Công ty du lịch đồng ý đáp ứng yêu cầu ấy, nhưng cũng muốn kiếm được nhiều tiền nhất. Công ty nhận 1 bythaler cho mỗi kilômét

của hành trình xe buýt. Khi đi giữa hai điểm tham quan liên tiếp trong lịch trình, xe buýt luôn chọn một đường đi ngắn nhất dọc theo các con phố của Bytetown. Ngoài ra, người quản lý của một số điểm tham quan còn trả tiền cho công ty khi đoàn ghé thăm điểm đó.

Hãy giúp công ty lập một hành trình thỏa yêu cầu của Byteasar và mang lại lợi nhuận lớn nhất. Lưu ý rằng đi ngang qua một điểm tham quan mà không dừng lại không được tính là đã ghé thăm điểm đó.

13.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n, m ($2 \leq n, m \leq 1000$), lần lượt là số đường đông–tây và số đường bắc–nam.

n dòng tiếp theo mô tả các điểm tham quan ở Bytetown. Dòng thứ i gồm m số nguyên $w_{i,j}$ ($0 \leq w_{i,j} \leq 10^6$), là hệ số hấp dẫn của điểm tham quan nằm tại giao lộ của đường đông–tây thứ i với đường bắc–nam thứ j . Giá trị hấp dẫn bằng 0 nghĩa là giao lộ tương ứng không có điểm tham quan. Bảo đảm rằng Bytetown có ít nhất một điểm tham quan.

n dòng tiếp theo, mỗi dòng gồm m số nguyên $c_{i,j}$ ($0 \leq c_{i,j} \leq 10^9$). Giá trị $c_{i,j}$, tức số thứ j trên dòng thứ i , là số tiền (tính bằng bythaler) công ty nhận được khi đưa đoàn tới điểm có hệ số hấp dẫn $w_{i,j}$. Nếu giao lộ không có điểm tham quan thì giá trị tương ứng $c_{i,j}$ bằng 0.

13.1.2 Dữ liệu ra

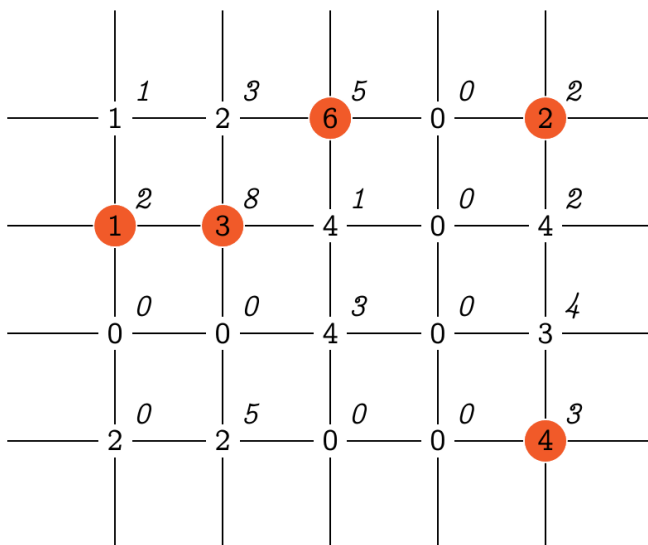
In một số nguyên duy nhất: lợi nhuận lớn nhất, tính bằng bythaler, mà công ty có thể thu được từ một hành trình ghé một số điểm tham quan theo thứ tự hệ số hấp dẫn tăng **nghiêm ngặt**.

13.1.3 Ví dụ

Input	Output
4 5	39
1 2 6 0 2	

1 3 4 0 4
 0 0 4 0 3
 2 2 0 0 4
 1 3 5 0 2
 2 8 1 0 2
 0 0 3 0 4
 0 5 0 0 3

Trong minh họa của ví dụ ở bản gốc, các số kiểu chữ thường là hệ số hấp dẫn, còn các số in nghiêng là khoản thu khi ghé điểm tương ứng. Hành trình tối ưu ghé năm điểm có các khoản thu lần lượt là 2, 2, 8, 3, 5, và tổng tiền nhận theo quãng đường xe chạy là 19. Vì vậy lợi nhuận bằng $2 + 2 + 8 + 3 + 5 + 19 = 39$.



Hình 13.1: Lộ trình tối ưu trong ví dụ; số thường là hệ số hấp dẫn, số nghiêng là khoản thu tại điểm tham quan.

13.2 Lời giải

13.2.1 Đồ thị có hướng ẩn trong đề bài

Hãy xem mỗi giao lộ có điểm tham quan là một đỉnh. Có một cạnh có hướng từ u đến w khi hệ số hấp dẫn ở w lớn hơn nghiêm ngặt hệ số hấp dẫn ở u . Từ đây, gọi hệ số hấp dẫn là **hạng** của đỉnh. Đồ thị này không có chu trình: sau mỗi cạnh, hạng tăng lên, nên không thể quay về đỉnh đã đi qua.

Đồ thị còn có hai loại trọng số:

- Trọng số của một đỉnh là khoản tiền công ty nhận được khi ghé điểm tham quan tại giao lộ đó.
- Trọng số của cạnh uw là khoảng cách Manhattan giữa hai giao lộ, tức số kilômét của một đường đi ngắn nhất trên lưới.

Ta cần tìm đường đi có tổng trọng số đỉnh và cạnh lớn nhất. Đây chính là lợi nhuận của hành trình tương ứng.

Về nguyên tắc, có thể dùng quy hoạch động trên đồ thị có hướng không chu trình: duyệt đỉnh theo thứ tự tô-pô, và với mỗi đỉnh w , thử mọi cạnh đi vào w để tính giá trị tốt nhất của một đường kết thúc tại w . Tuy nhiên, không thể xây đồ thị một cách tường minh. Nếu mọi đỉnh có hạng khác nhau thì giữa mỗi cặp đỉnh có đúng một cạnh có hướng. Với $n = m = 1000$, có đến $nm = 10^6$ đỉnh và xấp xỉ

$$\frac{(nm)(nm - 1)}{2} \approx 5 \cdot 10^{11}$$

cạnh.

Nén các giá trị hấp dẫn dương khác nhau thành các hạng liên tiếp $1, 2, \dots, r$. Gọi k_i là số đỉnh có hạng i . Một đường hợp lệ chứa nhiều nhất một đỉnh của mỗi hạng. Hơn nữa, một đường tối ưu có thể được chọn để chứa đúng một đỉnh ở mỗi hạng: nếu đang thiếu hạng i , chèn một đỉnh bất kỳ có hạng đó vào giữa hai hạng kề nhau. Khoản thu tại đỉnh không âm, và do bất đẳng thức tam giác, tổng khoảng cách sau khi chèn không giảm. Vì vậy tổng trọng số không giảm.

Thuật toán gồm r pha. Ở pha i , ta tính giá trị tốt nhất cho các đỉnh hạng i , chỉ từ các đỉnh hạng $i - 1$. Pha đầu tiên rất đơn giản:

giá trị của một đỉnh chỉ là trọng số của chính nó. Nếu pha i thực hiện trong thời gian $O(k_{i-1} + k_i)$, tổng thời gian sẽ là

$$O(k_1) + O(k_1 + k_2) + \dots + O(k_{r-1} + k_r) = O(k_1 + k_2 + \dots + k_r) = O(nm).$$

Phần còn lại là cách thực hiện một pha trong thời gian tuyến tính theo số đỉnh tham gia.

13.2.2 Một pha: đi sang phải và lên trên

Xét trước một trong bốn hướng chéo, cụ thể là từ trái-dưới sang phải-trên. Gán mỗi giao lộ tọa độ (x, y) . Với một đỉnh v , ký hiệu $\text{best}(v)$ là trọng số lớn nhất của một đường kết thúc ở v . Trong pha i , các giá trị $\text{best}(u)$ đã biết cho mọi đỉnh u hạng $i - 1$.

Ta sẽ tính một giá trị ứng viên cho từng đỉnh w hạng i với hai tính chất:

- nó **không nhỏ hơn** đường tốt nhất đến w mà đoạn cuối đi sang phải và lên trên;
- nó **không lớn hơn** $\text{best}(w)$ thực sự.

Nếu đoạn cuối của một đường tối ưu đến w đi sang phải và lên trên, hai cận này trùng nhau; do đó ta sẽ thu được đúng $\text{best}(w)$. Lấy giá trị lớn nhất trong bốn hướng sẽ cho đáp án của pha.

Trước hết cần một nhận xét đơn giản.

Bổ đề 1. Khoảng cách giữa hai điểm x_u, y_u và x_w, y_w không nhỏ hơn

$$x_w - x_u + y_w - y_u.$$

Chứng minh. Khoảng cách chính xác là

$$|x_w - x_u| + |y_w - y_u|.$$

Với mọi số thực z , ta có $|z| \geq z$. Áp dụng bất đẳng thức này cho cả hai hiệu tọa độ là xong. $\square$

Xét $w = (x_w, y_w)$ hạng i và $u = (x_u, y_u)$ hạng $i - 1$. Nếu đi từ u tới w sang phải và lên trên, tức $x_u \leq x_w$ và $y_u \leq y_w$, thì đường qua u có trọng số

$$\begin{aligned} & \text{best}(u) + |x_w - x_u| + |y_w - y_u| + \text{weight}(w) \\ &= \text{best}(u) + x_w - x_u + y_w - y_u + \text{weight}(w). \end{aligned}$$

Do đó, để chọn u , chỉ cần cực đại hóa

$$\text{useful}(u) := \text{best}(u) - x_u - y_u. \quad (1)$$

Khi ấy giá trị của đường dừng đến w là

$$\text{weight}(w) + x_w + y_w + \text{useful}(u). \quad (2)$$

Sắp các đỉnh hạng $i - 1$ và i theo thứ tự không giảm của $x + y$, rồi quét theo thứ tự đó. Ta luôn lưu đỉnh đã gặp có hạng $i - 1$ có useful lớn nhất. Khi gặp một đỉnh hạng $i - 1$, cập nhật đỉnh tốt nhất đang lưu theo (1). Khi gặp một đỉnh hạng i , dùng đỉnh đang lưu để tạo ứng viên theo (2), rồi cập nhật best của nó nếu ứng viên tốt hơn.

Tại sao phép quét này đúng, dù đỉnh u tốt nhất đã lưu có thể không thực sự nằm trái-dưới w ? Đặt u là đỉnh đã dùng để dựng ứng viên cho w .

- Nếu u thực sự ở trái-dưới w , (1) chọn chính xác tiền nhiệm tốt nhất trong hướng đang xét, nên ứng viên là tối ưu trong số các đường có đoạn cuối đi sang phải và lên trên.
- Nếu không, thì vẫn không có đỉnh u' nào ở trái-dưới w cho giá trị tốt hơn. Thật vậy, một đỉnh như thế luôn có $x_{u'} + y_{u'} \leq x_w + y_w$, nên đã xuất hiện trước w trong thứ tự quét; $\text{useful}(u)$ là lớn nhất trong các đỉnh đã xuất hiện.
- Mặt khác, ứng viên không thể lớn hơn đáp án thật. Nó dùng $x_w - x_u + y_w - y_u$ làm quãng đường; theo Bổ đề 1, giá trị này không vượt quá khoảng cách thật từ u tới w . Vì thế trọng số của đường thật đi qua u không nhỏ hơn ứng viên.

Ba ý trên chính là hai tính chất đã yêu cầu: ứng viên không bỏ sót lời giải tối ưu theo hướng phải–lên, đồng thời cũng không thể là một cận trên sai cho giá trị tối ưu toàn cục.

13.2.3 Ba hướng còn lại và sắp xếp hiệu quả

Làm hoàn toàn tương tự cho ba hướng chéo còn lại: phải–xuống, trái–xuống và trái–lên. Sau khi xét cả bốn hướng, lấy giá trị lớn nhất cho mỗi đỉnh hạng i .

Nếu thực hiện riêng từng pha, ta có vẻ phải sắp xếp các đỉnh bốn lần. Thực ra chỉ cần hai kiểu thứ tự, vì thứ tự cho hướng trái–xuống là thứ tự đảo của hướng phải–lên; tương tự, hai hướng còn lại cũng là hai thứ tự ngược nhau. Hai khóa sắp xếp cần dùng là tổng và hiệu tọa độ.

Ta còn có thể tránh hoàn toàn thừa số logarit. Tổng tọa độ không vượt quá $m + n$ (sau khi chọn gốc tọa độ phù hợp), nên có thể sắp xếp đếm trong thời gian $O(k + W)$ cho k khóa thuộc đoạn $[1, W]$. Ban đầu, sắp xếp tất cả các đỉnh theo tổng tọa độ và riêng theo hiệu tọa độ; rồi nhóm chúng theo hạng. Như vậy, với mỗi hạng ta có hai danh sách đã được sắp theo hai khóa cần thiết. Trong pha i , chỉ cần trộn tuyến tính hai danh sách của hạng $i - 1$ và i , sau đó quét chúng như trên.

Do đó, lời giải chạy trong thời gian tối ưu $O(nm)$, đúng bằng kích thước dữ liệu vào, và dùng bộ nhớ $O(nm)$. Một cài đặt nên dùng số nguyên 64 bit cho các giá trị quy hoạch động và đáp án.

Dãy số

Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 128 MB

Mã bài: cia

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/cia>

14.1 Đề bài

Gọi dãy số nguyên $a_1, a_2, \dots, a_n$ là **k -chẵn** nếu tổng của mọi đoạn con liên tiếp gồm đúng k phần tử của dãy là số chẵn.

Với một dãy đã cho, hãy xác định cần thay đổi ít nhất bao nhiêu phần tử để dãy trở thành k -chẵn.

14.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n và k ($1 \leq k \leq n \leq 1\,000\,000$), lần lượt là độ dài dãy và tham số “chẵn”.

Dòng thứ hai chứa n số nguyên $a_1, a_2, \dots, a_n$, trong đó $0 \leq a_i \leq 1\,000\,000\,000$.

14.1.2 Dữ liệu ra

In một số nguyên duy nhất: số phần tử ít nhất cần thay đổi để dãy đã cho trở thành k -chẵn.

14.1.3 Ví dụ

Với dữ liệu vào:

```
8 3
1 2 3 4 5 6 7 8
```

kết quả đúng là:

```
3
```

Còn với dữ liệu vào:

```
8 3
2 4 2 4 2 4 2 4
```

kết quả đúng là:

```
0
```

14.2 Lời giải

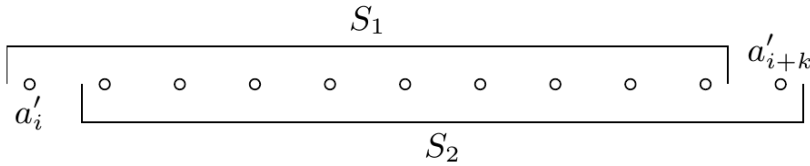
Ta cần biến đổi dãy sao cho nó trở thành k -chẵn. Trước hết, giá trị chính xác của các phần tử không quan trọng; chỉ **tính chẵn lẻ** của chúng mới quan trọng. Vì vậy, có thể thay mỗi phần tử bằng phần dư khi chia cho 2, rồi chỉ xét các dãy nhị phân gồm 0 và 1.

Ta dễ biết những đoạn độ dài k nào hiện có tổng lẻ. Mỗi đoạn như vậy buộc ta phải đổi tính chẵn lẻ của ít nhất một phần tử, nhưng không rõ nên chọn phần tử nào. Cách tiếp cận này dẫn tới các thuật toán kinh nghiệm và khó bảo đảm đúng.

Ở bài này, tốt hơn là tập trung vào cấu trúc của dãy đích. Ký hiệu dãy kết quả cần tìm là $a'_1, \dots, a'_n$. Mọi đoạn liên tiếp độ dài k của dãy này phải có tổng chẵn. Xét hai đoạn liên tiếp

$$a'_i, \dots, a'_{i+k-1} \quad \text{và} \quad a'_{i+1}, \dots, a'_{i+k},$$

có tổng lần lượt là S_1 và S_2 . Chúng chỉ khác nhau ở phần tử bị bỏ đi và phần tử được thêm vào, nên



Hình 14.1: Hai đoạn liên tiếp độ dài k trong dãy kết quả.

$$S_2 = S_1 + a'_{i+k} - a'_i.$$

Vì S_1 và S_2 đều chẵn, a'_i và a'_{i+k} phải cùng tính chẵn lẻ. Mà chúng đều là bit, do đó

$$a'_i = a'_{i+k} \quad (1 \leq i \leq n - k).$$

Nói cách khác, dãy đích phải **tuần hoàn với chu kỳ k** .

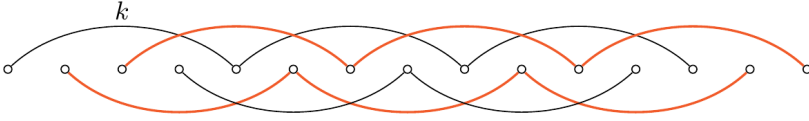
14.2.1 Biến dãy thành dãy tuần hoàn

Trước hết, xét bài toán: biến dãy nhị phân $a_1, \dots, a_n$ thành một dãy có chu kỳ k , với ít lần thay đổi nhất.

Chia các vị trí thành k **chuỗi**: chuỗi thứ r gồm các vị trí

$$r, r + k, r + 2k, \dots \quad (1 \leq r \leq k).$$

Mỗi chuỗi phải trở thành hằng, vì các phần tử cách nhau k vị trí phải bằng nhau. Trong mỗi chuỗi, hãy đếm số số 0 và số số 1:



Hình 14.2: Các chuỗi gồm những phần tử có chỉ số cách nhau đúng k .

- nếu có nhiều 0 hơn, đổi toàn bộ các 1 thành 0;
- nếu có nhiều 1 hơn, đổi toàn bộ các 0 thành 1;
- nếu hai số lượng bằng nhau, chọn một trong hai giá trị bất kỳ.

Như vậy, với số thay đổi nhỏ nhất, ta thu được một dãy tuần hoàn chu kỳ k .

Tuy nhiên, dãy đó chưa chắc đã là đáp án. Một dãy nhị phân k -chẵn bắt buộc phải có chu kỳ k , nhưng điều ngược lại không đúng: tính tuần hoàn chỉ đảm bảo tất cả các đoạn độ dài k có **cùng tính chẵn lẻ**. Các tổng ấy có thể đều chẵn hoặc đều lẻ. Nếu chúng đều chẵn thì ta đã xong; nếu chúng đều lẻ thì cần xử lý thêm.

Khi đó, ta chỉ có thể đảo giá trị của toàn bộ một chuỗi. Đảo một chuỗi bất kỳ sẽ đổi tính chẵn lẻ của tổng trong đoạn đầu tiên gồm k phần tử, nên tất cả các đoạn độ dài k sẽ có tổng chẵn. Để số thay đổi tăng ít nhất, phải chọn chuỗi có số 0 và số 1 gần nhau nhất, tức là tối thiểu hóa trị tuyệt đối của hiệu giữa hai số lượng này.

Thật vậy, giả sử chuỗi được chọn có z số 0 và j số 1, với $z \leq j$; trường hợp $z \geq j$ đối xứng. Ở lời giải tuần hoàn tham lam ban đầu, chuỗi này có z lần thay đổi (đổi các số 0 thành 1). Sau khi đảo lựa chọn của chuỗi, số thay đổi sẽ là j . Tổng số thay đổi tăng thêm $j - z$, hay tổng quát là

$$|j - z|.$$

Do đó, khi cần đảo một chuỗi, ta chọn chuỗi có $|j - z|$ nhỏ nhất.

14.2.2 Ví dụ minh họa

Xét

$$a = 101000001010011000$$

với $k = 4$. Viết dãy thành các hàng liên tiếp có bốn phần tử; mỗi cột khi đó là một chuỗi:

1	0	1	0
0	0	0	0
1	0	1	0
0	1	1	0
0	0		

Đưa mọi chuỗi về hàng với số thay đổi ít nhất cho dãy tuần hoàn sau:

0	0	1	0
0	0	1	0
0	0	1	0
0	0	1	0
0	0		

Trong dãy này, mọi đoạn độ dài 4 có tổng bằng 1, là số lẻ. Ta phải đảo một cột. Rẻ nhất là đảo cột đầu tiên; ta nhận được một dãy đích, trong đó các ô in đậm là năm vị trí khác với dãy ban đầu:

1	0	1	0
1	0	1	0
1	0	1	0
1	0	1	0
1	0		

14.2.3 Chứng minh tính đúng đắn

Ta đã chứng minh rằng mọi dãy k -chẵn nhị phân phải tuần hoàn với chu kỳ k . Với mỗi chuỗi vị trí đồng dư modulo k , bất kỳ dãy

tuần hoàn nào cũng buộc toàn bộ chuỗi phải là 0 hoặc toàn bộ là 1. Vì vậy, chọn giá trị xuất hiện nhiều hơn trong từng chuỗi rõ ràng là cách duy nhất tối thiểu hóa độ lệch số thay đổi trong chuỗi đó; tổng của các lựa chọn này tối thiểu hóa số thay đổi để tạo ra một dãy chu kỳ k .

Nếu tổng của một chu kỳ là chẵn, mọi đoạn độ dài k đều chính là một hoán vị vòng của chu kỳ ấy, nên đều có tổng chẵn. Dãy thu được là hợp lệ và tối ưu.

Nếu tổng của một chu kỳ là lẻ, dãy chưa hợp lệ. Trong mọi dãy chu kỳ k , để đổi tính chẵn lẻ của tổng một chu kỳ, phải đảo lựa chọn ở một số lẻ các chuỗi. Đảo hai chuỗi trở lên không thể rẻ hơn việc chỉ đảo chuỗi có chi phí tăng nhỏ nhất: chi phí đảo mỗi chuỗi là không âm $|j - z|$. Vì thế chỉ cần đảo một chuỗi có $|j - z|$ nhỏ nhất. Sau phép đảo, tổng mỗi đoạn dài k đổi từ lẻ sang chẵn, và số thay đổi tăng ít nhất có thể. Thuật toán do đó cho số thay đổi nhỏ nhất.

14.2.4 Độ phức tạp

Duyệt dãy một lần để đếm số 0, số 1 trong từng chuỗi, rồi duyệt k chuỗi để cộng đáp án và tìm chi phí đảo nhỏ nhất. Tổng thời gian là $O(n + k) = O(n)$, và bộ nhớ bổ sung là $O(k)$.

DNA ★★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 128 MB

Mã bài: dna

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/dna>

15.1 Đề bài

Nhà khoa học điên Byteasar muốn tạo ra một loài sinh vật mới. Vì vậy, ông quyết định sửa đổi bộ gen của một con chuột Bytean.

Một mã DNA là một dãy các chữ cái A, C, G, T. Kế hoạch của Byteasar rất đơn giản: từ DNA của con chuột, ông sẽ tạo một DNA mới có cùng độ dài nhưng **ít giống** DNA của chuột nhất có thể. Độ giống nhau giữa hai DNA là độ dài của **dãy con chung dài nhất** của chúng.

Dãy con chung dài nhất của hai từ x, y là từ dài nhất có thể nhận được từ cả x lẫn y bằng cách xóa một số chữ cái (có thể không xóa chữ nào) khỏi mỗi từ. Lưu ý rằng hai từ có thể có nhiều dãy con chung dài nhất: chẳng hạn, các dãy con chung dài nhất của CACCA và CAAC là CAA và CAC.

Hãy viết chương trình tìm DNA cần tạo.

15.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên n ($1 \leq n \leq 10\,000$), là độ dài DNA của con chuột Bytean.

Dòng thứ hai chứa DNA của chuột: một dãy gồm n chữ cái in hoa thuộc tập $\{A, C, G, T\}$.

15.1.2 Dữ liệu ra

Dòng đầu in một số nguyên: độ giống nhau giữa DNA của chuột và DNA do chương trình tìm được.

Dòng thứ hai in một dãy n ký tự A, C, G, T: một DNA có độ giống với DNA đầu vào nhỏ nhất có thể. Nếu có nhiều đáp án đúng, được phép in bất kỳ đáp án nào.

15.1.3 Ví dụ

Với dữ liệu vào:

```
4
GACT
```

một kết quả đúng là:

```
1
TCAG
```

15.2 Lời giải

Khi mô tả lời giải cho một bài toán, thông thường ta cố gắng trình bày chuỗi suy luận dần dần đến thuật toán. Lần này sẽ khác: với bài DNA, ta đơn giản là phải **nghĩ ra** lời giải. Có lẽ nên giải thử vài ví dụ trên giấy, suy ngẫm một lúc hoặc tin vào trực giác; cuối cùng, ta vẫn phải đoán được thuật toán.

Lời giải có thể diễn tả bằng một câu: tìm chữ cái xuất hiện **ít nhất** trong DNA của chuột rồi in chữ cái đó n lần. Đặc biệt, nếu một trong

các chữ A, C, G, T không xuất hiện trong DNA đầu vào, hãy in chữ ấy n lần. Khi đó dãy con chung dài nhất của DNA vừa tạo và DNA chuột là dãy rỗng — ngắn nhất có thể.

15.2.1 Chứng minh tính đúng đắn

Ký hiệu $\text{LCS}(x, y)$ là độ dài dãy con chung dài nhất của hai từ x, y . Gọi w là DNA chuột đầu vào. Giả sử chữ cái xuất hiện ít nhất trong w là A, và nó xuất hiện a lần. Ta xuất từ

$$y = AA \dots A$$

gồm n chữ A. Khi đó

$$\text{LCS}(w, y) = a,$$

vì y chỉ gồm các chữ A, còn w có đúng a chữ A.

Để chứng minh thuật toán tối ưu, cần chỉ ra rằng không thể làm tốt hơn, tức là với mọi từ x có độ dài n ,

$$\text{LCS}(w, x) \geq a.$$

Xét hai trường hợp.

Nếu x chứa ít nhất a lần chữ A, thì dãy gồm a chữ A là dãy con của cả w lẫn x . Suy ra ngay $\text{LCS}(w, x) \geq a$.

Trường hợp còn lại, A xuất hiện trong x ít hơn a lần, tức ít hơn số lần nó xuất hiện trong w . Khi đó, một trong ba chữ C, G, T phải xuất hiện trong x ít nhất nhiều bằng số lần nó xuất hiện trong w . Thật vậy, nếu cả bốn chữ A, C, G, T đều xuất hiện trong x ít hơn tương ứng trong w , thì x sẽ có ít chữ cái hơn w , mâu thuẫn vì cả hai đều có độ dài n .

Vì thế, có một chữ cái — giả sử là C — xuất hiện c lần trong w và ít nhất c lần trong x . Dãy gồm c chữ C là dãy con của cả hai từ. Hơn nữa $c \geq a$, vì A là chữ xuất hiện ít nhất trong w . Do đó

$$\text{LCS}(w, x) \geq c \geq a.$$

Vậy không DNA độ dài n nào có thể giống w ít hơn y ; thuật toán đã cho là đúng.

15.2.2 Thuật toán

Đếm số lần xuất hiện của bốn chữ cái. Chọn một chữ cái có số lần xuất hiện nhỏ nhất, gọi số đó là a . In a ở dòng đầu và in chữ cái đã chọn lặp lại n lần ở dòng thứ hai.

15.2.3 Độ phức tạp

Thuật toán chạy trong thời gian $O(n)$ và dùng $O(1)$ bộ nhớ phụ, không kể chuỗi kết quả dài n .

15.3 Lời cuối

Khi chuẩn bị Giải vô địch Ba Lan năm 2012, bài DNA được xem là ứng viên cho bài dễ nhất của cuộc thi. Dù sao đi nữa, toàn bộ lời giải chỉ gói trong vài từ. Hóa ra chúng tôi đã nhầm đáng kể, vì đã không tính đến ít nhất ba yếu tố khiến bài này khó hơn tưởng tượng.

Thứ nhất, có thể tìm ra lời giải tương đối dễ, **nhưng chỉ sau khi đã bắt đầu tìm trong các ý tưởng đơn giản**. Nói cách khác, bài này dễ nếu ta biết trước rằng nó dễ; tiếc là đề bài hoàn toàn không cho thấy điều đó.

Thứ hai, ví dụ được thiết kế khá đánh lừa: với chuỗi GACT, một đáp án mẫu là TCAG, tức chuỗi đầu vào viết ngược lại — mặc dù AAAA cũng là một đáp án tốt không kém.

Cuối cùng, giới hạn độ dài DNA của chuột chỉ là 10 000, nhỏ hơn nhiều so với đa số bài có lời giải tuyến tính. Đây có phải thêm một cái bẫy có chủ ý của ban tổ chức không? Không: giới hạn nhỏ như vậy cần thiết vì lý do kỹ thuật. Để kiểm tra một đáp án, chương trình chấm phải chạy thuật toán bậc hai tìm dãy con chung dài nhất. Vì vậy bản thân trình chấm chạy trong thời gian $\Theta(n^2)$, buộc kích

thước dữ liệu kiểm tra phải đủ nhỏ để các chương trình gửi trong cuộc thi được chấm trong vài giây.

Tính giá trị biểu thức ★★★★★

Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 128 MB

Mã bài: ewa

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/ewa>

16.1 Đề bài

Xét một biểu thức (E) chứa các hằng số nguyên từ (0) đến (9), các biến từ a đến z, và ba phép toán: cộng, nhân, và lũy thừa với số mũ là hằng số. Điều đáng chú ý là mỗi biến a, b, ..., z xuất hiện trong biểu thức (E) **nhều nhất một lần**.

Với một số nguyên tố (p), hãy cho biết đa thức được biểu diễn bởi (E) có bao nhiêu nghiệm modulo (p). Nói cách khác, cần đếm số cách gán cho các biến xuất hiện trong (E) những số nguyên từ (0) đến (p-1), sao cho giá trị của (E) chia hết cho (p). Số nghiệm có thể rất lớn, vì vậy chỉ cần in phần dư của nó khi chia cho (30,011).

Ví dụ, đa thức được biểu diễn bởi

$$E = ((a + y) \cdot (z + 8))^2$$

có (15) nghiệm modulo (p=3), trong đó có các nghiệm

$(a = 0, y = 0, z = 0), \quad (a = 1, y = 2, z = 0), \quad (a = 2, y = 0, z = 1).$

Về hình thức, biểu thức được định nghĩa đệ quy như sau:

- Mỗi hằng số nguyên $0, 1, \dots, 9$ là một biểu thức.
- Mỗi biến $a, b, \dots, z$ là một biểu thức.
- Nếu (A) và (B) là các biểu thức, thì $(A+B)$ và $(A*B)$ cũng là các biểu thức, lần lượt biểu diễn tổng và tích của (A,B) .
- Nếu (A) là biểu thức và (B) là hằng số nguyên từ (2) đến (9) , thì (A^B) cũng là một biểu thức, biểu diễn (A) mũ (B) .

16.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên tố (p) $((2 \leq p < 15,000))$.

Dòng thứ hai chứa một biểu thức (E) đúng theo đặc tả trên. Biểu thức là một chuỗi dài không quá (300) ký tự thuộc tập $0-9, a-z, +, *, ^, (,)$; chuỗi không chứa khoảng trắng.

16.1.2 Dữ liệu ra

Gọi (k) là số nghiệm modulo (p) của đa thức (E) . In một số nguyên không âm là phần dư của (k) khi chia cho $(30,011)$.

16.1.3 Ví dụ

Dữ liệu vào

3
 $((a+y)*(z+8))^2$

Dữ liệu ra

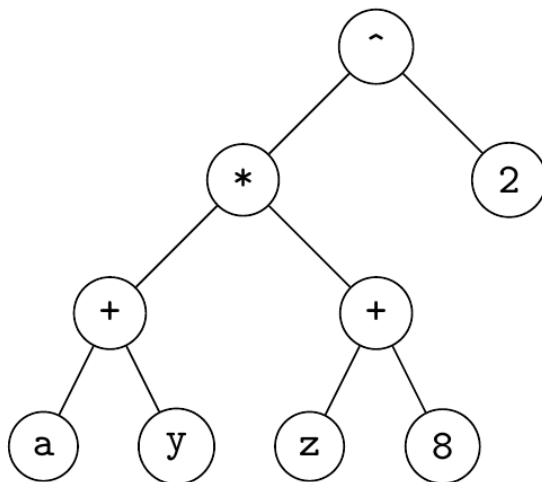
15

16.2 Lời giải

Thoạt nhìn, đây có vẻ là một bài toán thuần túy toán học. Thực ra lời giải chủ yếu dựa trên các ý tưởng thuật toán. Điều kiện mỗi biến chỉ xuất hiện nhiều nhất một lần là gợi ý quan trọng: trong toán học, một hạn chế kiểu này ít gặp; trong xử lý biểu thức bằng máy tính, nó giúp các phần con trở nên độc lập.

16.2.1 Phân tích cú pháp và cây biểu thức

Trước hết, chuyển chuỗi đầu vào thành **cây biểu thức**. Lá biểu diễn hằng số hoặc biến; nút trong biểu diễn phép cộng, nhân, hoặc lũy thừa. Chẳng hạn, với biểu thức ví dụ, cây có dạng:



Hình 16.1: Cây của biểu thức $E = ((a + y) \cdot (z + 8))^2$.

Việc chuyển văn bản thành một biểu diễn cấu trúc như vậy thường được gọi là *parsing* (phân tích cú pháp). Trong bài này, nó đặc biệt đơn giản vì:

1. mỗi hằng số và mỗi biến chỉ chiếm đúng một ký tự;
2. mọi biểu thức biểu diễn một phép toán đều được bao bởi một cặp ngoặc.

Ta dùng một hàm đệ quy nhận vị trí ký tự đầu tiên của một biểu thức con và trả về vị trí ký tự cuối cùng của nó, đồng thời xây dựng nút gốc của cây con đó.

- Nếu ký tự đầu là một chữ cái hoặc chữ số, biểu thức con chỉ gồm ký tự ấy; tạo một lá và trả về chính vị trí đó.
- Ngược lại, ký tự đầu là (. Gọi đệ quy để xây cây con trái bắt đầu ngay sau dấu ngoặc mở. Ký tự ngay sau phần mô tả cây con trái là toán tử của nút hiện tại. Sau đó gọi đệ quy để xây cây con phải, bắt đầu ngay sau toán tử. Cuối cùng, ký tự sau cây con phải chính là dấu) đóng, và vị trí này được trả về.

Mỗi ký tự chỉ được xử lý số lần hằng số, nên quá trình này mất thời gian tuyến tính theo độ dài biểu thức. Cây thu được cũng có kích thước tuyến tính.

16.2.2 Quy hoạch động trên cây

Khi đã có cây biểu thức, bài toán tự nhiên tách thành những bài toán con ứng với các cây con. Đây là lúc điều kiện không lặp biến được dùng một cách quyết định: hai cây con rời nhau chứa hai tập biến rời nhau, do đó các phép gán biến cho chúng độc lập.

Chỉ lưu số nghiệm của mỗi cây con là chưa đủ. Với mỗi nút u , ta lưu mảng

$$t_u[0], t_u[1], \dots, t_u[p-1],$$

trong đó $t_u[k]$ là số phép gán cho các biến xuất hiện trong cây con tại u khiến giá trị cây con đồng dư $k \pmod{p}$. Mọi phần tử mảng được tính modulo

$$q = 30\,011.$$

Các trường hợp cơ sở

- Nếu u là lá mang hằng số $c \in \{0, \dots, 9\}$, đặt $c' = c \bmod p$. Khi đó

$$t_u[c'] = 1, \quad t_u[k] = 0 \quad (k \neq c').$$

- Nếu u là một biến, thì

$$t_u[k] = 1 \quad (0 \leq k < p),$$

vì biến đó có đúng một cách nhận mỗi giá trị k modulo p .

Các phép toán

Giả sử u có hai con v, w .

Với phép cộng, ta có

$$t_u[k] = \sum_{i+j \equiv k \pmod{p}} t_v[i] t_w[j]. \quad (1)$$

Với phép nhân, ta có

$$t_u[k] = \sum_{i \cdot j \equiv k \pmod{p}} t_v[i] t_w[j]. \quad (2)$$

Nếu u là nút lũy thừa, có con v và số mũ $b \in \{2, \dots, 9\}$, thì

$$t_u[k] = \sum_{i^b \equiv k \pmod{p}} t_v[i]. \quad (3)$$

Vì sao các công thức đúng? Xét (1). Để tổng hai biểu thức có phần dư k , phần dư của chúng phải là một cặp (i, j) thỏa $i + j \equiv k \pmod{p}$. Có $t_v[i]$ cách gán biến ở cây trái cho phần dư i , và $t_w[j]$ cách ở cây phải cho phần dư j . Hai lựa chọn có thể ghép tự do vì hai cây không dùng chung biến, nên có đúng $t_v[i] t_w[j]$ cách. Cộng

trên mọi cặp phù hợp cho (1). Lập luận y hệt cho (2); còn (3) chỉ gom tất cả phần dư i mà khi nâng lên mũ b cho ra k .

Sau khi tính xong nút gốc r , đáp án là $t_r[0]$.

16.2.3 Vì sao cách trực tiếp chưa đủ nhanh?

Gọi n là số nút của cây. Tổng kích thước mọi mảng t_u là $O(np)$. Các lá và các nút lũy thừa đều xử lý được trong $O(p)$: với lũy thừa, duyệt từng i , tính $k = i^b \bmod p$, rồi cộng $t_v[i]$ vào $t_u[k]$.

Nếu thực hiện (1) và (2) trực tiếp, mỗi nút cộng hoặc nhân tốn $O(p^2)$. Với phép cộng, với mỗi k và i , chỉ cần lấy $j = (k - i) \bmod p$. Với phép nhân và $i \neq 0$, có thể lấy $j = ki^{-1} \bmod p$, vì mọi phần dư khác 0 modulo số nguyên tố đều có nghịch đảo. Dù vậy, độ phức tạp tệ nhất vẫn là

$$O(np^2),$$

quá lớn khi p gần 15 000.

Ta cần tăng tốc hai phép tích chập này bằng FFT.

16.2.4 FFT cho nút cộng

FFT cho phép tính các hệ số của tích hai đa thức bậc nhỏ hơn m trong $O(m \log m)$. Các giá trị trung gian trong FFT không nhất thiết là số nguyên; ở đây cần dùng kiểu thực có khoảng 63 bit chính xác, chẳng hạn long double trong GCC. Thật vậy, trong bài này $m < p$, còn các hệ số đầu vào không vượt q , nên hệ số kết quả nhiều nhất cùng bậc với $m \max a_i \max b_i$, vẫn nằm trong phạm vi có thể làm tròn chính xác với độ chính xác này.

Xét một nút cộng u , và lập hai đa thức

$$A(x) = \sum_{i=0}^{p-1} t_v[i]x^i, \quad B(x) = \sum_{i=0}^{p-1} t_w[i]x^i.$$

Tích của chúng là

$$C(x) = A(x)B(x) = \sum_{i=0}^{2p-2} c[i]x^i.$$

Mỗi tích $t_v[i]x^i \cdot t_w[j]x^j$ đóng góp vào hệ số $c[i+j]$. Nếu $i+j \equiv k \pmod{p}$, thì $i+j$ chỉ có thể là k hoặc $k+p$. Do đó, sau một phép nhân đa thức bằng FFT,

$$t_u[k] = c[k] + c[k+p], \quad (4)$$

trong đó có thể xem hệ số ngoài phạm vi của C là 0. Nút cộng được xử lý trong $O(p \log p)$.

16.2.5 Biến phép nhân thành phép cộng bằng generator

Phép nhân tưởng chừng khác hẳn, nhưng có thể chuyển về dạng tương tự. Ta dùng một kết quả chuẩn của lý thuyết số.

Bổ đề. Với mọi số nguyên tố p , tồn tại số nguyên dương g sao cho mỗi phần dư khác 0 modulo p được biểu diễn duy nhất dưới dạng

$$g^i \bmod p \quad \text{với } i \in \{0, 1, \dots, p-2\}.$$

Số g được gọi là một **generator modulo p** (hay căn nguyên thủy modulo p).

Xét (2) trước hết với $k > 0$; khi đó cả i, j trong mọi cặp đóng góp cũng khác 0. Viết

$$i \equiv g^{i'} \pmod{p}, \quad j \equiv g^{j'} \pmod{p}, \quad \text{và } k \equiv g^{k'} \pmod{p}.$$

Điều kiện $ij \equiv k \pmod{p}$ trở thành

$$g^{i'+j'} \equiv g^{k'} \pmod{p},$$

tức là, nhờ tính duy nhất của biểu diễn theo generator,

$$i' + j' \equiv k' \pmod{p-1}.$$

Đôi chỉ số mảng theo

$$t'_u[x] = t_u[g^x \bmod p]$$

(tương tự với v, w). Ta nhận được

$$t'_u[k'] = \sum_{i'+j' \equiv k' \pmod{p-1}} t'_v[i'] t'_w[j']. \quad (5)$$

Đây chính là công thức của nút cộng, chỉ khác là modulo $p-1$ thay vì modulo p . Vì vậy, một phép FFT nữa cho phép tính toàn bộ $t'_u[k']$ trong $O(p \log p)$, và từ đó suy ra mọi $t_u[k]$ với $k > 0$.

Trường hợp $k = 0$ có thể xử lý riêng trong $O(p)$. Tích bằng 0 khi ít nhất một thừa số bằng 0, nên chẳng hạn dùng trực tiếp

$$t_u[0] = t_v[0] \sum_{j=0}^{p-1} t_w[j] + t_w[0] \sum_{i=1}^{p-1} t_v[i] \pmod{q}.$$

Công thức này đếm mỗi cặp có thừa số trái bằng 0, rồi thêm các cặp có thừa số phải bằng 0 nhưng thừa số trái khác 0, nên không đếm đôi cặp $(0, 0)$.

16.2.6 Tìm generator

Trong các giới hạn bài này, không cần thuật toán lý thuyết số tinh vi để tìm g . Thử lần lượt các ứng viên $2, 3, \dots$. Với mỗi ứng viên x , tính các lũy thừa liên tiếp

$$x^0, x^1, \dots, x^{p-2} \pmod{p}$$

và dùng mảng đánh dấu để kiểm tra xem có phần dư nào lặp trước khi đạt mũ $p-1$ hay không. Nếu không lặp, x là generator.

Phương pháp vét cạn này có chặn tệ nhất $O(p^2)$, nhưng trên thực tế rất nhanh nếu dừng ngay ở generator đầu tiên. Nó chỉ thực hiện một lần, không phải cho từng nút của cây.

16.2.7 Chứng minh tính đúng đắn

Ta chứng minh bằng quy nạp theo cây rằng sau khi xử lý mỗi nút u , mảng t_u lưu đúng số phép gán tạo ra từng phần dư modulo p .

- Với lá hằng và lá biến, điều này đúng trực tiếp theo định nghĩa các mảng ở phần cơ sở.
- Giả sử điều đó đúng với các cây con của một nút cộng hoặc nhân. Các tập biến của hai cây con rời nhau, do mỗi biến toàn cục chỉ xuất hiện một lần. Vì thế mỗi cặp phép gán con ghép thành đúng một phép gán cho nút cha. Phân hoạch các cặp theo phần dư của hai giá trị cho đúng các công thức (1) và (2), nên mảng ở nút cha cũng đúng.
- Với nút lũy thừa, mỗi phép gán cho cây con cho một phần dư i , sau đó xác định duy nhất phần dư $i^b \bmod p$ của nút cha. Gộp các phép gán cùng cho ra k chính là (3).

Phép FFT không làm thay đổi công thức cần tính: (4) chỉ nhóm các cặp chỉ số có tổng bằng k hoặc $k + p$, đúng bằng điều kiện tổng đồng dư k modulo p . Tương tự, generator tạo song ánh giữa các phần dư khác 0 và các số mũ modulo $p - 1$; vì $g^{i'} g^{j'} = g^{i'+j'}$, (5) nhóm chính xác những cặp có tích đồng dư với phần dư cần xét. Trường hợp 0 đã được tính riêng. Vậy mọi mảng đều được tính đúng, và $t_r[0]$ tại nút gốc là số phép gán khiến $E \equiv 0 \pmod{p}$, tức đáp án yêu cầu.

16.2.8 Độ phức tạp

Tạo cây tốn $O(|E|)$ thời gian và bộ nhớ. Với n nút của cây, mọi mảng quy hoạch động chiếm tổng cộng $O(np)$ bộ nhớ. Mỗi lá hoặc nút lũy thừa tốn $O(p)$; mỗi nút cộng hoặc nhân tốn $O(p \log p)$ nhờ FFT. Do đó thời gian tổng là

$$O(np \log p),$$

cộng thêm thời gian một lần để tìm generator (tệ nhất $O(p^2)$). Bộ nhớ dùng cho lời giải là $O(np)$, ngoài bộ đệm $O(p)$ phục vụ FFT.

Công thức 1 ★★★★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 128 MB

Mã bài: for

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/for>

17.1 Đề bài

Cậu bé Bytie rất thích xem các cuộc đua Công thức 1 hằng năm trên đường đua nổi giữa Bytetown và Byteburg. Những khoảnh khắc hào hứng nhất đối với cậu là các pha vượt xe; cậu muốn được xem càng nhiều pha vượt càng tốt.

Bytie mơ về một cuộc đua có (n) xe Công thức 1. Với mỗi (i) ($(1 \leq i \leq n)$), chiếc xe xuất phát ở vị trí thứ (i) phải thực hiện đúng a_i lần vượt trong cuộc đua. Để đơn giản, tại mỗi thời điểm nhiều nhất chỉ có một pha vượt, với đúng hai xe tham gia: một xe vượt chiếc xe **ngay phía trước** nó.

Hãy xác định liệu một cuộc đua như vậy có thể diễn ra hay không.

17.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên (t), là số bộ dữ liệu kiểm tra.

Mỗi bộ dữ liệu gồm hai dòng:

- Dòng đầu chứa số nguyên (n) ($(1 \leq n \leq 1,000,000)$), số xe tham gia.
- Dòng tiếp theo chứa (n) số nguyên $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$); a_i là số lần vượt mà xe xuất phát thứ (i) phải thực hiện.

Tổng kích thước của một tệp dữ liệu vào không vượt quá (20) MB.

17.1.2 Dữ liệu ra

In (t) dòng, mỗi dòng ứng với một bộ dữ liệu. In TAK nếu có thể tổ chức cuộc đua đã cho, và NIE nếu không thể.

17.1.3 Ví dụ

Với dữ liệu vào:

```
3
2
0 1
3
0 1 4
3
1 1 3
```

kết quả đúng là:

```
TAK
NIE
TAK
```

17.2 Lời giải

17.2.1 Số lần vượt tối đa của một xe

Cố định một xe (k). Giả sử mọi xe khác đều phải thực hiện đúng số lần vượt đã cho trong đầu vào. Ký hiệu s_k là **số lần vượt lớn nhất** mà xe (k) có thể thực hiện trong một cuộc đua thỏa giả thiết đó.

Trước hết xét xe cuối cùng, xe (n). Nó có thể lần lượt vượt tất cả (n-1) xe đứng trước, nên có ngay (n-1) lần vượt. Sau khi vượt xe (i), hai xe có thể thay phiên vượt nhau thêm a_i cặp thao tác theo thứ tự

$$i \text{ vượt } n, \quad n \text{ vượt } i.$$

Việc này cho xe (n) thêm a_i lần vượt, đồng thời dùng hết toàn bộ a_i lần vượt bắt buộc của xe (i). Đây là chiến lược tốt nhất, vì thế

$$s_n = \sum_{i=1}^{n-1} (a_i + 1).$$

Với xe ($k \neq n$), việc vượt các xe xuất phát trước nó hoàn toàn tương tự. Nhưng để vượt một xe xuất phát sau nó, xe đó phải vượt xe (k) trước; điều này không phải lúc nào cũng làm được. Ta tách

$$s_k = A_k + B_k,$$

trong đó A_k chỉ phụ thuộc vào các xe đứng trước (k), còn B_k chỉ phụ thuộc vào các xe đứng sau (k). Phần thứ nhất là

$$A_k = \sum_{i=1}^{k-1} (a_i + 1).$$

Để tính phần thứ hai, với ($i > k$), đặt $b_{k,i}$ là số lần vượt ít nhất mà xe (i) phải làm để đứng ngay sau xe (k), nếu mọi xe ở giữa sẽ vượt xe (k) khi chúng còn có thể làm vậy. Ta có

$$b_{k,k+1} = 0, \quad b_{k,i+1} = b_{k,i} + [a_i - b_{k,i} \leq 0] \quad (i > k).$$

Ở đây ([P]) là ngoặc Iverson: bằng (1) nếu mệnh đề (P) đúng, và bằng (0) nếu sai. Quả vậy, xe $(k+1)$ vốn đã ở ngay sau xe (k) . Để xe $(i+1)$ đứng ngay sau (k) , trước hết nó cần làm số lần vượt tương ứng với xe (i) ; ngoài ra, nó phải vượt xe (i) thêm một lần nếu xe (i) không còn đủ lượt để tự vượt xe (k) .

Mỗi xe đã đến ngay sau xe (k) có thể dùng các lượt vượt còn lại để luân phiên vượt với xe (k) . Do đó

$$B_k = \sum_{i=k+1}^n \max(0, a_i - b_{k,i}).$$

Các công thức này tính được một giá trị s_k trong $(O(n))$ thời gian.

17.2.2 Điều kiện cần và xe tới hạn

Điều kiện cần hiển nhiên để có lời giải là mỗi xe phải có thể thực hiện đủ số lượt được yêu cầu:

$$s_k \geq a_k \quad \text{với mọi } 1 \leq k \leq n. \quad (1)$$

Điều đáng chú ý là điều kiện này cũng là **đủ**; chứng minh sẽ được đưa ra sau. Trước hết, hãy kiểm tra nó nhanh.

Nếu tính trực tiếp mọi B_k , độ phức tạp sẽ là (On^2) . Ta có thể quan sát rằng A_k tính được cho mọi (k) trong $(O(n))$. Bất cứ xe nào đã thỏa $A_k \geq a_k$ đều tự động thỏa (1), không cần tính B_k . Những xe còn lại có $a_k > A_k$; do A_k lớn hơn tổng các a_i trước nó ít nhất $(k-1)$, các giá trị a_k tại những vị trí này tăng theo cấp số nhân. Vì vậy chỉ có $(O(\log \max_i a_i))$ xe cần tính B_k , dẫn đến một cách làm $(On \log \max_i a_i)$.

Ta còn có thể làm tốt hơn nữa. Gọi (m) là **vị trí cuối cùng** sao cho

$$a_m \geq A_m.$$

Xe m được gọi là **xe tới hạn**. Nó hoặc dùng hết mọi lượt vượt của các xe xuất phát trước nó, hoặc bắt buộc phải bị một xe xuất phát

sau nó vượt. Hóa ra (1) chỉ có thể sai tại xe tới hạn. Vì thế toàn bộ điều kiện (1) tương đương đúng với một bất đẳng thức duy nhất:

$$s_m \geq a_m. \quad (2)$$

Thật vậy, xét một $k \neq m$.

- Nếu $k > m$, từ tính cuối cùng của m ta có $a_k < A_k$, nên

$$s_k = A_k + B_k \geq A_k > a_k.$$

- Nếu $k < m$, do $a_m \geq A_m \geq a_k + m - 1$, và luôn có $b_{k,m} \leq m - 2$, nên

$$\begin{aligned} s_k &\geq B_k \\ &\geq \max(0, a_m - b_{k,m}) \\ &\geq \max(0, (a_k + m - 1) - (m - 2)) > a_k. \end{aligned} \quad (3)$$

Do đó chỉ cần tìm m , rồi tính s_m bằng các công thức của phần trước. Cả hai bước đều là một lần duyệt, nên thuật toán chạy trong $O(n)$ thời gian.

17.2.3 Thuật toán quyết định

Với mỗi bộ kiểm tra:

1. Duyệt từ trái sang phải, duy trì A_i , và lưu vị trí cuối cùng m thỏa $a_m \geq A_m$. Sau đó m là xe tới hạn.
2. Ta đã có A_m . Duyệt các xe $i = m + 1, \dots, n$, duy trì $b = b_{m,i}$, và cộng $\max(0, a_i - b)$ vào B_m . Sau khi xét xe i , cập nhật $b \leftarrow b + [a_i - b \leq 0]$.
3. In TAK khi $A_m + B_m \geq a_m$; ngược lại in NIE.

Các tổng có thể đạt cỡ $n(10^9 + 1)$, vì vậy cần dùng kiểu số nguyên 64 bit.

17.2.4 Chứng minh tính đúng đắn

Phần còn lại chứng minh rằng (1) là điều kiện đủ. Chứng minh có tính xây dựng: nó chỉ ra thứ tự các pha vượt.

Với một dãy yêu cầu hiện tại $a_1, \dots, a_n$ đang thỏa (1), xác định xe tới hạn (m). Luôn thực hiện lượt vượt bằng **xe đầu tiên còn lượt vượt và đang ở sau xe tới hạn**. Nếu không có xe nào như vậy, cho chính xe tới hạn vượt. Sau mỗi lượt, số lượt còn phải thực hiện của xe đã vượt giảm đi một, và thứ tự hai xe kề nhau hoán đổi. Xe tới hạn có thể thay đổi trong quá trình này.

Ta sẽ chứng minh rằng một lượt theo quy tắc trên luôn tạo ra một dãy mới $a'_1, \dots, a'_n$ vẫn thỏa (1). Khi ấy lặp lại quy tắc sẽ giảm tổng $\sum_i a_i$ từng đơn vị, nên cuối cùng đạt dãy toàn (0), tức là dừng được toàn bộ cuộc đua.

Trường hợp 1: phía sau xe tới hạn không còn lượt nào

Giả sử $a_i = 0$ với mọi $(i > m)$, bao gồm cả trường hợp $(m = n)$. Khi đó $B_m = 0$. Mặt khác, do (m) tới hạn và (1) đúng,

$$A_m = s_m - B_m = s_m \geq a_m \geq A_m.$$

Suy ra $a_m = A_m$. Xe (m) có thể dùng đúng chiến lược đã mô tả ở đầu lời giải: lần lượt vượt mọi xe phía trước, và với xe (i) thực hiện chuỗi luân phiên gồm a_i lượt của xe (i) rồi xe (m). Vì vậy tồn tại cách hoàn tất phần đua còn lại sau khi xe (m) thực hiện lượt vượt đầu tiên. Do (1) là điều kiện cần, dãy sau lượt đầu tiên vẫn phải thỏa (1).

Trường hợp 2: có xe phía sau còn lượt vượt

Gọi $(j > m)$ là chỉ số nhỏ nhất sao cho $a_j > 0$. Theo quy tắc, xe (j) sẽ vượt. Sau phép vượt, trong dãy theo thứ tự trên đường đua, hai giá trị kề nhau đổi chỗ và giá trị của xe (j) giảm một:

$$a'_i = a_i \ (i \notin \{j-1, j\}), \quad a'_{j-1} = a_j - 1, \quad a'_j = a_{j-1}.$$

Với mọi ($i > j$), ta có $A'_i = A_i - 1$. Vì ($i > m$), định nghĩa xe tới hạn cho $A_i > a_i$, nên

$$s'_i = A'_i + B'_i \geq A'_i = A_i - 1 \geq a_i = a'_i. \quad (4)$$

Vậy chỉ còn phải xét các xe không ở sau (j). Có hai khả năng.

Trường hợp 2A: xe (j) không vượt xe tới hạn

Điều này nghĩa là giữa (m) và (j) có ít nhất một xe. Trước và sau lượt vượt, đoạn liên quan có dạng

$$\begin{array}{ccccccccccc} a_1 & \dots & a_m & 0 & 0 & \dots & 0 & a_j & a_{j+1} & \dots & a_n \\ & & & & & & & \downarrow & & & \\ a_1 & \dots & a_m & 0 & 0 & \dots & a_{j-1} & 0 & a_{j+1} & \dots & a_n \end{array}$$

Vì $A_j > a_j$, ta có

$$s'_{j-1} = A'_{j-1} + B'_{j-1} \geq A'_{j-1} = A_j - 1 > a_j - 1 = a'_{j-1}.$$

Kết hợp với (4), và với việc mọi xe có chỉ số lớn hơn (m) nhưng giá trị bằng (0) đương nhiên thỏa (1), điều kiện đúng cho mọi xe sau (m).

Nếu ($i \leq m$), khi tính B_i , xe (j) dù sao cũng phải vượt toàn bộ các xe từ ($i+1$) đến (m); lượt vừa thực hiện không thay đổi điều đó cũng như không thay đổi phần phía sau (j). Do đó $B'_i = B_i$, đồng thời $A'_i = A_i$. Suy ra

$$s'_i = A'_i + B'_i = A_i + B_i = s_i \geq a_i = a'_i.$$

Trường hợp 2B: xe (j) vượt xe tới hạn

Khi đó ($j = m+1$), và phép vượt biến đổi đoạn liên quan như sau:

$$\begin{array}{ccccccccccc} a_1 & \dots & a_{(m-1)} & a_m & a_{(m+1)} & a_{(m+2)} & \dots & a_n \\ & & & & & \downarrow & & \\ a_1 & \dots & a_{(m-1)} & a_{(m+1)-1} & a_m & a_{(m+2)} & \dots & a_n \end{array}$$

Bất đẳng thức (4) đã xử lý mọi $(i > m + 1)$. Với $(i < m)$, ta có

$$a'_{m+1} = a_m \geq A_m \geq a_i + m - 1 = a'_i + m - 1.$$

Áp dụng đúng lập luận của (3) cho xe $(m + 1)$, suy ra $s'_i \geq a'_i$.

Do (m) là xe tới hạn cuối cùng,

$$a_{m+1} < A_{m+1} = A_m + (a_m + 1).$$

Vì thế, với chỉ số (m) ,

$$s'_m \geq A'_m + a'_{m+1} = A_m + a_m \geq a_{m+1} > a_{m+1} - 1 = a'_m.$$

Cuối cùng, vì các giá trị sau $(m + 1)$ không đổi nên $B'_{m+1} = B_{m+1}$. Ngoài ra, $b_{m,m+1} = 0$, $a_{m+1} > 0$, nên $b_{m,m+2} = 0 = b_{m+1,m+2}$, và do đó

$$B_m = a_{m+1} + B_{m+1}.$$

Suy ra

$$\begin{aligned} s'_{m+1} &= A'_m + (a'_m + 1) + B'_{m+1} \\ &= A_m + a_{m+1} + B_{m+1} \\ &= A_m + B_m = s_m \\ &\geq a_m = a'_{m+1}. \end{aligned}$$

Vậy trong mọi trường hợp, một lượt đi theo quy tắc vẫn giữ (1). Quy nạp theo tổng số lượt còn lại cho thấy nếu (1) đúng thì ta luôn dựng được cuộc đua hợp lệ. Kết hợp với tính cần thiết đã nêu, thuật toán in TAK khi và chỉ khi đáp án là có.

17.2.5 Độ phức tạp

Mỗi bộ kiểm tra được duyệt một số lần hằng số. Thời gian là $O(n)$; bộ nhớ phụ là $O(n)$ nếu lưu mảng (a) để thực hiện lượt duyệt thứ hai, hoặc $O(n-m)$ nếu chỉ lưu đoạn hậu tố cần thiết. Trong thực tế, $O(n)$ số nguyên 64 bit vẫn phù hợp giới hạn bộ nhớ.

Cứu khủng long ★★★★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 128 MB

Mã bài: gen

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/gen>

18.1 Đề bài

Tin nóng từ Bytetown: các nhà khảo cổ vừa phát hiện hóa thạch khủng long gần thành phố! Sau khi nghe tin, một số người dân Bytetown muốn tới lấy một vài chiếc xương cho riêng mình. Để bảo vệ những di tích vô giá này, thị trưởng Bytetown quyết định bảo vệ khu khai quật và đã thuê quân đội cho mục đích đó.

Đại tướng Byteasar đã bố trí (n) binh lính tại một số vị trí chiến lược trong khu khai quật. (Binh lính không thể đứng tùy tiện vì sẽ cản trở công việc của các nhà khảo cổ; ngoài ra, họ phải có tầm nhìn tốt để bảo vệ khu vực.) Ta gọi một điểm trong khu vực là **được bảo vệ** nếu, khi di chuyển từ điểm đó theo bất kỳ hướng nào, ta không thể tránh việc làm giảm khoảng cách tới ít nhất một người lính.

Byteasar vừa được điều đến một tân binh. Đại tướng quyết định đặt người lính này tại một trong (m) vị trí chiến lược còn trống. Với

mỗi vị trí có thể chọn, ông muốn biết tổng diện tích phần khu khai quật được bảo vệ.

18.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên (n) và (m) ($(3 \leq n \leq 100,000)$, $(1 \leq m \leq 100,000)$), lần lượt là số binh lính đã được bố trí và số vị trí chiến lược còn trống.

(n) dòng tiếp theo mô tả vị trí các binh lính. Dòng thứ (i) chứa hai số nguyên x_i, y_i ($-10^8 \leq x_i, y_i \leq 10^8$), là tọa độ của vị trí binh lính thứ (i) đang đứng trong hệ tọa độ Descartes.

(m) dòng tiếp theo mô tả, theo cùng định dạng, các vị trí chiến lược còn trống. Mọi điểm được nêu trong đầu vào đều phân biệt.

Có thể giả sử diện tích phần khu vực được bảo vệ bởi (n) binh lính ban đầu là dương.

18.1.2 Dữ liệu ra

In đúng (m) dòng. Dòng thứ (i) phải là tổng diện tích phần khu vực được bảo vệ khi tân binh được đặt tại vị trí chiến lược còn trống thứ (i) . Mỗi số phải được in với đúng một chữ số sau dấu thập phân.

18.1.3 Ví dụ

Với dữ liệu vào:

```
3 2
0 0
2 -1
1 2
3 1
1 0
```

kết quả đúng là:

```
5.0
2.5
```

18.2 Lời giải

18.2.1 Vùng được bảo vệ là bao lồi

Trước hết, hãy xác định hình dạng của vùng được bảo vệ với một cách bố trí binh lính cố định. Xét ba người lính bất kỳ ở các điểm p_i, p_j, p_k . Mọi điểm nằm trong tam giác $p_i p_j p_k$ đều được bảo vệ: từ một điểm bên trong tam giác, di chuyển theo bất kỳ hướng nào cũng sẽ làm ta gần hơn với ít nhất một đỉnh của tam giác. Vì thế, mọi điểm trong **bao lồi** của các vị trí binh lính đều được bảo vệ.

Đây cũng là toàn bộ vùng được bảo vệ. Một điểm ở bên ngoài bao lồi có thể được ngăn cách khỏi bao lồi bởi một đường thẳng. Di chuyển từ điểm ấy theo hướng ra xa đường thẳng sẽ đồng thời tăng khoảng cách tới mọi điểm của bao lồi, nên cũng tăng khoảng cách tới mọi người lính. Các điểm trên biên bao lồi thực ra không được bảo vệ, nhưng biên có diện tích bằng (0), nên điều này không ảnh hưởng đến đáp án.

Vì vậy, bài toán tương đương với: với mỗi điểm trống $p_j = (x_j, y_j)$, hãy tính diện tích bao lồi của các vị trí ban đầu cộng thêm p_j . Ta sẽ luôn làm việc với **gấp đôi diện tích**, để mọi phép tính đều nguyên.

Một cách làm đơn giản là, với từng điểm trống, tính lại bao lồi rồi tính diện tích đa giác. Điều này tốn $O(mn \log n)$. Nếu sắp xếp trước các điểm ban đầu, bao lồi của chúng có thể được dựng trong $O(n)$, và cách làm này cải thiện thành $O(n \log n + mn)$ — vẫn chưa đủ nhanh.

18.2.2 Diện tích sau khi thêm một điểm

Hãy tính trước bao lồi W của n vị trí ban đầu trong $O(n \log n)$. Gọi các đỉnh của nó, theo thứ tự ngược chiều kim đồng hồ, là

$$a_0, a_1, \dots, a_{k-1},$$

với quy ước $a_k = a_0$ và $a_{-1} = a_{k-1}$. Với hai điểm $a_i = (x_i, y_i)$ và $a_j = (x_j, y_j)$, ký hiệu tích có hướng là

$$a_i \times a_j = x_i y_j - y_i x_j.$$

Gấp đôi diện tích của (W) là công thức dây giày

$$\sum_{i=0}^{k-1} a_i \times a_{i+1}. \quad (1)$$

Cố định một điểm trống p_j . Nếu p_j nằm trong hoặc trên biên (W), bao lồi không đổi và ta trả về ngay diện tích của (W). Từ đây giả sử p_j ở bên ngoài (W).

Xét góc nhỏ nhất có đỉnh tại p_j chứa toàn bộ W . Hai cạnh của góc này tiếp xúc với W ; nếu một cạnh chứa nhiều điểm của W , chọn điểm gần p_j nhất. Hai điểm tiếp xúc là các đỉnh của W ; gọi chỉ số của chúng là $\alpha(j)$ và $\beta(j)$.

Bao lồi mới W_j gồm p_j , $a_{\alpha(j)}$, $a_{\beta(j)}$ và phần biên của W nối $a_{\alpha(j)}$ với $a_{\beta(j)}$ ở xa p_j . Nói cách khác, biên của W_j gồm hai cạnh $a_{\alpha(j)}p_j$, $p_j a_{\beta(j)}$ và một đoạn liên tiếp các cạnh của W .

Để cộng nhanh đoạn liên tiếp đó, đặt

$$d[0] = 0, \quad d[i+1] = d[i] + a_i \times a_{i+1} \quad \text{với } i = 0, \dots, k-1.$$

Khi đó, gấp đôi diện tích của W_j tính được trong thời gian hằng số:

$$a_{\alpha(j)} \times p_j + p_j \times a_{\beta(j)} + \begin{cases} d[k] - d[\beta(j)] + d[\alpha(j)], & \alpha(j) < \beta(j), \\ d[\alpha(j)] - d[\beta(j)], & \alpha(j) > \beta(j). \end{cases} \quad (2)$$

Vì thế, phần còn lại chỉ là tìm nhanh hai chỉ số tiếp xúc $\alpha(j)$ và $\beta(j)$ cho mọi điểm trống.

18.2.3 Quét góc để tìm các điểm tiếp xúc

Kéo dài mỗi cạnh $a_i a_{i+1}$ của (W) thành một tia bắt đầu tại a_i . Cụ thể, với “tia có đỉnh a_i ” ta hiểu tia cùng phương với $a_{i-1} a_i$, nhưng

bắt đầu từ a_i và đi tiếp ra ngoài đa giác. Các tia này chia phần ngoài của (W) thành các miền rời nhau, mỗi miền là một góc có đỉnh tại một a_i .

Gọi A_i là miền gồm tia có đỉnh a_i và phần trong của góc được tạo bởi tia đó với tia $a_i a_{i+1}$. Khi đó

$$p_j \in A_{\alpha(j)}.$$

Ta sẽ tìm tất cả các $\alpha(j)$ đồng thời. Chọn một điểm s nằm trong W , rồi quét góc tất cả các đỉnh của W và các điểm trống: quay ngược chiều kim đồng hồ một tia xuất phát từ s , gọi là *tia quét*, và dừng tại những lúc tia đi qua một đỉnh của W hoặc một điểm trống. Ta duy trì cạnh $a_\ell a_{\ell+1}$ của W đang bị tia quét cắt.

Nếu một đỉnh đa giác và một hay nhiều điểm trống thẳng hàng với (s), xử lý sự kiện ở đỉnh đa giác trước, rồi xử lý các điểm trống theo thứ tự bất kỳ. Khi gặp đỉnh a_i , chỉ cần cập nhật ($\ell = i$).

Khi gặp điểm trống p_j , cạnh đang được tia quét cắt là $a_\ell a_{\ell+1}$; cạnh này cũng cắt phần trong của đoạn sp_j . Các tia có đỉnh

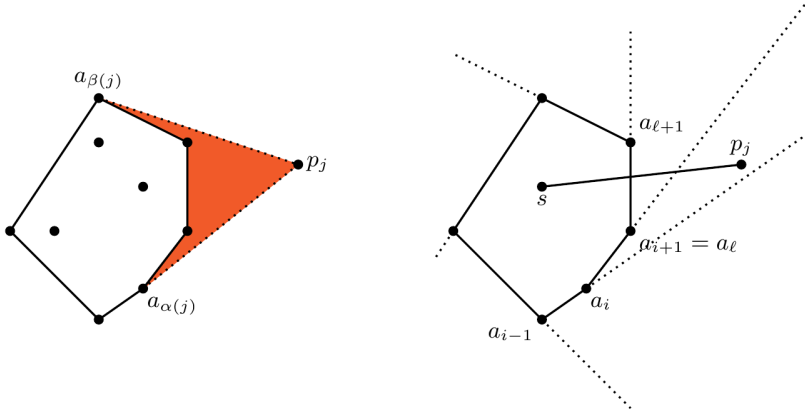
$$a_{\ell+1}, a_{\ell+2}, \dots, a_{\alpha(j)}$$

không cắt phần trong của đoạn sp_j , trong khi các tia có đỉnh

$$a_{\alpha(j)+1}, \dots, a_\ell$$

lại cắt đoạn này. (Các dãy chỉ số được hiểu theo vòng tròn; chẳng hạn với $x > y$, $a_x, \dots, a_y$ nghĩa là $a_x, \dots, a_{k-1}, a_0, \dots, a_y$.) Tính đơn điệu này cho phép tìm đúng chỗ chuyển, tức $\alpha(j)$, bằng tìm kiếm nhị phân. Mỗi lần kiểm tra chỉ cần xét xem tia tại một đỉnh cố định có cắt phần trong của đoạn sp_j hay không, việc này thực hiện bằng các kiểm tra hướng/tích có hướng trong $O(1)$.

Làm hoàn toàn đối xứng, quét theo hướng còn lại để tìm mọi $\beta(j)$.



Hình 18.1: Bên trái: đa giác W và hai điểm tới hạn của điểm trống p_j . Bên phải: điểm p_j nằm trong miền A_i .

18.2.4 Thuật toán và độ phức tạp

1. Dựng bao lồi (W) của các vị trí binh lính, sắp thứ tự các đỉnh ngược chiều kim đồng hồ, và tính mảng tổng tiền tố (d).
2. Với một điểm trống nằm trong hoặc trên biên (W), in diện tích ban đầu.
3. Với các điểm ngoài W , dùng hai lần quét góc như trên để tìm $\alpha(j)$ và $\beta(j)$.
4. Thế các chỉ số vào công thức (2), rồi chia gấp đôi diện tích cho (2) để in đáp án.

Việc dựng bao lồi mất $O(n \log n)$. Khi quét, ta sắp thứ tự góc của tổng cộng $n + m$ điểm, sau đó với mỗi điểm trống thực hiện một tìm kiếm nhị phân trên các đỉnh của bao lồi. Do đó mỗi lần quét tốn

$$O((n + m) \log(n + m)),$$

và cả thuật toán có độ phức tạp thời gian

$$O((n + m) \log(n + m)).$$

Tất cả tọa độ đều nguyên, nên gấp đôi diện tích là số nguyên; diện tích thực chỉ có thể kết thúc bằng .0 hoặc .5. Các tích có hướng và tổng diện tích cần được lưu bằng số nguyên có dấu 64 bit.

Hydra ★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 128 MB

Mã bài: hyd

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/hyd>

19.1 Đề bài

Cậu bé Bytie nhận được một trò chơi điện tử nhân dịp sinh nhật, mang tên *Những cuộc phiêu lưu kỳ thú của hiệp sĩ Byteasar*. Mục tiêu của trò chơi là điều khiển hiệp sĩ vượt qua rất nhiều thử thách, đánh bại kẻ xấu và phù thủy độc ác, rồi giải cứu các cô gái gặp nạn. Bytie đã hoàn thành gần hết các màn chơi. Giờ cậu bị kẹt ở màn cuối, nơi Byteasar phải chiến đấu với một con rắn khổng lồ: **Hydra Bytean**.

Byteasar sẽ dùng kiếm để chiến đấu với quái vật. Trò chơi có hai kiểu vung kiếm: Byteasar có thể **chặt đứt** đầu rắn, hoặc **nghiền nát** đầu rắn (dĩ nhiên đòn thứ hai tốn sức hơn). Chặt đứt đầu thì đơn giản hơn, nhưng khiến các đầu mới mọc lại từ cổ rắn. Hydra chỉ bị đánh bại khi nó không còn đầu nào và không đầu mới nào có thể mọc lại.

Hydra Bytean có n loại đầu, được đánh số từ 1 đến n . Ban đầu con rắn có đúng một đầu loại 1. Một đầu loại i ($1 \leq i \leq n$) được đặc

trưng bởi:

- u_i : số nhất kiểm cần để chặt đứt đầu đó;
- z_i : số nhất kiểm cần để nghiền nát đầu đó;
- một danh sách gồm r_i loại đầu $g_{i,1}, \dots, g_{i,r_i}$, là các đầu mọc thay cho đầu loại i nếu nó bị chặt đứt.

Hãy giúp Bytie tính số nhất kiểm ít nhất cần dùng để đánh bại Hydra.

19.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên n ($1 \leq n \leq 200\,000$), là số loại đầu của Hydra.

n dòng tiếp theo mô tả các loại đầu; dòng thứ i mô tả đầu loại i . Dòng này bắt đầu bằng ba số nguyên u_i, z_i, r_i ($1 \leq u_i < z_i \leq 10^9$, $1 \leq r_i$), tiếp theo là r_i số nguyên $g_{i,1}, \dots, g_{i,r_i}$ ($1 \leq g_{i,j} \leq n$). Tổng tất cả các giá trị r_i không vượt quá 1 000 000.

19.1.2 Dữ liệu ra

In một số nguyên duy nhất: số nhất kiểm tối thiểu cần để hoàn thành trò chơi.

19.1.3 Ví dụ

Với dữ liệu vào:

```
4
4 27 3 2 3 2
3 5 1 2
1 13 2 4 2
5 6 1 2
```

kết quả đúng là:

19.2 Lời giải

Ta sẽ tính nhiều hơn điều đề bài yêu cầu. Với từng loại đầu i , hãy xác định số nhát kiếm ít nhất để giết một Hydra mà đầu duy nhất của nó có loại i ; gọi giá trị đó là $\text{cost}[i]$. Đáp án cần in sẽ là $\text{cost}[1]$.

Để tiêu diệt Hydra có đầu loại i , có hai lựa chọn. Ta hoặc nghiền nát ngay đầu đó với z_i nhát kiếm, hoặc chặt nó với u_i nhát rồi độc lập tiêu diệt tất cả các đầu mọc lên thay thế. Gọi G_i là **danh sách** các đầu này; nếu một loại đầu xuất hiện nhiều lần trong danh sách thì mỗi lần xuất hiện đều phải được tính. Khi đó:

$$\text{cost}[i] = \min \left(z_i, u_i + \sum_{j \in G_i} \text{cost}[j] \right). \quad (1)$$

Vì mọi danh sách G_i đều không rỗng, một thuật toán đệ quy trực tiếp theo (1) chắc chắn sẽ rơi vào vòng lặp. Vấn đề là phải chọn được thứ tự tính các giá trị cost .

19.2.1 Một loại đầu có thể tính ngay

Xét một loại đầu i có giá trị z_i nhỏ nhất. Để giết Hydra bắt đầu với đầu loại i , cuối cùng ta phải nghiền nát ít nhất một đầu nào đó: chỉ chặt đầu thì sẽ luôn có đầu mới mọc ra. Với mọi loại đầu j , ta có $z_j \geq z_i$, nên bất cứ chiến lược nào cũng tốn ít nhất z_i nhát kiếm. Vì vậy, cách tốt nhất là nghiền nát đầu i ngay từ đầu, và

$$\text{cost}[i] = z_i.$$

Thuật toán dưới đây là sự tổng quát hóa của nhận xét này.

19.2.2 Các pha của thuật toán

Thuật toán hoạt động theo nhiều pha. Gọi C là tập các loại đầu mà giá trị cost đã được xác định; ban đầu C rỗng. Trước pha đầu tiên và sau mỗi pha, ta duy trì bất biến

$$G_i \subseteq C \implies i \in C.$$

Nói cách khác, nếu $\text{cost}[i]$ chưa biết ($i \notin C$), thì ít nhất một đầu trong danh sách G_i vẫn chưa có chi phí được xác định.

Ở đầu mỗi pha, chọn một loại đầu $i \notin C$ có z_i nhỏ nhất. Ta khẳng định rằng nghiền nát ngay đầu này là tối ưu, nên có thể đặt

$$\text{cost}[i] = z_i$$

và thêm i vào C .

Thật vậy, giả sử thay vào đó ta chặt đầu i . Do bất biến, trong các đầu mọc ra có một đầu $j_1 \notin C$. Nếu chặt đầu j_1 , lại có một đầu mới $j_2 \notin C$; cứ tiếp tục như vậy. Ta nhận được dãy vô hạn các loại đầu

$$i, j_1, j_2, \dots$$

không loại nào thuộc C . Muốn tiêu diệt Hydra, tại một thời điểm ta bắt buộc phải nghiền nát một đầu thuộc dãy này. Nhưng i có giá trị z nhỏ nhất trong tất cả các loại chưa thuộc C , nên lần nghiền nát ấy tổn ít nhất z_i nhất kiểm. Vì thế, chiến lược nào cũng tổn ít nhất z_i ; nghiền nát đầu i ngay lập tức đạt đúng cận này.

Sau khi thêm i vào C , ta khôi phục bất biến. Chẳng nào còn tìm được một loại đầu $k \notin C$ thỏa $G_k \subseteq C$, tính $\text{cost}[k]$ theo (1) rồi thêm k vào C . Những loại như vậy có thể được xử lý liên tiếp trong cùng pha.

19.2.3 Cài đặt hiệu quả

Để nhanh chóng tìm loại đầu có z_i nhỏ nhất ở đầu mỗi pha, ban đầu hãy sắp xếp các loại đầu theo thứ tự không giảm của z_i . Sau đó duyệt danh sách này theo thứ tự đó và bỏ qua một loại đầu nếu chi phí của nó đã được tính ở một pha trước.

Để thực hiện hiệu quả phần khôi phục bất biến, dựng một đồ thị có hướng gồm n đỉnh tương ứng với các loại đầu. Với mỗi lần xuất hiện $j \in G_i$, thêm một cạnh (j, i) . Cạnh đi vào đỉnh i thể hiện rằng chưa thể dùng công thức (1) để tính $\text{cost}[i]$, vì $\text{cost}[j]$ còn chưa biết.

Khi thêm loại đầu j vào C , xóa đỉnh j và tất cả các cạnh đi ra từ nó. Duy trì bậc vào của từng đỉnh. Mỗi khi bậc vào của một đỉnh i giảm về 0, mọi đầu trong G_i đã thuộc C , nên tính $\text{cost}[i]$ theo (1), thêm i vào C , rồi tiếp tục xóa nó và các cạnh đi ra. Để tính nhanh tổng trong (1), khi một đầu j được thêm vào C , cộng $\text{cost}[j]$ vào tổng đang chờ của mọi đỉnh i có cạnh (j, i) .

Mỗi loại đầu và mỗi cạnh chỉ bị xử lý hằng số lần. Việc sắp xếp theo z_i tốn $O(n \log n)$ thời gian. Vì vậy, với $R = \sum_i r_i$, tổng độ phức tạp là

$$O(n \log n + R)$$

về thời gian và

$$O(n + R)$$

về bộ nhớ.

Ghi chú. Về bản chất, thuật toán này gợi nhớ một phương pháp sắp xếp tô-pô trên đồ thị có hướng không chu trình, mặc dù đồ thị ở đây hoàn toàn có thể chứa chu trình.

Nghịch thế ★

Tác giả đề: Krzysztof Diks

Tác giả lời giải: Jakub Łącki

Giới hạn bộ nhớ: 128 MB

Mã bài: inw

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/inw>

20.1 Đề bài

Byteasar đã phát hiện một họ mới các đồ thị vô hướng có thể được biểu diễn bằng **nghịch thế**. Gọi

$$V = \{1, 2, \dots, n\}$$

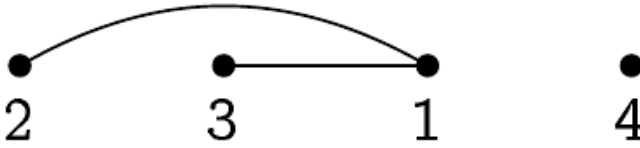
là tập đỉnh của đồ thị, và gọi

$$a_1, a_2, \dots, a_n$$

là một dãy gồm các số đôi một khác nhau thuộc tập V . Hai đỉnh a_i và a_j được nối bởi một cạnh khi cặp (i, j) là một nghịch thế của dãy, nghĩa là

$$i < j \quad \text{và} \quad a_i > a_j.$$

Ví dụ, với $n = 4$ và dãy 2, 3, 1, 4, ta thu được đồ thị sau:



Hình 20.1: Đồ thị nghịch thế của dãy 2, 3, 1, 4.

Các cạnh là $(2, 1)$ và $(3, 1)$; vì vậy hai cạnh này nối các đỉnh mang nhãn tương ứng như hình.

Byteasar muốn kiểm tra xem cách biểu diễn mình nghĩ ra có thật sự hữu ích không. Anh quyết định viết chương trình tìm mọi thành phần liên thông của đồ thị. Nhắc lại rằng hai đỉnh $u, v \in V$ thuộc cùng một thành phần liên thông nếu tồn tại một dãy đỉnh bắt đầu ở u , kết thúc ở v , sao cho mỗi hai đỉnh liên tiếp trong dãy đều được nối bởi một cạnh.

Trong ví dụ trên có hai thành phần liên thông: $\{1, 2, 3\}$ và $\{4\}$.

Hãy giúp Byteasar!

20.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n ($1 \leq n \leq 1\,000\,000$), là số đỉnh của đồ thị.

Dòng thứ hai chứa n số nguyên $a_1, a_2, \dots, a_n$.

20.1.2 Dữ liệu ra

Dòng đầu tiên phải in số thành phần liên thông của đồ thị; gọi số này là m .

Mỗi trong m dòng sau mô tả một thành phần liên thông. Trước hết in số k , là kích thước thành phần; sau đó in dãy tăng dần gồm k số hiệu đỉnh của thành phần ấy.

Các thành phần phải được in theo thứ tự sao cho các số hiệu đỉnh đầu tiên của chúng tạo thành một dãy tăng dần. Nói cách khác, nếu S và S' là hai thành phần liên thông, $u \in S$ và $v \in S'$ lần lượt là các đỉnh có số hiệu nhỏ nhất của chúng, và $u < v$, thì phải in S trước S' .

20.1.3 Ví dụ

Dữ liệu vào

```
4
2 3 1 4
```

Dữ liệu ra

```
2
3 1 2 3
1 4
```

20.2 Lời giải

Bài *Nghịch thể* là một trong những bài dễ nhất của Giải vô địch Ba Lan năm 2012. Dù đa số các đội giải được nó khá dễ dàng, có lẽ chỉ ít thí sinh nhận ra rằng bài có một lời giải cực kỳ đơn giản, có thể cài đặt chỉ trong vài dòng. Những người khác chọn một cách tiếp cận chuẩn nhưng phức tạp hơn; ở đây chúng ta sẽ không bàn đến cách đó.

Ta phải tìm các thành phần liên thông của một đồ thị được mã hóa bởi một hoán vị của các số từ 1 đến n . Bản thân đồ thị có thể có quá nhiều cạnh đến mức không thể lưu biểu diễn tường minh của nó trong giới hạn bộ nhớ. Vì vậy, ta phải thao tác trực tiếp trên biểu diễn đã cho trong đề.

20.2.1 Thành phần chứa a_1

Trước hết, hãy xem thành phần liên thông chứa a_1 trông như thế nào. Nếu $a_1 = 1$, đỉnh 1 tự tạo thành một thành phần một đỉnh: ở

bên phải của a_1 không có phần tử nào nhỏ hơn nó, nên không có cạnh nào đi ra từ đỉnh 1.

Nếu $a_1 > 1$ thì sao? Xét biểu diễn sau với $n = 10$:

4 1 8 2 7 3 5 6 10 9

Ta sẽ đánh dấu những phần tử mà ta biết là thuộc thành phần liên thông của đỉnh a_1 . Vì $a_1 = 4$, các đỉnh 1, 2, 3 đều nối trực tiếp với 4: chúng nhỏ hơn 4 và xuất hiện ở bên phải của 4. Vậy chúng nằm trong cùng thành phần:

[4] [1] 8 [2] 7 [3] 5 6 10 9

Lúc này thành phần chứa 1, 2, 3, 4, tức là mọi đỉnh có nhãn từ 1 đến a_1 . Tuy nhiên, đó không phải là bốn phần tử đầu của dãy: giữa 1 và 2 có đỉnh 8, giữa 2 và 3 có đỉnh 7. Hai đỉnh này cũng thuộc thành phần của a_1 : chúng có nhãn lớn hơn các đỉnh đã thêm trước đó, và ở bên phải chúng có ít nhất một đỉnh đã thuộc thành phần.

[4] [1] [8] [2] [7] [3] 5 6 10 9

Sau khi thêm 7 và 8, thành phần đã chứa đỉnh 8. Vì thế mọi đỉnh ở bên phải có nhãn nhỏ hơn 8 cũng phải thuộc thành phần; trong ví dụ là 5 và 6:

[4] [1] [8] [2] [7] [3] [5] [6] 10 9

Sau bước này, thành phần chứa mọi đỉnh có nhãn từ 1 đến 8, đồng thời chúng chính là tám phần tử đầu của dãy $a_1, \dots, a_{10}$. Để thấy không có cạnh nào từ thành phần này sang đỉnh bên ngoài. Một cạnh như vậy phải kết thúc ở một đỉnh nằm bên phải các đỉnh của thành phần, với nhãn nhỏ hơn 8. Nhưng tất cả các đỉnh có nhãn nhỏ hơn 8 đã thuộc thành phần rồi.

Tổng quát hơn, để xác định thành phần chứa a_1 , ta áp dụng luân phiên hai quy tắc sau:

1. Nếu đỉnh m thuộc thành phần thì các đỉnh $1, \dots, m-1$ cũng thuộc thành phần.

2. Nếu đỉnh a_k thuộc thành phần thì các đỉnh $a_1, \dots, a_{k-1}$ cũng thuộc thành phần.

Việc chứng minh hai tính chất này là một bài tập bổ ích. Có thể trước hết chứng minh quy tắc thứ hai, rồi giả sử thành phần đang xét gồm k phần tử đầu của dãy, từ đó chứng minh quy tắc thứ nhất. Lưu ý rằng các quy tắc này chỉ đúng khi ta đang tìm thành phần chứa đỉnh a_1 .

Khi không quy tắc nào còn làm thành phần liên thông lớn thêm, ta đã biết chính xác nội dung cuối cùng của nó. Thật vậy, quy tắc 2 cho thấy thành phần chứa một số lượng phần tử đầu của dãy; quy tắc 1 cho thấy các phần tử ấy chính là mọi số từ 1 đến một số m . Do đó thành phần chứa đúng m phần tử đầu của dãy $a_1, \dots, a_n$.

20.2.2 Thuật toán

Làm sao khai thác nhận xét trên để có lời giải hiệu quả? m phần tử đầu của dãy chỉ có thể tạo thành một thành phần liên thông nếu tập các phần tử đó là

$$\{1, \dots, m\}.$$

Để kiểm tra điều kiện này, dùng một tính chất rất đơn giản: với bất kỳ dãy gồm m số nguyên dương đôi một khác nhau, các phần tử của nó tạo thành tập $\{1, \dots, m\}$ khi và chỉ khi phần tử lớn nhất bằng m .

Vì vậy ta có thể phát hiện thời điểm m phần tử đầu tiên tạo thành $\{1, \dots, m\}$. Hơn nữa, ngay khi tìm được m nhỏ nhất như vậy, ta thật sự đã tìm được một thành phần liên thông: không có cạnh nào đi ra khỏi nó.

Thuật toán vì thế cực kỳ đơn giản. Duyệt dãy từ trái sang phải và duy trì giá trị lớn nhất `prefix_max` trong tiền tố hiện tại. Vì a là một hoán vị của $1, \dots, n$, tiền tố $a_1, \dots, a_i$ chứa đúng tập $\{1, \dots, i\}$ khi và chỉ khi `prefix_max == i`. Mỗi vị trí i thỏa điều kiện này là điểm kết thúc của một thành phần liên thông.

Nếu thành phần trước kết thúc tại $s - 1$, thành phần mới chứa đúng các nhãn liên tiếp $s, s + 1, \dots, i$. Vì định dạng đầu ra yêu cầu số thành phần trước, ta lưu mỗi thành phần bằng cặp đầu mút (s, i) , rồi in toàn bộ sau khi quét xong.

Một dạng giả mã trực tiếp là:

Pseudocode

```

components = empty list
start = 1
prefix_max = 0

for end = 1..n:
    prefix_max = max(prefix_max, a[end])
    if prefix_max == end:
        components.append((start, end))
        start = end + 1

print components.size
for (first, last) in components:
    size = last - first + 1
    print size followed by first, first+1, ..., last

```

Ta phải lưu các khoảng thành phần trước khi in, vì định dạng đầu ra yêu cầu số thành phần xuất hiện ở dòng đầu. Mỗi cặp $(first, last)$ biểu diễn toàn bộ các nhãn nguyên liên tiếp trong một thành phần, nên không cần lưu riêng từng đỉnh. Các thành phần được lưu và in theo đúng thứ tự yêu cầu, vì thành phần tiếp theo luôn có các nhãn lớn hơn toàn bộ nhãn của những thành phần trước.

Mỗi phần tử đầu vào chỉ được xem một lần, nên thuật toán chạy trong thời gian tuyến tính $O(n)$. Danh sách các khoảng dùng $O(c)$ bộ nhớ, với c là số thành phần liên thông và $c \leq n$. Nếu mảng đầu vào đã nằm trong bộ nhớ, cũng có thể tránh danh sách này bằng hai lượt quét: lượt đầu đếm thành phần, in số đếm, rồi lượt thứ hai in các thành phần.

Đề mai làm ★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 128 MB

Mã bài: jut

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/jut>

21.1 Đề bài

Trì hoãn (từ tiếng Latin *procrastinatio*, nghĩa là “đề dành cho ngày mai”): thói quen liên tục dời những việc quan trọng sang một lúc muộn hơn.

Byteasar có thói quen đề mọi việc đến phút chót. Có thể nói trì hoãn là tên đệm của anh. Tuy vậy, một khi đã hứa làm điều gì thì vẫn có thể tin cậy anh.

Sáng nay, Byteasar lập danh sách gồm n công việc phải hoàn thành trong thời gian tới. Công việc thứ i cần đúng d_i ngày **liên tiếp** để thực hiện và phải hoàn thành trong t_i ngày tới, tính từ hôm nay. Byteasar muốn biết mình có thể không làm gì trong bao lâu trước khi thật sự buộc phải bắt đầu. Hãy viết chương trình giúp anh tính điều đó; Byteasar có thể tự viết, nhưng như vậy trái với bản tính của anh.

21.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên n ($1 \leq n \leq 1\,000\,000$), số công việc Byteasar phải làm.

n dòng sau mô tả các công việc. Dòng thứ i chứa hai số nguyên d_i, t_i ($1 \leq d_i, t_i \leq 10^9$). Bảo đảm rằng Byteasar có thể hoàn thành mọi công việc đúng hạn.

21.1.2 Dữ liệu ra

In một số nguyên k : số ngày lớn nhất Byteasar có thể tránh làm việc. Nói cách khác, chậm nhất vào ngày thứ $k + 1$, anh phải bắt đầu một công việc nào đó để vẫn có thể hoàn thành toàn bộ kế hoạch đúng hạn.

21.1.3 Ví dụ

Dữ liệu vào

```
3
2 8
1 13
3 10
```

Dữ liệu ra

```
5
```

Giải thích. Byteasar nghỉ trong năm ngày đầu. Trong năm ngày tiếp theo, anh làm công việc thứ nhất rồi công việc thứ ba. Sau đó, anh dành một trong ba ngày kế tiếp để làm công việc thứ hai.

21.2 Lời giải

Mỗi công việc i có thời lượng d_i và hạn hoàn thành t_i . Ta biết chắc có một lịch khả thi; mục tiêu duy nhất là bắt đầu làm việc muộn nhất có thể.

Điều kiện “ d_i ngày liên tiếp” thực ra không làm bài toán khó hơn. Nếu một lịch nào đó chia công việc i thành nhiều đoạn, ta có thể ghép các đoạn ấy thành một khối liên tục, rồi dời những công việc được thực hiện xen giữa lên sớm tương ứng. Các hạn hoàn thành không bị vi phạm. Vì thế chỉ cần xét các lịch trong đó mỗi công việc là một khối liên tục.

21.2.1 Thứ tự công việc tối ưu

Các công việc nên được thực hiện theo thứ tự **không giảm của hạn hoàn thành**. Đây là quy tắc *earliest due date*.

Thật vậy, giả sử trong một lịch có hai công việc kề nhau A, B , trong đó $t_A > t_B$: công việc có hạn muộn hơn lại đứng ngay trước công việc có hạn sớm hơn. Hoán đổi hai khối công việc này:

- B hoàn thành sớm hơn trước, nên chắc chắn vẫn đúng hạn;
- A hoàn thành đúng thời điểm mà B từng hoàn thành; vì $t_A > t_B$, A cũng vẫn đúng hạn.

Phép đổi chỗ không làm thời điểm bắt đầu của lịch sớm đi. Lặp lại cho đến khi hết nghịch thế, ta có một lịch tốt không kém, với các hạn đã sắp xếp. Do đó, sau khi sắp xếp, ta giả sử

$$t_1 \leq t_2 \leq \dots \leq t_n.$$

Dưới đây là hai cách nhìn dẫn đến thuật toán.

21.2.2 Cách 1: quét từ đầu và theo dõi thời gian rồi

Hãy lần lượt xét các công việc theo thứ tự trên và luôn bố trí chúng muộn nhất có thể. Sau khi xét $i - 1$ công việc đầu, toàn bộ chúng phải hoàn thành trong t_{i-1} đơn vị thời gian đầu tiên.

Ta chia các khoảng rồi đang có thành hai phần:

- `initial_idle`: thời gian rồi ngay từ đầu, trước khi Byteasar làm bất kỳ công việc nào. Đây là đáp án ta muốn cực đại hóa.

- `gap_idle`: tổng thời gian rỗi nằm giữa các công việc đã xếp.

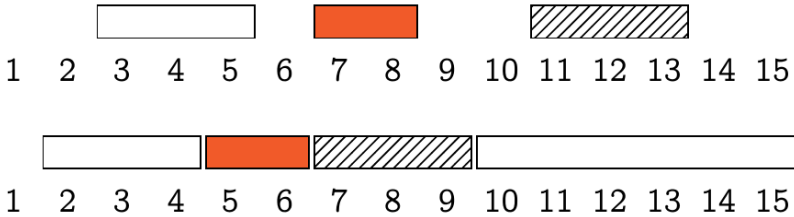
Khởi đầu với công việc đầu tiên:

$$\text{initial_idle} = t_1 - d_1, \quad \text{gap_idle} = 0.$$

Khi thêm công việc i , cửa sổ thời gian được nối từ t_{i-1} đến t_i , tức có thêm $t_i - t_{i-1}$ ngày. Sau khi đặt công việc mới dài d_i , cập nhật

$$\text{gap_idle} += t_i - t_{i-1} - d_i.$$

Nếu `gap_idle` vẫn không âm, khoảng trống mới đủ để chứa công việc, nên chỉ có thêm thời gian rỗi ở giữa các công việc. Nếu nó âm, cần “mượn” thời gian từ phần lịch trước: đẩy các công việc trước đó sang trái, bắt đầu từ công việc gần nhất. Hình dung các công việc như những toa tàu: đẩy toa ngoài cùng bên phải sang trái sẽ kéo dãn những toa trước nó.



Hình 21.1: Bước cuối khi xếp bốn công việc theo hạn hoàn thành; mỗi kiểu ô biểu diễn một công việc.

Những khoảng trống xen kẽ bị dùng hết trước; chỉ khi cần mới ăn vào thời gian rỗi ban đầu. Vì vậy ta chỉ cần làm:

Pseudocode

```
sort jobs by nondecreasing t
initial_idle = t[1] - d[1]
gap_idle = 0

for i = 2..n:
    gap_idle += t[i] - t[i-1] - d[i]
```

```

if gap_idle < 0:
    initial_idle += gap_idle
    gap_idle = 0

return initial_idle

```

Giá trị trả về chính là số ngày nghỉ lớn nhất ở đầu lịch.

21.2.3 Rút gọn thành một công thức

Có thể loại bỏ cả hai biến thời gian rồi. Xét một công việc j mà khi xử lý nó, ta phải giảm `initial_idle`. Khi đó, mọi khoảng trống ở giữa các công việc $1, \dots, j$ đã bị lấp kín; Byteasar làm liên tục từ sau thời gian nghỉ ban đầu đến thời điểm t_j . Do đó lúc này

$$\text{initial_idle} = t_j - (d_1 + d_2 + \dots + d_j). \quad (1)$$

Không biết trước j là công việc nào, nhưng đáp án không thể lớn hơn bất kỳ giá trị (1) nào: để hoàn thành j công việc đầu trước t_j , cần tổng cộng $d_1 + \dots + d_j$ ngày làm việc. Đồng thời, một trong các giá trị này chính là lần cuối `initial_idle` bị cập nhật. Vậy đáp án là

$$\min_{1 \leq i \leq n} \left(t_i - \sum_{j=1}^i d_j \right).$$

Đây là cách cài đặt ngắn gọn nhất: sau khi sắp xếp theo t_i , cộng dồn các d_i vào `total_duration` và lấy giá trị nhỏ nhất của $t_i - \text{total_duration}$.

21.2.4 Cách 2: đảo chiều thời gian

Cũng có thể lập lịch từ cuối về đầu. Đảo trục thời gian: mỗi hạn hoàn thành t_i biến thành thời điểm bắt đầu sớm nhất

$$t'_i = -t_i.$$

Ta nhận được bài toán tương đương: có các công việc thời lượng d_i , mỗi việc không được bắt đầu trước t'_i ; hãy hoàn thành tất cả sớm nhất có thể. Vì $t_1 \leq \dots \leq t_n$, sau khi đảo dấu ta duyệt các công việc theo thứ tự không giảm của t'_i , tức theo hạn ban đầu từ lớn xuống nhỏ.

Gọi `finish_time` là thời điểm hoàn thành công việc cuối cùng đã xếp trong thời gian đảo. Công thức quy hoạch rất trực tiếp:

$$\text{finish_time} \leftarrow \max(\text{finish_time}, t'_i) + d_i.$$

Ban đầu đặt `finish_time` = $-\infty$. Sau khi xếp hết, đáp án của bài toán gốc là

$$-\text{finish_time}.$$

Cách này tương đương với công thức ở phần trước, nhưng đôi khi dễ suy ra hơn khi quen với các bài lập lịch có *release time* (thời điểm sớm nhất được phép bắt đầu).

21.2.5 Độ đúng đắn

Ta chứng minh công thức đã rút gọn trả về đáp án.

Gọi

$$S_i = \sum_{j=1}^i d_j$$

là tổng thời lượng i công việc có hạn sớm nhất.

Bổ đề 1. Mọi lịch khả thi có số ngày nghỉ ban đầu k đều thỏa

$$k \leq t_i - S_i$$

với mọi i .

Chứng minh. Nhờ lập luận đổi chỗ ở trên, xét lịch theo thứ tự hạn không giảm. i công việc đầu cần đúng S_i ngày làm việc và tất cả phải xong không muộn hơn ngày t_i . Nếu đã nghỉ k ngày đầu, chỉ

còn $t_i - k$ ngày trước hạn đó, nên $S_i \leq t_i - k$. Chuyển về thu được bất đẳng thức cần chứng minh. $\square$

Bổ đề 2. Đặt

$$k^* = \min_i (t_i - S_i).$$

Nếu Byteasar nghỉ k^* ngày đầu rồi làm các công việc theo thứ tự hạn không giảm, không chèn thêm khoảng nghỉ, thì lịch hợp lệ.

Chứng minh. Công việc i hoàn thành ở thời điểm $k^* + S_i$. Theo định nghĩa của giá trị nhỏ nhất, $k^* \leq t_i - S_i$; do đó $k^* + S_i \leq t_i$. Điều này đúng với mọi i , nên mọi hạn đều được thỏa. $\square$

Định lý. k^* là số ngày nghỉ ban đầu lớn nhất có thể.

Chứng minh. Bổ đề 1 cho biết không lịch nào nghỉ quá k^* ngày. Bổ đề 2 xây dựng một lịch nghỉ đúng k^* ngày. Vậy k^* là tối ưu. $\square$

21.2.6 Độ phức tạp

Sắp xếp n công việc theo hạn mất $O(n \log n)$. Lần quét cộng dồn sau đó mất $O(n)$. Tổng thời gian là

$$O(n \log n),$$

và bộ nhớ phụ là $O(n)$ để lưu các công việc. Cần dùng số nguyên 64 bit: tổng thời lượng có thể đạt 10^{15} .

Thỏ ★★ ★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 256 MB

Mã bài: kro

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2012/kro>

22.1 Đề bài

Byteasar quyết định sống xanh hơn và trồng xà lách trong vườn. Như bạn có thể hình dung, những chú thỏ Bytean rất thích xà lách, nên chẳng có gì lạ khi chúng lập tức kéo đến khu vườn của Byteasar.

Trong vườn có n luống xà lách, được đánh số từ 1 đến n . Hai luống liên tiếp bất kỳ là kề nhau: với mọi $i = 1, 2, \dots, n - 1$, luống i kề luống $i + 1$; ngoài ra, luống n kề luống 1. Hiện tại có a_i con thỏ đang ở luống i và ăn xà lách của Byteasar.

Byteasar muốn đuổi khỏi vườn được nhiều thỏ nhất có thể. Ông sẽ dùng khẩu súng cũ của mình, trong đó có k viên đạn. Thỏ cực kỳ nhút nhát: mỗi khi Byteasar bắn vào luống i , toàn bộ thỏ tại luống đó sẽ vĩnh viễn chạy khỏi vườn. Hơn nữa, thỏ ở **cả hai luống** kề đều hoảng sợ và di chuyển sang luống kế tiếp ở phía còn lại — tức là luống kề với luống chúng đang ở, nhưng khác luống vừa bị bắn.

Hãy giúp Byteasar tìm số thỏ lớn nhất có thể đuổi khỏi vườn bằng không quá k phát súng.

22.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n và k ($5 \leq n \leq 2\,000$, $1 \leq k \leq n$), lần lượt là số luống xà lách và số viên đạn trong súng.

Dòng thứ hai chứa n số nguyên $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 1\,000\,000$), trong đó a_i là số thỏ ở luống thứ i .

22.1.2 Dữ liệu ra

In một số nguyên: số thỏ lớn nhất có thể đuổi khỏi vườn bằng không quá k phát súng.

22.1.3 Ví dụ

Dữ liệu vào

```
5 2
6 1 5 3 4
```

Dữ liệu ra

```
13
```

Giải thích. Đầu tiên Byteasar đuổi 6 con thỏ khỏi luống 1. Khi đó thỏ từ luống 5 di chuyển sang luống 4, còn thỏ từ luống 2 di chuyển sang luống 3. Sau đó ông bắn vào luống 4 và đuổi được 7 con thỏ.

22.2 Lời giải

Để mô tả ngắn gọn, từ đây ta gọi các luống là **ô**. Trước hết, hãy xác định số phát súng chắc chắn đủ để đuổi hết thỏ. Nếu n chẵn, ta chỉ việc lần lượt bắn vào mọi ô mang số lẻ. Thỏ từ các ô chẵn liên tiếp

sẽ nhảy sang những ô lẻ kế tiếp, rồi bị đuổi đi bởi các phát súng sau đó. Tương tự, khi n lẻ, chỉ cần

$$\left\lceil \frac{n}{2} \right\rceil$$

phát súng.

Quan sát này gợi ý rằng bắn vào mỗi ô cách một ô là một ý tưởng tốt. Ta sẽ chính xác hóa trực giác ấy.

22.2.1 Phân tích trường hợp tổng quát

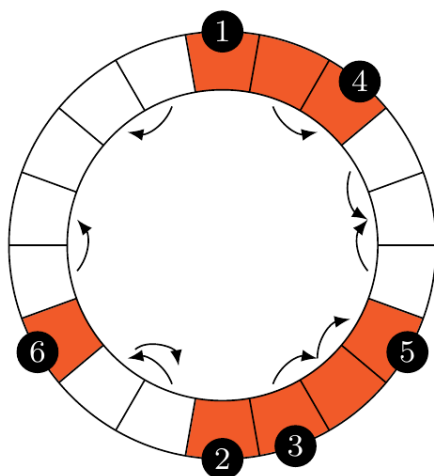
Hãy hình dung một lời giải tối ưu sử dụng đúng k phát súng. Gọi một ô là **sạch** nếu, sau k phát súng đó, các con thỏ ban đầu ở ô này đã bị đuổi đi thành công. Khái niệm này không chỉ bao gồm các ô bị bắn khi vẫn có thỏ: thỏ ban đầu ở một ô có thể đã nhảy sang ô khác rồi chỉ sau đó mới bị đuổi. Chia các ô trên vòng tròn thành các đoạn cực đại gồm toàn ô sạch.

Trong hình có ba đoạn sạch cực đại.

Ta đã xét trường hợp đuổi được tất cả thỏ: khi đó chỉ có một đoạn bao phủ toàn bộ các ô. Phân tích trên cho thấy $\lceil n/2 \rceil$ phát súng là đủ. Số này cũng là cần thiết. Thật vậy, nếu bắn ít hơn, một phép đếm đơn giản cho thấy giữa hai ô liên tiếp bị bắn nào đó phải có một khoảng trống gồm ít nhất hai ô. Khi đó không thể đuổi thỏ từ bất kỳ ô nào trong khoảng trống ấy.

Từ giờ, giả sử các đoạn sạch không phủ hết các ô. Chúng có thể có dạng nào? Chẳng hạn, liệu có thể có hai đoạn sạch chỉ cách nhau đúng một ô không sạch? Không. Giả sử có hai đoạn như thế. Muốn đuổi thỏ ban đầu ở ô ngoài cùng bên phải của đoạn bên trái, ta bắt buộc phải bắn vào chính ô ấy. Tương tự, phải bắn vào ô ngoài cùng bên trái của đoạn bên phải. Chỉ xét hai phát súng này: khi phát thứ nhất trong chúng được bắn, thỏ ở ô nằm giữa hai đoạn sẽ nhảy vào ô bị nhắm bởi phát thứ hai. Vì vậy chúng bị đuổi bởi phát thứ hai, mâu thuẫn với giả thiết ô giữa không sạch. Lập luận tương tự cũng cho thấy không thể có đúng một đoạn sạch dài $n - 1$.

Vì thế, các khoảng cách giữa những đoạn sạch cực đại đủ lớn để phát súng trong một đoạn không ảnh hưởng tới diễn biến ở các



Hình 22.1: Một vòng gồm $n = 18$ ô với $k = 6$ phát súng: vòng tròn đen ghi thứ tự bắn, mũi tên chỉ hướng thả nhảy, và các ô tô màu là ô sạch.

đoạn khác. Ta có thể xét từng đoạn độc lập. Lấy một đoạn gồm m ô, trước tiên giả sử m lẻ. Để đuổi hết thỏ trong đoạn này, khoảng trống giữa hai ô liên tiếp bị bắn không được chứa quá một ô. Do đó, cần và đủ đúng

$$\left\lceil \frac{m}{2} \right\rceil$$

phát súng, bắn vào mỗi ô cách một ô. Điều thú vị là các phát súng khi ấy có thể thực hiện theo bất kỳ thứ tự nào; ta không chứng minh riêng tính chất này.

Nếu m chẵn, cần $m/2 + 1$ phát súng. Nhưng không cần xét trường hợp này: cùng số phát súng đó có thể làm sạch một đoạn dài hơn gồm $m + 1$ ô.

Tóm lại:

- Nếu $k \geq \lceil n/2 \rceil$, ta có thể đuổi hết thỏ.
- Nếu $k < \lceil n/2 \rceil$, ta chỉ cần xét một số đoạn rời nhau từng đôi có độ dài lẻ, và trong mỗi đoạn bắn vào đúng mỗi ô cách một ô.

Vì vậy, trong trường hợp thứ hai, ta có thể giới hạn ở các lời giải thỏa hai tính chất sau:

1. Mọi cặp ô bị bắn cách nhau ít nhất một ô, tức chỉ số của chúng không kề nhau trên vòng tròn.
2. Không phát bắn nào làm thỏ từ một ô nhảy xa hơn ô kề nó.

Do đó, tại mọi thời điểm số thỏ ở một ô chỉ phụ thuộc vào các phát bắn trong phạm vi nhiều nhất hai ô quanh nó. Đây là nền tảng cho quy hoạch động dưới đây.

22.2.2 Quy hoạch động

Các quan sát trên cho phép xây dựng lời giải hiệu quả. Ta duyệt lần lượt các ô và ở mỗi ô xét khả năng bắn vào nó. Có một chi tiết nhỏ: các ô tạo thành vòng tròn, nên các phát bắn ở đầu dãy có thể ảnh

hưởng đến thỏ ở cuối dãy. Ta kiểm soát điều này bằng cách nhớ liệu đã bắn vào một trong hai ô đầu tiên hay chưa.

Trước tiên xét phiên bản trên **đoạn thẳng**, trong đó ô 1 không kề ô n . Trạng thái quy hoạch động được chỉ số hóa bởi:

- số ô đã xét $i \in \{1, \dots, n\}$;
- số phát súng đã dùng $j \in \{0, \dots, k\}$;
- thông tin về các phát bắn ở hai ô gần nhất.

Với mỗi trong $3n(k+1)$ trạng thái, ta lưu số thỏ lớn nhất đã đuổi được. Có ba loại trạng thái:

1. phát bắn cuối cùng là vào ô i ;
2. phát bắn cuối cùng là vào ô $i-1$;
3. phát bắn cuối cùng là vào một ô có số nhỏ hơn $i-1$, hoặc chưa bắn phát nào.

Gọi $t_a[i, j]$, với $a \in \{1, 2, 3\}$, là giá trị tốt nhất của trạng thái loại

a. Với $i > 1$, các công thức chuyển là

$$\begin{aligned} t_1[i, j] &= \max(t_2[i-1, j-1] + a_{i-1} + a_i, t_3[i-1, j-1] + a_i), \\ t_2[i, j] &= t_1[i-1, j], \\ t_3[i, j] &= \max(t_2[i-1, j], t_3[i-1, j]). \end{aligned}$$

Hãy giải thích công thức đầu tiên. Nó mô tả trường hợp bắn vào ô i . Nếu phát bắn trước đó là vào ô $i-2$ — tương ứng trạng thái $t_2[i-1, j-1]$ — thì thỏ từ ô $i-1$ đã chuyển sang ô i . Phát hiện tại vì thế đuổi được $a_{i-1} + a_i$ con. Nếu phát trước đó nhắm vào một ô nhỏ hơn $i-2$, hoặc chưa có phát bắn nào — trạng thái $t_3[i-1, j-1]$ — thì ô i vẫn chỉ có a_i con thỏ, nên ta chỉ đuổi được số đó. Theo các quan sát trước, không cần xét trường hợp phát bắn trước đó là vào ô $i-1$.

Để trở lại vòng tròn, mở rộng bảng thêm một chiều ghi nhận có phát bắn nào vào ô 1 hoặc 2 hay không, rồi xét thông tin này khi tính các trạng thái với $i \in \{n-1, n\}$. Mỗi trạng thái tính được trong thời gian hằng số, nên toàn bộ thuật toán chạy trong

$$O(nk)$$

thời gian.

Đường cao tốc ★★ ★

Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Łącki

Giới hạn bộ nhớ: 128 MB

Mã bài: aut

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/aut>

23.1 Đề bài

Công ty AutoByte đang xây dựng một trong các đường cao tốc của Byteania. Trước đây công ty chỉ thu phí ở đầu đường cao tốc. Tuy nhiên, Byteasar, chủ tịch mới của công ty, nhận ra rằng như thế khoản tiền thu được không phụ thuộc vào số dặm Byte mà khách hàng đã đi. Vì vậy, công ty dự định xây các trạm thu phí dọc toàn bộ đường cao tốc.

Trong một chuyến đi trên đường cao tốc, Byteasar đã dùng công-tơ-mét trên xe để ghi lại vị trí của toàn bộ n lối vào (vị trí một lối vào là khoảng cách từ nó đến đầu đường cao tốc). Công ty quyết định đặt $n + 1$ trạm thu phí cách đều nhau dọc theo đường cao tốc; tức là khoảng cách giữa mọi hai trạm liên tiếp đều bằng nhau. Đồng thời, giữa mỗi hai trạm phải có một lối vào, và giữa mỗi hai lối vào liên tiếp phải có một trạm thu phí. May mắn là cách bố trí các lối

vào hiện có cho phép sắp đặt như vậy.

Hãy tính khoảng cách nhỏ nhất và lớn nhất có thể giữa hai trạm thu phí. Chính xác hơn, cần tìm giá trị nhỏ nhất và lớn nhất của l sao cho tồn tại vị trí b_0 của trạm đầu tiên, và các trạm tiếp theo nằm tại

$$b_0 + l, b_0 + 2l, \dots, b_0 + nl.$$

Có thể một trạm được xác định theo cách trên trùng đúng vị trí của một lối vào. Khi đó, trạm sẽ được đặt sát lối vào, ngay trước hoặc ngay sau nó. Nói cách khác, vị trí của lối vào thứ j phải thuộc đoạn

$$[b_0 + (j - 1)l, b_0 + jl].$$

23.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n ($3 \leq n \leq 1\,000\,000$), là số lối vào đường cao tốc.

Dòng thứ hai chứa dãy tăng gồm n số nguyên $a_1, a_2, \dots, a_n$ ($0 \leq a_i \leq 10^9$). Các số này lần lượt là vị trí của các lối vào.

23.1.2 Dữ liệu ra

In hai số thực: khoảng cách nhỏ nhất và lớn nhất có thể giữa hai trạm thu phí liên tiếp, tính bằng dặm Byte. Có thể giả sử chênh lệch giữa hai giá trị này không nhỏ hơn 10^{-9} .

Kết quả được coi là đúng nếu nằm trong đoạn

$$[x(1 - \varepsilon) - \varepsilon, x(1 + \varepsilon) + \varepsilon],$$

trong đó x là đáp án đúng và $\varepsilon = 10^{-8}$. Vì vậy cả sai số tương đối lẫn sai số tuyệt đối không quá ε đều được chấp nhận.

23.1.3 Ví dụ

Dữ liệu vào

6

2 3 4 5 6 7

Dữ liệu ra

0.833333333333 1.250000000000

23.2 Lời giải

Trong lời giải, ta dùng một diễn giải hình học nên sẽ đổi ký hiệu của đề bài. Gọi x là khoảng cách giữa hai trạm liên tiếp, và y là vị trí của trạm đầu tiên. Khi đó $n + 1$ trạm được đặt ở các vị trí

$$y, y + x, y + 2x, \dots, y + n \cdot x.$$

Để thuận tiện, đánh số các trạm từ 0 đến n ; vì thế “trạm đầu tiên” là trạm số 0.

Một cách bố trí phải thỏa hai điều kiện: giữa mọi hai trạm có một lối vào, và giữa mọi hai lối vào có một trạm. Viết vị trí trạm thứ i là $y + i \cdot x$, với $0 \leq i \leq n$, và vị trí lối vào thứ i là a_i , với $1 \leq i \leq n$. Với $i = 0, \dots, n - 1$, trạm thứ i phải nằm trước lối vào thứ $i + 1$, còn trạm thứ $i + 1$ phải nằm sau lối vào ấy. Hai điều kiện là

$$y + i \cdot x \leq a_{i+1},$$

và

$$y + (i + 1)x \geq a_{i+1}.$$

Chuyển các hạng chứa x sang về phải, với mọi $i = 0, \dots, n - 1$ ta có

$$y \leq a_{i+1} - i \cdot x, \tag{1}$$

$$y \geq a_{i+1} - (i + 1)x. \tag{2}$$

Mọi nghiệm của bài toán thỏa tất cả các bất đẳng thức (1), (2), và ngược lại mọi nghiệm của các bất đẳng thức ấy chính là một bố trí hợp lệ.

23.2.1 Diễn giải hình học

Đồng nhất một bố trí có khoảng cách x và trạm đầu tại y với điểm (x, y) . Các bất đẳng thức (1) nói rằng nghiệm nằm trên hoặc dưới các đường thẳng

$$y = a_{i+1} - i \cdot x,$$

hay trong những nửa mặt phẳng bị các đường ấy chặn ở phía trên. Giao của các nửa mặt phẳng này bị chặn phía trên bởi hàm $f_1 : \mathbb{R} \rightarrow \mathbb{R}$, cho giới hạn trên chặt nhất của y tại mỗi x :

$$f_1(x) = \min_{i=0, \dots, n-1} \{a_{i+1} - i \cdot x\}. \quad (3)$$

Tương tự, các bất đẳng thức (2) xác định giao các nửa mặt phẳng bị chặn phía dưới bởi một hàm f_2 . Hàm f_1 là hàm lõm, còn f_2 là hàm lồi: trực quan, f_1 là bao dưới của các đường thẳng trong (3), còn f_2 là bao trên của các đường thẳng tương ứng với (2).

Tập các điểm biểu diễn nghiệm là các điểm nằm không cao hơn đồ thị f_1 , đồng thời không thấp hơn đồ thị f_2 . Ký hiệu miền nằm giữa hai đồ thị là S . Ta cần tìm trong S các điểm có hoành độ nhỏ nhất và lớn nhất.

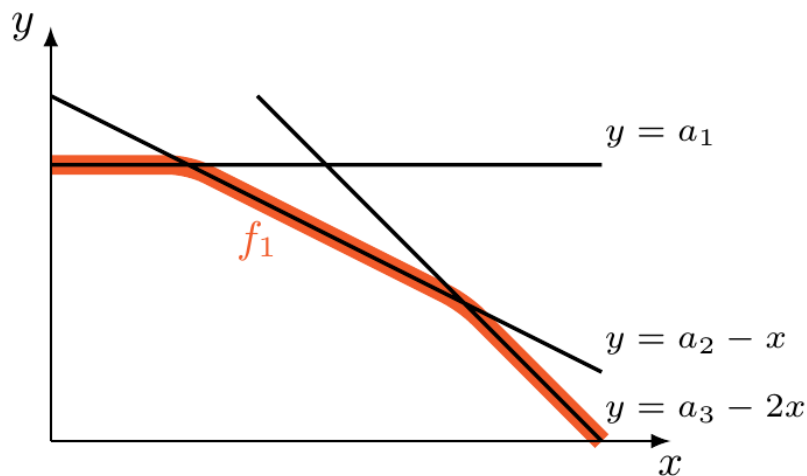
Trước hết, hãy tìm một điểm bất kỳ của S . Ta biết S không rỗng, nên với một $x \in \mathbb{R}$ nào đó phải có

$$f_1(x) \geq f_2(x).$$

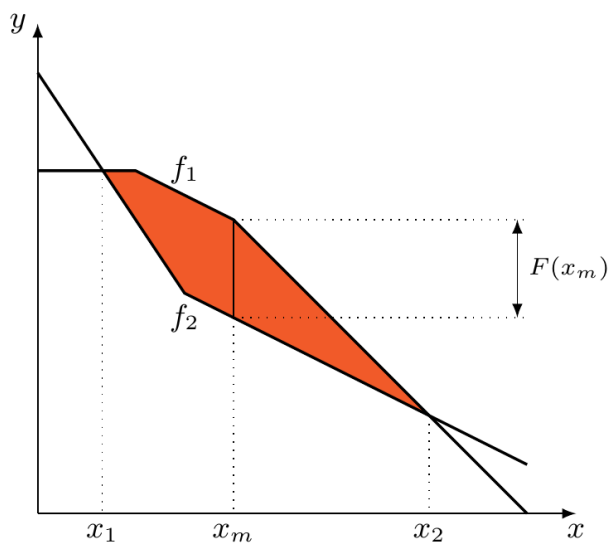
Đặt

$$F(x) := f_1(x) - f_2(x).$$

Vì f_1 lõm và f_2 lồi, F cũng là hàm lõm. Với mỗi x , ta tính được $F(x)$ trong $O(n)$ bằng (3) và công thức tương tự cho f_2 .



Hình 23.1: Hình 1. Ba đường thẳng giới hạn các nửa mặt phẳng của (1); đường gấp khúc màu cam là hàm f_1 , tức bao dưới của chúng.



Hình 23.2: Hình 2. Miền nghiệm S nằm giữa f_1 và f_2 ; x_1, x_2 là hai biên cần tìm, còn x_m cực đại hóa khoảng cách $F(x_m)$.

Điểm then chốt là dùng **tìm kiếm tam phân** để tìm hoành độ x_m làm cực đại $F(x_m)$. Chính xác hơn, sau hữu hạn vòng lặp ta tìm được một x_m^* gần tùy ý với x_m ; càng nhiều vòng lặp thì càng gần. Có thể bắt đầu tìm kiếm trên bất kỳ đoạn nào chứa x_m , chẳng hạn $[0, a_n]$. Với giá trị cực đại đó,

$$F(x_m) = f_1(x_m) - f_2(x_m) \geq 0.$$

Ta biết F không giảm trên $(-\infty, x_m]$ và không tăng trên $[x_m, \infty)$. Cần tìm x_1 nhỏ nhất và x_2 lớn nhất sao cho F không âm. Để tìm x_1 , chỉ cần dùng **tìm kiếm nhị phân** trên $[0, x_m]$ để tìm nghiệm $F(x_1) = 0$. Hoàn toàn tương tự, dùng tìm kiếm nhị phân ở phía bên phải x_m để tìm x_2 .

Độ phức tạp là

$$O(n \log M),$$

trong đó M biểu thị miền giá trị của dữ liệu và độ chính xác cần đạt. Thừa số $\log M$ xuất phát từ các phép tìm kiếm nhị phân và tam phân.

23.2.2 Lời giải ngắn hơn

Ta cũng có thể giải bài bằng một thuật toán đơn giản hơn, dù kém tự nhiên hơn. Với mỗi đáp án cần tìm, ta sẽ chỉ dùng một tìm kiếm nhị phân. Cố định một khoảng cách giữa hai trạm là x . Cần xác định một trong ba trường hợp: x hợp lệ (có thể đặt $n+1$ trạm với khoảng cách đó), x quá nhỏ, hoặc x quá lớn. Nếu kiểm tra này thực hiện trong $O(n)$, toàn bộ bài giải được trong $O(n \log M)$.

Với x cố định, lần lượt xét các trạm và duy trì một đoạn các vị trí có thể của trạm đầu, tức các giá trị y . Mỗi trạm/lối vào mới có thể thu hẹp đoạn ở cả hai phía, vì phải thêm hai bất đẳng thức (1), (2). Nếu sau khi xét toàn bộ các trạm, đoạn giá trị y còn không rỗng, x là hợp lệ.

Ngược lại, tại một thời điểm đoạn này phải rỗng: đầu phải đã đi sang trái đầu trái, hoặc ngược lại. Giả sử điều đó xảy ra khi xét

trạm số i , cách trạm đầu $i \cdot x$. Để trạm nằm trước lối vào số $i + 1$, bắt buộc

$$y \leq a_{i+1} - i \cdot x.$$

Gọi y_1 là giá trị y nhỏ nhất còn cho phép ngay trước khi thêm ràng buộc này. Nếu ngay cả y_1 cũng không thỏa bất đẳng thức trên, ta khẳng định x quá lớn.

Để chứng minh, xét một bố trí hợp lệ của $i - 1$ trạm đầu, vốn tồn tại trước khi đoạn bị rỗng. Dịch toàn bộ bố trí này sang trái hết mức có thể. Khi ấy trạm đầu nằm tại y_1 , và một trạm nào đó sẽ “tì” từ phía trái vào một lối vào ở vị trí a_j . Nếu không có trạm nào tì vào lối vào thì một y nhỏ hơn nữa vẫn hợp lệ, trái với định nghĩa của y_1 . Nếu ngay cả trong bố trí cực trái đó, trạm thứ i đã nằm sau lối vào a_{i+1} , tăng khoảng cách giữa các trạm chắc chắn không thể giúp ích: trạm đang tì vào a_j không thể bị dịch sang trái thêm. Vậy x đã chọn quá lớn.

Lập luận đối xứng cho thấy: nếu với y lớn nhất còn cho phép, ràng buộc dưới trong (2) không thỏa, thì x quá nhỏ.

Như vậy, trong $O(n)$ ta phân loại được giá trị x đang thử là quá nhỏ, quá lớn hay hợp lệ. Hai phép tìm kiếm nhị phân lập tức cho lời giải $O(n \log M)$.

23.2.3 Lời giải nhanh hơn

Điều thú vị là bài toán còn giải được trong thời gian tuyến tính. Bước đầu tiên là dựng hiệu quả các miền phía dưới đồ thị f_1 và phía trên đồ thị f_2 . Chỉ xét f_1 : ta cần dựng nó từ các bất đẳng thức (1). Có thể làm điều này bằng thuật toán tương tự thuật toán dựng bao lồi của một tập điểm.

Thuật toán bao lồi thông thường chạy trong $O(n \log n)$, nhưng thừa số logarit chỉ đến từ việc sắp xếp theo hệ số góc. Ở đây các hệ số góc là các số nguyên liên tiếp, nên có thể thực hiện bước “sắp xếp” đó trong thời gian tuyến tính. Sau khi dựng các miền giới hạn bởi f_1 và f_2 , ta xác định hiệu quả miền S , rồi tìm hai điểm cực biên của nó.

Mặc dù có độ phức tạp tốt hơn, lời giải này không được khuyến nghị để cài đặt: việc xử lý sai số biểu diễn số thực khá rắc rối. Vì thế, nên xem nó chủ yếu như một điều thú vị về mặt lý thuyết.

Bytehattan ★★ ★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 128 MB

Mã bài: baj

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/baj>

24.1 Đề bài

Bytehattan là một hòn đảo thuộc thủ đô của Byteania. Trên đảo thường xuyên diễn ra diễu hành, lễ hội và nhiều sự kiện khác. Chúng nhiều đến mức thường gây ùn tắc đường và ùn tắc giao thông nghiêm trọng. Byteasar làm việc tại tòa thị chính và được giao nhiệm vụ theo dõi giao thông trong thành phố.

Các con phố ở Bytehattan tạo thành một lưới đều kích thước $n \times n$. Hãy xem bản đồ Bytehattan như một hệ tọa độ: với mỗi cặp số nguyên x, y thỏa $1 \leq x, y \leq n$, tại điểm (x, y) có một giao lộ. Mọi hai giao lộ cách nhau đúng 1 đơn vị đều được nối bằng một con phố dài 1.

Byteasar nhận được các thông báo đóng đường. Mỗi thông báo nói rằng từ thời điểm đó một con phố sẽ không thể đi qua. Sau khi nhận thông báo về việc đóng một con phố, Byteasar phải xác định liệu vẫn có thể đi lại giữa hai giao lộ ở hai đầu phố đó chỉ

bằng những con phố chưa bị đóng hay không. Hãy giúp anh ấy viết chương trình hỗ trợ công việc này.

24.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n, k ($2 \leq n \leq 1500$, $1 \leq k \leq 2n(n-1)$), lần lượt là kích thước lưới phố của Bytehattan và số thông báo đóng phố.

Mỗi trong k dòng tiếp theo mô tả một lần đóng phố; các thông báo được cho theo thứ tự thời gian. Mỗi dòng gồm hai mô tả phố liên tiếp nhau. Trên thực tế, **chính xác một** trong hai phố ấy sẽ bị đóng. Nếu sau khi đóng con phố được mô tả ở dòng trước đó vẫn có thể đi lại giữa hai giao lộ ở hai đầu của nó, thì phố thứ nhất trong cặp hiện tại bị đóng. Ngược lại, phố thứ hai bị đóng. Lần đóng đầu tiên trong k lần đóng được mô tả luôn áp dụng cho phố thứ nhất của cặp đầu tiên. Mỗi con phố chỉ bị đóng nhiều nhất một lần.

Một mô tả phố gồm hai số nguyên a_i, b_i $1 \leq a_i, b_i \leq n$, sau đó là một chữ cái c_i với $c_i \in \{N, E\}$. Bộ ba này xác định một con phố có một đầu tại giao lộ a_i, b_i . Nếu $c_i = N$, đầu kia là $a_i, b_i + 1$; nếu $c_i = E$, đầu kia là $a_i + 1, b_i$. Khi $c_i = N$, ta luôn có $b_i < n$; tương tự, khi $c_i = E$, ta luôn có $a_i < n$.

Định dạng dữ liệu vào khác thường này cố ý buộc chương trình phải xử lý một lần đóng phố trước khi có thể xử lý lần đóng tiếp theo.

24.1.2 Dữ liệu ra

In đúng k dòng. Nếu sau lần đóng phố thứ i , vẫn có thể đi lại giữa hai giao lộ ở hai đầu của phố vừa đóng, dòng thứ i in TAK ("có" trong tiếng Ba Lan). Ngược lại, in NIE ("không").

24.1.3 Ví dụ

Dữ liệu vào

```

3 4
2 1 E 1 2 N
2 1 N 1 1 N
3 1 N 2 1 N
2 2 N 1 1 N

```

Dữ liệu ra

```

TAK
TAK
NIE
NIE

```

24.2 Lời giải

Trong bài Bytehatten, ta có một đồ thị vô hướng mà các cạnh lần lượt bị xóa. Sau mỗi lần xóa, cần biết hai đỉnh ở hai đầu cạnh vừa xóa có còn nằm trong cùng một thành phần liên thông hay không. Tương đương, ta cần kiểm tra xem việc xóa cạnh có làm tăng số thành phần liên thông hay không.

Bài toán duy trì số thành phần liên thông khá dễ khi **thêm** cạnh: chỉ cần dùng cấu trúc dữ liệu các tập rời nhau, còn gọi là **find-union** hay DSU. Tuy vấn đề của ta có vẻ ngược lại, DSU vẫn là công cụ cần thiết.

Trước hết, tại sao dữ liệu vào lại có định dạng khác thường này? Nếu chỉ cho danh sách các cạnh bị xóa, không những dữ liệu vào mà cả bài toán cũng đơn giản hơn: ta có thể đọc toàn bộ các lần xóa rồi mô phỏng quá trình theo thứ tự ngược. Khi đó các cạnh được **thêm** vào đồ thị; với mỗi cạnh thêm vào, chỉ cần kiểm tra nó có nối hai thành phần khác nhau hay không. DSU sẽ cho thuật toán thời gian tổng

$$O(n^2 + k\alpha(n)),$$

trong đó α là hàm Ackermann ngược. Định dạng ở đây không cho phép mẹo đó, nhưng đây là một kỹ thuật rất hữu ích cần ghi nhớ.

24.2.1 Các mặt của đồ thị phẳng

Chìa khóa là dạng đặc biệt của đồ thị: nó là một **đồ thị phẳng**. Nghĩa là có thể vẽ đồ thị trên mặt phẳng sao cho không có hai cạnh nào cắt nhau. Hãy xét cách vẽ tự nhiên theo lược của đề bài. Ban đầu, hình vẽ có $(n-1) \times (n-1)$ miền hữu hạn kích thước 1×1 , cùng một miền vô hạn ở bên ngoài. Những miền ấy được gọi là các **mặt**.

Ta dùng một sự kiện rất quan trọng về đồ thị phẳng. Với một cách vẽ bất kỳ của đồ thị phẳng, ký hiệu v là số đỉnh, e là số cạnh, f là số mặt và c là số thành phần liên thông. Khi đó công thức Euler cho

$$v - e + f = c + 1.$$

Công thức này là nền tảng của lời giải. Nó cho thấy để xác định số thành phần liên thông của một đồ thị phẳng, chỉ cần biết số đỉnh, số cạnh và số mặt. Trong bài này số đỉnh không thay đổi, luôn bằng n^2 . Số cạnh ban đầu là $2n(n-1)$ và giảm 1 sau mỗi lần xóa. Phần khó hơn là duy trì số mặt.

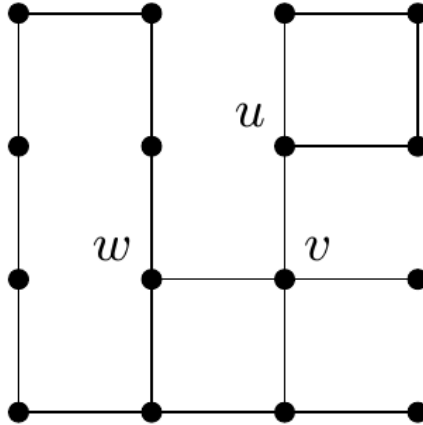
Ban đầu có $(n-1)^2 + 1$ mặt. Khi xóa một cạnh, hãy xét hai phía của cạnh đó thuộc về những mặt nào. Nếu chúng là hai mặt khác nhau thì sau khi xóa cạnh, hai mặt nhập lại thành một mặt.

Mỗi mặt của đồ thị sau một số lần xóa chỉ là hợp của một số mặt của đồ thị ban đầu. Hơn nữa, khi xóa thêm cạnh, các mặt chỉ có thể nhập lại với nhau, không bao giờ tách ra. Vì vậy ta có thể biểu diễn các mặt bằng DSU: mỗi tập DSU là một tập các mặt ban đầu. DSU cho phép kiểm tra hai phía của một cạnh đã thuộc cùng một mặt hay chưa, và nhập hai mặt khác nhau. Mỗi lần xóa cần nhiều nhất hai thao tác DSU, nên thời gian là

$$O(n^2 + k\alpha(n)).$$

Như vậy ta duy trì được số đỉnh, cạnh và mặt. Từ công thức Euler suy ra: nếu việc xóa cạnh **không làm giảm số mặt**, thì số thành phần liên thông tăng lên. Ngược lại, nếu hai mặt khác nhau nhập thành một, thì số thành phần liên thông không đổi. Do đó, sau khi xem hai phía của cạnh bằng DSU, ta lập tức in được đáp án.

Ví dụ, nếu cả hai phía của một cạnh uv là cùng một mặt, xóa uv sẽ làm tăng số thành phần liên thông. Còn nếu cạnh vw ngăn cách hai mặt khác nhau, xóa nó sẽ hợp nhất hai mặt đó và không làm tăng số thành phần liên thông.



Hình 24.1: Hình 1. Hai phía của cạnh uv thuộc cùng một mặt nên xóa nó làm tăng số thành phần liên thông; hai phía của vw thuộc hai mặt khác nhau nên xóa nó chỉ hợp nhất hai mặt.

24.2.2 Có thể làm gì thêm?

Bài toán chỉ hỏi về hai đầu của cạnh vừa xóa. Liệu có thể trả lời câu hỏi tổng quát hơn: sau các lần xóa, có tồn tại đường đi giữa hai đỉnh bất kỳ được chỉ định hay không? Hóa ra một thuật toán rất giống có thể làm được điều đó.

Duy trì mảng `component_id`, trong đó `component_id[v]` là mã của thành phần liên thông chứa đỉnh v . Các mã cụ thể có thể tùy ý, miễn là các đỉnh trong cùng một thành phần có cùng mã, còn các đỉnh ở hai thành phần khác nhau có mã khác nhau.

Sau khi xóa cạnh uw , nếu số thành phần liên thông không tăng thì không cần làm gì. Nếu có tăng, đồ thị tách thành hai thành phần C_u và C_w , lần lượt chứa u và w . Khi ấy, chạy đồng thời hai phép duyệt theo chiều sâu (DFS) bắt đầu tại u và w ; ngay khi một phép duyệt kết thúc, dừng phép còn lại. Bằng cách này ta tìm được thành phần nhỏ hơn trong hai thành phần mới. Không mất tính tổng quát, giả sử

$$|C_u| \leq |C_w|.$$

Khi đó C_u có kích thước không quá một nửa thành phần $C_u \cup C_w$ đã bị tách bởi việc xóa cạnh uw . Để cập nhật `component_id`, chỉ cần gán một mã mới, duy nhất cho mọi $x \in C_u$.

Lần cập nhật này tốn $O(|C_u|)$. Hai DFS chạy song song cũng tốn $O(|C_u|)$, vì chúng kết thúc cùng lúc. Ta dùng sự kiện rằng một thành phần liên thông phẳng có v đỉnh và e cạnh thì

$$e \leq 3v - 6.$$

Vì vậy thời gian của toàn bộ thuật toán tỉ lệ với tổng số lần thay đổi trong mảng `component_id`. Mỗi lần thay đổi `component_id[x]`, kích thước thành phần chứa x giảm ít nhất hai lần. Do đó mã của mỗi đỉnh thay đổi nhiều nhất $O(\log n)$ lần. Lưới có n^2 đỉnh, nên tổng số thay đổi là $O(n^2 \log n)$, và việc duy trì mảng `component_id` cũng chạy trong

$$O(n^2 \log n).$$

Khi đã biết mảng `component_id`, mỗi truy vấn về việc hai đỉnh có liên thông hay không được trả lời trong thời gian hằng số: chỉ cần so sánh hai mã. Nhờ vậy, với chi phí chỉ tăng không nhiều, ta giải được bài toán tổng quát hơn đáng kể so với bài gốc.

Thợ mộc ★★★★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 128 MB

Mã bài: cie

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/cie>

25.1 Đề bài

Byteasar muốn chơi cờ đam, nhưng bàn cờ của cậu đã bị thất lạc đâu đó. Cậu chỉ tìm được một tấm ván gỗ kích thước $n \times m$, được chia thành nm ô vuông bằng nhau. Mỗi ô được sơn trắng hoặc đen, nhưng cách phân bố màu trên tấm ván không nhất thiết tạo thành một bàn cờ đúng chuẩn. Vì vậy Byteasar quyết định tận dụng kỹ năng thợ mộc của mình: dùng cưa cắt ra một bàn cờ, tức một hình vuông gồm một số ô, sao cho mọi hai ô chung cạnh có màu khác nhau.

Không rõ liệu Byteasar có tìm được trên tấm ván một hình vuông có kích thước phù hợp hay không. Do đó, cậu quyết định cắt từ tấm ván **hai mảnh hình tam giác** rồi ghép chúng lại để tạo thành một bàn cờ. (Hai mảnh phải rời nhau, nhưng sau khi cắt có thể xoay chúng tùy ý.) Hãy giúp Byteasar tính kích thước lớn nhất của bàn cờ mà cậu có thể tạo được theo cách này.

Ví dụ trong đề gốc minh họa một tấm ván 4×5 và hai tam giác có thể ghép thành bàn cờ 3×3 .

25.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n và m ($1 \leq n, m \leq 1000$), là kích thước tấm ván. n dòng tiếp theo, mỗi dòng chứa m số nguyên: số thứ j của dòng thứ i ($1 \leq i \leq n, 1 \leq j \leq m$) mô tả màu của ô nằm ở giao của cột j và hàng i . Số 0 là ô trắng, số 1 là ô đen.

25.1.2 Dữ liệu ra

In một số nguyên duy nhất: kích thước lớn nhất của bàn cờ có thể thu được bằng cách cắt hai mảnh tam giác từ tấm ván rồi ghép chúng lại.

25.1.3 Ví dụ

Dữ liệu vào

```
4 5
1 1 0 1 1
0 1 0 1 0
1 0 1 0 0
0 0 1 1 0
```

Dữ liệu ra

```
3
```

Với dữ liệu vào sau:

```
3 3
1 1 1
1 1 0
0 1 0
```

kết quả đúng là:

```
2
```

25.2 Lời giải

Ta có một hình chữ nhật $n \times m$ gồm nm ô, mỗi ô đen hoặc trắng. Cần cắt từ đó hai tam giác để ghép thành một bàn cờ hình vuông có cạnh lớn nhất có thể.

25.2.1 Hình dạng của hai mảnh

Cách duy nhất để chia một hình vuông cạnh k thành hai tam giác là cắt theo một đường chéo. Ta nhận được hai tam giác vuông, mỗi tam giác có hai cạnh góc vuông cùng dài k . Các ô của mỗi tam giác được tô theo mẫu bàn cờ; hơn nữa, các ô nằm tại góc vuông của hai tam giác có cùng màu.

Do đó, chỉ cần cắt những tam giác có hai cạnh góc vuông song song với các cạnh của hình chữ nhật ban đầu. Hai tam giác được chọn phải rời nhau. Hai tam giác kiểu này trên mặt phẳng luôn có thể được ngăn cách bởi một đường thẳng: ngang, dọc, hoặc chéo theo một trong hai hướng. Vì thế, với mỗi đường thẳng như vậy và mỗi trong hai màu, ta muốn biết ở **mỗi phía** của đường thẳng, tam giác bàn cờ lớn nhất có góc vuông mang màu đã chọn là bao nhiêu. Ở đây, *kích thước* tam giác là độ dài hai cạnh góc vuông của nó.

Có $n + 1$ đường ngang, $m + 1$ đường dọc, và $n + m + 1$ đường cho mỗi trong hai hướng chéo. Tổng cộng là

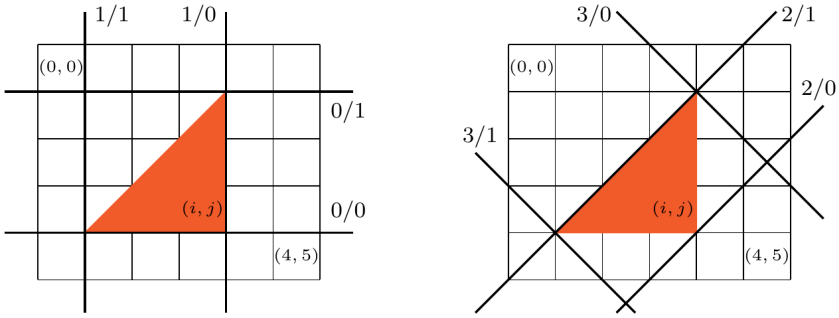
$$3n + 3m + 4$$

đường thẳng. (Để mô tả đơn giản hơn, ta cũng tính các đường trùng với cạnh của hình chữ nhật hoặc đi qua các góc của nó.) Nếu biết kích thước tam giác lớn nhất ở hai phía của mỗi đường, ta có thể xác định đáp án trong $O(n + m)$: với từng đường, ghép tam giác tốt nhất ở phía này với tam giác tốt nhất cùng màu góc vuông ở phía kia, rồi lấy giá trị lớn nhất có thể.

Một tam giác có một trong bốn hướng, tùy theo phía nào của đỉnh góc vuông chứa cạnh huyền. Phần còn lại mô tả thuật toán khi hướng đã cố định. Chạy nó bốn lần, mỗi lần quay toàn bộ hình chữ nhật 90° , sẽ xử lý đủ bốn hướng.

25.2.2 Các tam giác cực đại

Xét một tam giác có ô tại góc vuông mang tọa độ (i, j) ; nói rằng tam giác được *neo* tại ô đó. Cố định hướng sao cho ô neo có cả hai tọa độ lớn nhất trong các ô thuộc tam giác. Gọi tam giác là **cực đại** nếu nó có kích thước lớn nhất trong các tam giác bàn cờ neo tại ô ấy; nói cách khác, không thể mở rộng nó bằng cách đẩy cạnh huyền ra xa hơn.



Hình 25.1: Hình 1. Một hình chữ nhật 5×6 và tam giác kích thước 3 neo tại $(i, j) = (3, 3)$. Hình bên phải chỉ tám đường trong bốn hướng chạm tam giác; nhãn trên mỗi đường cho biết số thứ tự và phía.

Ta tính kích thước của các tam giác cực đại tại mọi ô trong $O(nm)$ bằng quy hoạch động đơn giản. Mọi tam giác như vậy có kích thước ít nhất 1. Tam giác cực đại neo ở (i, j) có kích thước $k \geq 2$ khi và chỉ khi:

- hai ô $(i - 1, j)$ và $(i, j - 1)$ có màu khác màu ô (i, j) ; và
- tam giác cực đại nhỏ hơn trong hai tam giác neo tại $(i - 1, j)$, $(i, j - 1)$ có kích thước $k - 1$.

Tương đương, nếu $d[i][j]$ là kích thước tam giác cực đại neo tại (i, j) , thì

$$d[i][j] = \begin{cases} 1 + \min(d[i - 1][j], d[i][j - 1]), & \text{nếu hai ô nêu trên tồn tại và đều} \\ 1, & \text{ngược lại.} \end{cases}$$

Với mỗi nửa mặt phẳng do một đường xét ở trên tạo ra, trước hết ta sẽ tìm tam giác cực đại lớn nhất nằm trong nửa mặt phẳng ấy và **chạm** đường thẳng. Có hai trường hợp: tam giác chạm đường tại đỉnh góc vuông, hoặc không. Từ đó ta sẽ suy ra tam giác bất kỳ lớn nhất chạm đường. Bước này cần thiết vì tam giác lớn nhất chạm một đường có thể không cực đại: nếu mở rộng nó, một phần tam giác sẽ vượt sang phía bên kia đường. Cuối cùng, ta suy ra tam giác lớn nhất nằm hoàn toàn ở mỗi phía của mọi đường.

25.2.3 Tam giác cực đại chạm các đường

Mỗi tam giác cực đại vừa tìm được chạm đúng tám đường trong tập đường đang xét; ba trong số đó được chạm ở đỉnh góc vuông. Mỗi đường được nhận diện bằng bốn thông tin:

1. hướng của đường;
2. số thứ tự của đường;
3. phía mà tam giác nằm;
4. màu của ô tại góc vuông;

và ta còn cần biết liệu đường có được chạm ở góc vuông hay không.

Đánh số các đường cùng một kiểu như sau. Với đường ngang (hướng 0) và dọc (hướng 1), số của đường là khoảng cách đến cạnh song song của hình chữ nhật nằm cùng phía với tam giác. Với hai hướng chéo (hướng 2, 3), số của đường là **hai lần** khoảng cách đến đỉnh xa nhất của hình chữ nhật nằm cùng phía với tam giác. Cách đánh số này gần như không đổi khi quay hình chữ nhật 90° : các đường hướng 0 đổi với hướng 1 cùng số, và hướng 2 đổi với hướng 3 cùng số. Nhờ vậy, có thể kết hợp kết quả của bốn lần quay một cách dễ dàng.

Xét tam giác cực đại kích thước k , neo tại (i, j) . Bảng dưới liệt kê tám đường mà tam giác chạm. Cột **phía** là phía của đường chứa tam giác, còn cột **chạm tại góc vuông** chỉ ra đường có đi qua đỉnh góc vuông không.

Hướng	Số đường	Phía	Chạm tại góc vuông
0	$i + 1$	0	có
0	$n - i + k - 1$	1	không
1	$j + 1$	0	có
1	$m - j + k - 1$	1	không
2	$i + j + 2$	0	có
2	$n + m - i - j + k - 2$	1	không
3	$n - i + j + k$	0	không
3	$m - j + i + k$	1	không

Vậy ta duyệt mọi tam giác cực đại và cập nhật giá trị lớn nhất cho mỗi tổ hợp của năm thông tin trên (hướng, số đường, phía, màu ở góc vuông, và cờ “chạm tại góc vuông”). Mỗi tam giác được xử lý trong thời gian hằng số, nên pha này tốn $O(nm)$.

25.2.4 Các tam giác còn lại

Giả sử đã có, với từng loại đường, kích thước lớn nhất của một tam giác cực đại chạm đường đó. Một lượt quét là đủ để tính kích thước lớn nhất của **mọi** tam giác chạm từng đường.

Quan sát then chốt: nếu dịch một đường đang chạm tam giác kích thước k về phía tam giác, ta nhận được một đường chạm một tam giác kích thước $k - 1$. Thông tin đường có chạm tại góc vuông hay không là quan trọng, vì trong trường hợp chạm tại góc vuông, tam giác nhỏ hơn có màu ở góc vuông ngược lại. Ta thực hiện phép quét này bằng quy hoạch động, duyệt các đường của một kiểu theo thứ tự **giảm** dần của số đường.

Sau đó, thêm một lượt quét cho phép tính kích thước lớn nhất của mọi tam giác nằm trong từng nửa mặt phẳng, kể cả khi chúng không chạm đường biên. Lần này dùng quan sát hiển nhiên rằng: khi dời một đường ra xa tam giác kích thước k , đường đó trở thành biên của một nửa mặt phẳng vẫn chứa nguyên tam giác kích thước k . Vì vậy ta duyệt các đường của một kiểu theo thứ tự **tăng** dần của số đường.

Cả hai lượt quét, trên tất cả các loại đường và trạng thái màu/phía, mất $O(nm)$.

25.2.5 Tổng kết

Sau bốn hướng, ta có thuật toán thời gian và bộ nhớ

$$O(nm).$$

Về ý tưởng, lời giải không khó: phần chính là quy hoạch động. Tuy nhiên, cài đặt khá thử thách. Cần xét cẩn thận mọi trường hợp, thao tác chính xác với không gian trạng thái nhiều chiều (đối với một tam giác cực đại chạm đường phải lưu năm thông tin: hướng, phía, số đường, màu ô tại góc vuông, và cờ cho biết đường có chạm ô đó), đồng thời chọn cách biểu diễn trạng thái giúp xử lý thuận tiện các trường hợp khác nhau — đặc biệt là cách đánh số đường.

Biểu tình ★★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 128 MB

Mã bài: dem

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/dem>

26.1 Đề bài

Chủ nhật tới, Byteville sẽ tổ chức Ngày Byte thường niên — một trong những lễ hội quan trọng nhất của Byteania. Tuy nhiên, mọi dấu hiệu đều cho thấy lễ kỷ niệm năm nay sẽ không chỉ là một hội hè gia đình yên bình.

Gần đây, người dân Byteville chia rẽ sâu sắc về một vấn đề then chốt. Một số người cho rằng, theo truyền thống, một byte luôn phải bằng tám bit. Nhưng những người ủng hộ tiến bộ lại thích các byte 16 bit có sức chứa lớn hơn nhiều. Những người khác nhìn vấn đề nghiêm khắc hơn hẳn và rất muốn tuyên bố rằng byte luôn chỉ được có 4 bit. Cuối cùng, Byteville còn có những phong trào lật đổ ít quan trọng hơn, mà thành viên của chúng chủ trương rằng số bit trong một byte không nên là lũy thừa của hai, thậm chí không nhất thiết phải là số chẵn! Mỗi hội đoàn đều dự định tổ chức một cuộc biểu

tình vào Chủ nhật để thuyết phục người dân Byteville về quan điểm của mình.

Nhiều người dân Byteville lo rằng số lượng biểu tình như thế có thể làm gián đoạn lễ hội Ngày Byte. Thị trưởng Byteville nhận thấy rằng cấm một số cuộc biểu tình có thể đem lại cho ông sự ủng hộ đáng kể của công chúng. Vì quyết định ấy dễ gây tranh cãi, ông quyết định chỉ hủy nhiều nhất hai cuộc biểu tình. Đồng thời, ông muốn chọn các cuộc biểu tình cần hủy sao cho tổng thời gian có bất kỳ cuộc biểu tình nào diễn ra trong thành phố, sau khi hủy, là ngắn nhất có thể. Hãy giúp thị trưởng xác định ông có thể tạo thêm được bao nhiêu thời gian không có biểu tình trong thành phố.

26.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n ($2 \leq n \leq 500\,000$), là số cuộc biểu tình đã được lên kế hoạch. Mỗi trong n dòng tiếp theo mô tả một cuộc biểu tình: dòng thứ i gồm hai số nguyên a_i, b_i $0 \leq a_i < b_i \leq 10^9$, nghĩa là cuộc biểu tình thứ i bắt đầu sau lúc mặt trời mọc a_i byte-phút và kết thúc sau lúc mặt trời mọc b_i byte-phút.

26.1.2 Dữ liệu ra

In đúng một số nguyên không âm: khoảng thời gian diễn ra biểu tình có thể được rút ngắn nhiều nhất bao nhiêu nếu thị trưởng Byteville hủy nhiều nhất hai cuộc biểu tình.

26.1.3 Ví dụ

Dữ liệu vào

```
5
0 9
1 4
2 5
7 9
6 7
```

Dữ liệu ra

4

Thị trưởng Byteville nên không cấp phép cho cuộc biểu tình thứ nhất và thứ tư.

26.2 Lời giải

Trước hết, hãy xét một nhận xét đơn giản. Nếu trong một khoảng thời gian có ít nhất ba cuộc biểu tình diễn ra, thị trưởng không thể tác động gì đến khoảng ấy: ngay cả sau quyết định của ông, vẫn có ít nhất một cuộc biểu tình đang diễn ra. Vì vậy, ta chỉ quan tâm các khoảng thời gian mà nhiều nhất hai cuộc biểu tình được lên kế hoạch, bởi chỉ tại đó ta mới có thể thay đổi được điều gì đó.

Gọi $t_1(i)$ là tổng thời lượng mà cuộc biểu tình duy nhất đang diễn ra là cuộc i . Gọi $t_2(i, j)$ là thời lượng mà chính xác hai cuộc biểu tình i và j diễn ra đồng thời, không có cuộc nào khác. Khi đó, nếu thị trưởng hủy các cuộc biểu tình i và j , ông sẽ tạo được đúng

$$t_1(i) + t_1(j) + t_2(i, j)$$

thời gian không có biểu tình.

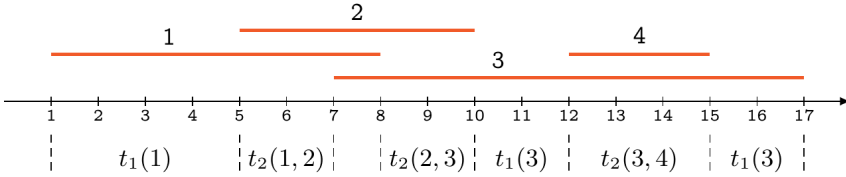
Hình minh họa trong nguồn gốc xét bốn cuộc biểu tình trên các khoảng $[1, 8]$, $[5, 10]$, $[7, 17]$ và $[12, 15]$. Khi đó

$$t_1(1) = 4, \quad t_1(2) = 0, \quad t_1(3) = 4, \quad t_1(4) = 0,$$

và

$$t_2(1, 2) = t_2(2, 3) = 2, \quad t_2(3, 4) = 3.$$

Trong ví dụ này, phương án tối ưu là hủy cuộc biểu tình 1 và 3: đây là hai cuộc có giá trị $t_1(\cdot)$ lớn nhất. Lưu ý rằng $t_2(1, 3) = 0$.



Hình 26.1: Hình 1. Bốn cuộc biểu tình trên trục thời gian và các đoạn đóng góp vào t_1 , t_2 .

26.2.1 Tính các giá trị t_1 và t_2

Ta tạo một mảng gồm $2n$ sự kiện dạng “cuộc biểu tình i bắt đầu/kết thúc tại thời điểm t ” rồi sắp xếp nó không giảm theo thời điểm. Sau đó duyệt các sự kiện đã sắp xếp, tức di chuyển theo trục thời gian từ trái sang phải, đồng thời duy trì tập các cuộc biểu tình đang diễn ra.

Khi gặp sự kiện tiếp theo, nếu hiện tại chỉ có đúng một cuộc biểu tình i đang diễn ra, ta tăng $t_1(i)$ thêm khoảng thời gian đã trôi qua kể từ sự kiện trước. Nếu hiện tại có đúng hai cuộc biểu tình i và j , ta tăng $t_2(i, j)$, và đối xứng cũng tăng $t_2(j, i)$, thêm đúng khoảng thời gian đó.

Để lưu tập các cuộc biểu tình đang diễn ra, chẳng hạn ta dùng cấu trúc set của thư viện STL. Các giá trị $t_1(i)$ có thể đặt trong một mảng n phần tử. Ta không thể làm tương tự với các giá trị $t_2(i, j)$, vì mảng đó sẽ có kích thước n^2 . Tuy vậy, trong quá trình xử lý các sự kiện, ta thực hiện phép tăng một giá trị $t_2(\cdot, \cdot)$ nhiều nhất $4n$ lần. Vì thế, có không quá $4n$ cặp biểu tình i, j thỏa $t_2(i, j) > 0$. Ta có thể lưu các giá trị này bằng cấu trúc map của STL.

Ta sắp xếp $2n$ sự kiện, rồi với mỗi sự kiện thực hiện một số hằng số thao tác trên cấu trúc dữ liệu, mỗi thao tác tốn $O(\log n)$. Do đó, việc xác định toàn bộ $t_1(i)$ và $t_2(i, j)$ tốn thời gian

$$O(n \log n).$$

26.2.2 Bài toán đồ thị

Bài toán có một cách diễn giải đồ thị hữu ích. Hãy tưởng tượng một đồ thị đầy đủ vô hướng: mọi cặp đỉnh đều được nối bởi một cạnh. Các đỉnh tương ứng với các cuộc biểu tình. Ta gán trọng số cho đỉnh và cạnh: đỉnh ứng với cuộc biểu tình i có trọng số $t_1(i)$, còn cạnh nối hai đỉnh i, j có trọng số $t_2(i, j)$. Ta cần tìm một cặp đỉnh sao cho tổng trọng số của chúng và trọng số cạnh nối chúng là lớn nhất. Điều quan trọng là đồ thị này có biểu diễn kích thước $O(n)$, vì chỉ có nhiều nhất $4n$ cạnh mang trọng số dương.

Từ đây, ta chỉ tập trung vào bài toán đồ thị. Với hai đỉnh i, j , ký hiệu tổng trọng số của chúng và trọng số cạnh nối chúng là

$$T(i, j) := t_1(i) + t_1(j) + t_2(i, j).$$

Có rất nhiều cặp đỉnh i, j , nhưng chỉ $4n$ cạnh có trọng số dương, và ta đã xác định các cặp đó ở pha đầu của thuật toán. Vậy ta có thể duyệt tất cả những cạnh này và chọn cạnh ij cực đại hóa $T(i, j)$. Liệu vậy đã xong? Chưa: đây chưa chắc là lời giải tối ưu, vì giá trị lớn nhất của $T(i, j)$ có thể đạt được ở hai đỉnh được nối bởi một cạnh trọng số 0.

Để xét trường hợp này, ta tìm hai đỉnh i_0, j_0 có trọng số lớn nhất, tức cực đại hóa

$$t_1(i_0) + t_1(j_0).$$

Kết quả cuối cùng là phương án tốt hơn trong hai phương án vừa xét.

Cần lưu ý một chi tiết: trong trường hợp thứ hai, ta tìm hai trọng số t_1 lớn nhất, nên cuối cùng có thể nhận được hai đỉnh i_0, j_0 nối với nhau bằng một cạnh trọng số dương. Điều đó có gây vấn đề không? Thực ra không. Nếu chỉ xét những cặp đỉnh i, j được nối bởi cạnh trọng số 0, chắc chắn ta không thể có kết quả tốt hơn, vì khi ấy ta tối đa hóa trên một tập nhỏ hơn. Mặt khác, nếu i_0 và j_0 nối với nhau bằng một cạnh trọng số dương, cặp này cũng đã được xét ở pha đầu của thuật toán. Hơn nữa, lợi ích từ cặp đó là

$$t_1(i_0) + t_1(j_0) + t_2(i_0, j_0),$$

lớn hơn $t_1(i_0) + t_1(j_0)$, vì $t_2(i_0, j_0) > 0$.

Như vậy, ta đã chứng minh tính đúng đắn của thuật toán. Một phân tích đơn giản cho thời gian chạy bị chặn bởi

$$O(n \log n).$$

26.3 Lời cuối

Khi chuẩn bị bài này cho cuộc thi, chúng tôi từng cân nhắc thêm một yêu cầu thứ hai: chọn hai cuộc biểu tình mà sau khi hủy, tổng thời gian có biểu tình trong thành phố là **dài nhất**. Trong diễn giải đồ thị, khi đó ta sẽ tìm cặp đỉnh i, j tối thiểu hóa

$$T(i, j) = t_1(i) + t_1(j) + t_2(i, j).$$

Thoạt nhìn, bài toán có vẻ đối xứng với bài toán đang xét, nhưng thực tế khó hơn một chút. Giống như trước, lời giải sẽ có hai phần: xét $O(n)$ cặp đỉnh được nối bằng cạnh trọng số dương — phần này không thay đổi — rồi chọn trong các cặp còn lại một cặp tối thiểu hóa tổng trọng số t_1 của hai đỉnh.

Tuy nhiên, lần này ở phần thứ hai ta không thể chỉ chọn hai đỉnh có trọng số nhỏ nhất, bởi cạnh trọng số dương có thể nối chúng và khiến chúng không cho kết quả nhỏ nhất. May mắn là có thể tiếp cận phần này theo cách khác và thu được thuật toán chạy trong $O(n \log n)$. Việc hoàn thiện ý tưởng này được để lại cho bạn đọc suy ngẫm.

Kỳ thi ★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 128 MB

Mã bài: egz

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/egz>

27.1 Đề bài

Giáo sư Bajtoni đang chuẩn bị một kỳ thi về Lý thuyết bit và byte. Ông đã chuẩn bị n câu hỏi. Mỗi câu được giáo sư gán một hệ số độ khó dự kiến, là một số tự nhiên trong khoảng từ 1 đến n . Mọi câu hỏi đều có hệ số độ khó khác nhau.

Giờ đây giáo sư đang cân nhắc thứ tự các câu hỏi trong đề thi. Ông muốn kiểm tra xem sinh viên của mình có thể tự đánh giá độ khó của câu hỏi hay không. Vì vậy, ông định sắp các câu hỏi sao cho hệ số độ khó của mọi hai câu liên tiếp chênh nhau ít nhất k . Hãy giúp giáo sư tìm một thứ tự như thế.

27.1.1 Dữ liệu vào

Dòng duy nhất của dữ liệu vào chứa hai số nguyên n, k

$$2 \leq n \leq 1\,000\,000, \quad 1 \leq k \leq n,$$

lần lượt là số câu hỏi giáo sư đã chuẩn bị và cận dưới cho chênh lệch độ khó của hai câu liên tiếp trong đề thi.

27.1.2 Dữ liệu ra

In một dòng chứa thứ tự các hệ số độ khó cần tìm: một dãy gồm n số tự nhiên đôi một khác nhau trong khoảng từ 1 đến n , sao cho mọi hai số liên tiếp chênh nhau ít nhất k .

Nếu có nhiều đáp án đúng, in bất kỳ một đáp án nào. Nếu không tồn tại thứ tự cần tìm, chỉ in từ NIE (nghĩa là “không” trong tiếng Ba Lan).

27.1.3 Ví dụ

Dữ liệu vào

5 2

Một dữ liệu ra đúng có thể là

2 4 1 5 3

Trong khi đó, với dữ liệu vào

5 4

thì dữ liệu ra đúng là

NIE

27.2 Lời giải

Trong bài dễ nhất của kỳ thi năm 2013, ta cần tìm một hoán vị của các số từ 1 đến n , sao cho hiệu của mọi hai số kề nhau có giá trị tuyệt đối ít nhất k ; hoặc kết luận rằng hoán vị đó không tồn tại.

Trước hết, với một giá trị n cố định, không cần chuẩn bị các hoán vị riêng cho mọi giá trị có thể của k . Một hoán vị thỏa yêu cầu với tham số k cũng thỏa yêu cầu với mọi tham số nhỏ hơn. Do đó, chỉ cần tìm giá trị lớn nhất k_n mà tại đó một hoán vị tồn tại, rồi in cùng hoán vị ấy cho mọi $k \leq k_n$. Với $k > k_n$, in NIE.

Hãy thử vài hoán vị cho các giá trị n nhỏ. Ta sẽ nhận ra một cách tạo hoán vị có hiệu giữa các phần tử kề nhau lớn: chia các số từ 1 đến n thành hai dãy gần bằng nhau, một dãy gồm các số nhỏ và dãy kia gồm các số lớn, rồi xen kẽ hai dãy. Như vậy, mỗi cặp số kề nhau luôn gồm một số nhỏ và một số lớn.

Cụ thể, chia các số thành hai dãy

$$1, 2, \dots, \left\lfloor \frac{n+1}{2} \right\rfloor$$

và

$$\left\lfloor \frac{n+1}{2} \right\rfloor + 1, \dots, n,$$

rồi xen kẽ chúng sao cho số

$$\left\lfloor \frac{n+1}{2} \right\rfloor$$

đứng cuối hoán vị.

Ví dụ, khi $n = 6$, ta xen kẽ các dãy 1, 2, 3 và 4, 5, 6, thu được

$$4 \quad 1 \quad 5 \quad 2 \quad 6 \quad 3.$$

Khi $n = 7$, ta thu được

$$1 \quad 5 \quad 2 \quad 6 \quad 3 \quad 7 \quad 4.$$

Tổng quát hơn, nếu n chẵn thì bắt đầu dãy kết quả bằng số nhỏ nhất trong các số lớn; nếu n lẻ thì bắt đầu bằng 1. Để thấy hiệu tuyệt đối của mọi hai phần tử kề nhau trong hoán vị này bằng

$$\left\lfloor \frac{n}{2} \right\rfloor$$

hoặc

$$\left\lfloor \frac{n}{2} \right\rfloor + 1.$$

Không thể làm tốt hơn; vì thế ta có thể chọn

$$k_n = \left\lfloor \frac{n}{2} \right\rfloor.$$

Để chứng minh điều này, xét số

$$\left\lfloor \frac{n+1}{2} \right\rfloor.$$

Ở một vị trí nào đó trong hoán vị, ta bắt buộc phải in số này. Ngay cạnh nó có một số khác trong khoảng từ 1 đến n . Hiệu giữa hai số ấy không thể vượt quá

$$\left\lfloor \frac{n}{2} \right\rfloor.$$

Quả thật, trong trường hợp thuận lợi nhất, số đứng cạnh là n , và hiệu đúng bằng

$$\left\lfloor \frac{n}{2} \right\rfloor.$$

Vậy không có hoán vị nào thỏa điều kiện với $k > \lfloor n/2 \rfloor$.

27.2.1 Thuật toán

Đặt $m = \lfloor n/2 \rfloor$.

- Nếu $k > m$, in NIE.
- Nếu $k \leq m$ và n chẵn, in

$$m+1, 1, m+2, 2, \dots, 2m, m.$$

- Nếu $k \leq m$ và n lẻ, in

$1, m+2, 2, m+3, \dots, m, 2m+1, m+1.$

Hoán vị được xây dựng có chênh lệch giữa các số kề nhau ít nhất $m \geq k$, nên luôn hợp lệ.

27.2.2 Độ phức tạp

Thời gian chạy bị chi phối bởi việc in n số, do đó là

$$O(n).$$

Ngoài bộ nhớ cần cho đầu ra, thuật toán chỉ dùng $O(1)$ bộ nhớ phụ.

Máy bắn tốc độ ★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Bộ nhớ: 256 MB

Mã bài: fot

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/fot>

28.1 Đề bài

Thị trường Bytetown dự định lắp đặt các máy bắn tốc độ trong thành phố. Bytetown có n giao lộ, được đánh số từ 1 đến n , và $n - 1$ đoạn đường hai chiều. Mỗi đoạn đường nối hai giao lộ. Mạng lưới đường sá cho phép đi từ bất kỳ giao lộ nào đến bất kỳ giao lộ nào khác.

Máy bắn tốc độ chỉ được đặt tại giao lộ, nhiều nhất một máy tại mỗi giao lộ, và thị trường muốn đặt được nhiều máy nhất. Tuy nhiên, để các tài xế Bytean không quá bức bối, ông quy định rằng trên mỗi tuyến đường đi theo các con đường của Bytetown mà không đi qua một giao lộ nào quá một lần, chỉ được có nhiều nhất k máy bắn tốc độ, kể cả các máy ở hai đầu tuyến đường. Hãy xác định các giao lộ cần đặt máy.

28.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n, k ($1 \leq n, k \leq 1\,000\,000$), lần lượt là số giao lộ của Bytetown và số máy bắn tốc độ tối đa có thể xuất hiện trên một tuyến đường.

$n - 1$ dòng tiếp theo mô tả mạng đường sá. Dòng thứ i chứa hai số nguyên a_i, b_i ($1 \leq a_i, b_i \leq n$), nghĩa là có một đoạn đường hai chiều nối giao lộ a_i với giao lộ b_i .

28.1.2 Dữ liệu ra

Dòng đầu in số nguyên m : số máy bắn tốc độ lớn nhất có thể đặt tại Bytetown. Dòng thứ hai in m số nguyên là các số hiệu giao lộ cần đặt máy. Nếu có nhiều đáp án, có thể in bất kỳ đáp án nào.

28.1.3 Ví dụ

Dữ liệu vào

```
5 2
1 3
2 3
3 4
4 5
```

Một dữ liệu ra hợp lệ

```
3
1 2 4
```

28.2 Lời giải

Đổi bài toán sang ngôn ngữ lý thuyết đồ thị: cho một cây n đỉnh, hãy đánh dấu nhiều đỉnh nhất sao cho trên mỗi đường đi đơn có nhiều nhất k đỉnh được đánh dấu.

Ta chỉ cần kiểm tra các đường đi **cực đại**, tức là không thể kéo dài thêm ở cả hai đầu. Thật vậy, mọi đường đi khác đều nằm trong

một đường đi cực đại; nếu đường lớn hơn chứa không quá k đỉnh được đánh dấu thì đường con của nó cũng vậy. Trong một cây, mỗi đường đi cực đại bắt đầu và kết thúc ở lá.

Trên một đường đi đơn lẻ, chọn k đỉnh nào cũng như nhau. Nhưng khi hai đường đi dùng chung một số cạnh, đánh dấu các đỉnh chỉ thuộc một trong hai đường có lợi hơn đánh dấu các đỉnh chung. Nói không chính thức: nên ưu tiên các đỉnh gần lá, vì chúng thuộc ít đường đi hơn các đỉnh ở gần “trung tâm” cây; nhờ đó ta có thể đánh dấu nhiều đỉnh hơn.

28.2.1 Các lớp của cây và thuật toán tham lam

Chia các đỉnh thành các **lớp** bằng quá trình bóc lá:

- Lớp 1 gồm tất cả các lá của cây.
- Lớp 2 gồm các đỉnh trở thành lá sau khi xóa mọi đỉnh lớp 1.
- Tổng quát, lớp i gồm những đỉnh trở thành lá sau khi xóa các lớp $1, 2, \dots, i - 1$.

Nếu k chẵn, đánh dấu mọi đỉnh thuộc $k/2$ lớp đầu tiên. Nếu cây có ít hơn $k/2$ lớp thì hiển nhiên đánh dấu toàn bộ các đỉnh.

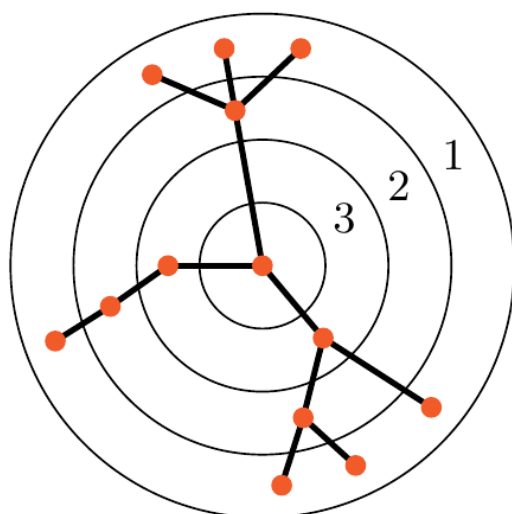
Nếu k lẻ, đánh dấu các đỉnh thuộc $\lfloor k/2 \rfloor$ lớp đầu tiên, rồi đánh dấu thêm **một** đỉnh bất kỳ còn lại, nếu còn đỉnh nào. (Nếu không còn đỉnh nào thì toàn bộ cây đã được đánh dấu.)

Xét một đường đi từ lá đến lá và ghi các số lớp của các đỉnh trên đường đi theo thứ tự. Mỗi đỉnh kề với nhiều nhất một đỉnh thuộc lớp cao hơn; hơn nữa, hai đỉnh cùng lớp không kề nhau. Vì vậy dãy số lớp ban đầu tăng nghiêm ngặt, rồi giảm nghiêm ngặt. Suy ra trên bất kỳ đường đi nào có nhiều nhất hai đỉnh của mỗi lớp.

Do đó:

- nếu k chẵn, $k/2$ lớp đầu chứa trên mỗi đường đi nhiều nhất

$$2 \cdot \frac{k}{2} = k;$$



Hình 28.1: Hình 1. Phân lớp các đỉnh của cây bằng cách bóc dần các lá; các đỉnh màu cam là ví dụ về những đỉnh được đánh dấu.

- nếu k lẻ, $\lfloor k/2 \rfloor$ lớp đầu chứa nhiều nhất

$$2 \left\lfloor \frac{k}{2} \right\rfloor = k - 1$$

đỉnh được đánh dấu trên mỗi đường đi. Ta còn có thể đánh dấu thêm một đỉnh bất kỳ.

Vậy tập đỉnh mà thuật toán chọn luôn hợp lệ.

28.2.2 Cài đặt trong thời gian tuyến tính

Tính bậc của từng đỉnh. Đưa mọi đỉnh hiện là lá vào hàng đợi Q_1 ; khi cây chỉ gồm một đỉnh, coi đỉnh đó là lá để xử lý riêng trường hợp này.

Thực hiện $\lfloor k/2 \rfloor$ pha. Ở pha i , đánh dấu và xóa toàn bộ các đỉnh trong hàng đợi Q_i , tức lớp i . Với mỗi cạnh nối một đỉnh vừa xóa tới đỉnh v chưa xóa, giảm bậc hiện thời của v . Nếu bậc của v trở thành 1, đưa v vào Q_{i+1} . Khi k lẻ, sau các pha đó, lấy một đỉnh chưa bị xóa bất kỳ và đánh dấu nó.

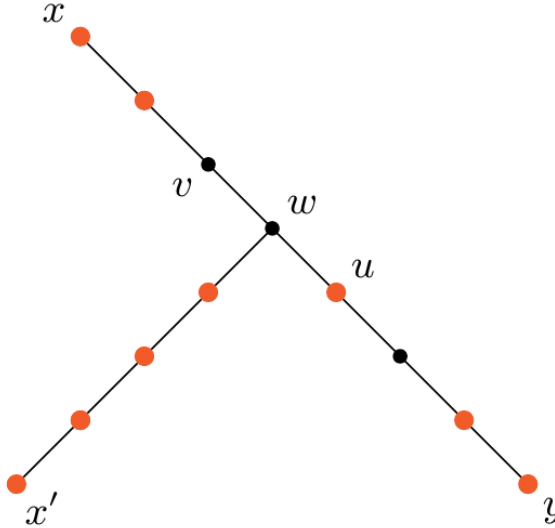
Mỗi đỉnh và mỗi cạnh được xử lý số lần hằng số, nên thời gian chạy là $O(n)$, bộ nhớ cũng là $O(n)$.

28.2.3 Chứng minh tối ưu

Trước hết, giả sử k chẵn và đặt K là tập đỉnh thuộc $k/2$ lớp đầu. Lấy một nghiệm tối ưu bất kỳ. Nếu nó bỏ sót một đỉnh $v \in K$, chọn một đỉnh như vậy có số lớp nhỏ nhất, gọi số lớp là ℓ . Phải có một đỉnh được đánh dấu u ở lớp cao hơn ℓ ; nếu không, nghiệm này có ít đỉnh hơn tập K do thuật toán tạo ra. Chọn u sao cho giữa v và u không có đỉnh nào được đánh dấu.

Ta chứng minh rằng thay việc đánh dấu u bằng đánh dấu v vẫn cho một nghiệm hợp lệ. Ký hiệu P_{ab} là đường đi giữa hai đỉnh a, b . Xét một đường lá-lá $P_{xx'}$. Nếu nó không qua v , hoặc cũng qua u , phép thay thế không làm tăng số đỉnh được đánh dấu trên đường đó. Còn lại, giả sử $P_{xx'}$ qua v nhưng không qua u .

Lấy một đường từ lá x đến lá y đi qua cả v và u . Nếu không tồn tại, có một đường từ x' đến một lá đi qua cả hai; chỉ cần đổi vai trò x, x' . Gọi w là đỉnh chung cuối cùng của $P_{xx'}$ và P_{xy} . Khi đó w nằm trên đường đi từ v tới u .



Hình 28.2: Hình 2. Minh họa phép đổi trong chứng minh với $k = 6$: các đỉnh màu cam được đánh dấu, và các đường đi liên quan gặp nhau tại w .

Hai đường $P_{xx'}$ và $P_{x'y}$ có chung đoạn $P_{x'w}$, không bị ảnh hưởng bởi phép thay thế. Trước thay thế, đoạn P_{wy} có ít nhất $\ell - 1$ đỉnh được đánh dấu ở các lớp thấp hơn ℓ , cộng thêm u , nên có ít nhất ℓ đỉnh được đánh dấu. Sau thay thế, đoạn P_{wx} có đúng ℓ đỉnh được đánh dấu: một đỉnh tại mỗi lớp $1, \dots, \ell - 1$ về phía lá x , cùng với v . Không có đỉnh đánh dấu nào giữa v, u theo cách chọn u .

Vì vậy số đỉnh đánh dấu trên $P_{xx'}$ sau thay thế không vượt số đỉnh đánh dấu trên $P_{x'y}$ trước thay thế. Đường thứ hai có không quá k đỉnh đánh dấu, nên đường thứ nhất cũng vậy. Ta có thể lặp phép thay thế để thu được một nghiệm tối ưu đánh dấu toàn bộ K .

Do thuật toán đánh dấu đúng K , nó tối ưu khi k chẵn.

Khi $k = 2t + 1$ lẻ, lập luận thay thế tương tự cho thấy có một nghiệm tối ưu chứa toàn bộ t lớp đầu. Sau khi bỏ các lớp đó, không thể chọn hai đỉnh còn lại: nếu chọn hai đỉnh như vậy, đường lá-lá đi qua chúng có hai nhánh ngoài, mỗi nhánh đi qua t lớp đã chọn, nên sẽ chứa ít nhất $2t + 2 = k + 1$ đỉnh đánh dấu. Vì thế một nghiệm hợp lệ có nhiều nhất $|K| + 1$ đỉnh. Thuật toán chọn đúng $|K| + 1$ đỉnh nếu còn đỉnh, hoặc chọn toàn bộ cây nếu không còn; do đó nó cũng tối ưu khi k lẻ.

Trò chơi bi ★★★★★

Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 128 MB

Mã bài: gra

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/gra>

29.1 Đề bài

Bajtek và Bitek quyết định chơi bi. Trong một bình có một số chẵn viên bi. Mỗi viên mang đúng một chữ số. Luật chơi rất đơn giản: hai người luân phiên lấy ngẫu nhiên một viên bi ra khỏi bình. Trò chơi kết thúc khi bình hết bi. Người có tích các chữ số trên những viên bi mình nhận được lớn hơn sẽ thắng.

Hai cậu bé rất thích trò chơi này. Cả hai đều đầy tham vọng và thực sự thích chiến thắng, nên kết quả hòa không làm ai hài lòng. Bajtek và Bitek muốn tránh kết cục ấy bằng mọi giá. Hãy viết chương trình kiểm tra xem, với tập bi ban đầu cho trước trong bình, trò chơi **có thể** kết thúc hòa hay không.

29.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên t ($1 \leq t \leq 1000$), là số bộ dữ liệu cần xét.

Mỗi trong t dòng sau chứa mười số nguyên không âm $k_0, \dots, k_9$ $0 \leq k_i \leq 10^{15}$, trong đó k_i là số viên bi mang chữ số i . Trong mỗi bộ dữ liệu, tổng các số k_i là số chẵn dương.

29.1.2 Dữ liệu ra

In t dòng, mỗi dòng là câu trả lời cho một bộ dữ liệu. Nếu trò chơi tương ứng có thể kết thúc hòa, in TAK (nghĩa là “có” trong tiếng Ba Lan); nếu không, in NIE.

29.1.3 Ví dụ

Dữ liệu vào

```
5
0 1 0 1 1 4 1 0 5 1
0 1 1 0 3 0 0 0 0 3
1 1 0 4 0 0 2 0 0 2
100000 100000 100000 100000 100000 100000 100000 100000
↪ 100000 100000
0 99999 99999 100000 100000 100000 100000 100000 100000
↪ 100000
```

Dữ liệu ra

```
TAK
NIE
NIE
TAK
NIE
```


29.2 Lời giải

Trước hết xét các chữ số 0, 5, 7. Chữ số 0 hiển nhiên là trường hợp đặc biệt. Nếu có ít nhất hai viên bi mang 0, đáp án chắc chắn là TAK: mỗi người nhận một viên 0, nên cả hai tích đều bằng 0. Nếu chỉ có đúng một viên 0, đáp án ngay lập tức là NIE. Từ đây, ta giả sử không có viên 0 nào.

Tiếp theo xét 5 và 7. Nếu số bi mang một trong hai chữ số này là lẻ, đáp án là NIE: để hai tích bằng nhau, số thừa số nguyên tố 5, tương ứng 7, phải được chia đều. Nếu cả hai số lượng đều chẵn, mọi bi mang 5 hoặc 7 bắt buộc phải được chia đều cho hai người; sau đó ta không cần quan tâm chúng nữa.

Mọi chữ số còn lại đều là tích của các lũy thừa của 2 và 3. Với

$$i \in \{1, 2, 3, 4, 6, 8, 9\},$$

gọi a_i là số bi chữ số i mà người chơi thứ nhất lấy, và $b_i = k_i - a_i$ là số bi loại đó của người chơi thứ hai. Lời giải dựa trên giả thuyết tự nhiên sau, giúp thu hẹp đáng kể không gian tìm kiếm.

Giả thuyết 1. Nếu tồn tại một cách chia dẫn đến hòa, thì tồn tại một cách chia dẫn đến hòa sao cho với mọi i ,

$$|a_i - b_i| \leq 6. \quad (1)$$

Tạm chấp nhận giả thuyết này. Khi một chữ số i xuất hiện quá 6 lần, ta có thể bỏ đi hai viên bi mang i . Lặp lại thao tác ấy, nhờ Giả thuyết 1 ta thu được một bài toán tương đương mà mỗi chữ số xuất hiện nhiều nhất 6 lần. Khi đó chỉ cần duyệt mọi cách chia thỏa (1), rồi kiểm tra ba điều kiện hòa:

$$\begin{cases} a_1 + a_2 + a_3 + a_4 + a_6 + a_8 + a_9 = b_1 + b_2 + b_3 + b_4 + b_6 + b_8 + b_9, \\ a_2 + 2a_4 + a_6 + 3a_8 = b_2 + 2b_4 + b_6 + 3b_8, \\ a_3 + a_6 + 2a_9 = b_3 + b_6 + 2b_9. \end{cases}$$

Điều kiện đầu kiểm tra hai người nhận cùng số viên bi. Hai điều

kiện sau kiểm tra số mũ của các thừa số nguyên tố 2 và 3 trong hai tích là bằng nhau.

Ta dùng quay lui (*backtracking*) để duyệt các cách chia thỏa (1). Với mỗi i , có nhiều nhất 7 lựa chọn cho a_i (và khi đó b_i được xác định), do đó số khả năng nhiều nhất là

$$7^7 \approx 800\,000.$$

Có thể thêm một tối ưu đơn giản nhưng quan trọng: trước khi quay lui, kiểm tra xem tổng số mũ của 2 và của 3 trên toàn bộ b_i có chẵn không. Cụ thể, nếu một trong hai số

$$k_2 + 2k_4 + k_6 + 3k_8, \quad k_3 + k_6 + 2k_9$$

lẻ thì trả lời NIE ngay.

Phần còn lại chứng minh Giả thuyết 1, bằng một chứng minh có hỗ trợ máy tính.

29.3 Chứng minh tính đúng đắn

Ta đổi cách mô tả một cách chia. Thay vì hai dãy a_i, b_i , dùng dãy số nguyên

$$x_i = a_i - b_i.$$

Ta gọi bộ dữ liệu

$$(k_1, k_2, k_3, k_4, k_6, k_8, k_9)$$

là một **cấu hình**; còn bộ

$$(x_1, x_2, x_3, x_4, x_6, x_8, x_9)$$

biểu diễn một cách chia tiềm năng là một **hệ**.

Một hệ là **hệ hòa** nếu

$$\begin{cases} x_1 + x_2 + x_3 + x_4 + x_6 + x_8 + x_9 = 0, \\ x_2 + 2x_4 + x_6 + 3x_8 = 0, \\ x_3 + x_6 + 2x_9 = 0. \end{cases}$$

Nói cấu hình k_i **chấp nhận** hệ x_i nếu, với mọi i ,

$$|x_i| \leq k_i \quad \text{và} \quad x_i \equiv k_i \pmod{2}.$$

Gọi một hệ là **bị chặn bởi 6** nếu $|x_i| \leq 6$ với mọi i . Toàn bộ chứng minh quy về mệnh đề sau.

Định lý 1. Với mọi cấu hình $k_1, k_2, k_3, k_4, k_6, k_8, k_9$ có lời giải, tồn tại một hệ hòa bị chặn bởi 6, được cấu hình ấy chấp nhận.

Thật vậy, nếu định lý đúng, với một cấu hình có lời giải ta lấy

$$a_i = \frac{k_i + x_i}{2}, \quad b_i = \frac{k_i - x_i}{2}.$$

Đây là các số nguyên không âm và tạo nên một cách chia hòa thỏa (1). Khi ấy ta cũng gọi hệ x_i là một lời giải của cấu hình k_i .

29.3.1 Giảm về các cấu hình nhỏ

Định lý 1 ở dạng trên chưa thể kiểm chứng hữu hạn bằng máy tính. Ta sẽ giới hạn tập cấu hình cần xét.

Nói cấu hình k_i được **sinh bởi** cấu hình k'_i nếu:

- k_i chấp nhận k'_i ;
- $k'_i \leq 6$ với mọi i ;
- hai cấu hình trùng nhau ở các thành phần nhỏ: nếu $k_i \leq 4$ hoặc $k'_i \leq 4$, thì $k_i = k'_i$.

Vì sao có định nghĩa này? Bắt đầu từ k_i , chừng nào có hơn 6 bi của một loại, bỏ hai bi loại đó. Cấu hình thu được là một k'_i sinh ra k_i . Do đó:

Quan sát 1. Mọi cấu hình k_i đều được sinh bởi một cấu hình nào đó k'_i .

Nếu cấu hình sinh có lời giải, lời giải đó hiển nhiên bị chặn bởi 6. Ta sẽ chỉ cần khảo sát các cấu hình bị chặn bởi 6. Mong muốn của ta là hai cấu hình có quan hệ sinh phải tương đương về việc có lời giải. Ta sẽ thiết lập điều này từng bước.

Gọi cấu hình k_i là **hiển nhiên không thể** nếu thỏa ít nhất một trong các điều kiện sau:

1. $\sum_i k_i$ là lẻ.
2. Tổng số mũ của 2 hoặc của 3 trong toàn bộ bi là lẻ; tức một trong hai số

$$k_2 + 2k_4 + k_6 + 3k_8, \quad k_3 + k_6 + 2k_9$$

là lẻ.

3. k_4 lẻ và $k_2 = k_6 = k_8 = 0$. Khi đó số mũ của 2 chỉ đến từ các viên 4, nên không thể chia đều.
4. k_9 lẻ và $k_3 = k_6 = 0$. Khi đó số mũ của 3 chỉ đến từ các viên 9, nên không thể chia đều.
5. $k_2 + k_4 + k_6 = 1$. Sau khi chia các viên 8, số mũ của 2 của cả hai người đều chia hết cho 3; vì vậy không thể phân đúng một viên bổ sung mang 2, 4 hoặc 6.

Rõ ràng một cấu hình hiển nhiên không thể thì không có lời giải.

Bổ đề 1. Giả sử k_i được sinh bởi k'_i . Nếu k'_i có lời giải, thì lời giải đó cũng là lời giải cho k_i . Nếu k'_i hiển nhiên không thể, thì k_i cũng hiển nhiên không thể.

Chứng minh. Nếu x_i là lời giải của k'_i , thì nó là một hệ hòa được k'_i chấp nhận. Với mọi i , ta có $k_i \geq k'_i$ và k_i, k'_i cùng tính chẵn lẻ. Vì thế x_i cũng là hệ hòa được k_i chấp nhận.

Ngược lại, nếu k'_i thỏa một trong năm điều kiện trên thì k_i cũng thỏa điều kiện ấy. Với (1) và (2), điều này do các thành phần tương ứng cùng chẵn lẻ; với (3)–(5), còn dùng thêm việc hai cấu hình bằng nhau ở các thành phần nhỏ. $\square$

Như vậy, ta đã xác định được kết quả cho các cấu hình có lời giải và các cấu hình hiển nhiên không thể. Nếu mọi cấu hình đều thuộc một trong hai loại này thì chứng minh đã xong. Ta kiểm tra phần còn lại bằng chương trình.

29.3.2 Kiểm chứng các cấu hình bị chặn bởi 6

Trước hết, sinh tất cả các hệ hòa bị chặn bởi 6. Có đúng 4497 hệ như vậy. Với mọi cấu hình bị chặn bởi 6, kiểm tra rằng nó hoặc hiển nhiên không thể, hoặc chấp nhận một trong các hệ đã sinh.

Ở bước thứ hai, ta chỉ cần xét cấu hình có chấp nhận một hệ hay không, nên dấu của các x_i không còn quan trọng. Vì vậy, thay từng thành phần của hệ bằng trị tuyệt đối rồi loại trùng, chỉ còn 2216 hệ khác nhau cần sử dụng. Điều này làm phép kiểm tra nhanh gần gấp đôi.

Khi chạy chương trình, vẫn xuất hiện vài trăm cấu hình khó: chúng không hiển nhiên không thể, nhưng không tìm được lời giải trong các hệ vừa liệt kê. Ta sẽ chứng minh mọi cấu hình được sinh bởi các cấu hình khó này cũng không có lời giải, bằng khử Gauss.

Cố định một cấu hình khó k'_i . Giả sử ngược lại có một hệ hòa x_i được một cấu hình k_i sinh bởi k'_i chấp nhận. Các phương trình của hệ hòa luôn phải đúng. Ngoài ra, với mọi i mà $k'_i \leq 4$, ta có

$$x_i \in \{-k'_i, \dots, k'_i\}, \quad x_i \equiv k'_i \pmod{2}.$$

Ta duyệt mọi tổ hợp khả dĩ cho các x_i bị ràng buộc theo cách này, rồi giải hệ phương trình thực nhận được. Nếu mỗi hệ hoặc vô nghiệm, hoặc chỉ có một nghiệm mà một tọa độ không nguyên hay nguyên nhưng khác tính chẵn lẻ với k'_i tương ứng, thì không cấu hình nào được sinh bởi k'_i có lời giải.

Ví dụ về phép kiểm tra

Xét cấu hình khó

$$(k'_1, k'_2, k'_3, k'_4, k'_6, k'_8, k'_9) = (0, 6, 1, 0, 1, 5, 3).$$

Ta lập các hệ:

$$\begin{cases} x_1 + x_2 + x_3 + x_4 + x_6 + x_8 + x_9 = 0, \\ x_2 + 2x_4 + x_6 + 3x_8 = 0, \\ x_3 + x_6 + 2x_9 = 0, \\ x_1 = 0, \\ x_3 = \pm 1, \\ x_4 = 0, \\ x_6 = \pm 1, \\ x_9 \in \{\pm 1, \pm 3\}. \end{cases}$$

Có tổng cộng $2 \cdot 2 \cdot 4 = 16$ hệ cần xét. Hầu hết vô nghiệm. Từ phương trình thứ ba cùng các giới hạn của x_3, x_6, x_9 , một hệ chỉ có thể có nghiệm khi

$$(x_3, x_6, x_9) = (1, 1, -1)$$

hoặc

$$(x_3, x_6, x_9) = (-1, -1, 1).$$

Trong trường hợp đầu, hai phương trình đầu sau khi thế có dạng

$$\begin{cases} 0 + x_2 + 1 + 0 + 1 + x_8 - 1 = 0, \\ x_2 + 2 \cdot 0 + 1 + 3x_8 = 0, \end{cases}$$

tức là

$$\begin{cases} x_2 + x_8 = -1, \\ x_2 + 3x_8 = -1. \end{cases}$$

Nghiệm duy nhất là $x_2 = -1, x_8 = 0$. Nhưng nó không được cấu hình ban đầu chấp nhận, và cũng không được bất kỳ cấu hình nào sinh bởi nó chấp nhận: x_2 khác tính chẵn lẻ với k'_2 , và x_8 cũng khác tính chẵn lẻ với k'_8 .

Chương trình kiểm tra như vậy xử lý được mọi cấu hình khó, trừ đúng hai cấu hình “cứng đầu” sau:

$$(k'_1, k'_2, k'_3, k'_4, k'_6, k'_8, k'_9) = (5, 0, 0, 5, 6, 0, 6),$$

và

$$(k'_1, k'_2, k'_3, k'_4, k'_6, k'_8, k'_9) = (5, 0, 0, 6, 6, 0, 5).$$

Với mỗi cấu hình này, có một hệ phương trình có vô số nghiệm nên cách kiểm tra trên không kết luận được. Ta xử lý hai trường hợp cuối bằng tay; trường hợp thứ hai hoàn toàn tương tự trường hợp thứ nhất.

Trong mọi cấu hình được sinh bởi cấu hình thứ nhất, số bi mang 4 là lẻ, còn số bi mang 6 và 9 là chẵn. Giả sử một cấu hình như vậy có lời giải $x_1, x_2, x_3, x_4, x_6, x_8, x_9$. Khi đó x_9 chẵn, vì k'_9 chẵn. Từ điều kiện về số mũ của 3, suy ra x_6 phải chia hết cho 4. Khi đó, từ điều kiện về số mũ của 2, x_4 phải chẵn. Điều này mâu thuẫn với việc k'_4 lẻ, vì một hệ được chấp nhận phải có $x_4 \equiv k'_4 \pmod{2}$.

Mâu thuẫn trên xử lý cả hai cấu hình còn lại (trường hợp thứ hai đối xứng tương tự), và hoàn tất chứng minh có hỗ trợ máy tính của Định lý 1. Do Định lý 1 kéo theo Giả thuyết 1, thuật toán đã nêu luôn trả lời đúng. $\square$

29.3.3 Độ phức tạp

Sau các kiểm tra hằng số, phần quay lui xét nhiều nhất $7^7 \approx 800\,000$ trạng thái cho mỗi bộ dữ liệu; mỗi trạng thái kiểm tra trong thời gian hằng số. Vì vậy thời gian là $O(7^7)$ cho một bộ dữ liệu (một hằng số độc lập với k_i), và bộ nhớ phụ là $O(1)$, không kể ngăn xếp quay lui có độ sâu 7.

Người hùng ★★ ★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 256 MB

Mã bài: her

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/her>

30.1 Đề bài

Byteotheus, người hùng Byteania nổi tiếng nhất, lại một lần nữa chiến thắng trở về sau trận chiến. Trong khi thủy thủ đoàn đang chất những chiến lợi phẩm lên tàu, Byteotheus ngồi trong khoang để lên đường về hòn đảo quê hương Bitaca. Đây không phải việc dễ dàng: nhiều vị thần ghen tị với sự mến mộ mà dân chúng dành cho anh và rất muốn làm anh mất mặt. May mắn thay, một số vị thần lại có thiện cảm với anh, đặc biệt là nữ thần Bythena. Chính nàng đã gửi cho Byteotheus một giấc mơ vào đêm qua, cảnh báo anh về những hiểm nguy có thể gặp trên đường.

Trên Biển Byteania có n hòn đảo, được đánh số từ 1 đến n . Hiện tàu của Byteotheus ở đảo 1, còn đích đến Bitaca là đảo n . Một số cặp đảo được nối bởi các hải trình một chiều. Mỗi đảo là điểm xuất phát của nhiều nhất 10 hải trình. Đánh số các hải trình từ 1 đến m ; hải trình thứ i đi từ đảo a_i đến đảo b_i , và mất đúng d_i ngày. Nếu

tàu khởi hành theo hải trình thứ i từ đảo a_i vào lúc bình minh của ngày j , nó đến đảo b_i vào lúc bình minh của ngày $j + d_i$.

Tàu được phép chờ tại bất kỳ đảo nào lâu tùy ý trước khi lên đường tiếp. Tuy nhiên, sau khi đã rời một đảo theo một hải trình, tàu không thể đổi hướng hay đi lâu hơn thời gian của hải trình đó. Byteotheus có thể bắt đầu chuyến đi từ đảo 1 sớm nhất vào lúc bình minh của ngày thứ nhất.

Cảnh báo của Bythema rất cụ thể. Nàng cho Byteotheus một danh sách chính xác gồm p cạm bẫy do các vị thần dựng lên. Mỗi cạm bẫy nằm trên một đảo và hoạt động trong một khoảng thời gian. Chính xác hơn, cạm bẫy thứ i ở đảo w_i , hoạt động từ ngày s_i đến hết ngày k_i . Chúng cực kỳ nguy hiểm: nếu tàu của Byteotheus có mặt ở một đảo đang có cạm bẫy hoạt động, không ai sống sót. May thay, Bitaca không có cạm bẫy; đồng thời trên đảo 1, không có cạm bẫy nào hoạt động trong ngày đầu tiên.

Byteotheus muốn chọn đường về nhà sao cho tránh được mọi cạm bẫy. Hãy xác định số ngày ít nhất cần thiết để về Bitaca an toàn.

30.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n, m

$$2 \leq n \leq 100\,000, \quad 1 \leq m \leq 1\,000\,000,$$

lần lượt là số đảo và số hải trình.

m dòng tiếp theo mô tả các hải trình. Dòng thứ i chứa ba số nguyên a_i, b_i, d_i

$$1 \leq a_i, b_i \leq n, \quad a_i \neq b_i, \quad 1 \leq d_i \leq 10^9,$$

nghĩa là hải trình thứ i đi từ a_i đến b_i , mất d_i ngày. Mọi hải trình đều là một chiều; từ mỗi đảo có nhiều nhất 10 hải trình xuất phát.

Dòng kế tiếp chứa số nguyên p

$$0 \leq p \leq 100\,000,$$

là số cạm bẫy. p dòng sau mô tả các cạm bẫy. Dòng thứ i chứa ba số nguyên w_i, s_i, k_i

$$1 \leq w_i < n, \quad 1 \leq s_i \leq k_i \leq 10^9,$$

nghĩa là cạm bẫy thứ i nằm ở đảo w_i , hoạt động từ ngày s_i đến hết ngày k_i . Nếu $w_i = 1$ thì luôn có $s_i > 1$.

30.1.2 Dữ liệu ra

Nếu không thể lập một hành trình tránh mọi cạm bẫy, in NIE (nghĩa là “không” trong tiếng Ba Lan).

Ngược lại, in một số nguyên d : số ngày nhỏ nhất để hoàn thành chuyến đi. Nói cách khác, tàu đến Bitaca vào lúc bình minh của ngày $d + 1$.

30.1.3 Ví dụ

Dữ liệu vào

```
5 6
1 2 3
1 4 13
2 3 1
2 4 2
3 2 2
4 5 1
5
1 2 4
1 8 8
2 6 7
2 10 11
4 6 7
```

Dữ liệu ra

10

Byteotheus khởi hành từ đảo 1 lúc bình minh ngày thứ nhất và đến đảo 2 vào ngày thứ tư. Anh chờ tại đó một ngày rồi đi đến đảo 3. Đến đảo 3 vào ngày thứ sáu, anh lập tức quay về đảo 2; từ đây, vào ngày thứ tám, anh đi tới đảo 4. Anh đến đảo 4 vào ngày thứ mười và cuối cùng đến Bitaca vào ngày thứ mười một.

30.2 Lời giải

Đây là một biến thể khác của bài toán đường đi ngắn nhất trong đồ thị. Lời giải dựa trên thuật toán Dijkstra, vốn tìm khoảng cách ngắn nhất từ một đỉnh xuất phát trong đồ thị có hướng với trọng số không âm.

30.2.1 Vì sao không thể dùng Dijkstra trực tiếp?

Trong Dijkstra thông thường, với mỗi đỉnh v ta duy trì $d[v]$: độ dài ngắn nhất của một đường đi từ đỉnh xuất phát đến v đã được xây dựng. Ban đầu, $d[\text{start}] = 0$, các giá trị khác là ∞ . Ta chia các đỉnh thành đã xử lý và chưa xử lý. Ở mỗi bước, chọn đỉnh chưa xử lý u có $d[u]$ nhỏ nhất, *relax* mọi cạnh đi ra từ u , rồi đánh dấu u đã xử lý. Relax cạnh có hướng uw nghĩa là thử cải thiện $d[w]$ bằng $d[u]$ cộng trọng số cạnh uw .

Nếu chỉ tạo một đỉnh cho mỗi đảo và một cạnh cho mỗi hải trình, ta có n đỉnh, m cạnh. Nhưng cạm bẫy khiến không phải lúc nào cũng được phép ở một đảo, nên không thể áp dụng nguyên vẹn Dijkstra trên đồ thị này.

Với mỗi đảo, các khoảng thời gian cạm bẫy hoạt động chia thời gian thành những **khoảng được phép**: các khoảng cực đại mà ta được phép hiện diện trên đảo. Sau khi gộp các khoảng cạm bẫy chồng lấn trên từng đảo, số khoảng được phép trên tất cả các đảo là

$$O(n + p).$$

Đây sẽ là các đỉnh của đồ thị mới.

30.2.2 Đỉnh là một khoảng thời gian được phép

Xét một khoảng được phép $v = [a, b]$ tại đảo s . Ta gán cho nó giá trị $d[v]$: thời điểm sớm nhất $t \in [a, b]$ mà ta có thể đến đảo s trong khoảng này. Nếu không thể đến, đặt $d[v] = \infty$.

Điểm quan trọng là: nếu đến được đảo s lúc $t \in [a, b]$, ta cũng có thể ở đó tại mọi thời điểm

$$t' \in [t, b],$$

vì ta chỉ cần chờ trên đảo. Ban đầu, mọi giá trị là ∞ , trừ khoảng được phép ở đảo 1 chứa thời điểm bắt đầu chuyến đi; giá trị của khoảng đó là 0. Mọi khoảng đều bắt đầu ở trạng thái chưa xử lý.

Ta vẫn chạy Dijkstra: ở mỗi bước, chọn khoảng chưa xử lý $v = [a, b]$ có $d[v]$ nhỏ nhất. Giả sử v thuộc đảo s , và

$$t = d[v].$$

Xét một hải trình đi từ s đến đảo r , có độ dài ℓ . Ta có thể khởi hành ở bất cứ thời điểm nào từ t đến b , nên có thể đến đảo r ở bất cứ thời điểm nào trong đoạn

$$[t + \ell, b + \ell]. \quad (1)$$

Vì vậy, cần relax tất cả các khoảng được phép của đảo r có giao khác rỗng với đoạn (1). Nếu các khoảng của mỗi đảo được sắp theo đầu trái (hoặc đầu phải; vì chúng rời nhau nên hai cách là tương đương), ta có thể duyệt chính xác các khoảng liên quan.

Với khoảng giao đầu tiên $[a_1, b_1]$, thời điểm đến sớm nhất là

$$\max(a_1, t + \ell).$$

Các khoảng giao tiếp theo sẽ được xử lý bằng một quan sát mạnh hơn.

30.2.3 Quan sát giúp thuật toán đủ nhanh

Ngay thơ mà nói, trên một đảo có thể có p khoảng được phép, và có thể có rất nhiều lần relax hải trình đi vào đảo ấy. Nếu mỗi lần đều duyệt lại tất cả p khoảng, tổng công việc có thể lên đến bậc np , quá lớn.

Quay lại đoạn đến được $[t + \ell, b + \ell]$. Giả sử nó giao với các khoảng được phép của đảo đích

$$[a_1, b_1], [a_2, b_2], \dots, [a_k, b_k],$$

trong đó

$$a_1 \leq b_1 < a_2 \leq b_2 < \dots < a_k \leq b_k.$$

Với mọi khoảng từ khoảng thứ hai trở đi, ta đến được ngay lúc nó bắt đầu:

$$d[a_i, b_i] \leq a_i \quad (i = 2, \dots, k).$$

Nhưng không thể đến một khoảng thời gian được phép sớm hơn đầu trái của chính nó. Do đó, sau khi gán giá trị a_i , khoảng này đã nhận khoảng cách tối ưu vĩnh viễn và **không bao giờ cần được xét lại khi relax các cạnh vào đảo đó**. Ta loại nó khỏi tập các khoảng còn có thể được relax. Chỉ khoảng giao đầu tiên có thể còn cần được xét trong các lần relax sau.

Đó là lý do tổng số lần duyệt các khoảng không bị nhân lên theo số hải trình.

30.2.4 Cài đặt và phân tích độ phức tạp

Đối với mỗi đảo, lưu các khoảng được phép chưa bị “chốt” trong một cấu trúc set của STL, sắp theo đầu trái. Một phép relax hải trình gồm ba phần:

1. Tìm khoảng đầu tiên $[a_1, b_1]$ giao với đoạn đến được (1). Việc này mất $O(\log p)$.

2. Relax $[a_1, b_1]$. Thao tác này có thể cần cập nhật hàng đợi ưu tiên của Dijkstra, nên tính là $O(\log(n + p))$.
3. Duyệt các khoảng giao còn lại $[a_2, b_2], \dots, [a_k, b_k]$, gán khoảng cách của chúng bằng đầu trái tương ứng, đưa chúng vào hàng đợi ưu tiên, rồi xóa chúng khỏi set của đảo đích.

Mỗi khoảng được phép, khi đã là khoảng không phải đầu tiên trong một phép relax, chỉ bị xóa như vậy nhiều nhất một lần. Do đó tổng chi phí của phần 3 trên mọi hải trình là

$$O((n + p) \log(n + p)).$$

Một khoảng được phép đã được lấy ra bởi Dijkstra chỉ relax các hải trình từ đảo của nó. Mỗi đảo có không quá 10 hải trình đi ra, nên tổng số phép relax là $O(n + p)$. Vì thế tổng chi phí của hai phần đầu cũng là

$$O((n + p) \log(n + p)).$$

Cuối cùng, Dijkstra thực hiện $O(n + p)$ thao tác lấy phần tử nhỏ nhất từ hàng đợi ưu tiên, mỗi thao tác mất $O(\log(n + p))$. Việc dựng các khoảng được phép ban đầu mất

$$O(n + p \log p),$$

do cần gom và sắp xếp các khoảng chạm bấy. Tổng độ phức tạp của thuật toán là

$$O((n + p) \log(n + p)).$$

Bản cài đặt là phần khó nhất của bài: cần xử lý cẩn thận các đầu mút khoảng, phần tử đã bị xóa khỏi set, và các bản ghi cũ trong hàng đợi ưu tiên. Đây cũng là lý do trong kỳ thi, chỉ có bốn đội giải được bài *Heros*.

Kỹ thuật di truyền ★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 128 MB

Mã bài: inz

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/inz>

31.1 Đề bài

Các nhà cổ sinh vật học Bytean vừa khai quật được vài mẫu hổ phách, bên trong có những con muỗi cổ bị mắc kẹt. Phân tích mẫu vật cho thấy chúng thuộc kỷ Jura, nên có lẽ từng tiếp xúc với các loài bò sát khổng lồ đã thống trị vùng đất Bytean khi ấy. Điều đó gợi ý cho các nhà di truyền học một ý tưởng thú vị: thử khôi phục vật liệu di truyền của **byteoraptor** từ máu muỗi.

Giống như mọi sinh vật Bytean, bộ gen byteoraptor là một dãy gồm một số **byteo-amino acid**. Để đơn giản, loại của mỗi byteo-amino acid được ký hiệu bằng một số tự nhiên. Bộ gen có tính dư thừa: mỗi loại byteo-amino acid được lặp đúng k lần; đặc biệt, độ dài của mọi bộ gen hợp lệ là bội của k . Nói cách khác, nếu chia bộ gen thành các khối liên tiếp, mỗi khối gồm k byteo-amino acid, thì mọi byteo-amino acid trong cùng một khối có cùng loại.

Các nhà di truyền học đã tách được từ máu một con muỗi một dãy nghi vấn gồm n byteo-amino acid. Không may, dãy này có thể không phải một bộ gen hợp lệ: nó có thể đã bị lẫn các byteo-amino acid ngoại lai. Họ muốn kiểm tra giả thuyết ấy bằng cách xóa ít **nhất** số byteo-amino acid khỏi dãy sao cho phần còn lại là một bộ gen hợp lệ. Nếu có nhiều kết quả tốt như nhau, họ quan tâm đến bộ gen đứng sớm nhất theo thứ tự từ điển. Hãy giúp họ thực hiện khám phá này.

Hai dãy khác nhau l_1, l_2 có cùng độ dài được so sánh theo thứ tự từ điển tại vị trí đầu tiên mà chúng khác nhau; dãy có số nhỏ hơn ở vị trí đó đứng trước.

31.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n, k

$$1 \leq n \leq 1\,000\,000, \quad 2 \leq k \leq 1\,000\,000,$$

lần lượt là độ dài dãy đã tách được và mức độ dư thừa của một bộ gen hợp lệ.

Dòng thứ hai chứa n số nguyên

$$g_1, \dots, g_n \quad (1 \leq g_i \leq 1\,000\,000),$$

là các loại byteo-amino acid liên tiếp trong dãy.

31.1.2 Dữ liệu ra

In hai dòng. Dòng đầu là số nguyên m

$$0 \leq m \leq n,$$

là độ dài lớn nhất của một bộ gen hợp lệ có thể nhận được sau khi xóa một số phần tử của dãy đã cho.

Dòng thứ hai in m số mô tả các loại byteo-amino acid liên tiếp trong bộ gen hợp lệ đó. Nếu có nhiều lời giải, in lời giải nhỏ nhất

theo thứ tự từ điển. Nếu $m = 0$, tức không tách được bộ gen không rỗng nào, dòng thứ hai phải để trống.

31.1.3 Ví dụ

Dữ liệu vào

```
16 3
3 2 3 1 3 1 1 2 4 2 1 1 2 2 2 2
```

Dữ liệu ra

```
9
1 1 1 2 2 2 2 2 2
```

31.2 Lời giải

Trong phần này, bỏ qua bối cảnh khảo cổ. Ta có dãy

$$g_1, g_2, \dots, g_n$$

gồm các số nguyên từ 1 đến M , và số k . Một dãy được gọi là **hợp lệ** nếu độ dài của nó chia hết cho k , và khi chia thành các khối liên tiếp gồm k phần tử thì mọi phần tử của từng khối bằng nhau. Ta cần tìm dãy con hợp lệ dài nhất của dãy đầu vào; trong các dãy dài nhất, chọn dãy nhỏ nhất theo thứ tự từ điển.

31.2.1 Tìm độ dài tối ưu

Chỉ tìm độ dài của dãy con cần chọn là khá dễ. Duyệt dãy từ trái sang phải và đếm số lần xuất hiện của mỗi giá trị kể từ sau khối đã chọn gần nhất. Ngay khi đã thấy k lần xuất hiện của một giá trị x , chọn đúng k lần xuất hiện ấy làm một khối

$$x, x, \dots, x,$$

rồi đặt lại mọi bộ đếm về 0 và tiếp tục duyệt phần đuôi còn lại.

Đây là tham lam đúng cho **số khối**: thời điểm có thể hoàn thành khối đầu tiên được chọn là sớm nhất có thể. Vì vậy phần dãy còn lại là dài nhất có thể trong mọi cách chọn khối đầu tiên; áp dụng lập luận tương tự cho phần đuôi cho ta số khối tối đa. Gọi độ dài kết quả là s .

Tuy nhiên, cách này không nhất thiết cho dãy nhỏ nhất từ điển. Ví dụ với $k = 3$ và dãy

$$4, 4, 1, 1, 4, 1, 7, 7, 2, 7,$$

thuật toán trên lập tức gặp ba số 4, chọn khối $(4, 4, 4)$, rồi chọn $(7, 7, 7)$. Nó tạo dãy

$$4, 4, 4, 7, 7, 7.$$

Nhưng dãy con

$$1, 1, 1, 7, 7, 7$$

cũng có độ dài 6 và nhỏ hơn theo thứ tự từ điển. Lỗi là ta quyết định khối đầu tiên khi vừa thấy số 4 thứ ba, trong khi số 1 thứ ba chỉ xuất hiện muộn hơn một chút. Mặt khác, nếu cứ trì hoãn việc chọn khối đầu tiên, ta có thể không còn nhận được độ dài tối ưu.

31.2.2 Biết phần đuôi còn làm được bao nhiêu

Với $i = 1, \dots, n$, đặt d_i là độ dài của dãy con hợp lệ dài nhất có thể lấy từ hậu tố

$$g_i, g_{i+1}, \dots, g_n.$$

Đặt thêm $d_{n+1} = 0$. Hiển nhiên dãy

$$d_1, d_2, \dots, d_n$$

không tăng. Các giá trị này có thể tính trong thời gian tuyến tính: áp dụng đúng thuật toán tham lam tìm độ dài ở trên cho dãy đảo ngược

$$g_n, g_{n-1}, \dots, g_1.$$

Sau khi đã biết $s = d_1$, xét cách dựng khối đầu tiên của kết quả. Giả sử phần tử cuối cùng của khối này nằm không muộn hơn vị trí j . Để toàn bộ kết quả vẫn có độ dài s , trong phần còn lại phải lấy được dãy hợp lệ dài $s - k$. Do đó cần có

$$d_{j+1} = s - k. \quad (1)$$

Lấy **chỉ số lớn nhất** j thỏa (1). Khi đó khối đầu tiên có thể được tạo từ k lần xuất hiện của cùng một giá trị trong đoạn

$$g_1, \dots, g_j.$$

Ngược lại, khối đầu tiên của bất cứ nghiệm tối ưu nào không thể dùng g_{j+1} hoặc phần tử ở sau nó: nếu làm vậy, sau khối đầu tiên không còn đủ độ dài $s - k$ cho phần đuôi. Vì thế, để tối thiểu hóa từ điển, chỉ việc chọn **giá trị nhỏ nhất xuất hiện ít nhất k lần** trong $g_1, \dots, g_j$, rồi lấy k lần xuất hiện đầu tiên của giá trị đó.

Sau khi đã chọn xong khối đầu, bỏ qua mọi phần tử đến lần xuất hiện thứ k vừa chọn và lặp lại lập luận trên cho phần dãy còn lại. Lúc này độ dài cần còn lại giảm từ s xuống $s - k$, rồi tiếp tục giảm k sau mỗi khối. Nhờ vậy ta vừa luôn giữ được khả năng đạt độ dài tối ưu, vừa chọn nhỏ nhất từ điển ở khối đầu tiên chưa xác định.

Điều này cũng chứng minh tính đúng đắn bằng quy nạp theo số khối còn phải tạo: điều kiện (1) đặc trưng chính xác giới hạn xa nhất mà khối hiện tại được phép vươn tới; trong giới hạn đó, giá trị nhỏ nhất đủ k lần là lựa chọn từ điển tốt nhất. Phần đuôi sau lần xuất hiện thứ k của nó lại là cùng một bài toán con.

31.2.3 Cài đặt dựng dãy

Ta duyệt dãy đầu vào từ vị trí hiện thời và có mảng frequency, trong đó frequency[x] là số lần giá trị x đã xuất hiện trong đoạn đang xét. Khi gặp một vị trí vẫn có thể là phần tử cuối của khối hiện tại — tức vị trí cuối cùng có

$$d_{i+1} = \text{độ dài còn cần} - k,$$

— ta phải biết số nhỏ nhất xuất hiện ít nhất k lần cho tới đó.

Thông tin này có thể được duy trì ngay trong lúc quét. Mỗi khi một giá trị x xuất hiện đúng lần thứ k , đưa x vào tập các giá trị đã đủ k lần và cập nhật giá trị nhỏ nhất của tập. Đồng thời ghi lại vị trí xuất hiện thứ k của x . Tại giới hạn cuối cùng nêu trên, giá trị nhỏ nhất đang có chính là loại tạo khối tiếp theo; vị trí đã ghi của nó cho biết nơi phải bắt đầu lần quét kế tiếp.

Sau khi chốt một khối, các bộ đếm phải trở về 0 để mô tả đoạn mới. Có thể thực hiện bằng cách duyệt lại phần đoạn vừa xử lý và giảm các bộ đếm tương ứng. Cách này không làm thuật toán chậm đi: các đoạn bị duyệt lại khi chốt các khối là rời nhau.

Một cách cài đặt gọn hơn cho thao tác đặt lại này được trình bày ở phần cuối.

31.2.4 Độ phức tạp

Mỗi phần tử được xử lý trong thời gian hằng số. Có vẻ như việc quay lại để xóa các bộ đếm sau mỗi khối có thể khiến thời gian lớn hơn tuyến tính, nhưng không phải vậy. Nếu sau khi xử lý đến g_j ta quay lại ngay sau g_ℓ , thì

$$d_{j+1} = d_{\ell+1}.$$

Nếu lần quay lại tiếp theo diễn ra sau khi đã xử lý $g_{j'}$, với $j' > j$, thì

$$d_{j'+1} < d_{j+1}.$$

Vì vậy các đoạn chỉ số bị duyệt lại ở các lần quay lại là đôi một rời nhau. Tổng số bước của toàn bộ thuật toán là tuyến tính theo n .

Ta dùng một mảng kích thước M để đếm tần suất các giá trị. Do đó thời gian chạy là

$$O(n + M),$$

và bộ nhớ dùng là $O(n + M)$, gồm mảng d , dãy đầu vào và các mảng phục vụ đếm.

31.3 Mẹo cài đặt: mảng có thể xóa về 0 tức thời

Trong lời giải, ta thường phải xóa mọi giá trị của mảng đếm mà không muốn duyệt toàn bộ M ô. Có một kỹ thuật tổng quát tạo ra một **mảng có thể xóa về 0**, hỗ trợ mỗi thao tác sau trong thời gian $O(1)$:

- đọc một ô;
- ghi một ô;
- xóa về 0 toàn bộ mảng.

Với một mảng độ dài M , dùng hai mảng thường `value` và `stamp`, đều dài M , cùng biến số nguyên `epoch`. `value[p]` lưu dữ liệu từng ghi vào ô p ; `stamp[p]` lưu số hiệu thời kỳ (*epoch*) khi ô đó được ghi lần gần nhất. Mỗi lần xóa toàn bộ mảng, chỉ tăng `epoch` lên 1.

Khi ghi giá trị v vào ô p , đặt

$$\text{value}[p] = v, \quad \text{stamp}[p] = \text{epoch}.$$

Khi đọc ô p , nếu

$$\text{stamp}[p] = \text{epoch},$$

thì trả về `value[p]`; nếu không, trả về 0. Thật vậy, trong trường hợp sau, lần ghi gần nhất xảy ra trước thời kỳ hiện tại, nên ô này đã bị xóa kể từ lần ghi đó. Ban đầu, khởi tạo hai mảng và `epoch` bằng 0.

Như vậy, tăng một biến duy nhất khiến mọi giá trị cũ trở nên “lỗi thời”, nên về mặt logic mọi ô đều bằng 0. Kỹ thuật này tránh việc phải quét cả mảng khi chuyển sang khối tiếp theo.

Janošík ★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 128 MB

Mã bài: jan

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/jan>

32.1 Đề bài

Như mọi người đều biết, Janošík lấy của người giàu để chia cho người nghèo. Chàng cùng toán cướp của mình vừa đánh cướp một đoàn xe chở vàng tới lâu đài của bá tước và thu được n chiếc tráp. Sau khi chuyển chiến lợi phẩm về hang, họ phát hiện chiếc tráp thứ i , với $i = 1, 2, \dots, n$, chứa đúng i túi vàng.

Khi một người nghèo đến xin Janošík vài đồng vàng ducat, chàng thực hiện thủ tục sau. Trước tiên, chàng chọn một chiếc tráp không rỗng chứa ít túi vàng nhất.

- Nếu tráp chứa đúng một túi vàng, Janošík trao túi ấy cho người nghèo, và người đó vui vẻ ra về.
- Nếu tráp chứa nhiều hơn một túi và số túi là lẻ, Janošík bỏ một túi vào túi áo của mình rồi bắt đầu lại toàn bộ thủ tục.
- Nếu số túi trong tráp là chẵn, Janošík lấy đúng một nửa số túi

ra, chuyển chúng sang một chiếc tráp rỗng — may thay, trong hang có thừa tráp rỗng — rồi bắt đầu lại toàn bộ thủ tục.

Vì thế, nếu một người nghèo đến khi Janošík vẫn còn ít nhất một tráp không rỗng, sau khi chàng áp dụng thủ tục có thể nhiều lần, người đó chắc chắn sẽ nhận được một túi vàng. Người nghèo tiếp tục đến hang của Janošík cho tới khi tất cả các tráp đều rỗng.

Những tên cướp trong toán của Janošík lo rằng cách hành xử ấy sẽ làm tổn hại thanh danh của giới cướp. Chúng muốn biết sau khi mọi chiếc tráp được vét sạch, có bao nhiêu túi vàng chiến lợi phẩm nằm lại trong túi áo của thủ lĩnh.

32.1.1 Dữ liệu vào

Dòng duy nhất chứa một số nguyên n :

$$1 \leq n \leq 10^9,$$

là số tráp mà toán cướp của Janošík đã lấy được.

32.1.2 Dữ liệu ra

In một số nguyên duy nhất: số túi vàng còn lại trong túi áo của Janošík sau khi tất cả các tráp đã rỗng.

32.1.3 Ví dụ

Dữ liệu vào

5

Dữ liệu ra

2

32.2 Lời giải

32.2.1 Mô phỏng trực tiếp

Đề bài mô tả một thuật toán và yêu cầu tính kết quả của thuật toán ấy trên dữ liệu đã cho. Cách cài đặt trực tiếp không khó: ta chỉ việc mô phỏng lần lượt các hành động của Janošík.

Ta lưu cách phân bố hiện tại của các túi vàng bằng một danh sách: với mỗi tráp đang chứa i túi, danh sách có một phần tử mang giá trị i . Ban đầu, danh sách gồm

$$1, 2, \dots, n.$$

Ngoài ra, biến `pocket_bags` lưu số túi vàng hiện nằm trong túi áo của Janošík.

Mỗi bước của thuật toán diễn ra như sau. Trước tiên, tìm phần tử nhỏ nhất trong danh sách và gọi giá trị của nó là j , rồi xóa phần tử ấy.

- Nếu $j = 1$, không cần làm gì thêm trong bước này: túi vàng được trao cho một người nghèo.
- Nếu $j > 1$ và j lẻ, tăng `pocket_bags` lên một và chèn $j - 1$ vào danh sách.
- Nếu j chẵn, chèn hai phần tử có giá trị $j/2$ vào danh sách.

Lặp lại cho tới khi danh sách rỗng.

Ta có thể thực hiện mỗi bước bằng một số hằng thao tác nếu luôn giữ danh sách theo thứ tự không giảm. Khi ấy, phần tử nhỏ nhất j nằm ngay đầu danh sách. Các phần tử mới, dù là $j - 1$ hay $j/2$, đều nhỏ hơn j , do đó cũng nhỏ hơn mọi phần tử còn lại; ta có thể chèn chúng vào đầu mà không phá vỡ thứ tự.

Cách mô phỏng này luôn đúng nhưng quá chậm khi n lớn. Trong mỗi bước, ta thực hiện một trong hai loại thao tác:

- **(A)** giảm tổng số túi trong các tráp đi một;
- **(B)** chia số túi của một tráp thành hai phần bằng nhau.

Muốn làm rỗng tất cả các tráp, mỗi túi vàng ban đầu cuối cùng đều phải trải qua đúng một thao tác (A), bất kể nó được trao cho người nghèo hay bỏ vào túi áo Janošík. Tổng số túi ban đầu là

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}.$$

Do đó, riêng thao tác (A) đã được thực hiện đúng $n(n+1)/2$ lần. Với $n = 1\,000\,000\,000$, con số này xấp xỉ $5 \cdot 10^{17}$; ngay cả nếu mỗi thao tác chỉ tốn vài nano giây, chương trình vẫn chạy hàng chục năm. Ta thậm chí chưa cần đếm thao tác (B), dù có thể chứng minh số lần thực hiện nó không vượt quá $n(n+1)/2$. Vì vậy độ phức tạp thời gian của mô phỏng là

$$\Theta(n^2).$$

Ta cần tính kết quả của thủ tục nhanh hơn.

32.2.2 Không làm cùng một việc hai lần

Mỗi khi Janošík chọn một tráp chứa i túi, chàng sẽ phân phối hết toàn bộ vàng bắt nguồn từ tráp ấy trước khi chọn bất kỳ tráp nào khác đang chứa ít nhất i túi. Trong cả quá trình, Janošík có thể nhiều lần gặp các tráp chứa i túi, nhưng lần nào chàng cũng xử lý chúng theo đúng cùng một cách. Bởi vậy, số túi cuối cùng đi vào túi áo Janošík từ một tráp chứa i túi luôn như nhau.

Gọi $d(i)$ là số túi bắt nguồn từ một tráp ban đầu có i túi và cuối cùng nằm trong túi áo Janošík. Ta có hệ thức truy hồi

$$d(i) = \begin{cases} 0, & i = 1, \\ 1 + d(i-1), & i > 1 \text{ và } i \text{ lẻ}, \\ 2d(i/2), & i \text{ chẵn}. \end{cases} \quad (1)$$

Nếu tính lần lượt $d(1), d(2), \dots, d(n)$ và lưu chúng trong một mảng, mỗi giá trị $d(i)$ được tính trong thời gian hằng số vì nó chỉ phụ thuộc vào $d(i-1)$ hoặc $d(i/2)$, vốn đã được tính trước.

Do $d(i) \leq i$ — Janošík không thể lấy nhiều túi hơn số có trong tráp — mỗi phần tử của mảng có thể là một số nguyên 32 bit. Tuy nhiên, tổng đáp án

$$d(1) + d(2) + \dots + d(n)$$

phải được tính bằng số nguyên 64 bit.

Phương pháp này chạy trong thời gian $\Theta(n)$, nhưng cũng cần bộ nhớ $\Theta(n)$. Với $n = 10^9$, chương trình mất vài giây và mảng 32 bit chiếm khoảng 3,7 GB, gần gấp 30 lần giới hạn bộ nhớ của bài. Ta vẫn cần một lời giải khéo hơn.

32.2.3 Giải hệ thức truy hồi

Hệ thức (1) khá đơn giản. Ta có thể tính $d(i)$ cho một vài giá trị nhỏ rồi thử nhận ra quy luật:

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$d(i)$	0	0	1	0	1	2	3	0	1	2	3	4	5	6	7	8
$i - d(i)$	1	2	2	4	4	4	4	8	8	8	8	8	8	8	8	8

Bảng này gợi ý công thức

$$d(i) = i - 2^{\lfloor \log_2 i \rfloor}, \quad (2)$$

trong đó $2^{\lfloor \log_2 i \rfloor}$ là lũy thừa của hai lớn nhất không vượt quá i . Nói theo biểu diễn nhị phân, tính $d(i)$ chính là xóa bit 1 có trọng số lớn nhất của i .

Ta chứng minh (2) thỏa hệ thức (1) bằng quy nạp mạnh theo i . Với $i = 1$, ta có $\log_2 1 = 0$, nên

$$1 - 2^0 = 0 = d(1).$$

Giả sử $i \geq 2$ và công thức đúng với mọi $j < i$.

Nếu i chẵn, theo giả thuyết quy nạp,

$$\begin{aligned}
d(i) &= 2d(i/2) \\
&= 2 \left(\frac{i}{2} - 2^{\lfloor \log_2(i/2) \rfloor} \right) \\
&= 2 \left(\frac{i}{2} - 2^{\lfloor \log_2 i \rfloor - 1} \right) \\
&= i - 2^{\lfloor \log_2 i \rfloor}.
\end{aligned}$$

Nếu i lẻ, cũng theo giả thuyết quy nạp,

$$\begin{aligned}
d(i) &= 1 + d(i-1) \\
&= 1 + (i-1 - 2^{\lfloor \log_2(i-1) \rfloor}) \\
&= i - 2^{\lfloor \log_2 i \rfloor}.
\end{aligned}$$

Đẳng thức cuối đúng vì với $i > 1$ lẻ, lũy thừa của hai lớn nhất không vượt quá i cũng không vượt quá $i-1$; do đó

$$\lfloor \log_2(i-1) \rfloor = \lfloor \log_2 i \rfloor.$$

Vậy (2) đúng với mọi $i \geq 1$.

Giờ mỗi $d(i)$ có thể được tính độc lập, nên không còn cần mảng và bộ nhớ giảm xuống $\Theta(1)$. Ta có thể duyệt $i = 1, 2, \dots, n$, đồng thời giữ lũy thừa của hai lớn nhất không vượt quá i , cập nhật nó khi i đi qua 1, 2, 4, 8, Thời gian là $\Theta(n)$. Tuy nhiên, lời giải ấy vẫn hơi chậm so với giới hạn thời gian của bài.

32.2.4 Rút gọn tổng

Nếu cần tính riêng mọi giá trị $d(1), d(2), \dots, d(n)$, ta không thể tránh ít nhất n thao tác. Nhưng bài chỉ hỏi tổng của chúng, và tổng này có thể được tính nhanh hơn:

$$\begin{aligned}
\sum_{i=1}^n d(i) &= \sum_{i=1}^n (i - 2^{\lfloor \log_2 i \rfloor}) \\
&= \frac{n(n+1)}{2} - \sum_{i=1}^n 2^{\lfloor \log_2 i \rfloor}.
\end{aligned}$$

Trong tổng cuối, mỗi lũy thừa 2^j xuất hiện đúng 2^j lần, ứng với

$$i = 2^j, 2^j + 1, \dots, 2^{j+1} - 1,$$

ngoại trừ lũy thừa lớn nhất $2^{\lfloor \log_2 n \rfloor}$, vì nhóm cuối có thể chưa đầy. Số phần tử thực sự của nhóm j là

$$\min(n - 2^j + 1, 2^j).$$

Vì vậy, đáp án là

$$\sum_{i=1}^n d(i) = \frac{n(n+1)}{2} - \sum_{j=0}^{\lfloor \log_2 n \rfloor} 2^j \min(n - 2^j + 1, 2^j).$$

Ta chỉ cần duyệt các lũy thừa

$$1, 2, 4, 8, \dots, 2^{\lfloor \log_2 n \rfloor}.$$

Với $n = 10^9$, toàn bộ công thức chỉ có 31 nhóm lũy thừa. Có thể cài đặt vòng lặp bằng một biến p , ban đầu $p = 1$; ở mỗi bước trừ

$$p \cdot \min(n - p + 1, p)$$

khỏi $n(n+1)/2$, rồi nhân đôi p , và dừng sau khi $p > n$. Mọi phép tính phải dùng số nguyên 64 bit.

Thuật toán cuối cùng có độ phức tạp

$$\Theta(\log n) \text{ thời gian và } \Theta(1) \text{ bộ nhớ.}$$

Chăn đi biển ★★ ★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Eryk Kopczyński

Giới hạn bộ nhớ: 128 MB

Mã bài: koc

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2013/koc>

33.1 Đề bài

Mùa hè này, cư dân Byteburg kéo nhau đông đúc tới bãi biển thành phố bên hồ Byteania để tắm nắng. Mỗi người mang theo một chiếc chăn của hãng Byteasar và Son, một nhất mùa này. Mọi chiếc chăn đều có cùng kích thước $a \times b$ (dù hoa văn khác nhau), và người tắm nắng luôn trải chăn sao cho cạnh dài hơn vuông góc với bờ hồ.

Giáo sư Byteoni, cũng là một người đi tắm nắng năm nay, nhận thấy rằng mỗi cư dân luôn trải chăn ở đúng vị trí yêu thích của riêng mình. Dù mọi người đến và rời bãi biển vào các thời điểm khác nhau, ông chưa từng nghe có ai chiếm vị trí yêu thích của người khác bằng chiếc chăn của mình. Điều đó khiến giáo sư tò mò và quyết định nghiên cứu hiện tượng này.

Ông lập một hệ tọa độ trên bãi biển và ghi lại, với mỗi trong n cư dân, tọa độ vị trí mà người đó luôn trải chăn. Hệ tọa độ được chọn

sao cho trục OX song song với các cạnh độ dài a , còn trục OY song song với các cạnh độ dài b của mọi chiếc chăn. Ban đầu giáo sư định tính diện tích giao của từng cặp chăn. Sau đó ông nhận ra chỉ cần giá trị trung bình của các diện tích ấy. Nói cách khác, ông cần kỳ vọng diện tích giao của hai chiếc chăn thuộc về hai cư dân Byteburg khác nhau, được chọn ngẫu nhiên. Hãy giúp ông tính giá trị đó.

33.1.1 Dữ liệu vào

Dòng đầu chứa ba số nguyên n, a, b :

$$2 \leq n \leq 200\,000, \quad 1 \leq a, b \leq 1\,000\,000,$$

lần lượt là số cư dân và hai kích thước của chăn.

Mỗi trong n dòng tiếp theo chứa hai số nguyên x_i, y_i :

$$0 \leq x_i, y_i \leq 1\,000\,000,$$

là tọa độ góc dưới bên trái của chiếc chăn mà cư dân thứ i luôn trải.

33.1.2 Dữ liệu ra

In một số thực: diện tích giao trung bình của các cặp chăn thuộc về hai cư dân Byteburg khác nhau.

Kết quả được chấp nhận nếu thuộc đoạn

$$[x - 10^{-2}, x + 10^{-2}],$$

trong đó x là đáp án chính xác.

33.1.3 Ví dụ

Dữ liệu vào

```
4 3 5
0 0
```

2 1
3 3
0 5

Dữ liệu ra

1.833333333

Ở ví dụ này, sáu cặp chẵn có diện tích giao lần lượt là 4, 0, 0, 1, 6, 0. Vì thế đáp án chính xác là

$$\frac{4 + 0 + 0 + 1 + 6 + 0}{6} = \frac{11}{6}.$$

33.2 Lời giải

33.2.1 Đổi thứ tự cộng

Gọi S là tổng diện tích giao trên mọi cặp chẵn không thứ tự. Nếu duyệt từng cặp $i < j$ và cộng trực tiếp diện tích giao của chúng, đáp án cần in sẽ là

$$\frac{S}{\binom{n}{2}}.$$

Không thể duyệt $\binom{n}{2}$ cặp khi n lớn. Thay vào đó, xét từng ô đơn vị của mặt phẳng. Nếu một ô đang bị phủ bởi đúng m chiếc chẵn, nó thuộc phần giao của đúng

$$\binom{m}{2}$$

cặp chẵn. Do đó, đóng góp của ô ấy vào S là $\binom{m}{2}$. Chỉ cần tính tổng các đóng góp này trên toàn mặt phẳng.

33.2.2 Quét theo trục x

Dùng đường quét thẳng đứng đi từ trái sang phải. Giữa hai hoành độ có sự kiện liên tiếp, tập chẵn cắt đường quét không thay đổi. Gọi

$$s = \sum_{\text{ô đơn vị trong cột hiện tại}} \binom{m}{2},$$

tức tổng đóng góp trên một dải đơn vị theo chiều x . Nếu đường quét nhảy một quãng d mà không đi qua cạnh đứng nào của chẵn, ta tăng

$$S += s \cdot d.$$

Mỗi chiếc chẵn (x, y) tạo hai sự kiện: bắt đầu tại x , kết thúc tại $x + a$. Có thể xem nó hoạt động trên nửa khoảng $[x, x + a)$, nên tại mỗi hoành độ sự kiện ta cộng đóng góp từ đoạn trước, rồi xóa các chẵn kết thúc và thêm các chẵn bắt đầu. Cách quy ước ở đầu mút không ảnh hưởng diện tích.

Khi đường quét đang cắt một chiếc chẵn có tung độ đáy y , ta lưu y trong đa tập D . Vấn đề còn lại: thêm hoặc xóa một giá trị y khỏi D thì cập nhật s thế nào?

33.2.3 Đóng góp khi thêm một chiếc chẵn

Giả sử ta thêm chẵn mới có đáy y . Với một chẵn đang có đáy y' , độ dài giao theo chiều dọc là

$$m(y') = \max(0, b - |y - y'|).$$

Đó chính là phần cần cộng vào s cho cặp gồm chẵn mới và chẵn có đáy y' . Vì vậy đặt

$$\Delta = \sum_{y' \in D} \max(0, b - |y - y'|).$$

Sau khi tính Δ , tăng s thêm Δ , rồi chèn y vào D . Khi một chiếc chẵn kết thúc, làm ngược lại: trước tiên bỏ y khỏi D , tính tổng Δ

với các phần tử còn lại, rồi giảm s đi Δ . Nhờ vậy không bao giờ tính một cặp hai lần.

Chỉ các y' thuộc đoạn $[y - b, y + b]$ mới có đóng góp. Tách đoạn này tại y , ta được

$$\begin{aligned}\Delta &= \sum_{\substack{y' \in D \\ y-b \leq y' \leq y}} (b - y + y') + \sum_{\substack{y' \in D \\ y < y' \leq y+b}} (b + y - y') \\ &= \left(\sum_{[y-b, y]} y' \right) - \left(\sum_{(y, y+b]} y' \right) \\ &\quad + (b - y) \#([y - b, y]) + (b + y) \#((y, y + b]).\end{aligned}\tag{1}$$

Vì vậy, để tính Δ , chỉ cần trả lời trên một khoảng giá trị y :

1. có bao nhiêu phần tử của D nằm trong khoảng đó;
2. tổng các phần tử đó bằng bao nhiêu.

33.2.4 Cấu trúc dữ liệu

Dùng hai cây đoạn (hoặc hai Fenwick tree sau khi nén tọa độ):

- cây thứ nhất lưu số lần xuất hiện của mỗi tung độ;
- cây thứ hai lưu tổng các tung độ ấy.

Mỗi truy vấn tổng đoạn, truy vấn số lượng đoạn, chèn, và xóa đều mất $O(\log M)$, với M là miền tọa độ được lưu. Do y_i và b bị chặn, có thể xây cây trực tiếp trên một miền $O(10^6)$; cách gọn hơn là nén các tọa độ y_i và dùng tìm kiếm nhị phân để xác định những tọa độ nằm trong các khoảng của (1).

Với mỗi sự kiện, dùng hai cây để tính Δ , cập nhật s , và cập nhật đa tập D . Sau khi quét xong, S đã là tổng diện tích giao của mọi cặp chẵn. In

$$\frac{S}{\binom{n}{2}}$$

dưới dạng số thực.

33.2.5 Độ phức tạp và độ chính xác

Có $2n$ sự kiện. Sắp xếp chúng mất $O(n \log n)$; mỗi sự kiện thực hiện hằng số thao tác cây, mất $O(\log M)$. Tổng thời gian là

$$O(n \log n + n \log M),$$

hay $O(n \log n)$ nếu nén tọa độ. Bộ nhớ là $O(n)$ khi nén tọa độ (hoặc $O(M)$ với cây trên miền tọa độ trực tiếp).

Giá trị S có thể vượt quá long long: tổng diện tích giao có thể vào cỡ $n^2 ab$. Không nên chỉ dùng double để tích lũy vì sai số có thể lớn. Có thể dùng long double, kiểu `__int128` của GCC, hoặc mô phỏng số nguyên 128-bit bằng hai số nguyên 64-bit. Các lựa chọn này đều đủ chính xác cho yêu cầu của bài.

Luật sư ★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 256 MB

Mã bài: adw

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/adw>

34.1 Đề bài

Luật sư Byteasar, đồng sở hữu văn phòng luật Byteasar và Cộng sự, là một trong những thành viên được săn đón nhất của đoàn luật sư Byteotia. Vì thế, chẳng có gì lạ khi ông luôn vô cùng bận rộn. Mỗi ngày ông có rất nhiều cuộc họp và từ lâu đã không còn kiểm soát được liệu mình có thể tham dự tất cả hay không. Do đó, ông thuê một thư ký để giúp mình sắp xếp mớ hỗn độn này.

Byteasar quyết định rằng mỗi ngày ông sẽ chỉ tham dự hai cuộc họp, nhưng phải tham dự trọn vẹn từ đầu đến cuối. Những cuộc họp còn lại sẽ do các trợ lý đảm nhiệm; văn phòng của ông không hề thiếu trợ lý.

Đáng tiếc là đôi khi, ngay cả việc tìm hai cuộc họp không trùng thời gian trong lịch trình dày đặc của Byteasar cũng rất khó. Ta quy ước hai cuộc họp không trùng thời gian nếu một cuộc bắt đầu **sau**

khi cuộc kia đã kết thúc. Hãy giúp thư ký của Byteasar giải quyết vấn đề này.

34.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n, m

$$2 \leq n \leq 500\,000, \quad 1 \leq m \leq 20,$$

lần lượt là số cuộc họp trong lịch trình của Byteasar và số ngày mà lịch trình bao gồm.

Mỗi dòng trong n dòng tiếp theo mô tả một cuộc họp bằng ba số nguyên a_i, b_i, d_i

$$1 \leq a_i < b_i \leq 80\,000\,000, \quad 1 \leq d_i \leq m.$$

Các số này cho biết vào ngày d_i , Byteasar có một cuộc họp bắt đầu đúng a_i mili giây sau nửa đêm và kết thúc b_i mili giây sau nửa đêm.

34.1.2 Dữ liệu ra

In m dòng. Dòng thứ i cho biết Byteasar có thể tham dự hai cuộc họp trong ngày thứ i hay không.

Nếu không thể, in một từ NIE (nghĩa là “không” trong tiếng Ba Lan). Nếu có thể, in từ TAK (nghĩa là “có” trong tiếng Ba Lan), theo sau là chỉ số của hai cuộc họp mà Byteasar có thể tham dự. Các cuộc họp được đánh số từ 1 đến n theo thứ tự xuất hiện trong dữ liệu vào.

Cuộc họp được in trước phải bắt đầu sớm hơn. Cuộc họp thứ hai phải bắt đầu ít nhất một mili giây sau khi cuộc họp thứ nhất kết thúc.

Nếu có nhiều đáp án đúng, được phép in bất kỳ đáp án nào.

34.1.3 Ví dụ

Dữ liệu vào

```

6 3
3 5 1
2 4 2
1 8 1
6 7 3
3 5 2
7 12 1

```

Dữ liệu ra

```

TAK 1 6
NIE
NIE

```

34.2 Lời giải

Xét riêng từng ngày trong m ngày. Với một ngày cố định, bài toán có thể phát biểu ngắn gọn như sau: cho một tập các đoạn, cần tìm hai đoạn sao cho đầu mút phải của đoạn thứ nhất nhỏ hơn hẳn đầu mút trái của đoạn thứ hai. Nếu tồn tại hai đoạn như vậy, ta in TAK cùng chỉ số của chúng; nếu không, ta in NIE.

Một cách trực tiếp là duyệt mọi cặp đoạn. Với mỗi cặp $[a_i, b_i]$ và $[a_j, b_j]$, ta kiểm tra xem $b_i < a_j$ hay không. Ngay khi tìm thấy một cặp thỏa mãn, ta dừng và in đáp án.

Vấn đề là số cặp có thể rất lớn. Trong trường hợp xấu nhất, có 500 000 đoạn và mọi cặp đoạn đều giao nhau, chẳng hạn khi tất cả các đoạn giống hệt nhau. Khi ấy ta phải xét khoảng

$$500\,000^2 = 2,5 \cdot 10^{11}$$

cặp. Ngay cả với giả định khá lạc quan rằng có thể kiểm tra nửa tỉ cặp mỗi giây, chương trình vẫn chạy hơn tám phút.

Ta có thể chọn hai ứng viên cực trị thay vì duyệt mọi cặp. Gọi p là đoạn có **đầu mút phải nhỏ nhất**, và gọi q là đoạn có **đầu mút trái lớn nhất**. Nói cách khác,

$$b_p = \min_i b_i, \quad a_q = \max_i a_i.$$

Nếu

$$b_p < a_q,$$

thì p, q chính là một đáp án: cuộc họp p kết thúc trước khi cuộc họp q bắt đầu. Hơn nữa, vì $a_p < b_p < a_q$, cuộc họp p cũng bắt đầu sớm hơn cuộc họp q , đúng với thứ tự đầu ra yêu cầu.

Ngược lại, giả sử $b_p \geq a_q$. Do a_q là đầu mút trái lớn nhất, mọi đoạn đều có đầu mút trái không vượt quá a_q . Do b_p là đầu mút phải nhỏ nhất, mọi đoạn đều có đầu mút phải không nhỏ hơn b_p . Vì vậy, mọi đoạn đều chứa toàn bộ đoạn

$$[a_q, b_p].$$

Đặc biệt, mọi hai đoạn đều giao nhau, nên không thể tồn tại hai cuộc họp cần tìm.

Như vậy, điều kiện $b_p < a_q$ vừa đủ vừa cần. Để tìm p và q , chỉ cần duyệt danh sách các cuộc họp của ngày đó đúng một lần.

34.2.1 Thuật toán

Với mỗi ngày, duy trì:

- cuộc họp p có thời điểm kết thúc nhỏ nhất;
- cuộc họp q có thời điểm bắt đầu lớn nhất.

Khi đọc mỗi cuộc họp, cập nhật hai ứng viên của ngày tương ứng. Sau khi đọc xong:

- nếu ngày nào có $b_p < a_q$, in TAK, rồi in chỉ số của p và q ;
- nếu không, in NIE.

Không cần xử lý riêng trường hợp $p = q$: nếu đó là cùng một cuộc họp thì $a_p < b_p$, nên bất đẳng thức $b_p < a_q = a_p$ không thể đúng.

34.2.2 Độ phức tạp

Mỗi cuộc họp được xử lý đúng một lần. Vì vậy, tổng thời gian là

$$O(n + m),$$

trong đó $O(m)$ là thời gian xuất kết quả cho m ngày. Nếu chỉ lưu hai ứng viên cho mỗi ngày, thuật toán dùng $O(m)$ bộ nhớ phụ.

Xăng ★★ ★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Adam Karczmarz

Giới hạn bộ nhớ: 256 MB

Mã bài: ben

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/ben>

35.1 Đề bài

Bajtazar làm việc trong bộ phận hậu cần của tập đoàn dầu khí Bytean khổng lồ Bajtoil. Công việc của anh là lập kế hoạch vận chuyển nhiên liệu đến các trạm xăng.

Byteotia có (n) giao lộ, được đánh số từ (1) đến (n) , cùng (m) con đường hai chiều nối một số cặp giao lộ. Tại một số giao lộ có trạm xăng của Bajtoil.

Đội xe vận tải của Bajtoil gồm những xe bồn có bình nhiên liệu với nhiều dung tích khác nhau. Mỗi xe tiêu thụ (1) lít xăng trên mỗi kilômét di chuyển. Vì vậy, có thể xem một xe với bình chứa (b) lít đi được nhiều nhất (b) kilômét mà không cần tiếp nhiên liệu. Tài xế không được dùng lượng xăng nằm trong bồn hàng của xe, nhưng có thể đổ đầy bình nhiên liệu miễn phí tại bất kỳ trạm xăng Bajtoil nào.

Trong công việc, Bajtazar phải nhiều lần trả lời câu hỏi sau: một xe bồn có bình nhiên liệu dung tích (b) lít có thể đi từ trạm xăng tại giao lộ (x) đến trạm xăng tại giao lộ (y) hay không? Xe đó không thể đi quá (b) kilômét liên tiếp mà không gặp một trạm xăng Bajtoil. Mọi chuyến đi đều bắt đầu và kết thúc tại giao lộ có trạm xăng Bajtoil.

Hãy giúp Bajtazar tự động trả lời các truy vấn hậu cần này.

35.1.1 Dữ liệu vào

Dòng đầu chứa ba số nguyên (n,s,m)

$$2 \leq s \leq n \leq 200\,000, \quad 1 \leq m \leq 200\,000,$$

lần lượt là số giao lộ, số trạm xăng và số con đường ở Byteotia.

Dòng thứ hai chứa (s) số nguyên đôi một khác nhau

$$c_1, c_2, \dots, c_s \quad (1 \leq c_i \leq n),$$

cho biết các giao lộ có trạm xăng Bajtoil.

Mỗi trong (m) dòng tiếp theo chứa ba số nguyên u_i, v_i, d_i

$$1 \leq u_i, v_i \leq n, \quad u_i \neq v_i, \quad 1 \leq d_i \leq 10\,000,$$

nghĩa là con đường thứ (i) dài d_i kilômét và nối hai giao lộ u_i, v_i . Giữa mỗi cặp giao lộ có nhiều nhất một con đường.

Dòng kế tiếp chứa số nguyên (q) ($(1 \leq q \leq 200,000)$), là số truy vấn. Mỗi trong (q) dòng sau chứa ba số nguyên x_i, y_i, b_i

$$1 \leq x_i, y_i \leq n, \quad x_i \neq y_i, \quad 1 \leq b_i \leq 2 \cdot 10^9,$$

mô tả truy vấn liệu một xe bồn có bình nhiên liệu dung tích b_i lít có thể đi từ trạm tại giao lộ x_i đến trạm tại giao lộ y_i hay không. Bảo đảm cả x_i lẫn y_i đều có trạm xăng Bajtoil.

35.1.2 Dữ liệu ra

In đúng (q) dòng. Dòng thứ (i) chứa từ TAK (có) hoặc NIE (không), tùy theo xe có bình nhiên liệu dung tích b_i lít có thể đi từ giao lộ x_i đến giao lộ y_i hay không.

35.1.3 Ví dụ

Dữ liệu vào

```
6 4 5
1 5 2 6
1 3 1
2 3 2
3 4 3
4 5 5
6 4 5
4
1 2 4
2 6 9
1 5 9
6 5 8
```

Dữ liệu ra

```
TAK
TAK
TAK
NIE
```

35.2 Lời giải

Ta xem mạng lưới đường sá là đồ thị $G = (V, E)$, trong đó các đỉnh ứng với giao lộ và các cạnh ứng với đường. Với cạnh $(u, v) \in E$, ký hiệu $c(u, v)$ là chiều dài con đường từ u đến v , tính bằng kilômét. Ngoài ra, ta có tập đỉnh đặc biệt $S \subseteq V$ gồm các giao lộ có trạm xăng.

Mỗi truy vấn cho hai đỉnh $x, y \in S$ và một số b , rồi hỏi liệu có đường đi P từ x đến y sao cho khi cắt P tại mọi đỉnh thuộc S , mỗi đoạn thu được đều dài không quá b . Ta gọi một đường đi như vậy là **tốt**, và ký hiệu truy vấn là (x, y, b) .

35.2.1 Đặc trưng của một đường đi tốt

Trước hết, giả sử chỉ cần trả lời một truy vấn (x, y, b) . Với mỗi đỉnh v , gọi $d_S(v)$ là khoảng cách từ v đến đỉnh gần nhất thuộc S . Đặc biệt, $d_S(v) = 0$ nếu $v \in S$. Với mỗi cạnh $(u, v) \in E$, đặt

$$c^*(u, v) := d_S(u) + c(u, v) + d_S(v).$$

Ta sẽ chứng minh rằng, với một giá trị b cố định, tồn tại đường đi tốt từ x đến y khi và chỉ khi giữa x và y có một đường đi chỉ gồm những cạnh (u, v) thỏa

$$c^*(u, v) \leq b. \quad (1)$$

Trước tiên, giả sử một cạnh (u, v) có $c^*(u, v) > b$. Không đường đi tốt nào giữa hai đỉnh thuộc S có thể dùng cạnh này. Thật vậy, nếu một hành trình đi qua (u, v) , nó phải chứa một đoạn gồm ba phần liên tiếp:

1. từ một đỉnh $s_1 \in S$ đến u , dài ít nhất $d_S(u)$;
2. cạnh (u, v) , dài $c(u, v)$;
3. từ v đến một đỉnh $s_2 \in S$, dài ít nhất $d_S(v)$.

Đoạn không có trạm xăng ở bên trong ấy dài ít nhất

$$d_S(u) + c(u, v) + d_S(v) = c^*(u, v) > b,$$

trái với định nghĩa của đường đi tốt.

Theo chiều ngược lại, giả sử trong G có đường đi

$$x = v_1, v_2, \dots, v_p = y$$

mà $c^*(v_i, v_{i+1}) \leq b$ với mọi $i = 1, \dots, p-1$. Với mỗi đỉnh v , chọn một đỉnh $l_S(v) \in S$ gần v nhất, tức khoảng cách từ v đến $l_S(v)$ đúng bằng $d_S(v)$. Vì $x, y \in S$, ta có thể chọn $l_S(x) = x$ và $l_S(y) = y$.

Giờ hãy ghép các đoạn sau thành một hành trình P : với mỗi $i = 1, \dots, p-1$, đi theo một đường ngắn nhất từ $l_S(v_i)$ đến v_i , đi qua cạnh (v_i, v_{i+1}) , rồi theo một đường ngắn nhất từ v_{i+1} đến $l_S(v_{i+1})$. Mỗi đoạn như vậy bắt đầu và kết thúc tại trạm xăng, đồng thời có độ dài

$$d_S(v_i) + c(v_i, v_{i+1}) + d_S(v_{i+1}) = c^*(v_i, v_{i+1}) \leq b.$$

Nói các đoạn này lại cho ta một đường đi tốt từ x đến y . Hành trình có thể đi lặp qua một số cạnh hay đỉnh, nhưng điều đó không ảnh hưởng đến tính hợp lệ.

Vậy với b cố định, tồn tại đường đi tốt từ x đến y khi và chỉ khi x và y nằm trong cùng một thành phần liên thông của đồ thị

$$G_b = (V, \{(u, v) \in E : c^*(u, v) \leq b\}). \quad (2)$$

35.2.2 Thuật toán offline

Bước đầu tiên là tính $c^*(u, v)$ cho mọi cạnh. Muốn vậy, ta cần biết $d_S(v)$ với mọi $v \in V$. Hãy thêm vào G một **siêu đỉnh** z , rồi nối z với mỗi đỉnh thuộc S bằng một cạnh độ dài 0, tạo thành đồ thị G' . Khi đó, $d_S(v)$ chính là khoảng cách từ z đến v trong G' .

Do mọi độ dài cạnh đều không âm, chạy Dijkstra từ z sẽ tìm được toàn bộ các giá trị $d_S(\cdot)$ trong thời gian

$$O((n + m) \log n).$$

Sau đó, tính trực tiếp $c^*(u, v)$ cho tất cả các cạnh.

Ta không buộc phải trả lời một truy vấn trước khi đọc truy vấn kế tiếp. Vì vậy, hãy đọc toàn bộ truy vấn, sắp xếp chúng không giảm theo b_i , rồi trả lời theo thứ tự đó. Giả sử sau khi sắp xếp ta có

$$(x_1, y_1, b_1), \dots, (x_q, y_q, b_q), \quad b_1 \leq b_2 \leq \dots \leq b_q.$$

Đồng thời, sắp xếp các cạnh không giảm theo trọng số $c^*(u, v)$. Ban đầu đặt H là đồ thị trên tập đỉnh V nhưng chưa có cạnh. Khi xử lý truy vấn thứ i , thêm vào H tất cả những cạnh chưa thêm có

$$c^*(u, v) \leq b_i.$$

Lúc này H chính là G_{b_i} . Theo (2), câu trả lời là TAK khi và chỉ khi x_i, y_i thuộc cùng một thành phần liên thông của H .

Ta duy trì các thành phần liên thông của H bằng cấu trúc tập hợp rời nhau DSU (còn gọi là *union-find*). Thêm cạnh tương ứng với phép hợp nhất hai tập; kiểm tra hai đỉnh có nối với nhau hay không tương ứng với hai phép tìm đại diện. Với gộp theo hạng/kích thước và nén đường đi, tổng thời gian cho $m + q$ thao tác là

$$O((m + q) \log^* n),$$

nhỏ hơn $O((m + q) \log n)$. Phải lưu chỉ số ban đầu của mỗi truy vấn để đưa các câu trả lời về đúng thứ tự xuất hiện trong dữ liệu vào.

Tính cả Dijkstra và hai lần sắp xếp, toàn bộ thuật toán chạy trong

$$O((n + m + q) \log(n + q))$$

thời gian và dùng $O(n + m + q)$ bộ nhớ.

35.3 Trả lời truy vấn trực tuyến

Thuật toán trên đủ để giải bài trong cuộc thi, nhưng nó cần biết trước mọi truy vấn và xử lý chúng theo dung tích bình nhiên liệu tăng dần. Trong thực tế, một trạm có thể bắt ngờ hết xăng và việc giao hàng phải được lập tức lên kế hoạch; cũng không thể giả sử theo thời gian người ta chỉ gửi đến các xe có bình ngày càng lớn.

Ta sẽ sửa lời giải để có thể trả lời độc lập mỗi truy vấn trong thời gian $O(\log n)$. Nhắc lại rằng truy vấn (x, y, b) tương đương với việc kiểm tra x, y có thuộc cùng thành phần của G_b hay không.

Mục tiêu là xây dựng một rừng F gồm các cây có gốc, có trọng số cạnh, trên tập đỉnh V , thỏa hai tính chất:

1. Với mọi b và mọi $v, w \in V$, hai đỉnh v, w thuộc cùng thành phần của G_b khi và chỉ khi trong F có đường đi giữa chúng và không cạnh nào trên đường ấy có trọng số lớn hơn b .
2. Chiều cao của mọi cây trong F không vượt quá $\log_2 n$.

Nếu chỉ cần tính chất thứ nhất, ta có thể dùng nhận xét quen thuộc sau.

Nhận xét 1. Xét một đồ thị vô hướng có trọng số, hai đỉnh bất kỳ v, w và một số b . Trong đồ thị có đường đi từ v đến w chỉ gồm những cạnh trọng số không quá b khi và chỉ khi trong một rừng khung nhỏ nhất của đồ thị cũng có một đường đi như vậy.

Ở đây rừng khung nhỏ nhất được tạo bằng cách tìm một cây khung nhỏ nhất trong từng thành phần liên thông. Vì thế, rừng khung nhỏ nhất của G với trọng số cạnh $c^*(u, v)$ là một ứng viên cho tính chất thứ nhất, nhưng không nhất thiết có chiều cao lôgarit. Ta sẽ dùng một cách xây dựng hơi khác.

Giả sử trước mắt đã có rừng F thỏa cả hai tính chất. Với mỗi đỉnh v không phải gốc, gọi $p(v)$ là cha của nó. Bắt đầu tại v , liên tục đi lên cha chừng nào đỉnh hiện tại chưa phải gốc và cạnh nối nó với cha có trọng số không quá b . Gọi đỉnh dừng lại là $r_b(v)$.

Hai đỉnh v, w được nối trong F bằng một đường mà mọi trọng số cạnh đều không quá b khi và chỉ khi

$$r_b(v) = r_b(w). \quad (3)$$

Tính chất thứ nhất biến (3) thành điều kiện tương đương với việc v, w cùng thành phần của G_b . Tính chất thứ hai cho phép tính mỗi $r_b(\cdot)$ bằng cách đi lên nhiều nhất $O(\log n)$ cạnh. Do đó, một truy vấn được trả lời trong $O(\log n)$. Ta chỉ còn phải xây dựng F .

35.3.1 Xây dựng rừng

Nếu xóa khỏi một rừng thỏa hai tính chất trên mọi cạnh có trọng số lớn hơn một giá trị B , ta được một rừng mà tính chất thứ nhất

vẫn đúng với mọi $b \leq B$. Gọi rừng ấy là F_B . Ta sẽ xây dựng F_B với các giá trị B tăng dần bằng cách xét các cạnh theo trọng số $c^*(u, v)$ tăng dần. Cuối cùng thu được $F = F_C$, trong đó C là trọng số c^* lớn nhất.

Khởi đầu là rừng rỗng F_0 , không có cạnh; khi ấy $p(v)$ chưa xác định với mọi v . Với mỗi đỉnh v , ta còn duy trì $h(v)$, chiều cao của cây con có gốc tại v . Ban đầu $h(v) = 0$. Rừng F_0 hiển nhiên thỏa tính chất thứ nhất đối với mọi $b \leq 0$.

Để dễ trình bày, trước hết giả sử các trọng số $c^*(u, v)$ đôi một khác nhau. Xét cạnh (u, v) kế tiếp có trọng số $B = c^*(u, v)$. Trong rừng tạo bởi các cạnh nhẹ hơn B , tìm

$$x = r_{B-}(u), \quad y = r_{B-}(v),$$

tức gốc hai cây hiện chứa u, v . Việc này tốn $O(h(x) + h(y))$ thời gian bằng cách đi theo các cạnh của rừng. Không mất tính tổng quát, giả sử $h(x) \geq h(y)$.

Nếu $x = y$, cạnh mới không làm thay đổi thành phần liên thông, nên không cần làm gì. Nếu $x \neq y$, nối hai cây bằng cách đặt

$$p(y) = x$$

và gán trọng số B cho cạnh mới từ y đến x . Nếu $h(x) = h(y)$, tăng $h(x)$ thêm 1; nếu không, chiều cao của cây gốc x giữ nguyên. Đây chính là phép hợp nhất theo hạng, nhưng các cạnh trong rừng nối hai **gốc thành phần**, thay vì nhất thiết sao chép cạnh (u, v) gốc. Lập luận quy nạp trực tiếp theo các cạnh đã xét cho thấy rừng mới tiếp tục thỏa tính chất thứ nhất với mọi ngưỡng đến B .

Nếu nhiều cạnh có cùng trọng số, chỉ cần xử lý chúng liên tiếp ở cùng ngưỡng B . Các phép nối gốc đều được gán trọng số B ; sau khi xử lý hết nhóm, rừng thu được biểu diễn đúng các thành phần của G_B . Lập luận trên không thay đổi.

35.3.2 Giới hạn chiều cao

Ta chứng minh rằng tại mọi thời điểm, cây con có gốc tại v chứa ít nhất

$$2^{h(v)}$$

đỉnh. Ban đầu $h(v) = 0$, mỗi đỉnh đứng riêng nên mệnh đề đúng. Chiều cao không bao giờ giảm. Giá trị $h(v)$ chỉ tăng khi ta gắn vào v một cây khác, có gốc $w \neq v$, với $h(w) = h(v)$. Trước khi nối, theo giả thiết quy nạp, mỗi cây có ít nhất $2^{h(v)}$ đỉnh. Sau khi nối, cây gốc v có ít nhất

$$2^{h(v)} + 2^{h(v)} = 2^{h(v)+1}$$

đỉnh, đúng với chiều cao mới.

Do cả rừng chỉ có n đỉnh,

$$2^{h(v)} \leq n, \quad \text{nên} \quad h(v) \leq \log_2 n.$$

Vì thế tính chất thứ hai được bảo đảm. Rừng F dùng $O(n)$ bộ nhớ và được xây dựng trong tổng thời gian

$$O(n + m \log n),$$

bao gồm việc sắp xếp cạnh và đi lên các cây có chiều cao lôgarit. Sau bước tiền xử lý Dijkstra để tính c^* và xây dựng F , mỗi truy vấn có thể được đọc rồi trả lời ngay, độc lập với các truy vấn khác, trong thời gian

$$O(\log n).$$

Giá cả ★★

Tác giả đề: Jakub Radoszewski

Tác giả lời giải: Jakub Radoszewski

Giới hạn bộ nhớ: 256 MB

Mã bài: cen

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/cen>

36.1 Đề bài

Bajtazar làm nhân viên thu mua cho một nhà hàng ở Byteotia. Mỗi tối, anh nhận từ quản lý danh sách những thực phẩm phải mua vào sáng hôm sau. Bajtazar phải mua đúng một đơn vị của mỗi sản phẩm trong danh sách. Quản lý luôn yêu cầu tổng chi phí mua sắm phải nhỏ nhất có thể.

Buổi tối, Bajtazar ngồi trước máy tính và kiểm tra giá của tất cả sản phẩm cần mua tại các kho bán buôn thực phẩm trong vùng. Anh cũng biết chi phí đi từ nhà hàng đến từng kho rồi quay về. Bây giờ, Bajtazar phải quyết định sẽ mua những sản phẩm nào tại mỗi kho.

Với mỗi kho mà Bajtazar quyết định mua hàng, anh sẽ đi từ nhà hàng đến kho đó, mua sắm rồi lập tức chở hàng về nhà hàng. May thay, cốp xe đủ lớn nên anh không cần ghé một kho nhiều hơn một lần: toàn bộ hàng mua tại đó có thể được chở về trong một chuyến. Thực phẩm rất dễ hỏng, vì vậy trong một chuyến đi Bajtazar chỉ có

thẻ mua hàng tại một kho.

Hãy viết chương trình giúp Bajtazar tính cách mua toàn bộ sản phẩm với chi phí nhỏ nhất.

36.1.1 Dữ liệu vào

Dòng đầu chứa hai số nguyên n, m

$$1 \leq n \leq 100, \quad 1 \leq m \leq 16,$$

lần lượt là số kho bán buôn và số sản phẩm Bajtazar phải mua. n dòng tiếp theo mô tả giá tại từng kho.

Số đầu tiên trên dòng thứ i là d_i ($1 \leq d_i \leq 1\,000\,000$), biểu thị chi phí đi từ nhà hàng đến kho thứ i , kể cả lượt về. Tiếp theo là m số nguyên

$$c_{i,1}, c_{i,2}, \dots, c_{i,m} \quad (1 \leq c_{i,j} \leq 1\,000\,000),$$

trong đó $c_{i,j}$ là giá của sản phẩm thứ j tại kho thứ i .

36.1.2 Dữ liệu ra

In một dòng chứa một số nguyên: tổng giá các sản phẩm và chi phí đi lại đến những kho được chọn trong phương án mua sắm rẻ nhất.

36.1.3 Ví dụ

Dữ liệu vào

```
3 4
5 7 3 7 9
2 1 20 3 2
8 1 20 1 1
```

Dữ liệu ra

Giải thích. Bajtazar mua sản phẩm số 2 tại kho thứ nhất và mua tất cả sản phẩm còn lại tại kho thứ hai. Vì thế, anh không cần ghé kho thứ ba.

36.2 Lời giải

Ta cần lập kế hoạch mua m loại thực phẩm. Có n kho và kho nào cũng bán đủ mọi loại sản phẩm. Biết giá của từng sản phẩm tại từng kho cùng chi phí đi lại đến mỗi kho, ta muốn tìm một kế hoạch có tổng chi phí nhỏ nhất. Không nhất thiết phải ghé tất cả các kho.

Ta biết rằng có nhiều nhất 16 sản phẩm và không quá 100 kho.

36.2.1 Lời giải thứ nhất

Đặt

$$N = \{1, \dots, n\}, \quad M = \{1, \dots, m\}.$$

Giới hạn $m \leq 16$ gợi ý rằng thuật toán có thể có độ phức tạp hàm mũ theo m . Một phương pháp chuẩn là quy hoạch động với các trạng thái tương ứng với những tập con của tập sản phẩm. Với mỗi $A \subseteq M$, ta muốn tìm chi phí nhỏ nhất để mua các sản phẩm thuộc A . Ký hiệu giá trị này là $\text{minCost}[A]$. Ta tính $\text{minCost}[A]$ cho các tập có kích thước tăng dần và lưu lại mọi giá trị đã tính.

Làm thế nào để tính $\text{minCost}[A]$? Nếu $A \neq \emptyset$, việc mua tất cả sản phẩm trong A đòi hỏi một số chuyến ghé kho. Hãy xét kho được ghé cuối cùng và những sản phẩm mua tại đó. Theo ký hiệu trong đề bài, $c_{i,j}$ là giá sản phẩm j tại kho i , còn d_i là chi phí đi lại đến kho i . Đặt

$$\text{purchaseCost}[i, B] = d_i + \sum_{j \in B} c_{i,j} \quad (1)$$

là tổng chi phí mua tất cả sản phẩm thuộc B tại kho i , kể cả chi phí đi lại. Khi đó:

$$\text{minCost}[A] = \min\{\text{purchaseCost}[i, B] + \text{minCost}[A \setminus B] : i \in N, B \subseteq A, B \neq \emptyset\}. \quad (2)$$

Trong công thức này, i là kho được ghé cuối cùng, còn B là tập sản phẩm mua tại kho đó. Công thức cũng xét cả những kế hoạch ghé một kho nhiều lần (kho i có thể đã xuất hiện khi tính kết quả cho $A \setminus B$), nhưng làm vậy chắc chắn không có lợi.

Như vậy ta đã có một lời giải đầu tiên, dù còn khá trừu tượng. Trước khi bàn đến chi tiết cài đặt, hãy ước lượng sơ bộ độ phức tạp. Trong công thức (2), có 2^m khả năng cho A ; với mỗi tập như vậy, ta xét n kho cùng không quá $2^{|A|} \leq 2^m$ tập con sản phẩm. Cận đơn giản là $O(4^m n)$, chưa đủ tốt. Tuy nhiên, từ chính công thức (2), ta có thể xây dựng một thuật toán nhanh hơn đáng kể.

36.2.2 Biểu diễn tập con

Ta biểu diễn các tập con của M bằng những số nguyên từ 0 đến $2^m - 1$. Mỗi số trong khoảng này được viết bằng đúng m bit. Bit thứ i của mã tập A bật khi và chỉ khi $i \in A$.

Với cách biểu diễn này, các phép toán bit trên mã số tương ứng với những phép toán tập hợp cơ bản. Chẳng hạn, giao của hai tập được tính bằng phép and trên hai mã. Nếu một tập là tập con của tập kia, hiệu của chúng tương ứng với phép trừ thông thường giữa hai mã. Cách biểu diễn này cũng cho phép đánh chỉ số hiệu quả cho các mảng trong lời giải, đặc biệt là mảng `minCost`.

Khi điền mảng `minCost`, lúc xét A ta phải có sẵn kết quả của mọi tập con của A . Ta có thể xử lý các tập theo số phần tử tăng dần, nhưng với cách mã hóa trên còn có một cách đơn giản hơn: nếu $B \subseteq A$, mã của B không lớn hơn mã của A . Vì vậy chỉ cần duyệt các tập theo mã tăng dần, từ 0 đến $2^m - 1$.

Từ đây về sau, để đơn giản, ta đồng nhất một tập với mã của nó.

36.2.3 Các tập con của một tập con

Trong phân tích trên, ta chặn thời gian tính một giá trị $\text{minCost}[A]$ bởi $O(2^m n)$, trong khi con số thực sự “chỉ” là $O(2^{|A|} n)$. Sự khác biệt này hóa ra rất quan trọng.

Thuật toán duyệt mọi tập con A của M , rồi với mỗi A , duyệt mọi tập con không rỗng B của nó. Có thể nhìn quá trình này như việc duyệt mọi cặp (A, B) thỏa

$$B \subseteq A \subseteq M.$$

Ta đếm chính xác số cặp như vậy. Mỗi cặp có thể được mã hóa bằng cách tô mỗi phần tử của $M = \{1, \dots, m\}$ bằng một trong ba màu:

- màu thứ nhất cho các phần tử thuộc B ;
- màu thứ hai cho các phần tử thuộc $A \setminus B$;
- màu thứ ba cho các phần tử thuộc $M \setminus A$.

Vì

$$B \cup (A \setminus B) \cup (M \setminus A) = M$$

và ba tập này đôi một rời nhau, mỗi phần tử nhận đúng một trong ba màu. Mỗi cặp (A, B) sinh ra một cách tô màu khác nhau và ngược lại. Có 3^m cách tô màu các phần tử của M , nên cũng có đúng 3^m cặp cần xét. Nhờ đó, ta thu được một cận tốt hơn đáng kể cho độ phức tạp thời gian.

Tuy nhiên, để cài đặt thực sự đạt độ phức tạp phụ thuộc vào 3^m , thay vì 4^m , ta phải chỉ duyệt những cặp thỏa $B \subseteq A \subseteq M$. Điều này rất đáng kể: 4^{16} lớn hơn bốn tỉ, còn 3^{16} chỉ xấp xỉ bốn mươi ba triệu.

Với một tập A cố định, ta duyệt các tập con theo mã giảm dần. Bắt đầu từ $B = A$; ở mỗi bước, tập con kế tiếp được tính bằng

Trong mã giả dưới đây, setMask là mã của A , còn subsetMask là mã của tập con đang xét.

Pseudocode

```
subsetMask := setMask
while subsetMask > 0 do
    process subsetMask
    subsetMask := (subsetMask - 1) and setMask
```

Chẳng hạn, với $A = \{1, 2, 4, 6\}$, các tập con do thuật toán sinh ra, viết dưới dạng nhị phân và đọc theo từng cột, là:

101011	100011	001011	000011
101010	100010	001010	000010
101001	100001	001001	000001
101000	100000	001000	000000

Hãy giải thích vì sao thuật toán đúng. Rõ ràng, mọi tập sinh ra đều là tập con của A . Vòng lặp không thể chạy vô hạn, vì sau mỗi bước, số B giảm ít nhất 1, còn phép and chỉ có thể làm nó nhỏ hơn nữa. Ta chỉ cần chứng minh rằng không có tập con nào của A bị bỏ sót.

Về trực giác, nếu A gồm một số phần tử liên tiếp bắt đầu từ phần tử 1, phép and hoàn toàn không cần thiết và thuật toán hiển nhiên sinh ra mọi tập con của A . Nếu A có “khoảng trống”, tức tồn tại $x \notin A$ trong khi một phần tử lớn hơn x lại thuộc A , thì sau mỗi vòng lặp, các bit ứng với những khoảng trống ấy trong B sẽ bị xóa. Vì vậy chúng không ảnh hưởng đến điều xảy ra trên những bit ứng với phần tử của A khi trừ 1, nên có thể hoàn toàn bỏ qua chúng.

Ta có thể chứng minh hình thức bằng bổ đề sau.

Bổ đề 1. Giả sử $B \subseteq A$, $B \neq \emptyset$, và đặt

$$B' = (B - 1) \text{ and } A.$$

Khi đó B' là tập con của A có mã lớn nhất nhưng vẫn nhỏ hơn mã của B .

Chứng minh. Viết $A = \{a_1, \dots, a_k\}$, với $a_1 < a_2 < \dots < a_k$. Trước hết, giả sử $a_1 \in B$. Khi đó tập con của A đứng ngay trước B là $B \setminus \{a_1\}$. Nếu $a_1 \notin B$, gọi a_i ($i > 1$) là phần tử nhỏ nhất của B . Khi ấy tập đứng ngay trước B là

$$(B \cup \{a_1, \dots, a_{i-1}\}) \setminus \{a_i\}.$$

Trong cả hai trường hợp, B' đúng là tập đứng ngay trước cần tìm. $\square$

Vậy việc dùng thuật toán hiệu quả để sinh mọi tập con của một tập cho ta lời giải có độ phức tạp thời gian

$$O(3^m n).$$

36.2.4 Một cải tiến nhỏ và lời giải chuẩn

Ta có thể cải thiện thêm bằng cách loại bỏ thừa số n khỏi phân hàm mũ của thuật toán.

Trở lại công thức (2): ta đang xét mọi kho i và mọi tập sản phẩm B có thể mua tại đó. Nhưng với mỗi B , chỉ cần xét một kho duy nhất — kho mà ta có thể mua tập sản phẩm ấy với giá rẻ nhất. Đặt

$$\text{minimumPurchaseCost}[B] = \min\{\text{purchaseCost}[i, B] : i \in N\}. \quad (3)$$

Khi đó công thức (2) trở thành

$$\text{minCost}[A] = \min\{\text{minimumPurchaseCost}[B] + \text{minCost}[A \setminus B] : B \subseteq A, B \neq A\} \quad (4)$$

Tính tất cả giá trị $\text{minCost}[A]$ theo cách này chỉ mất $O(3^m)$ thời gian.

Ta tính trước mọi giá trị $\text{minimumPurchaseCost}[B]$ trực tiếp từ (3) và (1) trong thời gian $O(2^m mn)$. Cũng có thể giảm phần tiền xử lý này xuống $O(2^m n)$ bằng cách tính $\text{purchaseCost}[i, A]$ từ những giá trị đã biết, nhưng điều đó không làm thay đổi hiệu năng tổng thể cần thiết. Những lời giải kiểu này đã được chấp nhận trong kỳ thi.

Do đó, lời giải chuẩn chạy trong

$$O(3^m + 2^m mn)$$

thời gian và dùng $O(2^m)$ bộ nhớ nếu xử lý lần lượt từng kho khi tính `minimumPurchaseCost`.

36.2.5 Lời giải thay thế

Còn có một lời giải tốt hơn về cơ sở hàm mũ: thừa số hàm mũ của nó chỉ là 2^m . Lời giải này cũng dùng quy hoạch động trên các tập con của tập sản phẩm. Lần này ta lưu nhiều trạng thái hơn, đổi lại có thể xét lần lượt từng kho và từng sản phẩm.

Đặt $\text{minCost}[i, A]$ là chi phí nhỏ nhất để mua các sản phẩm thuộc $A \subseteq M$, với điều kiện chỉ được dùng các kho mang số $1, \dots, i$. Tiếp theo, đặt $\text{minCostWithWarehouse}[i, A]$ là chi phí nhỏ nhất với cùng điều kiện, nhưng bắt buộc phải mua hàng tại kho i .

Các giá trị ban đầu là

$$\text{minCost}[0, \emptyset] = 0, \quad \text{minCost}[0, A] = \infty \quad \text{với } A \neq \emptyset.$$

Mục tiêu là tính $\text{minCost}[n, M]$. Với mỗi kho $i = 1, \dots, n$, trước tiên ta tính các giá trị $\text{minCostWithWarehouse}[i, \cdot]$ từ $\text{minCost}[i-1, \cdot]$, rồi tính

$$\text{minCost}[i, A] = \min(\text{minCost}[i-1, A], \text{minCostWithWarehouse}[i, A]).$$

Công thức nói rằng nếu muốn mua tập A tại các kho $1, \dots, i$, hoặc ta không đến kho i và trả $\text{minCost}[i-1, A]$, hoặc ta có đến kho ấy và trả $\text{minCostWithWarehouse}[i, A]$.

Ta còn phải tính $\text{minCostWithWarehouse}[i, \cdot]$. Nhắc lại rằng đây là chi phí tối ưu của một kế hoạch có ghé kho i , và có thể ghé cả những kho khác. Giả sử ta mua sản phẩm $j \in A$ tại kho i . Với sản phẩm này — nói chung là với mỗi sản phẩm mua tại kho i — ta có lựa chọn chuyển trạng thái

$$\text{minCostWithWarehouse}[i, A] = \text{minCostWithWarehouse}[i, A \setminus \{j\}] + c_{i,j}.$$

Vì chưa biết sản phẩm nào được mua tại kho i , ta chọn khả năng tốt nhất:

$$\text{minCostWithWarehouse}[i, A] := \min\{\text{minCostWithWarehouse}[i, A \setminus \{j\}] + c_{i,j}\}$$

Cần bổ sung trường hợp cơ sở: ta đi đến kho i , trả phí đi lại, nhưng chưa mua sản phẩm nào ở đó. Khi ấy

$$\text{minCostWithWarehouse}[i, A] = \text{minCost}[i - 1, A] + d_i.$$

Gộp hai khả năng trên nghĩa là trước tiên gán giá trị cơ sở, rồi lần lượt giảm nó bằng các chuyển trạng thái theo từng sản phẩm. Sau đây là mã giả đầy đủ. Ta đồng nhất các tập con của M với mã từ 0 đến $2^m - 1$. Với sản phẩm $j \in \{1, \dots, m\}$, bit của nó có giá trị 2^{j-1} .

Trong mã giả, $\text{travelCost}[\text{warehouse}]$ và $\text{price}[\text{warehouse}, \text{product}]$ lần lượt tương ứng với d_i và $c_{i,j}$.

Pseudocode

```

minCost[0, 0] := 0
for productMask := 1 to 2^m - 1 do
    minCost[0, productMask] := infinity

for warehouse := 1 to n do
    for productMask := 0 to 2^m - 1 do
        minCostWithWarehouse[warehouse, productMask] :=
            minCost[warehouse - 1, productMask] +
            travelCost[warehouse]
        for product := 1 to m do
            productBit := 2^(product - 1)
            if (productMask and productBit) != 0 then
                minCostWithWarehouse[warehouse, productMask] := min(
                    minCostWithWarehouse[warehouse, productMask],
                    minCostWithWarehouse[warehouse, productMask -
                        productBit]
                    + price[warehouse, product]
                )
        for productMask := 0 to 2^m - 1 do
            minCost[warehouse, productMask] := min(
                minCost[warehouse - 1, productMask],

```

```

        minCostWithWarehouse[warehouse, productMask]
    )
return minCost[n, 2^m - 1]

```

Độ phức tạp thời gian của lời giải này hiển nhiên là

$$O(2^m mn).$$

Nếu lưu toàn bộ bảng, độ phức tạp bộ nhớ là $O(2^m n)$, nhưng rất dễ giảm về $O(2^m)$. Tại mỗi bước, ta chỉ dùng các giá trị $\text{minCost}[i, \cdot]$ của hai chỉ số i liên tiếp, và chỉ một lớp i của bảng $\text{minCostWithWarehouse}$. Vì thế, chỉ cần một bảng gồm hai hàng và có thể thay mọi truy cập $\text{minCost}[i, A]$ bằng $\text{minCost}[i \bmod 2, A]$.

Ta còn có thể tiết kiệm thêm bộ nhớ bằng cách không tạo riêng mảng $\text{minCostWithWarehouse}$. Nếu thay mọi tham chiếu đến mảng này trong mã giả bằng tham chiếu đến mảng minCost , đồng thời cập nhật đúng theo thứ tự mã A tăng dần, thuật toán vẫn đúng. Việc kiểm chứng chi tiết được dành cho bạn đọc.

Ước số ★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 256 MB

Mã bài: dzi

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/dzi>

37.1 Đề bài

Cho một dãy gồm n số nguyên

$$a_1, a_2, \dots, a_n.$$

Hãy xác định số cặp có thứ tự (i, j) sao cho

$$i, j \in \{1, \dots, n\}, \quad i \neq j,$$

và a_i là ước của a_j .

37.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n

$$1 \leq n \leq 2\,000\,000.$$

Dòng thứ hai chứa dãy n số nguyên

$$a_1, a_2, \dots, a_n, \quad 1 \leq a_i \leq 2\,000\,000.$$

37.1.2 Dữ liệu ra

In một số nguyên duy nhất: số cặp cần tìm.

37.1.3 Ví dụ

Dữ liệu vào

5
2 4 5 2 6

Dữ liệu ra

6

Có sáu cặp thỏa mãn:

$$(1, 2), (1, 4), (1, 5), (4, 1), (4, 2), (4, 5).$$

37.2 Lời giải

Trước hết, hãy xét bài toán cho dãy

$$1, 2, \dots, M.$$

Ta muốn đếm số cặp số (i, j) sao cho

$$1 \leq i, j \leq M,$$

i là ước của j , và $i \neq j$. Khi đó nhất thiết $i < j$. Chẳng hạn, với $M = 9$, có 14 cặp như vậy:

$$(1, 2), (1, 3), (1, 4), (1, 5), (1, 6), (1, 7), (1, 8), (1, 9), \\ (2, 4), (2, 6), (2, 8), (3, 6), (3, 9), (4, 8).$$

Để duyệt tất cả các cặp này, với mỗi số i từ 1 đến M , ta duyệt những bội của i lớn hơn i nhưng không vượt quá M . Với một giá trị cố định $1 \leq i \leq M$, có đúng

$$\left\lfloor \frac{M}{i} \right\rfloor - 1$$

bội như vậy. Ta trừ 1 vì chỉ quan tâm đến các bội lớn hơn i , nên phải loại chính i .

Tổng số cặp được duyệt có thể chặn trên như sau:

$$\sum_{i=1}^M \left(\left\lfloor \frac{M}{i} \right\rfloor - 1 \right) \leq \sum_{i=1}^M \frac{M}{i} = M \sum_{i=1}^M \frac{1}{i} = O(M \log M).$$

Ở bước cuối, ta dùng một tính chất quan trọng: tổng $\sum_{i=1}^M \frac{1}{i}$ rất gần $\log M$, trong đó $\log$ là lôgarit tự nhiên. “Rất gần” ở đây có nghĩa là với mọi M , độ chênh lệch giữa tổng này và $\log M$ không vượt quá 1.

Ý chính của lập luận trên là như sau. Gọi S là tập các cặp (i, j) thỏa

$$1 \leq i < j \leq M$$

và i là ước của j . Ta đã chứng minh rằng S tương đối nhỏ:

$$|S| = O(M \log M).$$

Hơn nữa, có thể duyệt hiệu quả mọi phần tử của S : với mỗi i , chỉ cần sinh lần lượt các bội của i không vượt quá M .

Từ đây chỉ còn một bước ngắn để giải bài toán cho dãy tổng quát $a_1, \dots, a_n$. Mỗi cặp phần tử của dãy được tính vào đáp án đều ứng

với một cặp giá trị trong S , ngoại trừ trường hợp hai giá trị bằng nhau mà ta sẽ xử lý riêng. Vì vậy, ta có thể đảo ngược cách nhìn: duyệt mọi cặp $(i, j) \in S$, rồi với mỗi cặp, tính xem có bao nhiêu cặp phần tử của dãy $a_1, \dots, a_n$ mang hai giá trị (i, j) .

Ví dụ, giả sử đang xét $(3, 12) \in S$. Nếu số 3 xuất hiện 5 lần trong dãy còn số 12 xuất hiện 7 lần, thì có đúng

$$5 \cdot 7 = 35$$

cặp chỉ số tương ứng. Vì vậy, ta cộng 35 vào đáp án. Để thực hiện việc này hiệu quả, trước hết cần đếm số lần xuất hiện của từng giá trị từ 1 đến M trong dãy.

37.2.1 Thuật toán

Từ đây, đặt

$$M = \max(a_1, a_2, \dots, a_n).$$

Ta xây dựng mảng frequency, trong đó $\text{frequency}[i]$ là số phần tử của dãy có giá trị bằng i .

Tiếp theo, duyệt lần lượt các ô của mảng frequency. Mỗi khi gặp $\text{frequency}[i] \neq 0$, ta duyệt các bội k của i sao cho

$$i < k \leq M.$$

Với mỗi k như vậy, i là ước của k , nên cộng

$$\text{frequency}[i] \cdot \text{frequency}[k]$$

vào đáp án.

Cuối cùng, cần tính cả những cặp phần tử có giá trị bằng nhau. Vì mọi số dương đều là ước của chính nó, $\text{frequency}[i]$ phần tử có giá trị i tạo ra

$$\text{frequency}[i](\text{frequency}[i] - 1)$$

cặp có thứ tự gồm hai chỉ số khác nhau. Ta cộng đại lượng này vào đáp án với mọi i .

Ghi chú hiệu đính: Bản tiếng Ba Lan in công thức $\text{frequency}[i](\text{frequency}[i] - 1)/2$, vốn chỉ đếm cặp không thứ tự. Đề bài yêu cầu cặp có thứ tự; công thức đúng phải là $\text{frequency}[i](\text{frequency}[i] - 1)$. Ví dụ của đề cũng xác nhận cách đếm này: hai phần tử bằng 2 tạo cả hai cặp $(1, 4)$ và $(4, 1)$.

Có thể mô tả phần đếm chính bằng mã giả:

Trong mã giả, maxValue là M , còn $\text{frequency}[\text{value}]$ là số lần giá trị value xuất hiện trong dãy.

Pseudocode

```
answer = 0
for divisor = 1 .. maxValue:
    answer += frequency[divisor] * (frequency[divisor] - 1)
    for multiple = 2*divisor, 3*divisor, ... while multiple <=
        maxValue:
            answer += frequency[divisor] * frequency[multiple]
return answer
```

Vì đáp án có thể đạt cỡ $n(n - 1)$, cần dùng kiểu số nguyên 64 bit.

37.2.2 Chứng minh tính đúng đắn

Ta chia mọi cặp chỉ số hợp lệ (p, q) thành hai loại.

- Nếu $a_p = a_q = i$, cặp đó được tính trong hạng

$$\text{frequency}[i](\text{frequency}[i] - 1).$$

Hạng này đếm đúng mọi cách chọn có thứ tự hai vị trí khác nhau mang giá trị i .

- Nếu $a_p \neq a_q$, đặt $i = a_p$ và $k = a_q$. Vì $i \mid k$ và hai số dương khác nhau, ta có $i < k$; do đó k xuất hiện đúng một lần trong vòng duyệt các bội $2i, 3i, \dots$. Thuật toán cộng $\text{frequency}[i] \text{frequency}[k]$, chính là số cặp chỉ số có thứ tự mang hai giá trị (i, k) .

Hai loại không giao nhau, và mỗi cặp hợp lệ được tính đúng một lần. Ngược lại, mọi cặp mà thuật toán tính đều gồm hai giá trị bằng nhau hoặc gồm i và một bội k của i , nên đều thỏa điều kiện đề bài. Vì vậy, thuật toán trả về đúng đáp án.

37.2.3 Độ phức tạp

Việc đọc dãy và đếm tần suất tốn $O(n + M)$ thời gian, tính cả khởi tạo mảng. Phân duyệt các bội tốn

$$O(M \log M)$$

thời gian theo phân tích ở trên. Do đó, tổng thời gian là

$$O(n + M \log M),$$

và bộ nhớ phụ là

$$O(M).$$

37.3 Lời kết

Chìa khóa của bài Ước số là tính chất

$$\sum_{i=1}^n \frac{1}{i} = O(\log n).$$

Đây là một tính chất đáng nhớ vì nó hữu ích trong nhiều tình huống, ngay cả khi bài toán không trực tiếp nói về ước số hay lý thuyết số. Một ví dụ là **bài toán người sưu tập phiếu thưởng**.

Người sưu tập tìm các nhãn dán hình siêu anh hùng trong những hộp ngũ cốc; trong phiên bản cổ điển, đó là các phiếu thưởng. Mỗi hộp chứa đúng một nhãn dán, được chọn ngẫu nhiên đều trong n loại có thể có. Trung bình người ấy phải mở bao nhiêu hộp để thu thập đủ cả n loại?

Người sưu tập nhận nhãn đầu tiên ngay khi mở hộp đầu tiên. Trái lại, khi đã có $n - 1$ loại, trung bình phải mở thêm n hộp mới gặp loại cuối cùng còn thiếu.

Tổng quát, nếu đã có $i - 1$ loại khác nhau, xác suất để hộp kế tiếp chứa một loại mới là

$$\frac{n - i + 1}{n}.$$

Vì vậy, số hộp trung bình cần mở để nhận thêm một loại mới là

$$\frac{n}{n - i + 1}.$$

Cộng các kỳ vọng này với mọi i , ta được

$$\sum_{i=1}^n \frac{n}{n - i + 1} = n \sum_{i=1}^n \frac{1}{n - i + 1} = n \sum_{i=1}^n \frac{1}{i} = O(n \log n).$$

Do đó, trung bình người sưu tập phải mua $O(n \log n)$ hộp ngũ cốc trước khi có đủ cả n loại nhãn dán.

Nim Euclid ★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Adam Karczmarz

Giới hạn bộ nhớ: 256 MB

Mã bài: euk

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/euk>

38.1 Đề bài

Euclid và Pythagoras là biệt danh của hai người Byteotia nổi tiếng vì niềm say mê những câu đố toán học. Gần đây, họ thường dành buổi tối để chơi trò chơi sau.

Trên bàn có một đồng gồm n viên đá. Hai người lần lượt thực hiện nước đi. Trong lượt của Euclid:

- nếu đồng có ít nhất p viên đá, anh lấy đi một bội dương tùy ý của p , nhưng không được lấy nhiều hơn số đá đang có;
- nếu đồng có ít hơn p viên đá, anh phải thêm đúng p viên vào đồng.

Nước đi của Pythagoras cũng tương tự, nhưng dùng số q : anh lấy đi một bội dương tùy ý của q , hoặc thêm đúng q viên khi đồng có ít hơn q viên.

Người làm đồng đá trở thành rồng sẽ thắng. Euclid đi trước.

Hai người muốn biết liệu mình đã hiểu trò chơi này một cách hoàn hảo hay chưa. Hãy viết chương trình xác định kết quả của ván đấu khi cả hai đều chơi tối ưu.

38.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên t

$$1 \leq t \leq 1000,$$

là số bộ test. Mỗi bộ test gồm một dòng chứa ba số nguyên p, q, n

$$1 \leq p, q, n \leq 10^9.$$

38.1.2 Dữ liệu ra

In đúng t dòng, mỗi dòng là đáp án cho một bộ test. Đáp án là một trong ba chữ cái:

- E nếu Euclid có thể bảo đảm chiến thắng, bất kể Pythagoras chơi thế nào;
- P nếu Pythagoras có thể bảo đảm chiến thắng, bất kể Euclid chơi thế nào;
- R nếu ván đấu sẽ kéo dài vô hạn, tức là hòa.

38.1.3 Ví dụ

Dữ liệu vào

```
4
3 2 1
2 3 1
3 4 5
2 4 3
```

Dữ liệu ra

P
P
E
R

Giải thích. Trong bộ test đầu tiên, ở lượt đầu Euclid buộc phải thêm 3 viên đá. Khi đó Pythagoras có thể lấy cả 4 viên và giành chiến thắng.

38.2 Lời giải

Đặt

$$d = \gcd(p, q).$$

Trước hết, nhận xét rằng mọi nước đi của cả hai người, dù thêm hay lấy đá, đều thay đổi số đá trong đồng một lượng là bội của d . Vì vậy, số dư

$$r = n \bmod d$$

không đổi trong suốt ván đấu. Nếu ban đầu $r \neq 0$, không người nào có thể làm đồng đá trở thành rỗng, nên ván đấu kéo dài vô hạn. Như ta sẽ thấy, đây là trường hợp hòa duy nhất.

Từ giờ, giả sử $r = 0$. Trong suốt ván đấu, số đá luôn chia hết cho d . Ta có thể chia cả n, p, q cho d ; phép biến đổi này không làm thay đổi kết quả, đồng thời khiến p và q nguyên tố cùng nhau.

Nếu sau khi chia mà $p = q$, do hai số nguyên tố cùng nhau nên nhất thiết

$$p = q = 1.$$

Người đi trước hiển nhiên thắng bằng cách lấy hết đá ngay trong lượt đầu. Do Euclid đi trước, đáp án là E.

Ta còn phải xử lý trường hợp

$$\gcd(p, q) = 1, \quad p \neq q.$$

Để phân tích thuận tiện, tạm quên tên hai người và ai là người đi trước. Ta gọi người có thể lấy một số đá chia hết cho p là người chơi P , còn người kia là người chơi Q . Ta xét riêng trường hợp P đi trước và Q đi trước. Nhờ vậy, không mất tính tổng quát, ta có thể giả sử

$$p < q.$$

38.2.1 Phân tích các trường hợp

Lời giải dựa trên bốn trường hợp; mỗi trường hợp sau sẽ được quy về những trường hợp trước.

Trường hợp A: Q đi trước và $n < q$

Ta sẽ chứng minh rằng P có thể thắng bằng cách luôn buộc Q phải thêm q viên đá.

Trong lượt đầu, Q buộc phải thêm q viên. Sau đó P lấy số đá lớn nhất được phép để trong đồng còn

$$(n + q) \bmod p < p < q$$

viên. Nếu số dư bằng 0, P thắng ngay. Nếu không, Q lại buộc phải thêm q viên.

Tiếp tục như vậy, Q luôn chỉ có thể thêm q viên nên chắc chắn không thể làm rỗng đồng. Sau lượt thứ k của P , trong đồng còn

$$(n + kq) \bmod p$$

viên, miễn là ván đấu chưa kết thúc.

Ta chỉ cần chứng minh tồn tại $k \geq 1$ sao cho

$$n + kq \equiv 0 \pmod{p}.$$

Vì $\gcd(p, q) = 1$, thuật toán Euclid mở rộng tìm được các số nguyên ℓ_0, k_0 thỏa

$$\ell_0 p + k_0 q = 1,$$

hay

$$k_0 q \equiv 1 \pmod{p}.$$

Chọn

$$k = (-nk_0) \bmod p + p.$$

Khi đó $k \geq p \geq 1$, và

$$n + kq \equiv n - nk_0 q + pq \equiv 0 \pmod{p}.$$

Vì vậy, ván đầu cuối cùng sẽ kết thúc với chiến thắng của P .

Trường hợp B: P đi trước và $n \geq p$

P có thể thắng. Nếu $p \mid n$, người này lấy hết đá ngay ở lượt đầu. Nếu không, P để lại

$$n \bmod p < p < q$$

viên và đưa trò chơi về trường hợp A, trong đó đối thủ Q là người đi tiếp.

Trường hợp C: P đi trước và $n < p$

Ta sẽ chứng minh rằng P thắng khi và chỉ khi

$$(q - p) \nmid n.$$

Trong lượt đầu, P buộc phải thêm p viên. Nếu $n + p < q$, nước đi này đưa trò chơi về trường hợp A và P thắng.

Nếu không, ta có

$$q \leq n + p < 2q,$$

nên Q buộc phải lấy đúng q viên. Sau hai lượt, đồng còn $n + p - q$ viên. Nếu số này bằng 0, Q thắng. Nếu lượt thứ ba đưa trò chơi về trường hợp A thì P thắng; nếu không, sau lượt thứ tư còn

$$n - 2(q - p)$$

viên.

Lặp lại lập luận này. Nếu coi P là người thắng ngay khi sau lượt của mình đồng còn ít hơn q viên, theo trường hợp A, thì mọi nước đi tiếp theo đều được xác định duy nhất: P luôn thêm p viên và Q luôn lấy q viên. Sau lượt thứ $2k$, đồng còn

$$n - k(q - p)$$

viên; sau lượt thứ $2k + 1$, đồng còn

$$n - k(q - p) + p$$

viên, miễn là ván đấu chưa kết thúc.

Đặt

$$\ell = \left\lfloor \frac{n}{q - p} \right\rfloor.$$

Người chơi Q không thể thắng trước lượt thứ 2ℓ , bởi với mọi số nguyên $\ell' < \ell$,

$$n - \ell'(q - p) > n - \ell(q - p) \geq 0.$$

Tương tự, P không thể thắng trước lượt thứ $2\ell + 1$, vì

$$\begin{aligned} n - \ell'(q - p) + p &\geq n - (\ell - 1)(q - p) + p \\ &= n - \ell(q - p) + q \\ &\geq q. \end{aligned}$$

Nếu $q - p \mid n$, Q làm rỗng đồng ở lượt thứ 2ℓ và thắng; ta vừa chứng minh rằng P không thể thắng trước đó.

Ngược lại, nếu $q - p \nmid n$, viết

$$n = \ell(q - p) + r, \quad 0 < r < q - p.$$

Sau lượt thứ $2\ell + 1$, số đá còn lại là

$$\begin{aligned} n - \ell(q - p) + p &= r + p \\ &< (q - p) + p = q. \end{aligned}$$

Do đó P thắng theo trường hợp A. Q không thể làm rỗng đồng sớm hơn vì $q - p$ không chia hết n .

Trường hợp D: Q đi trước và $n \geq q$

Trong trường hợp này, P thua khi và chỉ khi số dư

$$r = n \bmod q$$

đồng thời thỏa

$$r < p \quad \text{và} \quad (q - p) \mid r.$$

Viết

$$n = kq + r, \quad 0 \leq r < q.$$

Nếu Q lấy ít hơn kq viên, đồng còn ít nhất

$$q + r > p$$

viên. Nước đi đó đưa trò chơi về trường hợp B, nên P thắng. Vì vậy, để có cơ hội thắng, Q buộc phải để lại đúng r viên.

Nếu $r \geq p$, ta lại có trường hợp B và P thắng. Nếu $r < p$, ta rơi vào trường hợp C: P thắng khi và chỉ khi $q - p$ không chia hết r . Điều này chứng minh điều kiện đã nêu.

38.2.2 Thuật toán

Với mỗi bộ test có các giá trị ban đầu p_E, q_P, n , trong đó p_E là tham số của Euclid và q_P là tham số của Pythagoras:

1. Tính $d = \gcd(p_E, q_P)$. Nếu $d \nmid n$, in R.
2. Chia p_E, q_P, n cho d .
3. Nếu $p_E = q_P$, in E.
4. Nếu $p_E < q_P$, Euclid chính là người chơi P và đi trước:
 - nếu $n \geq p_E$, in E theo trường hợp B;
 - nếu $n < p_E$, in E khi $(q_P - p_E) \nmid n$, ngược lại in P, theo trường hợp C.
5. Nếu $p_E > q_P$, Pythagoras là người chơi P , Euclid là người chơi Q , và Q đi trước. Đặt $p = q_P, q = p_E$:
 - nếu $n < q$, in P theo trường hợp A;
 - nếu $n \geq q$, đặt $r = n \bmod q$. In E khi $r < p$ và $(q - p) \mid r$; ngược lại in P, theo trường hợp D.

Có thể viết phần quyết định bằng mã giả:

Trong mã giả, euclidParameter, pythagorasParameter và pileSize lần lượt là p_E, q_P và n .

Pseudocode

```
d = gcd(euclidParameter, pythagorasParameter)
if pileSize mod d != 0:
    answer = 'R'
else:
    euclidParameter = euclidParameter / d
    pythagorasParameter = pythagorasParameter / d
    pileSize = pileSize / d

    if euclidParameter == pythagorasParameter:
        answer = 'E'
    else if euclidParameter < pythagorasParameter:
        if pileSize >= euclidParameter or
           pileSize mod (pythagorasParameter - euclidParameter) != 0:
            answer = 'E'
        else:
```

```

        answer = 'P'
    else:
        smallerParameter = pythagorasParameter
        largerParameter = euclidParameter
        if pileSize < largerParameter:
            answer = 'P'
        else:
            remainder = pileSize mod largerParameter
            if remainder < smallerParameter and
                remainder mod (largerParameter - smallerParameter) ==
                0:
                answer = 'E'
            else:
                answer = 'P'

    return answer

```

38.2.3 Chứng minh tính đúng đắn

Nếu $d = \gcd(p_E, q_P)$ không chia hết n , số đá luôn đồng dư với n theo môđun d , nên không thể bằng 0; thuật toán trả về R là đúng.

Nếu $d \mid n$, việc chia cả ba giá trị cho d bảo toàn chính xác mọi nước đi và điều kiện thắng. Sau phép chuẩn hóa, hai tham số nguyên tố cùng nhau. Khi chúng bằng nhau, cả hai đều bằng 1, nên Euclid thắng ngay.

Khi hai tham số khác nhau, ta gán người sở hữu tham số nhỏ hơn là P , người còn lại là Q . Bốn trường hợp A–D bao phủ mọi trạng thái ban đầu theo hai tiêu chí: người đi trước là P hay Q , và số đá nhỏ hơn hay không nhỏ hơn tham số của người đi trước. Các chứng minh ở trên xác định chính xác người thắng trong từng trường hợp. Thuật toán chỉ việc ánh xạ kết quả đó trở lại tên Euclid và Pythagoras, nên trả về đúng kết quả của ván đấu.

38.2.4 Độ phức tạp

Với mỗi bộ test, tính

$$d = \gcd(p, q)$$

bằng thuật toán Euclid tồn

$$O(\log \min(p, q))$$

thời gian. Mọi phép kiểm tra còn lại đều tốn $O(1)$. Thuật toán dùng $O(1)$ bộ nhớ phụ cho mỗi bộ test.

Cột trụ ★★★★★

Tác giả đề: Jakub Radoszewski, Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 256 MB

Mã bài: fil

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/fil>

39.1 Đề bài

Bajtazar là người quản lý một nhà kho lớn. Dự đoán mùa đông sắp tới sẽ rất khắc nghiệt, anh quyết định lắp hệ thống sưởi dưới sàn nhà kho.

Mặt bằng nhà kho là một hình chữ nhật có kích thước chẵn $n \times m$, được chia thành các ô vuông đơn vị. Phần lớn các ô là diện tích kho, nhưng một số ô bị những cột trụ lớn chiếm chỗ; các cột trụ giúp gia cố kết cấu nhà kho. Trên mặt bằng, mỗi cột trụ chiếm một hình vuông 2×2 , gồm bốn ô vuông đơn vị. Các cột không nằm quá sát nhau: khoảng cách Euclid giữa tâm của hai cột bất kỳ ít nhất là 6. Ngoài ra, tâm mỗi cột cách mỗi bức tường ngoài của nhà kho ít nhất 3 đơn vị.

Hệ thống sưởi sử dụng một ống dẫn nhiệt duy nhất đặt dưới sàn. Ống phải đi qua tâm của tất cả các ô vuông đơn vị, trừ những ô bị

cột trụ chiếm. Mỗi đoạn ống phải song song với một trong các bức tường, và ống chỉ được đổi hướng tại tâm ô vuông đơn vị. Điểm đầu và điểm cuối của ống phải trùng nhau. Tại điểm đó, nước đã nguội được dẫn ra ngoài và nước nóng được bơm vào ống.

Bajtazar nhờ bạn lập kế hoạch đường đi của ống dưới sàn. Để hỗ trợ, anh đặt một hệ tọa độ Descartes trên mặt bằng nhà kho: hoành độ thuộc đoạn $[0, n]$, tung độ thuộc đoạn $[0, m]$. Tọa độ tâm của mọi ô vuông đơn vị đều có dạng $k + \frac{1}{2}$, với $k \in \mathbb{N}$.

39.1.1 Dữ liệu vào

Dòng đầu chứa ba số nguyên n, m, f

$$1 \leq n, m \leq 1000,$$

trong đó n và m đều chẵn. Các số này lần lượt biểu thị hai kích thước của nhà kho và số cột trụ.

Mỗi dòng trong f dòng tiếp theo chứa hai số nguyên x_i, y_i

$$0 \leq x_i \leq n, \quad 0 \leq y_i \leq m,$$

là tọa độ tâm của cột trụ thứ i . Dữ liệu bảo đảm các cột thỏa mãn những điều kiện về khoảng cách đã nêu.

39.1.2 Dữ liệu ra

Ở dòng đầu, in một từ TAK (có) hoặc NIE (không), tùy theo có thể lắp hệ thống sưởi đúng yêu cầu của Bajtazar hay không.

Nếu câu trả lời là TAK, dòng thứ hai phải mô tả một phương án đặt ống bằng một chuỗi gồm $nm - 4f$ chữ cái. Quy ước điểm bắt đầu của ống là $(\frac{1}{2}, \frac{1}{2})$. Mỗi đoạn tiếp theo được mã hóa như sau:

- vectơ $[0, 1]$ bằng chữ G;
- vectơ $[0, -1]$ bằng chữ D;
- vectơ $[1, 0]$ bằng chữ P;
- vectơ $[-1, 0]$ bằng chữ L.

chiếm; hai đỉnh được nối bằng cạnh khi hai ô tương ứng có chung một cạnh. Bài toán yêu cầu tìm một **chu trình Hamilton**, tức một chu trình đi qua mỗi đỉnh đúng một lần.

Trong trường hợp tổng quát, nghĩa là khi cần một thuật toán giải được trên đồ thị bất kỳ, tìm chu trình Hamilton là một bài toán NP-đầy đủ. Vì thế, chưa có thuật toán hiệu quả nào được biết đến cho trường hợp tổng quát. Ta phải khai thác dạng đặc biệt của đồ thị trong bài này. Đồ thị ấy là đồ thị phẳng, nhưng chỉ riêng tính phẳng vẫn chưa đủ, vì tìm chu trình Hamilton trên đồ thị phẳng bất kỳ cũng là bài toán NP-đầy đủ.

39.2.1 Chia mặt bằng thành các mảnh

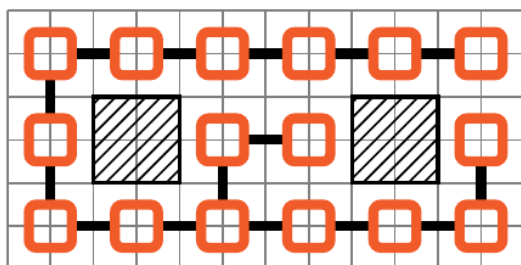
Trước hết, xét một phiên bản đơn giản hơn: khoảng cách từ tâm mỗi cột đến **mỗi** cạnh của hình chữ nhật đều là số lẻ. Nói cách khác, nếu ghép các ô đơn vị thành những mảnh 2×2 , thì mỗi mảnh hoặc bị một cột trụ phủ kín, hoặc hoàn toàn không chứa ô nào thuộc cột trụ.

Trong mỗi mảnh loại thứ hai, ta đặt một chu trình độ dài 4, như ở Hình 1. Tiếp theo, lập một đồ thị mới: mỗi mảnh là một đỉnh, còn hai đỉnh kề nhau khi hai mảnh có chung một cạnh. Tìm một cây khung bất kỳ của đồ thị các mảnh. So sánh Hình 1 với hình trong ví dụ, ta dễ thấy cách dùng các cạnh của cây khung để nối những chu trình nhỏ thành một chu trình duy nhất: với mỗi cạnh cây khung, bỏ hai cạnh song song nằm hai phía của biên chung rồi nối chéo qua biên ấy. Mỗi phép ghép biến hai chu trình thành một mà vẫn giữ bậc của mọi ô bằng 2. Sau khi xử lý toàn bộ cây khung, ta thu được chu trình Hamilton cần tìm.

Thuật toán này chạy trong thời gian

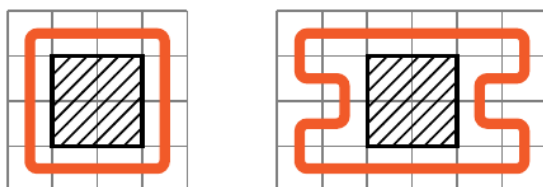
$$O(nm + f).$$

Với bài toán đầy đủ, ta phải cho phép chia hình chữ nhật thành các mảnh kém đều đặn hơn. Nếu khoảng cách từ tâm cột đến các cạnh hình chữ nhật đều là số chẵn, ta đặt cột trong một mảnh 4×4 . Nếu khoảng cách theo hai phương có tính chẵn lẻ khác nhau, ta



Hình 39.2: Hình 1. Nhà kho được chia thành các mảnh 2 nhân 2. Mỗi mảnh không có cột chứa một chu trình bốn ô; các cạnh đen đậm tạo thành cây khung chỉ ra nơi ghép các chu trình nhỏ thành một chu trình lớn.

dùng mảnh 4×6 hoặc 6×4 , như Hình 2. Điều kiện về khoảng cách giữa tâm các cột bảo đảm những mảnh này không chồng lên nhau. Phần còn lại — dựng cây khung và ghép các chu trình — hoàn toàn tương tự.



k 2. Otoczenie filaru cyklem w zależności od

Hình 39.3: Hình 2. Hai cách bao quanh một cột trụ bằng chu trình: cột nằm giữa một mảnh 4 nhân 4, hoặc giữa một mảnh chữ nhật 4 nhân 6; trường hợp 6 nhân 4 là hình quay của trường hợp thứ hai.

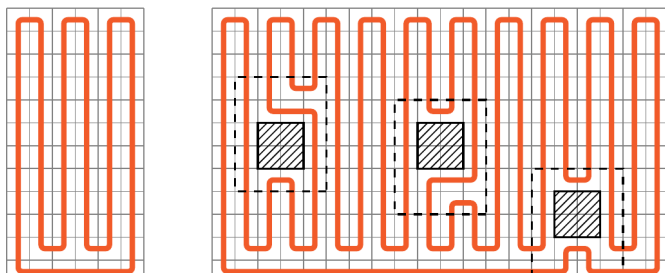
39.2.2 Cách dựng đơn giản hơn

Bài toán còn có một lời giải đơn giản hơn. Đầu tiên, phủ toàn bộ hình chữ nhật bằng một chu trình dạng “lò sưởi” như bên trái Hình 3. Chu trình đi dọc các ô sát cạnh dưới, rồi lần lượt ngoằn ngoèo qua từng cột ô và đổi hướng sau mỗi cột.

Sau đó, lần lượt đặt từng cột trụ vào và sửa chu trình cục bộ sau mỗi lần đặt. Phần bên phải Hình 3 minh họa ba trường hợp cần xét:

- ở trường hợp bên trái, khoảng cách từ tâm cột đến cạnh trái của hình chữ nhật là số lẻ;
- ở hai trường hợp còn lại, khoảng cách ấy là số chẵn;
- trường hợp cuối cùng dành cho cột có tâm cách cạnh dưới đúng 3 đơn vị.

Mỗi phép sửa chỉ thay đổi một đoạn nhỏ của chu trình quanh cột mới: nó chuyển đường ống ra khỏi bốn ô bị cột chiếm và nối lại hai đầu đường đi sao cho kết quả vẫn là một chu trình duy nhất. Nhờ các điều kiện khoảng cách của đề bài, vùng sửa quanh các cột khác nhau không xung đột.



Hình 39.4: Hình 3. Bên trái là chu trình “lò sưởi” ban đầu trên hình chữ nhật không có cột. Bên phải là ba mẫu sửa cục bộ quanh cột trụ, ứng với khoảng cách lẻ tới cạnh trái, khoảng cách chẵn tới cạnh trái, và trường hợp cột cách cạnh dưới đúng 3 đơn vị.

Một điều thú vị là nếu tăng khoảng cách tối thiểu từ tâm cột đến biên hình chữ nhật từ 3 lên 4, lời giải còn đơn giản hơn. Khi đó

không những trường hợp thứ ba biến mất, mà ta còn có thể cài đặt theo một cách khác: đặt tất cả các cột trước, rồi bắt đầu từ góc dưới bên phải và dựng chu trình một cách tham lam. Ở mỗi bước, luôn chọn nước đi khả dụng đầu tiên trong thứ tự:

left, up, down, right

Có thể dễ dàng kiểm tra rằng trên hình chữ nhật ở Hình 3 sau khi bỏ cột dưới cùng, thuật toán sửa cục bộ và thuật toán tham lam tạo ra cùng một kết quả.

39.2.3 Độ phức tạp

Cả cách chia thành các mảnh rồi ghép theo cây khung lẫn cách sửa cục bộ chu trình “lò sưởi” đều chỉ cần xử lý một lượng công việc hằng số cho mỗi ô và mỗi cột. Vì vậy, độ phức tạp thời gian là

$$O(nm + f),$$

và có thể lưu cấu trúc đường ống trong $O(nm)$ bộ nhớ.

Nóng lên toàn cầu ★★ ★

Tác giả đề: Jacek Tomasiewicz, Tomasz Idziaszek

Tác giả lời giải: Tomasz Idziaszek

Giới hạn bộ nhớ: 256 MB

Mã bài: glo

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/glo>

40.1 Đề bài

Giáo sư Byteoni đang chuẩn bị một báo cáo cho Ủy ban Liên Byteotia về Biến đổi Khí hậu. Báo cáo phải chỉ ra một cách rõ ràng, dứt khoát và không còn chút nghi ngờ nào về tác động của người dân Byteotia đối với biến đổi khí hậu trong khu vực. Tuy giáo sư có khá nhiều dữ liệu thực nghiệm, chỉ những lập luận có cơ sở thôi vẫn chưa đủ để thâm nhập vào truyền thông đại chúng; trình bày dữ liệu sao cho thật rõ ràng và đúng trọng tâm cũng quan trọng không kém. Vì vậy, ông muốn cân nhắc kỹ để lựa chọn dữ liệu đưa lên biểu đồ chính của báo cáo.

Biểu đồ chủ chốt sẽ thể hiện nhiệt độ không khí trung bình qua các năm. Giáo sư có số liệu nhiệt độ trung bình hằng năm của n năm gần đây. Ông muốn chú thích biểu đồ bằng một câu đại loại như: “nhiệt độ thấp nhất vào năm $r_{\min}$, còn nhiệt độ cao nhất vào năm

$r_{\max}$, do đó có thể thấy rõ ràng...”. Đáng tiếc, ông lo rằng cùng một nhiệt độ nhỏ nhất hoặc lớn nhất có thể xuất hiện nhiều lần trong giai đoạn ấy, làm cho phát biểu trên kém thuyết phục.

Do đó, giáo sư quyết định chỉ trình bày một phần dữ liệu. Ông muốn chọn một khoảng năm sao cho trong khoảng đó có **đúng một** năm đạt nhiệt độ nhỏ nhất của khoảng và **đúng một** năm đạt nhiệt độ lớn nhất của khoảng. Khoảng được chọn không nhất thiết phải chứa năm có nhiệt độ trung bình nhỏ nhất hay lớn nhất trên toàn bộ n năm; thậm chí có thể không chứa cả hai. Dĩ nhiên, giáo sư muốn đưa lên biểu đồ càng nhiều dữ liệu càng tốt, nên ông quan tâm đến khoảng năm dài nhất.

40.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n

$$1 \leq n \leq 500\,000,$$

là số năm mà giáo sư có dữ liệu nhiệt độ trung bình.

Dòng thứ hai chứa n số nguyên

$$t_1, t_2, \dots, t_n, \quad -10^9 \leq t_i \leq 10^9.$$

Số t_i là nhiệt độ trung bình trong năm thứ i .

40.1.2 Dữ liệu ra

In ra hai số nguyên l và k . Chúng có nghĩa là khoảng dài nhất thỏa mãn điều kiện của giáo sư có độ dài l năm, và k là năm sớm nhất mà một khoảng dài như vậy có thể bắt đầu.

40.1.3 Ví dụ

Dữ liệu vào

10

8 3 2 5 2 3 4 6 3 6

Dữ liệu ra

6 4

Giải thích. Biểu đồ sẽ trình bày các nhiệt độ

5, 2, 3, 4, 6, 3.

Trong khoảng này, nhiệt độ nhỏ nhất 2 xuất hiện đúng một lần và nhiệt độ lớn nhất 6 cũng xuất hiện đúng một lần.

40.2 Lời giải

Cho dãy $t_1, \dots, t_n$. Ta cần tìm dãy con liên tiếp dài nhất mà trong đó cả phần tử nhỏ nhất lẫn phần tử lớn nhất đều xuất hiện đúng một lần. Vì mỗi dãy con liên tiếp được xác định bởi một khoảng chỉ số, từ đây ta sẽ nói về việc tìm một khoảng như vậy.

40.2.1 Cố định vị trí của cực tiểu duy nhất

Cố định một vị trí i , và xét khoảng dài nhất T_i chứa t_i với t_i là cực tiểu duy nhất của khoảng.

Xét tất cả các vị trí có giá trị không lớn hơn t_i . Ngoại trừ chính i , không vị trí nào trong số đó có thể thuộc T_i . Gọi:

- $l_{i,1}$ là vị trí lớn nhất nhỏ hơn i sao cho $t_{l_{i,1}} \leq t_i$;
- $r_{i,1}$ là vị trí nhỏ nhất lớn hơn i sao cho $t_{r_{i,1}} \leq t_i$.

Đề hai vị trí này luôn tồn tại, đặt hai phần tử lân cận

$$t_0 = t_{n+1} = -\infty.$$

Khi đó T_i phải nằm hoàn toàn trong khoảng mở

$$(l_{i,1}, r_{i,1}).$$

Nếu khoảng này chỉ chứa vị trí i , hiển nhiên $T_i = [i, i]$. Trong phần còn lại, giả sử nó chứa ít nhất hai vị trí; khi ấy T_i cũng có ít nhất hai phần tử.

Gọi m_i là giá trị lớn nhất trong khoảng $l_{i,1}, r_{i,1}$. Ta có thể giả sử T_i chứa đúng một lần xuất hiện của m_i . Thật vậy, nếu một khoảng hợp lệ — có cực tiểu duy nhất tại i và cực đại duy nhất — chứa đúng một m_i , ta luôn có thể kéo dài nó mà vẫn giữ nguyên tính hợp lệ cho đến khi nó chứa một lần xuất hiện của m_i .

Các điều kiện vừa nêu cũng đã đủ để bảo đảm trong T_i có đúng một cực tiểu và một cực đại. Vì thế, T_i chính là khoảng dài nhất thỏa cả ba điều kiện:

- chứa vị trí i ;
- nằm trong khoảng mở $l_{i,1}, r_{i,1}$;
- chứa đúng một lần xuất hiện của m_i .

Để dựng T_i , ta xét hai khả năng: lần xuất hiện của m_i được giữ lại nằm bên trái hoặc bên phải i . Với mỗi khả năng, bắt đầu từ đoạn nối vị trí ấy với i , rồi mở rộng tối đa về cả hai phía. Việc mở rộng chỉ dừng ở biên của $l_{i,1}, r_{i,1}$, hoặc ngay trước một lần xuất hiện khác của m_i . Hai khả năng này tạo ra hai ứng viên cho T_i .

40.2.2 Sáu vị trí biên

Gọi $l_{i,2}$ và $l_{i,3}$, với

$$l_{i,3} < l_{i,2},$$

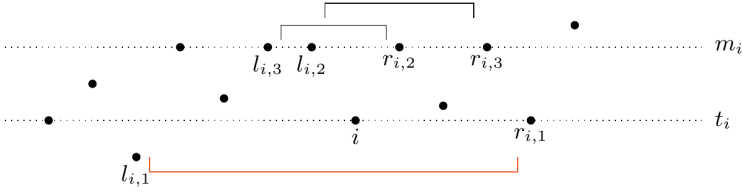
là hai vị trí ngoài cùng bên phải trong $l_{i,1}, i$ có giá trị bằng m_i . Nếu phía trái chỉ có một lần xuất hiện của m_i , đặt $l_{i,3} = l_{i,1}$. Nếu phía trái không có lần nào, đặt cả

$$l_{i,2} = l_{i,3} = l_{i,1}.$$

Định nghĩa đối xứng $r_{i,2}$ và $r_{i,3}$, với

$$r_{i,2} \leq r_{i,3},$$

là hai vị trí ngoài cùng bên trái trong $i, r_{i,1}$ có giá trị bằng m_i . Các vị trí không tồn tại được thay bằng $r_{i,1}$ theo quy ước tương tự.



Hình 40.1: Hình 1. Vị trí i , sáu vị trí biên $l_{i,1}, l_{i,2}, l_{i,3}, r_{i,2}, r_{i,3}, r_{i,1}$ và hai khoảng ứng viên cho T_i . Mỗi phần tử của dãy được biểu diễn bằng một chấm; độ cao của chấm biểu thị giá trị phần tử.

Vì T_i phải chứa i và đúng một lần xuất hiện của m_i , hai ứng viên duy nhất là hai khoảng mở

$$(l_{i,3}, r_{i,2}) \quad \text{và} \quad (l_{i,2}, r_{i,3}).$$

Nói theo chỉ số đóng, chúng lần lượt là

$$[l_{i,3} + 1, r_{i,2} - 1] \quad \text{và} \quad [l_{i,2} + 1, r_{i,3} - 1].$$

Ứng viên thứ nhất giữ lại lần xuất hiện m_i tại $l_{i,2}$, còn ứng viên thứ hai giữ lại lần xuất hiện tại $r_{i,2}$. Ta chọn ứng viên dài hơn làm T_i ; nếu bằng nhau, chọn ứng viên bắt đầu sớm hơn.

40.3 Cài đặt

40.3.1 Tìm $l_{i,1}$ và $r_{i,1}$

Ta có thể tính tất cả $l_{i,1}$ trong tổng thời gian $O(n)$ bằng một ngăn xếp đơn điệu.

Ta nói vị trí j **nhìn thấy được từ bên trái** tại vị trí i nếu

$$j < i, \quad t_j \leq t_i,$$

và mọi phần tử nằm giữa j và i đều không nhỏ hơn t_j . Theo định nghĩa này, $l_{i,1}$ chính là vị trí lớn nhất nhìn thấy được từ bên trái tại i .

Duyệt dãy từ trái sang phải. Sau khi xử lý $t_0, \dots, t_i$, ngăn xếp chứa vị trí i cùng tất cả các vị trí nhìn thấy được từ bên trái tại i , theo thứ tự tăng dần; vị trí lớn nhất nằm trên đỉnh.

Khi chuẩn bị xử lý t_i , loại khỏi đỉnh ngăn xếp mọi vị trí có giá trị lớn hơn t_i . Những vị trí ấy không thể là $l_{i,1}$. Hơn nữa, chúng cũng không thể là $l_{j,1}$ của bất kỳ $j > i$, bởi khi đó vị trí i luôn là một ứng viên gần hơn và tốt hơn. Sau khi loại xong, đỉnh ngăn xếp chính là $l_{i,1}$; cuối cùng đẩy i vào ngăn xếp.

Ta tính $r_{i,1}$ hoàn toàn đối xứng bằng cách duyệt từ phải sang trái.

40.3.2 Tìm bốn vị trí của cực đại

Một cách trực tiếp là dùng cây đoạn hỗ trợ truy vấn vị trí ngoài cùng của giá trị lớn nhất trên một khoảng. Nhờ đó có thể tìm $l_{i,2}, l_{i,3}$ và, bằng một cây đối xứng, $r_{i,2}, r_{i,3}$. Cách này cho thuật toán $O(n \log n)$.

Ta có thể làm đơn giản hơn và đạt tổng thời gian $O(n)$ bằng cách bổ sung thông tin cho ngăn xếp đơn điệu.

Giả sử ngăn xếp hiện chứa

$$a_0 = 0, a_1, \dots, a_s = i,$$

là các vị trí nhìn thấy được từ bên trái tại i . Kèm theo mỗi a_j , lưu một bản tóm tắt A_j của các vị trí đạt giá trị lớn nhất trong phần dãy nằm giữa a_{j-1} và a_j . Ta không cần lưu toàn bộ tập: chỉ cần

- giá trị lớn nhất trong phần ấy;
- hai vị trí ngoài cùng bên phải đạt giá trị lớn nhất đó.

Khi loại a_j khỏi ngăn xếp, phần dãy do A_j đại diện được nhập vào phần kế bên. Vì vậy, cập nhật A_{j-1} bằng cách xét thông tin đang có của A_{j-1} , chính vị trí a_j , và thông tin của A_j . Giữ lại giá trị lớn nhất trong ba nguồn cùng hai vị trí ngoài cùng bên phải đạt giá trị ấy. Mỗi lần nhập như vậy chỉ tốn $O(1)$.

Sau khi đã loại mọi phần tử lớn hơn t_i , bản tóm tắt ở đỉnh cho ta hai vị trí ngoài cùng bên phải đạt cực đại trong $l_{i,1}, i$, tức $l_{i,2}$ và $l_{i,3}$. Vì mỗi vị trí chỉ được đẩy vào và loại khỏi ngăn xếp nhiều nhất một lần, toàn bộ lượt duyệt tốn $O(n)$.

Duyệt đối xứng từ phải sang trái, đồng thời lưu hai vị trí ngoài cùng bên trái, ta thu được $r_{i,2}$ và $r_{i,3}$, cũng trong $O(n)$.

40.3.3 Hiệu chỉnh hai phía

Các cặp bên trái và bên phải được tính độc lập, nên giá trị cực đại tìm được ở hai phía có thể khác nhau. Chỉ cặp ứng với giá trị lớn hơn mới thực sự biểu diễn m_i :

- nếu cực đại bên trái nhỏ hơn cực đại bên phải, thay $l_{i,2}$ và $l_{i,3}$ bằng $l_{i,1}$;
- nếu cực đại bên phải nhỏ hơn cực đại bên trái, thay $r_{i,2}$ và $r_{i,3}$ bằng $r_{i,1}$;
- nếu hai cực đại bằng nhau, giữ nguyên cả hai cặp.

Nếu $l_{i,1}, r_{i,1}$ chỉ chứa i , xử lý riêng khoảng một phần tử $[i, i]$.

40.3.4 Thuật toán hoàn chỉnh

1. Tính $l_{i,j}$ và $r_{i,j}$ cho mọi i và mọi $j \in \{1, 2, 3\}$ bằng hai lượt quét với ngăn xếp tăng cường.
2. Với mỗi i , hiệu chỉnh cặp vị trí ở phía không đạt m_i .
3. Xét hai ứng viên

$$[l_{i,3} + 1, r_{i,2} - 1] \quad \text{và} \quad [l_{i,2} + 1, r_{i,3} - 1].$$

4. Trong mọi ứng viên của mọi i , chọn khoảng dài nhất; nếu có nhiều khoảng cùng độ dài, chọn khoảng có đầu trái nhỏ nhất.
5. Nếu khoảng được chọn là $[L, R]$, in

$$R - L + 1 \quad \text{và} \quad L.$$

Mọi khoảng hợp lệ có một cực tiểu duy nhất tại vị trí nào đó i , nên nó được xét khi xử lý T_i . Ngược lại, hai ứng viên dựng cho mỗi i đều nằm giữa hai phần tử không lớn hơn gần nhất, vì thế t_i là cực tiểu duy nhất; chúng đồng thời chứa đúng một m_i , nên cũng có cực đại duy nhất. Do đó thuật toán không bỏ sót đáp án và không tạo ra ứng viên sai.

40.3.5 Độ phức tạp

Mỗi phần tử được đẩy vào và lấy khỏi mỗi ngăn xếp nhiều nhất một lần, còn mọi cập nhật tóm tắt đều tốn thời gian hằng số. Vì vậy, độ phức tạp là

$$O(n)$$

thời gian và

$$O(n)$$

bộ nhớ.

Bản hit của mùa ★★★★★

Tác giả đề: Tomasz Kociumaka, Jakub Radoszewski, Wojciech Rytter, Tomasz Waleń

Tác giả lời giải: Tomasz Kociumaka

Giới hạn bộ nhớ: 256 MB

Mã bài: hit

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/hit>

41.1 Đề bài

Nhà máy In Byteotia (BPF) vừa nhận một đơn hàng lớn sản xuất giấy dán tường kẻ sọc — xu hướng đang thịnh hành trong thiết kế nội thất mùa này. Mỗi tấm giấy dán tường gồm n sọc dọc có cùng chiều rộng, được tô bằng ba màu đỏ, lục và lam. BPF chịu trách nhiệm cả thiết kế lẫn in ấn. Khách hàng đã chỉ định màu cho một số sọc; với các sọc còn lại, họ để BPF hoàn toàn tự do lựa chọn.

BPF in một số sọc liên tiếp bằng một **khuôn in**. Mỗi vị trí trên khuôn có một màu xác định, và khuôn có thể ngắn hơn toàn bộ tấm giấy. Mỗi lần áp khuôn, các sọc của khuôn phải trùng khít với các sọc của giấy, rồi tất cả các vị trí trên khuôn được in cùng lúc. Vì thế, một sọc trên giấy có thể bị in nhiều lần. Nếu một sọc được in bằng nhiều màu khác nhau, màu cuối cùng của nó sẽ là hỗn hợp các màu ấy. Khuôn chỉ dùng được theo một chiều và không được xoay hay

lật dưới bất kỳ hình thức nào.

Bất kể khiêu thẩm mỹ ra sao, ưu tiên hàng đầu của nhân viên BPF là thiết kế khuôn ngăn nhất có thể mà vẫn in được toàn bộ tấm giấy. Đối với những sọc đã được khách hàng chỉ định, họ phải dùng đúng màu thuần khiết, không pha thêm màu nào khác. Nói cách khác, mỗi lần khuôn phủ lên một sọc đã được chỉ định màu, vị trí tương ứng trên khuôn phải có chính xác màu đó. Không sọc nào trên giấy được phép để không màu.

41.1.1 Dữ liệu vào

Dòng đầu chứa số nguyên t

$$1 \leq t \leq 10,$$

là số bộ test. Mỗi dòng trong t dòng tiếp theo mô tả một bộ test bằng một chuỗi gồm các chữ cái in hoa R, G, B và dấu sao *. Các chữ cái biểu thị màu của sọc tương ứng, còn dấu sao biểu thị sọc mà khách hàng chưa chỉ định màu.

Chuỗi không rỗng, có độ dài không quá 3000, và chứa nhiều nhất 19 dấu sao.

41.1.2 Dữ liệu ra

Với mỗi bộ test, in một dòng chứa chuỗi chỉ gồm các chữ cái R, G, B: một khuôn có độ dài nhỏ nhất cho phép in tấm giấy theo yêu cầu. Nếu có nhiều đáp án đúng, được phép in bất kỳ đáp án nào.

41.1.3 Ví dụ

Dữ liệu vào

```
1
RRG*R*BRR**B
```

Dữ liệu ra

```
RRGB
```

41.2 Lời giải

Trước khi mô tả thuật toán, ta đưa ra một số khái niệm. Đối tượng cơ bản sẽ là **từ bộ phận** (*partial word*): ngoài các chữ cái thông thường — trong bài này là R, G, B — từ còn có thể chứa ký hiệu đại diện *, gọi là **ô trống** (*blank*).

Hai ký tự thuộc tập $\{R, G, B, *\}$ được gọi là **khớp** nếu chúng bằng nhau hoặc ít nhất một trong hai là ô trống. Ký hiệu quan hệ này là $\approx$. Ta mở rộng nó cho hai từ bộ phận có cùng độ dài bằng cách yêu cầu mọi cặp ký tự tương ứng đều khớp nhau.

Chẳng hạn,

$$R*B \approx RG* \quad \text{và} \quad R*B \approx *RB,$$

nhưng

$$RG* \not\approx *RB,$$

vì ở vị trí thứ hai có hai chữ cái khác nhau là G và R. Do đó, $\approx$ không có tính bắc cầu. Tuy nhiên, hai từ bộ phận khớp nhau khi và chỉ khi tồn tại một từ đầy đủ — không chứa ô trống — khớp với cả hai. Ta gọi từ ấy là một **phép điền chung**. Ví dụ, RGB là một phép điền chung của $R*B$ và $RG*$.

Nhờ quan hệ $\approx$, ta dễ dàng phát biểu bài toán tìm mẫu trong văn bản cho từ bộ phận: tìm mọi đoạn của văn bản khớp với mẫu. Tuy nhiên, mục tiêu của bài này là dựng **khuôn phủ** ngắn nhất, tức một mẫu mà các lần xuất hiện của nó phủ kín toàn bộ văn bản — bản thiết kế giấy dán tường. Ta còn yêu cầu khuôn phủ phải là một từ đầy đủ; nếu không, khuôn một ký tự * sẽ luôn là lời giải tầm thường.

Phiên bản bài toán này trên từ thông thường từng xuất hiện tại Olympic Tin học Ba Lan trong bài *Szablon* (vòng 2, kỳ OI thứ 12). Cần lưu ý rằng nếu xét mọi từ đầy đủ khớp với văn bản rồi lấy khuôn phủ ngắn nhất của chúng, ta chưa chắc thu được khuôn phủ ngắn nhất của chính văn bản bộ phận. Ngay ví dụ trong đề đã cho thấy điều đó.

41.2.1 Kiểm tra một khuôn phủ

Trong phần lời giải, các ký tự của từ được đánh số từ 0.

Trước hết, xét cách kiểm tra một mẫu S độ dài m có phải là khuôn phủ của từ bộ phận T độ dài n hay không. Muốn vậy, S phải phủ cả vị trí đầu lẫn vị trí cuối của T , nên nó phải khớp với một tiền tố và một hậu tố của T . Hơn nữa, hai lần xuất hiện liên tiếp của S phải chồng lên nhau hoặc nằm sát nhau; nếu không, các vị trí ở giữa sẽ không được phủ.

Để hình thức hóa các điều kiện ấy, đặt

$$\text{Occ}(S, T)$$

là tập các vị trí bắt đầu của những lần xuất hiện của S trong T . Với

$$A = \{a_1, \dots, a_k\}, \quad a_1 < \dots < a_k,$$

định nghĩa

$$\text{MaxGap}(A) = \max\{a_{i+1} - a_i : i = 1, \dots, k-1\}.$$

Nhận xét 1. Mẫu S là khuôn phủ của từ bộ phận T khi và chỉ khi đồng thời thỏa mãn:

1. $0 \in \text{Occ}(S, T)$;
2. $\text{MaxGap}(\text{Occ}(S, T) \cup \{|T|\}) \leq |S|$.

Vì vậy, chỉ cần tính tập $\text{Occ}(S, T)$, việc có thể làm dễ dàng trong thời gian $O(nm)$, rồi kiểm tra tiêu chuẩn trên.

Cũng tồn tại thuật toán $O(n \log n)$ cho bước này, nhưng như ta sẽ thấy, bài toán hiện tại không cần đến nó. Đây là một bài tập thú vị; có thể bắt đầu bằng trường hợp bảng chữ cái nhị phân.

41.2.2 Lời giải ngây thơ

Trong bài, ta không được cho sẵn một ứng viên S , mà phải tìm khuôn phủ ngắn nhất trong tất cả các từ. Điều kiện 1 của Nhận xét 1 giới hạn các ứng viên ở những từ khớp với một tiền tố nào đó của T .

Có $O(3^k n)$ ứng viên như vậy, trong đó k là số ô trống trong T : mỗi ứng viên được xác định duy nhất bởi độ dài và cách chọn một trong ba chữ cái cho từng ô trống của T . Dùng thuật toán kiểm tra ở trên cho từng ứng viên cho ta lời giải ngây thơ với thời gian

$$O(3^k n^3).$$

41.2.3 Cài đặt hiệu quả hơn

Nhiều ứng viên trong tập trên rất giống nhau. Thay vì tìm từng mẫu hoàn toàn độc lập, ta có thể dùng chung một phần phép tính. Thực tế, ta sẽ tính được tất cả các khuôn phủ trong thời gian về cơ bản tỉ lệ với số ứng viên.

Bước đầu tiên

Cố định một phép điền của T , tức một từ đầy đủ $\bar{T}$ khớp với T . Các ứng viên ứng với phép điền này chính là các tiền tố của $\bar{T}$.

Ta xử lý các tiền tố theo thứ tự độ dài tăng dần và duy trì tập lần xuất hiện Occ . Ở mỗi bước, tập này thu hẹp từ

$$\text{Occ}_{m-1} = \text{Occ}(\bar{T}[0..m-1], T)$$

thành

$$\text{Occ}_m = \text{Occ}(\bar{T}[0..m], T).$$

Ở đây, $\bar{T}[0..m]$ là đoạn gồm các ký tự tại vị trí $0, \dots, m$. Ta chỉ cần kiểm tra những lần xuất hiện trong Occ_{m-1} có thể kéo dài thêm ký tự $\bar{T}[m]$ hay không; việc này tốn thời gian tỉ lệ với kích thước của

Occ_{m-1} . Kiểm tra $\bar{T}[0..m]$ có phải khuôn phủ cũng cần duyệt Occ_m . Do đó, mỗi bước tốn $O(n)$, và toàn bộ thuật toán tốn

$$O(3^k n^2).$$

Mảng Pref

Ta có thể tiếp tục chia sẻ phép tính giữa các phép điền khác nhau. Khi kéo dài mẫu $\bar{T}[0..m-1]$ thành $\bar{T}[0..m]$, nếu $T[m] \neq *$, ký tự được thêm đơn giản là $T[m]$. Một lần xuất hiện tại vị trí cố định

$$i \in \text{Occ}_{m-1}$$

kéo dài được qua $T[m]$ khi và chỉ khi

$$T[i+m] \approx T[m].$$

Điều kiện này không phụ thuộc vào phép điền $\bar{T}$ đang xét, dù việc i có thuộc Occ_{m-1} hay không thì có thể phụ thuộc vào phép điền.

Suy ra, nếu $T[i+p] \approx T[p]$ thì chắc chắn $i \notin \text{Occ}_m$ với mọi $m \geq p$. Vì thế, ta định nghĩa mảng

$$\text{Pref}[i] = \min(\{n-i\} \cup \{p : T[i+p] \approx T[p]\}).$$

Giá trị này là độ dài tiền tố dài nhất của T xuất hiện tại vị trí i trong T . Có thể tính Pref trong $O(nk)$, nhưng thuật toán ngây thơ $O(n^2)$ đã đủ cho bài này. Ta còn cần mảng ngược Pref^{-1} , trong đó $\text{Pref}^{-1}[p]$ lưu mọi vị trí i thỏa $\text{Pref}[i] = p$.

Nếu $T[m] \neq *$, ta có

$$\text{Occ}_m = \text{Occ}_{m-1} \setminus \text{Pref}^{-1}[m],$$

nên có thể cập nhật Occ rất dễ dàng.

Ghi chú hiệu đính: Bản tiếng Ba Lan in $\text{Pref}^{-1}[m-1]$. Với quy ước $\text{Occ}_m = \text{Occ}(\bar{T}[0..m], T)$ ngay phía trên, vị trí bị loại khi lần không khớp đầu tiên nằm tại độ lệch m , nên chỉ số đúng là m .

Ta còn phải dùng Nhận xét 1 để kiểm tra $\bar{T}[0..m]$ có phải khuôn phủ hay không. Điều kiện 1 đã tự động đúng, vì mẫu là tiền tố; chỉ cần kiểm tra điều kiện 2. Vì vậy, ta cần duy trì

$$\text{MaxGap}(\text{Occ} \cup \{|T|\}).$$

Mỗi phép cập nhật chỉ xóa một vị trí khỏi $\text{Occ} \cup \{|T|\}$. Ta không bao giờ xóa 0 hoặc $|T|$, nên MaxGap chỉ có thể tăng. Khi xóa một vị trí, chỉ cần tìm phần tử đứng trước và đứng sau nó, rồi xét hiệu giữa hai phần tử ấy như một ứng viên mới cho MaxGap .

Để làm việc này hiệu quả, ta lưu $\text{Occ} \cup \{|T|\}$ bằng danh sách liên kết đôi, kèm một mảng mà tại vị trí i chứa con trỏ đến nút tương ứng của danh sách — hoặc null nếu i không thuộc tập.

Với một phép điền cố định, mỗi phần tử chỉ bị xóa khỏi Occ một lần, nên tổng chi phí thao tác trên Occ là $O(n)$. Việc liệt kê các vị trí cần xóa bằng Pref^{-1} cũng tốn tổng thời gian tuyến tính.

Riêng khi $T[m] = *$, không thể dùng Pref^{-1} . Khi đó, ta kiểm tra trực tiếp những vị trí i thỏa

$$\bar{T}[m] \approx T[i + m],$$

tổng cộng $O(kn)$ cho một phép điền. Cộng trên mọi phép điền và kể cả thời gian dựng Pref , ta được độ phức tạp

$$O(n^2 + 3^k kn).$$

41.2.4 Thuật toán phân nhánh

Một tối ưu nữa đến từ việc các tiền tố của hai phép điền khác nhau vẫn giống nhau cho tới ô trống đầu tiên được điền bằng hai chữ cái khác nhau. Trước khi m đi đến vị trí đó, ta có thể dùng chung toàn bộ phép tính.

Ta xây dựng một thuật toán kéo dài tiền tố $\bar{T}[0..m-1]$ bằng $T[m]$ nếu đó là chữ cái. Nếu $T[m] = *$, thuật toán phân thành ba nhánh, mỗi nhánh gán một giá trị khác nhau cho $\bar{T}[m]$. Mỗi lần phân nhánh phải sao chép trạng thái thuật toán — tức biểu diễn của $\text{Occ} \cup \{|T|\}$

— trong $O(n)$. Ngoài chi tiết này, mỗi nhánh hoạt động như lời giải trước.

Giữa hai lần phân nhánh liên tiếp, ta chỉ phải cập nhật 0cc theo cách trực tiếp đúng một lần, ngay sau khi phân nhánh. Vì vậy, không kể các lời gọi đệ quy, mỗi thể hiện của thuật toán tốn $O(n)$.

Gọi $C(n, j)$ là tổng thời gian của một thể hiện được gọi sau khi đã điền ô trống thứ j . Khi đó,

$$C(n, j) = \begin{cases} O(n), & j = k, \\ O(n) + 3C(n, j+1), & j < k. \end{cases}$$

Hệ thức này cho

$$C(n, 0) = O(3^k n).$$

Cộng thêm bước tiền xử lý, toàn bộ thuật toán chạy trong

$$O(n^2 + 3^k n).$$

41.2.5 Lời giải chuẩn

Thành phần chi phối trong lời giải hiện tại là $O(3^k n)$, tương ứng với số ứng viên phải kiểm tra. Ta cần thu nhỏ tập ứng viên.

Giả sử biết khuôn phủ cần tìm có độ dài không quá m . Khi đó chỉ có

$$O(3^{k_m} m)$$

ứng viên, trong đó k_m là số ô trống trong m ký tự đầu tiên của văn bản. Thuật toán phân nhánh khi ấy chạy trong

$$O(n^2 + 3^{k_m} n).$$

Bài toán đối xứng khi đảo ngược cả văn bản lẫn mẫu, nên ta cũng có thể định nghĩa k_m là số ô trống trong m ký tự cuối của văn bản.

Với

$$m = \left\lfloor \frac{n}{2} \right\rfloor,$$

ta có thể chọn một trong hai phía sao cho

$$k_m \leq \left\lfloor \frac{k}{2} \right\rfloor,$$

vì ít nhất một nửa của văn bản chứa không quá một nửa tổng số ô trống. Nhờ đó, ta tìm được mọi khuôn phủ có độ dài không quá $\lfloor n/2 \rfloor$ trong thời gian

$$O(n^2 + 3^{k/2}n),$$

đủ nhanh với các giới hạn của đề.

Vẫn có thể xảy ra trường hợp T không có khuôn phủ nào dài không quá $n/2$. May thay, việc kiểm tra một mẫu độ dài $m \geq n/2$ có phải khuôn phủ hay không lại rất đơn giản. Tiền tố và hậu tố độ dài m đã phủ toàn bộ văn bản, nên chỉ cần mẫu khớp với cả hai. Nói cách khác, khuôn phủ độ dài $m \geq n/2$ tồn tại khi và chỉ khi tiền tố và hậu tố độ dài m có một phép điền chung.

Theo nhận xét ở đầu lời giải, điều này tương đương với

$$T[0..m-1] \approx T[n-m..n-1],$$

hay

$$\text{Pref}[n-m] = m.$$

Khi điều kiện đúng, ta dựng phép điền chung một cách trực tiếp. Do đó, dùng mảng Pref , ta có thể tìm khuôn phủ ngắn nhất với $m \geq n/2$ trong thời gian tuyến tính.

Kết hợp với thuật toán cho các khuôn phủ ngắn hơn, lời giải chuẩn có độ phức tạp thời gian

$$O(n^2 + 3^{k/2}n).$$

Độ phức tạp bộ nhớ là

$$O(kn),$$

dù một cách cài đặt phân nhánh cẩn thận hơn có thể giảm xuống $O(n)$.

41.3 Lời kết

Thuật toán trên đủ nhanh cho bài, nhưng với k nhỏ thì vẫn còn xa tối ưu. Thực tế, tồn tại một thuật toán chạy trong

$$O\left(nk^4 + 2^{O(\sqrt{k} \log k)}\right) = O\left(nk^4 + k^{O(\sqrt{k})}\right),$$

ngay cả khi kích thước bảng chữ cái tăng tuyến tính theo n .

Bạn đọc quan tâm có thể xem bài báo *Covering Problems for Partial Words and for Indeterminate Strings* của M. Crochemore, C. S. Iliopoulos và các tác giả của bài toán này: <https://arxiv.org/pdf/1412.3696.pdf>.

Dàn dựng ★★★★★

Tác giả đề: Adam Karczmarz

Tác giả lời giải: Adam Karczmarz

Giới hạn bộ nhớ: 256 MB

Mã bài: ins

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/ins>

42.1 Đề bài

Steven Byteberg là một đạo diễn chuyên làm phim hành động. Hiện ông đang thực hiện một bộ phim mới về cuộc chiến giữa các băng đảng mafia Byteonia. Byteberg vẫn đang cân nhắc diễn biến cuối cùng của trường đoạn cao trào — một màn đấu súng ngoạn mục.

Có n tên gangster tham gia cảnh quay, được đánh số từ 1 đến n . Khi căng thẳng lên đến đỉnh điểm, mỗi tên rút súng và ngắm vào một tên khác. Không có hai tên nào cùng ngắm vào một người. Đám gangster tuy nghèo nhưng được huấn luyện bài bản: mỗi tên chỉ có thể bắn một phát, song phát đạn ấy luôn chính xác và chí mạng.

Đến một lúc nào đó, một tên không thể chịu nổi sức ép nữa và cuộc đấu súng bắt đầu.

Đạo diễn đã quy định thứ tự ban đầu mà các gangster phải bóp cò. Cụ thể, gangster i bắn gangster p_i đúng vào thời điểm t_i , trừ khi

đến lúc đó gangster i đã bị giết. Một gangster chết đúng vào thời điểm có người bắn về phía hắn.

Đạo diễn muốn biết cuối cảnh còn bao nhiêu gangster sống sót. Tuy nhiên, Byteberg vẫn chưa hoàn toàn chắc chắn về thứ tự nổ súng. Thỉnh thoảng ông yêu cầu thay đổi một giá trị t_i . Sau mỗi thay đổi, ông muốn biết số gangster sẽ sống sót theo lịch bắn mới, có tính đến tất cả những thay đổi trước đó.

42.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n

$$2 \leq n \leq 200\,000,$$

là số gangster tham gia cảnh quay.

Dòng thứ hai chứa n số nguyên

$$p_1, p_2, \dots, p_n$$

thỏa mãn

$$1 \leq p_i \leq n, \quad p_i \neq i,$$

và $p_i \neq p_j$ với mọi $i \neq j$. Giá trị p_i cho biết gangster i định bắn ai.

Dòng thứ ba chứa n số nguyên

$$u_1, u_2, \dots, u_n, \quad 1 \leq u_i \leq 10^9,$$

mô tả lịch bắn ban đầu: lúc đầu $t_i = u_i$.

Dòng thứ tư chứa một số nguyên q

$$0 \leq q \leq 200\,000,$$

là số lần thay đổi các giá trị $t_1, \dots, t_n$. Mỗi dòng trong q dòng tiếp theo chứa hai số nguyên k_i, v_i

$$1 \leq k_i \leq n, \quad 1 \leq v_i \leq 10^9,$$

cho biết thay đổi thứ i đặt $t_{k_i} = v_i$.

Tất cả các số

$$u_1, u_2, \dots, u_n, v_1, v_2, \dots, v_q$$

đôi một khác nhau.

42.1.2 Dữ liệu ra

In đúng $q + 1$ dòng. Dòng đầu chứa số gangster sống sót với lịch bắn ban đầu. Dòng thứ i trong q dòng tiếp theo chứa số gangster sống sót khi lịch bắn được xác định bởi $t_1, \dots, t_n$ sau khi thực hiện tất cả các thay đổi từ 1 đến i .

42.1.3 Ví dụ

Dữ liệu vào

```
4
2 3 4 1
1 2 3 4
3
1 8
2 7
3 6
```

Dữ liệu ra

```
2
2
1
1
```

42.2 Lời giải

Trước hết, ta phân tích cách các gangster ngấm bắn nhau. Không có hai gangster nào cùng ngấm vào một người, trong khi mỗi gangster

ngắm đúng một đối thủ. Do đó, mỗi gangster cũng là mục tiêu của đúng một gangster khác. Nói cách khác, ánh xạ $i \mapsto p_i$ là một hoán vị.

Vì $p_i \neq i$, hoán vị này phân rã thành một số chu trình, mỗi chu trình có độ dài ít nhất 2. Cụ thể, với mọi $i \in \{1, \dots, n\}$, tồn tại $m \geq 2$ gangster phân biệt $g_1, \dots, g_m$ sao cho:

- $g_1 = i$;
- với mọi $j \in \{1, \dots, m-1\}$, gangster g_j ngắm vào g_{j+1} ;
- gangster g_m ngắm vào g_1 .

Không có phát súng nào đi từ chu trình này sang chu trình khác. Vì vậy, một lần đổi lịch bắn chỉ có thể làm thay đổi kết quả trong đúng một chu trình. Ta có thể xử lý riêng từng chu trình rồi cộng số người sống sót của chúng. Việc phân rã hoán vị thành các chu trình mất $O(n)$ thời gian.

42.2.1 Một chu trình với lịch bắn cố định

Xét một chu trình gồm $m \geq 2$ gangster. Để đơn giản, đánh số lại họ theo thứ tự trên chu trình: gangster 1 ngắm gangster 2, gangster 2 ngắm gangster 3, v.v., còn gangster m ngắm gangster 1. Khi thuận tiện, ta viết gangster 0 để chỉ gangster m . Thời điểm gangster i dự kiến bắn là t_i .

Đếm số phát súng dễ hơn đếm trực tiếp số người sống sót. Mỗi phát súng thực sự được bắn ra đều hạ một mục tiêu, và các mục tiêu là khác nhau. Do đó, nếu có s phát súng thì chính xác $m - s$ gangster sống sót.

Ta gọi gangster i là một **người chắc chắn bắn** nếu

$$t_i < t_{i-1},$$

tức hấn dự kiến bóp cò trước người đang ngắm vào mình. Một người chắc chắn bắn sẽ kịp bắn bất kể sau đó bản thân có bị giết hay không.

Vì các thời điểm t_i đôi một khác nhau, mỗi chu trình có ít nhất một người chắc chắn bắn: đó là gangster có thời điểm bắn sớm nhất.

Chọn một người chắc chắn bắn i , rồi đi dọc chu trình theo hướng các phát súng cho tới người chắc chắn bắn kế tiếp j . Nếu cả chu trình chỉ có một người chắc chắn bắn thì $j = i$. Gọi

$$i = w_1, w_2, \dots, w_p$$

là các gangster gặp trên đường từ i đến ngay trước j . Vì $w_2, \dots, w_p$ không phải người chắc chắn bắn nên

$$t_{w_1} < t_{w_2} < \dots < t_{w_p}.$$

Gangster w_1 bắn và giết w_2 trước khi w_2 kịp bóp cò. Vì w_2 không bắn, w_3 sống đủ lâu để bắn; phát súng của w_3 lại giết w_4 trước lượt của w_4 , và cứ thế tiếp tục. Chính xác những gangster w_ℓ có chỉ số ℓ lẻ sẽ bắn.

Vì vậy, đoạn này đóng góp

$$d_i = \left\lfloor \frac{p+1}{2} \right\rfloor = \left\lceil \frac{p}{2} \right\rceil$$

phát súng. Các đoạn bắt đầu tại những người chắc chắn bắn tạo thành một phân hoạch của chu trình. Tổng số phát súng trong chu trình do đó bằng tổng các giá trị d_i của mọi người chắc chắn bắn.

42.2.2 Hỗ trợ các thay đổi

Kết quả trên dẫn thẳng đến thuật toán động. Với mỗi chu trình, ta duy trì tập có thứ tự gồm các người chắc chắn bắn, để có thể nhanh chóng tìm người chắc chắn bắn đứng trước và đứng sau một vị trí theo thứ tự vòng tròn. Đồng thời, ta duy trì tổng các đóng góp d_i ; tổng này chính là số phát súng của chu trình.

Khi thay đổi t_i , chỉ hai bất đẳng thức có thể đổi giá trị đúng/sai:

$$t_i < t_{i-1} \quad \text{và} \quad t_{i+1} < t_i.$$

Do đó, chỉ gangster i và người kế tiếp $i + 1$ trên chu trình có thể nhận hoặc mất trạng thái người chắc chắn bắn. Ngoài ra, độ dài đoạn — và vì thế giá trị d_s — của người chắc chắn bắn s đứng ngay trước i cũng có thể thay đổi. Như vậy, mỗi lần cập nhật chỉ tác động đến một số hằng số phân tử và một số hằng số giá trị đóng góp.

Có thể nhìn thao tác cục bộ cụ thể như sau:

- Khi chen một người chắc chắn bắn x nằm giữa hai người chắc chắn bắn liên tiếp a và b , đoạn từ a đến trước b bị tách thành hai đoạn. Ta bỏ đóng góp cũ của a , rồi thêm đóng góp mới của a và x .
- Khi xóa x , hai đoạn bắt đầu tại a và x được gộp lại. Ta bỏ hai đóng góp cũ rồi thêm đóng góp mới của a .

Khoảng cách dùng trong công thức $d = \lceil p/2 \rceil$ là số đỉnh từ đầu đoạn đến ngay trước người chắc chắn bắn kế tiếp, tính theo vòng tròn. Có thể lưu vị trí của mỗi gangster trong chu trình để tính khoảng cách này trong $O(1)$.

Sau khi cập nhật tổng số phát súng, số người sống sót trên toàn bộ các chu trình là

$$n - (\text{tổng số phát súng}).$$

42.2.3 Cài đặt

Tập người chắc chắn bắn của mỗi chu trình được lưu bằng một cây tìm kiếm nhị phân cân bằng, chẳng hạn set trong thư viện chuẩn C++ . Chen, xóa, cũng như tìm phần tử liền trước hoặc liền sau đều mất $O(\log n)$ thời gian. Khi đi qua đầu hoặc cuối set, ta quay vòng sang phần tử cuối hoặc đầu để phản ánh thứ tự chu trình.

Ban đầu, ta phân rã hoán vị và đánh số vị trí trong từng chu trình. Sau đó, ta liệt kê những người chắc chắn bắn theo thứ tự trên mỗi chu trình, dùng dãy đã sắp thứ tự ấy để khởi tạo cấu trúc, rồi tính các đóng góp. Toàn bộ bước khởi tạo mất $O(n)$ thời gian.

Mỗi thay đổi chỉ cần kiểm tra lại trạng thái của i và người kế tiếp, thực hiện một số hằng số thao tác trên cây cân bằng, rồi cập nhật tổng số phát súng. Vì thế, tổng độ phức tạp là

$$O(n + q \log n),$$

với q là số lần thay đổi. Bộ nhớ sử dụng là

$$O(n).$$

Hang động ★★

Tác giả đề: Tomasz Idziaszek

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 256 MB

Mã bài: jas

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/jas>

43.1 Đề bài

Một nhóm các nhà thám hiểm hang động đang lên kế hoạch khảo sát một hang động mới được phát hiện. Hang động gồm n căn phòng, được đánh số từ 1 đến n . Các căn phòng được nối với nhau bằng $n - 1$ hành lang sao cho từ bất kỳ phòng nào cũng có thể đi tới bất kỳ phòng nào khác. Mỗi hành lang nối đúng hai căn phòng.

Nhóm khảo sát gồm m nhà thám hiểm, được đánh số từ 1 đến m . Mỗi người đưa ra yêu cầu về khu vực hang động mà mình muốn khảo sát. Nhà thám hiểm i muốn bắt đầu tại phòng a_i , kết thúc tại phòng b_i , và trên đường đi qua nhiều nhất d_i hành lang. Mỗi lần đi qua một hành lang đều được tính, kể cả khi đó là hành lang đã đi qua trước đó.

Byteasar, trưởng đoàn thám hiểm, muốn tại một thời điểm nào đó tất cả các nhà nghiên cứu có thể gặp nhau để trao đổi những gì

họ quan sát được. Vì vậy, anh muốn biết liệu có thể chọn một căn phòng và lập lộ trình cho mọi nhà thám hiểm sao cho tất cả các lộ trình đều đi qua căn phòng đã chọn hay không. Dĩ nhiên, mỗi lộ trình vẫn phải thỏa mãn những yêu cầu ban đầu của người thực hiện nó.

43.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên t

$$1 \leq t \leq 1000,$$

là số bộ test. Tiếp theo là mô tả của từng bộ test.

Mỗi bộ test bắt đầu bằng một dòng chứa hai số nguyên n, m

$$2 \leq n, \quad m \leq 300\,000,$$

lần lượt là số căn phòng và số nhà thám hiểm.

$n - 1$ dòng tiếp theo mô tả các hành lang. Mỗi dòng chứa hai số nguyên u_i, w_i

$$1 \leq u_i, w_i \leq n,$$

cho biết phòng u_i và phòng w_i được nối trực tiếp bằng một hành lang.

m dòng tiếp theo mô tả yêu cầu của các nhà thám hiểm. Dòng thứ i chứa ba số nguyên a_i, b_i, d_i

$$1 \leq a_i, b_i \leq n, \quad 1 \leq d_i \leq 600\,000.$$

Nhà thám hiểm i bắt đầu tại phòng a_i , kết thúc tại phòng b_i , và đi qua hành lang nhiều nhất d_i lần. Dữ liệu bảo đảm luôn tồn tại một cách đi từ a_i đến b_i qua không quá d_i hành lang.

Tổng các giá trị n trên mọi bộ test không vượt quá 300 000; tổng các giá trị m cũng không vượt quá 300 000.

43.1.2 Dữ liệu ra

In đúng t dòng, mỗi dòng là đáp án cho một bộ test.

Nếu có thể lập lộ trình sao cho tất cả các nhà thám hiểm đều đi qua cùng một căn phòng, in TAK, rồi in số hiệu của một căn phòng có thể dùng làm nơi gặp mặt. Nếu không thể, chỉ in NIE.

Nếu có nhiều đáp án đúng, có thể in ra bất kỳ đáp án nào.

43.1.3 Ví dụ

Dữ liệu vào

```
2
5 3
1 2
2 3
2 4
3 5
1 4 2
5 5 5
3 2 1
3 2
1 2
2 3
1 1 2
3 3 1
```

Dữ liệu ra

```
TAK 2
NIE
```

43.2 Lời giải

Trước hết, ta đưa vào một số ký hiệu. Gọi $d(u, w)$ là khoảng cách giữa hai phòng u, w , tức số hành lang trên đường đi ngắn nhất giữa chúng.

Mỗi nhà thám hiểm được mô tả bởi bộ ba a_i, b_i, d_i : người đó đi từ a_i đến b_i và được phép đi qua nhiều nhất d_i hành lang. Vì vậy, người này có thể ghé thăm phòng x khi và chỉ khi

$$d(a_i, x) + d(x, b_i) \leq d_i. \quad (1)$$

Ta gọi tập các phòng mà nhà thám hiểm i có thể ghé thăm là **miền** i .

Nói theo ngôn ngữ lý thuyết đồ thị, dữ liệu cho một cây n đỉnh — sơ đồ hang động — cùng mô tả của m miền. Ta cần kiểm tra xem giao của tất cả các miền có chứa đỉnh nào hay không; nếu có, phải đưa ra một đỉnh như vậy.

43.2.1 Lời giải đơn giản

Trước tiên, có thể xây dựng một lời giải $O(n(n+m))$. Ta tính khoảng cách giữa mọi cặp đỉnh trong $O(n^2)$. Với mỗi trong m miền, duyệt qua mọi đỉnh và dùng công thức (1) để liệt kê các đỉnh thuộc miền đó trong $O(n)$. Cuối cùng, với mỗi đỉnh, đếm số miền chứa nó để lấy giao của tất cả các miền.

Lời giải này khá đơn giản, nhưng hiển nhiên chưa đủ nhanh.

43.3 Chia để trị

Lời giải tốt hơn đầu tiên sử dụng kỹ thuật chia để trị. Kỹ thuật này thường được áp dụng cho mảng: thực hiện lượng công việc tỉ lệ với kích thước mảng, chia mảng thành hai nửa, rồi giải đệ quy từng nửa. Để làm điều tương tự trên cây, ta cần biết cách chia cây thành những phần tương đối cân bằng.

Bổ đề 1. Cho cây T có n đỉnh. Trong thời gian $O(n)$, ta có thể tìm một đỉnh v sao cho sau khi xóa v , mọi cây thành phần còn lại đều có không quá $n/2$ đỉnh. Đỉnh v như vậy được gọi là một **trọng tâm** của cây.

Chứng minh. Gốc hóa T tại một đỉnh tùy ý, rồi tính kích thước cây con gốc tại mỗi đỉnh. Bước này tốn $O(n)$.

Bắt đầu ở gốc. Chẳng nào một cây con gốc tại con của đỉnh hiện tại còn có nhiều hơn $n/2$ đỉnh, ta chuyển xuống người con đó. Vì chỉ đi xuống cây nên quá trình sẽ dừng tại một đỉnh v .

Cây con gốc tại v có nhiều hơn $n/2$ đỉnh: ta bắt đầu tại gốc và chỉ chuyển vào một cây con có tính chất này. Nếu v không phải gốc, sau khi xóa v , thành phần chứa cha của v có kích thước nhỏ hơn $n/2$, bởi cây con gốc tại v có kích thước lớn hơn $n/2$. Mặt khác, mọi cây con gốc tại một người con của v có không quá $n/2$ đỉnh; nếu không, cuộc đi xuống chưa thể dừng tại v . Do đó v là một trọng tâm. $\square$

Ý tưởng tổng quát của lời giải chia để trị như sau. Trước tiên, tìm trọng tâm v của T , rồi kiểm tra xem v có thuộc giao của tất cả các miền hay không. Nếu có, ta đã tìm được đáp án. Nếu không, nhưng giao ấy khác rỗng, thì toàn bộ giao phải nằm trong đúng một cây thành phần thu được sau khi xóa v . Ta xác định cây thành phần đó rồi tiếp tục đệ quy trong nó.

Sau khi tìm trọng tâm v , tính khoảng cách từ v tới các đỉnh đang xét trong $O(n)$. Dùng các khoảng cách ấy và công thức (1), ta kiểm tra v có thuộc từng miền hay không trong tổng thời gian $O(m)$. Nếu v thuộc mọi miền, thuật toán dừng và trả về v .

Gọi $\mathcal{T}_v$ là tập các cây thành phần sinh ra khi xóa v khỏi T . Vì v không thuộc giao, tồn tại ít nhất một miền i , được xác định bởi a_i, b_i, d_i , không chứa v . Hai đỉnh a_i, b_i phải nằm trong cùng một cây $F \in \mathcal{T}_v$. Thật vậy, mọi đỉnh trên đường đi từ a_i đến b_i đều thuộc miền i . Nếu a_i, b_i thuộc hai thành phần khác nhau thì đường đi ấy phải qua v , trái với việc v không thuộc miền i .

Hơn nữa, toàn bộ miền i nằm trong F . Do đó, nếu giao của tất cả các miền khác rỗng, giao ấy cũng phải nằm trong F . Ta chỉ cần gọi đệ quy trên F , không cần xét các thành phần còn lại.

Một mức đệ quy có thể được cài đặt trong $O(n + m)$, với n là kích thước cây hiện đang xét. Theo định nghĩa trọng tâm, sau mỗi bước, kích thước cây giảm ít nhất một nửa. Vì vậy, độ sâu đệ quy là $O(\log n)$, cho cận thời gian trực tiếp

$$O((n + m) \log n).$$

Có thể siết cận này tốt hơn. Ở mỗi mức, lượng công việc xử lý

các đỉnh tỉ lệ với kích thước phần cây hiện tại, mà kích thước này giảm ít nhất một nửa sau mỗi bước. Vì

$$n + \frac{n}{2} + \frac{n}{4} + \dots \leq 2n,$$

tổng thời gian dành cho các đỉnh chỉ là $O(n)$. Tuy nhiên, ở mỗi mức ta vẫn có thể phải duyệt cả m miền. Do đó, tổng thời gian là

$$O(n + m \log n).$$

Điều quan trọng là sau khi tìm trọng tâm, ta không thể đơn giản gọi đệ quy trên **mọi** cây trong $\mathcal{T}_v$. Dù các cây thành phần là những phần rời nhau của cây đầu vào, một miền có thể giao với nhiều cây trong $\mathcal{T}_v$. Khi ấy, miền đó sẽ bị xử lý trong nhiều lời gọi đệ quy ở cùng một mức, khiến thời gian chạy tăng đáng kể. Lập luận ở trên cho phép ta chỉ đi vào một cây thành phần duy nhất.

43.4 Lời giải nhanh hơn

Bài toán còn có một thuật toán vừa nhanh hơn vừa đơn giản hơn; phần khó hơn nằm ở chứng minh tính đúng đắn.

Chọn một đỉnh v tùy ý. Ta định nghĩa **khoảng cách từ miền i đến v** là khoảng cách giữa v và đỉnh gần v nhất thuộc miền i . Với miền i được mô tả bởi a_i, b_i, d_i , khoảng cách này bằng

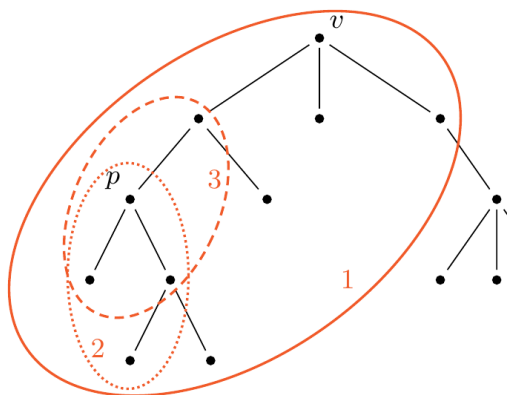
$$\max \left(0, \left\lceil \frac{d(a_i, v) + d(v, b_i) - d_i}{2} \right\rceil \right). \quad (2)$$

Công thức có thể thấy trực tiếp từ cấu trúc của cây. Đường đi từ a_i đến v và đường đi từ v đến b_i có một đoạn thừa đối xứng so với một lộ trình đi từ a_i đến b_i . Khi đi một cạnh từ v theo hướng tới miền, tổng $d(a_i, x) + d(x, b_i)$ giảm nhiều nhất 2. Vì thế, phần vượt quá giới hạn d_i phải được bù theo từng cặp đơn vị; phép làm tròn lên xử lý trường hợp phần vượt là số lẻ. Nếu v đã thuộc miền thì khoảng cách bằng 0.

Ý tưởng thuật toán chỉ gồm vài bước:

1. Tìm miền i nằm xa v nhất.
2. Trong miền i , tìm đỉnh p gần v nhất.
3. Kiểm tra xem p có thuộc mọi miền hay không.

Sự kiện then chốt là: **nếu giao của tất cả các miền khác rỗng, thì p thuộc giao ấy.** Điều này đúng với mọi lựa chọn đỉnh ban đầu v , dù những lựa chọn khác nhau có thể dẫn tới những đỉnh p khác nhau.



Hình 43.1: Hình 1. Cây được gốc hóa tại v cùng ba miền đánh số 1, 2, 3. Miền 2 nằm xa v nhất; p là đỉnh gần v nhất thuộc miền 2.

Để chứng minh sự kiện trên, gốc hóa toàn bộ cây tại v . Giao của mọi miền, nếu tồn tại, phải nằm trong cây con gốc tại p , bởi trong số các đỉnh của miền i , p là đỉnh gần gốc nhất và mọi đỉnh khác của miền i nằm trong cây con ấy.

Mặt khác, nếu một miền j chứa một đỉnh trong cây con gốc tại p , thì nó cũng phải chứa p . Nếu không, miền j sẽ nằm xa v hơn miền i , trái với cách chọn i . Vì vậy, nếu giao của tất cả các miền khác rỗng thì p thuộc mọi miền, tức chính p là một đáp án.

43.4.1 Cài đặt thuật toán tuyến tính

Thuật toán có thể được cài đặt trong thời gian $O(n + m)$ cho mỗi bộ test theo tổng kích thước dữ liệu:

1. Chọn một đỉnh v tùy ý và tính khoảng cách từ v tới mọi đỉnh của cây trong $O(n)$.
2. Dùng công thức (2) để tính khoảng cách từ v tới từng miền trong $O(1)$, rồi chọn một miền i có khoảng cách lớn nhất. Tổng thời gian của bước này là $O(m)$.
3. Tính khoảng cách từ a_i và từ b_i tới tất cả các đỉnh. Nhờ công thức (1), với mỗi đỉnh x , ta kiểm tra được trong $O(1)$ liệu x có thuộc miền i hay không.
4. Duyệt mọi đỉnh thuộc miền i và chọn đỉnh p có khoảng cách tới v nhỏ nhất. Bước này tốn $O(n)$.
5. Tính khoảng cách từ p tới mọi đỉnh. Với từng miền j , kiểm tra

$$d(a_j, p) + d(p, b_j) \leq d_j.$$

Nếu bất đẳng thức đúng với mọi j , in TAK p ; nếu có miền không chứa p , in NIE.

Mỗi lượt tính khoảng cách trên cây có thể thực hiện bằng DFS hoặc BFS trong $O(n)$. Ta chỉ thực hiện một số hằng các lượt duyệt cây và một số hằng các lượt duyệt danh sách miền. Vì thế, độ phức tạp của mỗi bộ test là

$$O(n + m)$$

về thời gian và

$$O(n + m)$$

về bộ nhớ. Với các giới hạn cộng dồn trong đề, tổng thời gian trên toàn bộ dữ liệu vào cũng là $O(\sum n + \sum m)$.

Thuyền trưởng ★★

Tác giả đề: Jakub Łacki

Tác giả lời giải: Jakub Łacki

Giới hạn bộ nhớ: 256 MB

Mã bài: kap

Đề trực tuyến: <https://oi.edu.pl/en/archive/amppz/2014/kap>

44.1 Đề bài

Thuyền trưởng Bajtazar cùng người phó thuyền trưởng không thể thay thế của mình, Bajtek, đi lại trên vùng biển Byteotia. Trên biển có n hòn đảo, được đánh số từ 1 đến n . Con tàu của thuyền trưởng đang neo tại đảo số 1, và trong chuyến thám hiểm này ông dự định đi tới đảo số n .

Trong suốt hành trình, con tàu luôn di chuyển theo một trong bốn hướng: bắc, nam, đông hoặc tây. Ở mỗi thời điểm, hoặc thuyền trưởng, hoặc phó thuyền trưởng đứng lái. Mỗi khi tàu rẽ một góc 90° , hai người lại đổi chỗ cho nhau tại bánh lái.

Trên đường đi, tàu có thể dừng tại các đảo khác. Sau mỗi lần dừng, thuyền trưởng được quyền quyết định mình có phải là người cầm lái đầu tiên hay không. Nói cách khác, trên mỗi chặng từ một đảo đến một đảo khác, một người điều khiển tàu khi tàu đi theo

hướng bắc hoặc nam, còn người kia điều khiển khi tàu đi theo hướng đông hoặc tây. Đặc biệt, nếu cả chặng chỉ đi theo đúng một trong bốn hướng thì chỉ một người phải cầm lái trên chặng đó.

Thuyền trưởng muốn lập lộ trình cho chuyến đi sắp tới và phân chia công việc sao cho tổng thời gian ông phải cầm lái là ít nhất. Ông không quan tâm lộ trình được chọn dài đến mức nào. Tàu di chuyển với vận tốc không đổi là một đơn vị mỗi giờ.

44.1.1 Dữ liệu vào

Dòng đầu chứa một số nguyên n

$$2 \leq n \leq 200\,000,$$

là số đảo trên biển.

Để đơn giản, ta đặt trên biển Byteotia một hệ tọa độ có hai trục song song với các hướng chính. Mỗi đảo được biểu diễn bởi một điểm. Trong n dòng tiếp theo, dòng thứ i chứa hai số nguyên x_i, y_i

$$0 \leq x_i, y_i \leq 1\,000\,000\,000,$$

là tọa độ của đảo thứ i . Không có hai đảo nào có cùng tọa độ.

44.1.2 Dữ liệu ra

In một số nguyên: số giờ ít nhất mà thuyền trưởng phải cầm lái trên hành trình từ đảo số 1 đến đảo số n .

44.1.3 Ví dụ

Dữ liệu vào

```
5
2 2
1 1
4 5
7 1
6 7
```

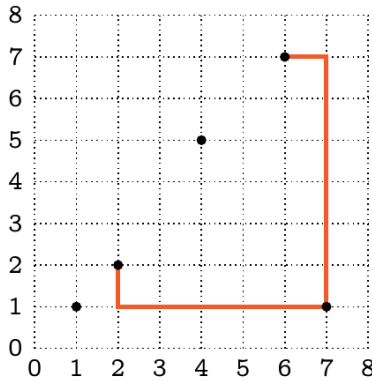
Dữ liệu ra

2

Giải thích. Thuyền trưởng có thể đi theo lộ trình qua đảo 4:

$$(2, 2) \rightarrow (7, 1) \rightarrow (6, 7).$$

Trên chặng từ đảo 1 đến đảo 4, ông chỉ cần lái trong một giờ khi tàu đi về phía nam. Trên chặng thứ hai, ông chỉ cần lái khi tàu đi về phía tây, cũng trong một giờ. Vì vậy, tổng thời gian ông phải cần lái là 2 giờ.



Hình 44.1: Hình minh họa ví dụ. Một lộ trình đạt đáp án đi từ $(2, 2)$ xuống $(2, 1)$, đi sang phải tới đảo 4 $= (7, 1)$, đi lên tới $(7, 7)$, rồi đi sang trái tới đảo 5 $= (6, 7)$.

44.2 Lời giải

Ta có n điểm

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

trên mặt phẳng và cần đi từ x_1, y_1 đến x_n, y_n .

44.2.1 Chi phí của một chặng

Xét một chặng từ đảo (x', y') đến đảo (x'', y'') . Tàu có thể hoàn thành chặng này bằng cách đi theo hai phương ngang và dọc. Sau khi dừng ở đảo đầu chặng, thuyền trưởng được chọn ai sẽ cầm lái trước; vì thế ông có thể chọn chỉ cầm lái trên phần di chuyển theo phương có độ dài ngắn hơn. Chi phí của chặng, tức số giờ thuyền trưởng phải cầm lái, là

$$w((x', y'), (x'', y'')) = \min(|x'' - x'|, |y'' - y'|).$$

Do đó, bài toán trở thành tìm một lộ trình từ điểm 1 đến điểm n có tổng chi phí nhỏ nhất, trong đó ta được phép chuyển trực tiếp giữa hai điểm bất kỳ với trọng số như trên.

Cách đo chi phí này khá khác thường. Nói chính xác hơn, nó không phải là một metric trên tập điểm, vì bất đẳng thức tam giác có thể không đúng.

Chẳng hạn, xét ba điểm

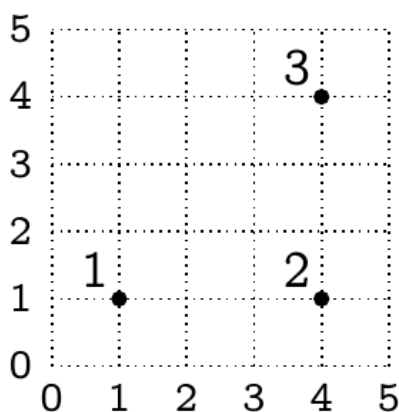
$$p_1 = (1, 1), \quad p_2 = (4, 1), \quad p_3 = (4, 4).$$

Đi trực tiếp từ p_1 đến p_3 tốn

$$\min(|4 - 1|, |4 - 1|) = 3,$$

nhưng đi từ p_1 qua p_2 rồi tới p_3 có tổng chi phí bằng 0. Như vậy, ghé một điểm trung gian có thể rẻ hơn hẳn đi thẳng tới đích.

Vì chi phí được đo theo một cách đặc biệt như vậy, một số trực giác quen thuộc về đường đi ngắn trên mặt phẳng không còn dùng được. Dù thế, lời giải cuối cùng lại khá đơn giản.

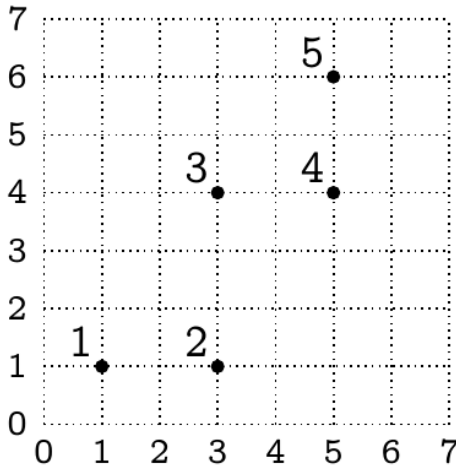


Hình 44.2: Hình 1. Ba điểm $p_1 = (1, 1)$, $p_2 = (4, 1)$, $p_3 = (4, 4)$: bước trực tiếp $p_1 \rightarrow p_3$ có chi phí 3, còn đường $p_1 \rightarrow p_2 \rightarrow p_3$ có chi phí 0.

Ta có thể thử đoán rằng một lộ trình tối ưu luôn chỉ cần nhiều nhất một điểm trung gian. Điều này không đúng. Ví dụ, với năm điểm

$$(1, 1), (3, 1), (3, 4), (5, 4), (5, 6),$$

có một lộ trình chi phí 0 từ điểm đầu tới điểm cuối, nhưng nó lần lượt phải đi qua cả ba điểm ở giữa.



Hình 44.3: Hình 2. Một “cầu thang” gồm năm điểm, trong đó mỗi cặp liên tiếp nằm trên cùng một đường ngang hoặc dọc. Đường từ điểm 1 tới điểm 5 có chi phí 0, nhưng dùng ba điểm trung gian.

44.2.2 Chèn các điểm trung gian mà không tăng chi phí

Ta có thể di chuyển miễn phí giữa hai điểm nằm trên cùng một đường thẳng đứng hoặc nằm ngang. Tổng quát hơn, việc di chuyển giữa hai điểm sẽ rẻ nếu hoành độ hoặc tung độ của chúng gần nhau. Ta sẽ hình thức hóa nhận xét này.

Giả sử ta đi từ

$$p_a = (x_a, y_a)$$

đến

$$p_c = (x_c, y_c),$$

với $x_a \leq x_c$, và bước này có chi phí

$$|x_c - x_a|,$$

nghĩa là $|x_c - x_a| \leq |y_c - y_a|$. Giả sử còn có một điểm

$$p_b = (x_b, y_b)$$

thỏa

$$x_a \leq x_b \leq x_c.$$

Khi ấy, ta có thể đi từ p_a tới p_c qua p_b mà không làm tăng chi phí. Thật vậy, bước thứ nhất tốn không quá

$$|x_b - x_a| = x_b - x_a,$$

còn bước thứ hai tốn không quá

$$|x_c - x_b| = x_c - x_b.$$

Vì vậy, tổng chi phí không vượt quá

$$(x_b - x_a) + (x_c - x_b) = x_c - x_a.$$

Suy ra, nếu bước từ p_a đến p_c có chi phí $|x_c - x_a|$, ta có thể chèn vào giữa bước ấy **tất cả** các điểm có hoành độ nằm giữa x_a và x_c , miễn là ghé chúng theo thứ tự hoành độ không giảm. Lộ trình sau khi chèn chắc chắn không đắt hơn bước đi trực tiếp. Lập luận hoàn toàn tương tự cũng đúng với tung độ.

44.2.3 Chỉ cần nối các điểm kề nhau theo từng tọa độ

Trước hết, tạm giả sử không có hai điểm nào nằm trên cùng một đường thẳng đứng hoặc nằm ngang. Xét một lộ trình chi phí nhỏ nhất giữa hai điểm bất kỳ. Mỗi bước của lộ trình đi từ x_i, y_i đến x_j, y_j , với chi phí bằng một trong hai giá trị

$$|x_j - x_i| \quad \text{hoặc} \quad |y_j - y_i|.$$

Theo nhận xét ở trên, ta có thể thay mỗi bước bằng một chuỗi bước đi qua tất cả các điểm có hoành độ nằm giữa x_i và x_j , hoặc qua tất cả các điểm có tung độ nằm giữa y_i và y_j . Trong cả hai trường hợp, tổng chi phí không tăng.

Lập hai mảng $t_x[1..n]$ và $t_y[1..n]$, đều chứa toàn bộ các điểm:

- trong t_x , các điểm được sắp theo hoành độ không giảm;
- trong t_y , các điểm được sắp theo tung độ không giảm.

Nhận xét vừa chứng minh dẫn tới kết luận then chốt: giữa hai điểm bất kỳ luôn tồn tại một lộ trình tối ưu mà ở mỗi bước, từ điểm hiện tại p , ta chỉ đi đến một trong nhiều nhất bốn điểm sau:

- điểm ngay trước p trong t_x ;
- điểm ngay sau p trong t_x ;
- điểm ngay trước p trong t_y ;
- điểm ngay sau p trong t_y .

44.2.4 Dựng đồ thị và chạy Dijkstra

Ta dựng một đồ thị vô hướng G , trong đó mỗi điểm là một đỉnh. Sau khi xây dựng t_x và t_y , nối bằng cạnh mọi cặp điểm nằm ở hai vị trí liên tiếp trong một trong hai mảng. Gán cho cạnh nối i và j trọng số

$$w(i, j) = \min(|x_i - x_j|, |y_i - y_j|).$$

Đồ thị có n đỉnh và không quá $2n - 2$ cạnh trước khi loại các cạnh trùng. Theo kết luận ở phần trước, đường đi có tổng trọng số nhỏ nhất giữa hai đỉnh trong G tương ứng với lộ trình rẻ nhất giữa hai điểm trên mặt phẳng:

- mọi đường đi trong G hiển nhiên là một lộ trình hợp lệ giữa các đảo;
- ngược lại, mỗi bước của một lộ trình tối ưu trong đồ thị đầy đủ có thể được thay bằng một chuỗi cạnh giữa các điểm liên tiếp trong t_x hoặc t_y , mà không làm tăng tổng chi phí.

Nếu có nhiều điểm nằm trên cùng một đường thẳng đứng hoặc nằm ngang thì thuật toán vẫn đúng. Khi sắp xếp, các điểm ấy đứng liên tiếp theo tọa độ tương ứng và được nối với nhau bởi một đường đi gồm toàn cạnh trọng số 0.

Cuối cùng, dùng thuật toán Dijkstra từ đỉnh 1. Khoảng cách ngắn nhất tìm được tới đỉnh n chính là số giờ ít nhất mà thuyền trưởng phải cầm lái.

44.2.5 Độ phức tạp

Sắp xếp hai mảng t_x, t_y tốn $O(n \log n)$ thời gian. Đồ thị có $O(n)$ cạnh, nên Dijkstra với hàng đợi ưu tiên cũng chạy trong $O(n \log n)$ thời gian.

Tổng độ phức tạp là

$$O(n \log n)$$

thời gian và

$$O(n)$$

bộ nhớ.

Thông tin nguồn

- *Looking for a Challenge 2*, bản tiếng Anh, Warszawa, 2019.
- *W poszukiwaniu wyzwań 2*, bản tiếng Ba Lan, ấn bản sửa đổi lần hai, phiên bản 2.1, Warszawa, 2019.